

Random-Key Optimizers

A tutorial

Mauricio G. C. Resende
U. of Washington & Fed. U. of São Paulo

Talk given at LION 8 ~ Ischia, Italy ~ June 9, 2024

Overview

- Encoding and decoding with vectors of random keys
- Examples of encoding and decoding with random keys
- Random-Key Optimizers
  - RKGA & BRKGA
  - Dual Annealing RKO
  - Continuous GRASP RKO



Part I

Encoding and decoding with vectors of random keys



Encoding & decoding with random keys

Encoding

- Solutions to combinatorial optimization problems can be encoded with an *n*-vector of random keys
- A random key is a randomly generated real number in the interval [0,1)
- An encoded solution is a point in the half-open real unit hypercube

Bean (1994)

Decoding

- A decoder is a deterministic algorithm (usually a heuristic)
  - INPUT: *n*-vector of random keys
  - OUTPUT: a feasible solution of the combinatorial optimization problem and its cost

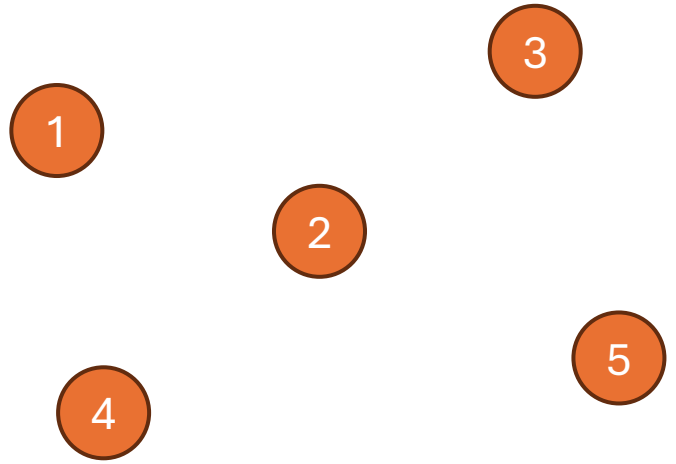
Encoding & decoding: TSP on *n* cities

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order
- OUTPUT: indices of sorted vector is permutation of cities visited in tour and total length L of tour



X = (0.085, 0.277, 0.149, 0.332, 0.148)
s[X] = (0.085, 0.148, 0.149, 0.277, 0.332)
π(s[X]) = (1, 5, 3, 2, 4)

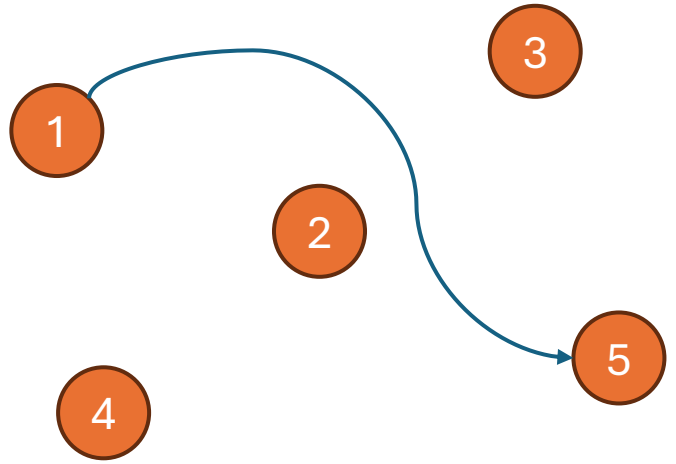
Encoding & decoding: TSP on *n* cities

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order
- OUTPUT: indices of sorted vector is permutation of cities visited in tour and total length L of tour



X = (0.085, 0.277, 0.149, 0.332, 0.148)
s[X] = (0.085, 0.148, 0.149, 0.277, 0.332)
π(s[X]) = (1, 5, 3, 2, 4)
L = l(1,5)

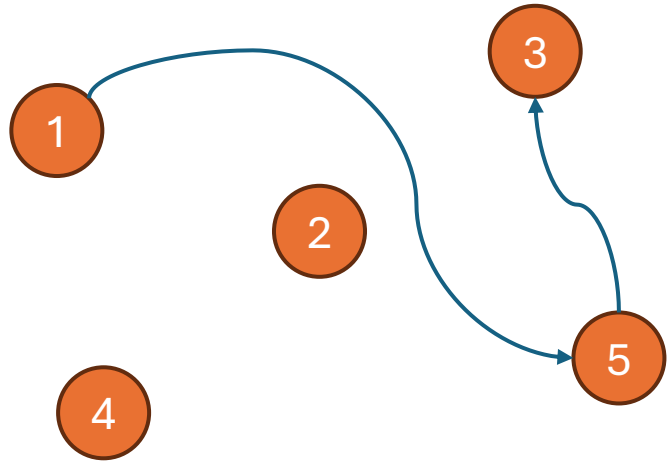
Encoding & decoding: TSP on *n* cities

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order
- OUTPUT: indices of sorted vector is permutation of cities visited in tour and total length L of tour



X = (0.085, 0.277, 0.149, 0.332, 0.148)
s[X] = (0.085, 0.148, 0.149, 0.277, 0.332)
π(s[X]) = (1, 5, 3, 2, 4)
L = l(1,5) + l(5,3)

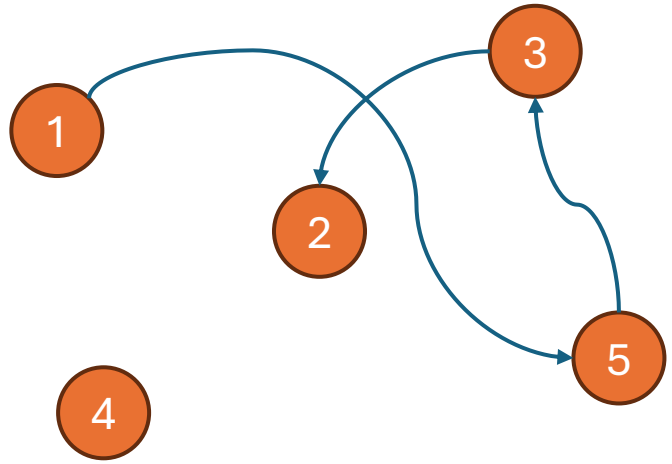
Encoding & decoding: TSP on *n* cities

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order
- OUTPUT: indices of sorted vector is permutation of cities visited in tour and total length L of tour



X = (0.085, 0.277, 0.149, 0.332, 0.148)
s[X] = (0.085, 0.148, 0.149, 0.277, 0.332)
π(s[X]) = (1, 5, 3, 2, 4)
L = l(1,5) + l(5,3) + l(3,2)

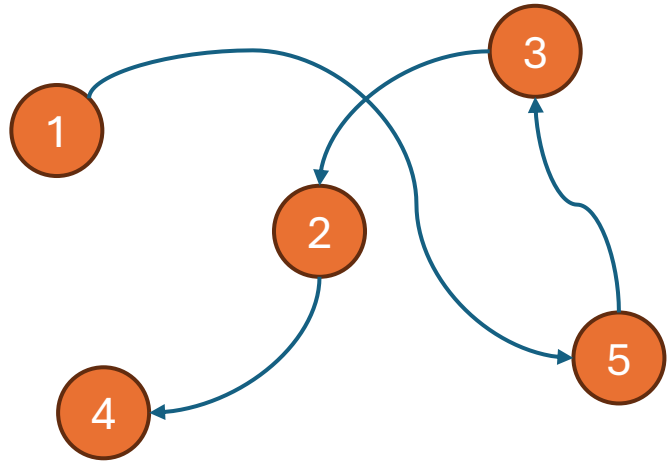
Encoding & decoding: TSP on *n* cities

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order
- OUTPUT: indices of sorted vector is permutation of cities visited in tour and total length L of tour



X = (0.085, 0.277, 0.149, 0.332, 0.148)
s[X] = (0.085, 0.148, 0.149, 0.277, 0.332)
π(s[X]) = (1, 5, 3, 2, 4)
L = l(1,5) + l(5,3) + l(3,2) + l(2,4)

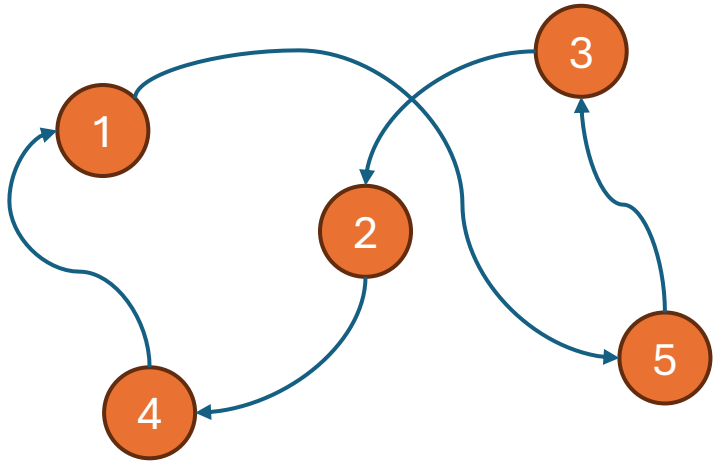
Encoding & decoding: TSP on *n* cities

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order
- OUTPUT: indices of sorted vector is permutation of cities visited in tour and total length L of tour



X = (0.085, 0.277, 0.149, 0.332, 0.148)
s[X] = (0.085, 0.148, 0.149, 0.277, 0.332)
π(s[X]) = (1, 5, 3, 2, 4)
L = l(1,5) + l(5,3) + l(3,2) + l(2,4) + l(4,1)

Encoding & decoding: Set covering with *n* elements

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

1									
1	1	0	1	0	1	0	0	0	◇
0	1	0	0	0	0	0	0	1	
0	0	1	0	0	0	1	1	0	
1	1	1	0	1	1	1	0	1	◇
0	0	0	1	1	1	0	1	0	

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, 5, 3, 6, 2, 8, 4, 9, 7)
|S| = 1

Encoding & decoding: Set covering with *n* elements

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

1				**1**					
1	1	0	1	0	1	0	0	0	◇
0	1	0	0	0	0	0	0	1	
0	0	1	0	0	0	1	1	0	
1	1	1	0	1	1	1	0	1	◇
0	0	0	1	1	1	0	1	0	◇

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, 3, 6, 2, 8, 4, 9, 7)
|S| = 2

Encoding & decoding: Set covering with *n* elements

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

1		1		1					
1	1	0	1	0	1	0	0	0	◇
0	1	0	0	0	0	0	0	1	
0	0	1	0	0	0	1	1	0	◇
1	1	1	0	1	1	1	0	1	◇
0	0	0	1	1	1	0	1	0	◇

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, 6, 2, 8, 4, 9, 7)
|S| = 3

Encoding & decoding: Set covering with *n* elements

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

1		1		1	1				
1	1	0	1	0	1	0	0	0	◇
0	1	0	0	0	0	0	0	1	
0	0	1	0	0	0	1	1	0	◇
1	1	1	0	1	1	1	0	1	◇
0	0	0	1	1	1	0	1	0	◇

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, 2, 8, 4, 9, 7)
|S| = 4

Encoding & decoding: Set covering with *n* elements

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

1	1	1		1	1				
1	1	0	1	0	1	0	0	0	◇
0	1	0	0	0	0	0	0	1	◇
0	0	1	0	0	0	1	1	0	◇
1	1	1	0	1	1	1	0	1	◇
0	0	0	1	1	1	0	1	0	◇

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, **2**, 8, 4, 9, 7)
|S| = 5

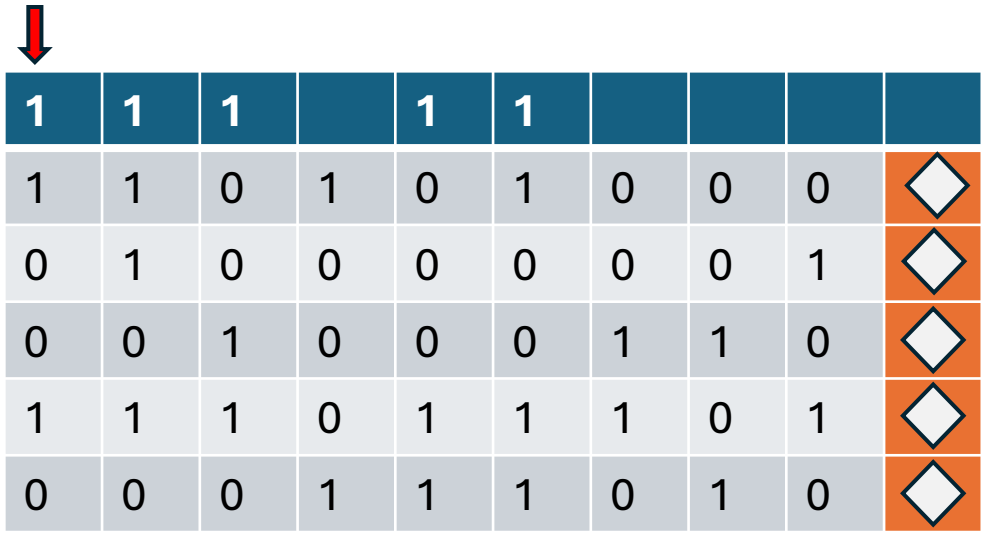
Encoding & decoding: Set covering with *n* elements

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

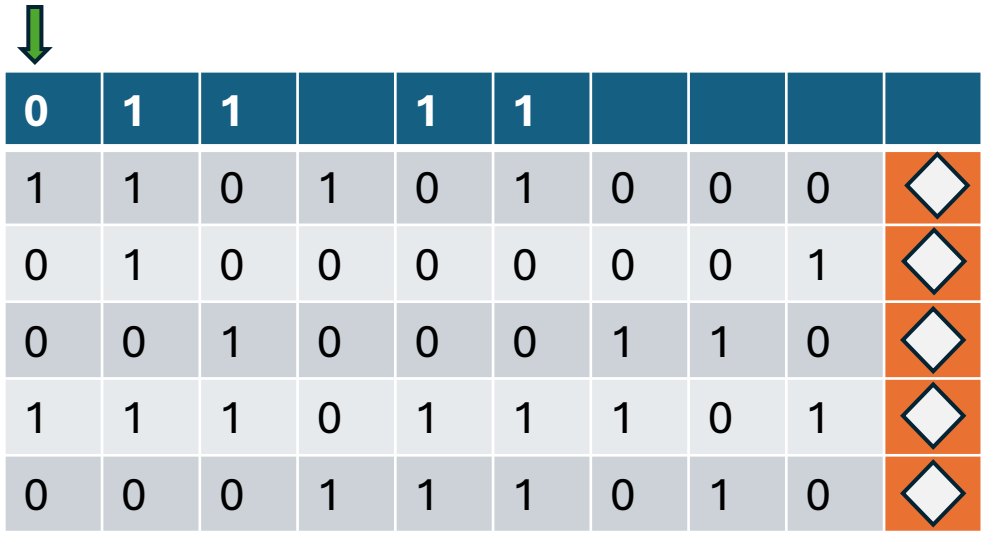
Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|



X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, **2**, 8, 4, 9, 7)
|S| = 5

Encoding & decoding: Set covering with *n* elements



Encoding

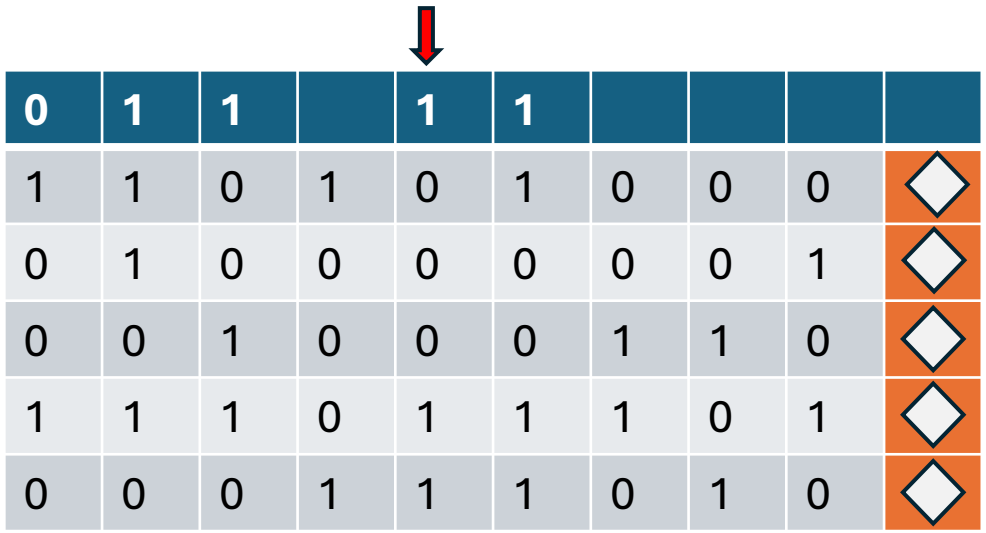
- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, **2**, 8, 4, 9, 7)
|S| = 4

Encoding & decoding: Set covering with *n* elements



Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, **2**, 8, 4, 9, 7)
|S| = 4

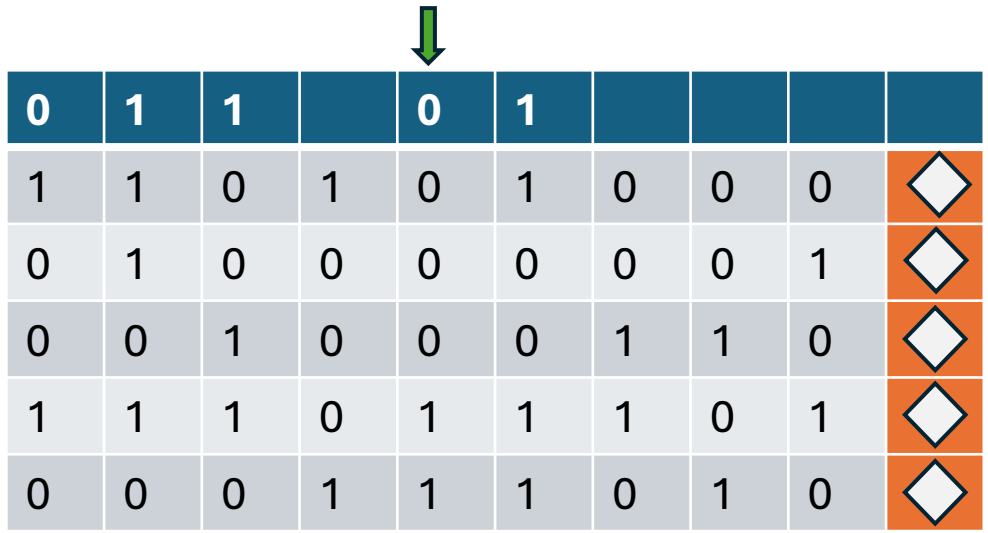
Encoding & decoding: Set covering with *n* elements

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

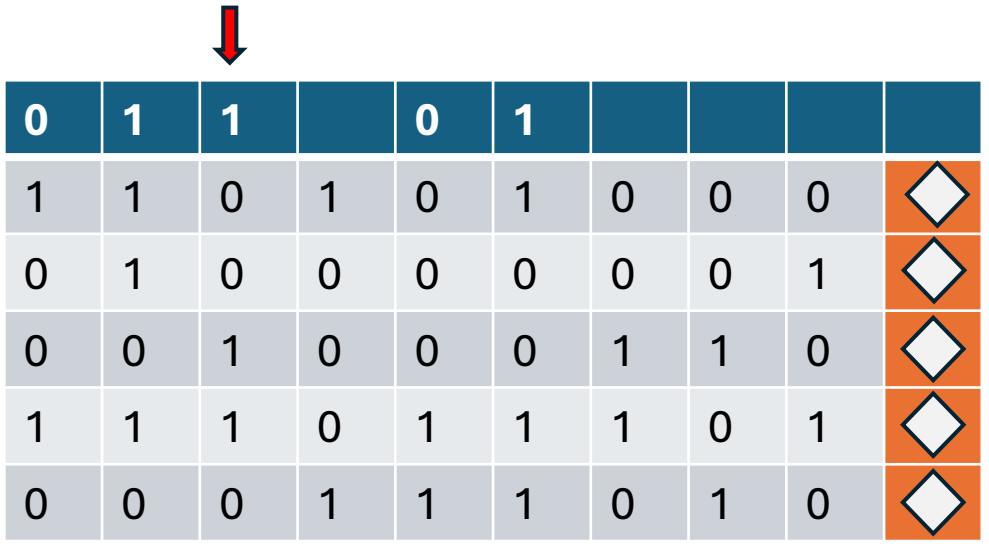
Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|



X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, **2**, 8, 4, 9, 7)
|S| = 3

Encoding & decoding: Set covering with *n* elements



Encoding

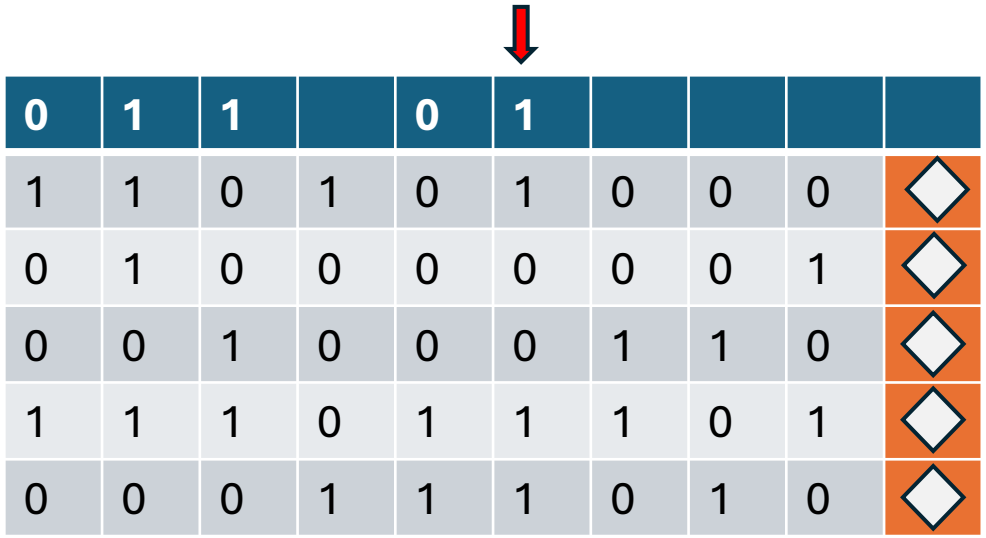
- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, **2**, 8, 4, 9, 7)
|S| = 3

Encoding & decoding: Set covering with *n* elements



Encoding

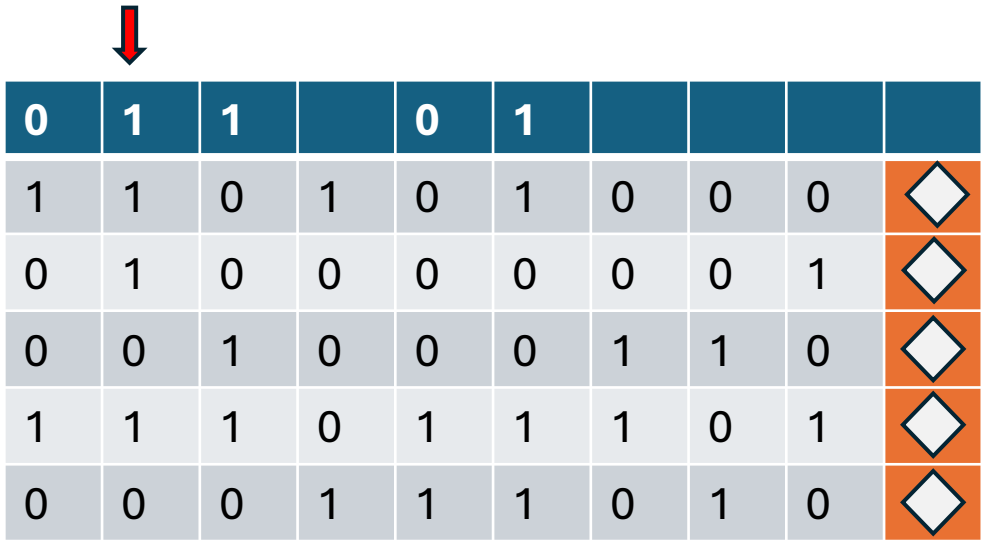
- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, **2**, 8, 4, 9, 7)
|S| = 3

Encoding & decoding: Set covering with *n* elements



Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, **2**, 8, 4, 9, 7)
|S| = 3

Encoding & decoding: Set covering with *n* elements

Encoding

- Solutions are encoded with an *n*-vector *X* of random keys

Decoder

- INPUT: *n*-vector of random keys
- Sort the *n* keys in increasing order. : Indices of sorted vector are sequence that elements are placed one at a time in the partial cover. Scan again removing superfluous elements.
- OUTPUT: indices S of selected elements in and total cover size |*S*|

1	2	3	4	5	6	7	8	9	
0	**1**	**1**		**0**	**1**				
1	1	0	1	0	1	0	0	0	◇
0	1	0	0	0	0	0	0	1	◇
0	0	1	0	0	0	1	1	0	◇
1	1	1	0	1	1	1	0	1	◇
0	0	0	1	1	1	0	1	0	◇

X = (.08, .28, .15, .33, .14, .16, .84, .31, .41)
s[X] = (.08, .14, .15, .16, .28, .31, .33, .41, .84)
π(s[X]) = (**1**, **5**, **3**, **6**, **2**, 8, 4, 9, 7)
|S| = 3

S = {2, 3, 6}

OSPF routing

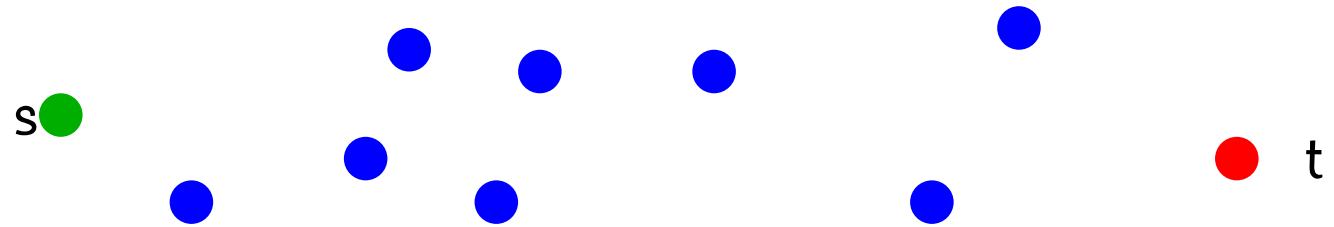
- Given a network G = (N,A), where N is the set of routers and A is the set of links.

OSPF routing

- Given a network G = (N,A), where N is the set of routers and A is the set of links.
- The OSPF (open shortest path first) routing protocol assumes each link a has a weight w(a) assigned to it so that a packet from a source router s to a destination router t is routed on a shortest weight path from s to t.

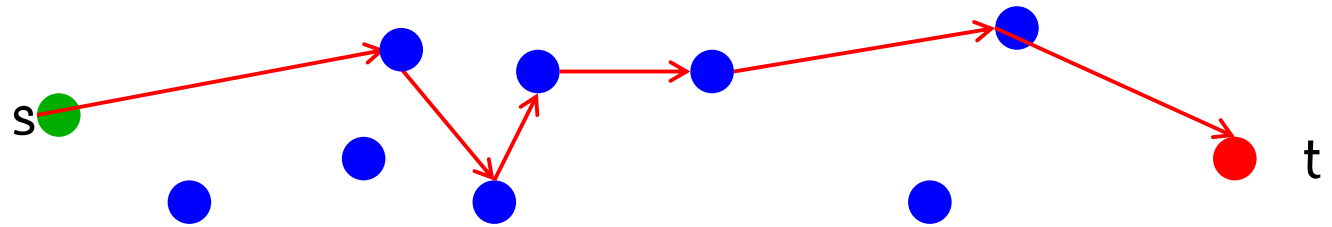
OSPF routing

- Given a network G = (N,A), where N is the set of routers and A is the set of links.
- The OSPF (open shortest path first) routing protocol assumes each link a has a weight w(a) assigned to it so that a packet from a source router s to a destination router t is routed on a shortest weight path from s to t.



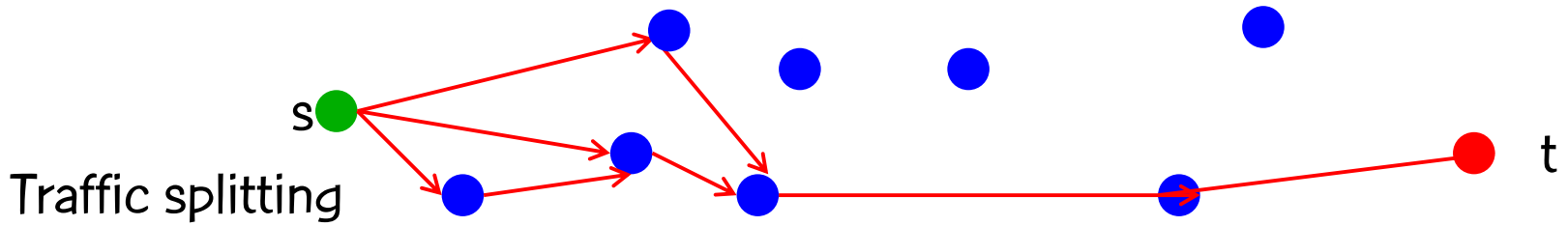
OSPF routing

- Given a network G = (N,A), where N is the set of routers and A is the set of links.
- The OSPF (open shortest path first) routing protocol assumes each link a has a weight w(a) assigned to it so that a packet from a source router s to a destination router t is routed on a shortest weight path from s to t.



OSPF routing

- Given a network G = (N,A), where N is the set of routers and A is the set of links.
- The OSPF (open shortest path first) routing protocol assumes each link a has a weight w(a) assigned to it so that a packet from a source router s to a destination router t is routed on a shortest weight path from s to t.



OSPF routing

- By setting OSPF weights appropriately, one can do traffic engineering, i.e. route traffic so as to optimize some objective (e.g. minimize congestion, maximize throughput, etc.).

OSPF routing

- Assign an integer weight $\in [1, w_{max}]$ to each link in AS. In general, $w_{max} = 65535 = 2^{16} - 1$.
- Each router computes tree of shortest weight paths to all other routers in the AS, with itself as the root, using Dijkstra's algorithm.

OSPF weight setting

- OSPF weights are assigned by network operator.
  - CISCO assigns, by default, a weight proportional to the inverse of the link bandwidth (Inv Cap).
  - If all weights are unit, the weight of a path is the number of hops in the path.
- We seek to find good OSPF weights.

OSPF weight setting problem

- Given a directed network $G = (N, A)$ with link capacities $c_a \in A$ and demand matrix $D = (d_{s,t})$ specifying a demand to be sent from node s to node t :
  - Assign weights $w_a \in [1, w_{max}]$ to each link $a \in A$, such that the objective function Φ is minimized when demand is routed according to the OSPF protocol.

Encoding/decoding for OSPF routing in IP networks

- Encoding:
  - A vector X of N random keys, where N is the number of links. The i-th random key corresponds to the i-th link weight.

Encoding/decoding for OSPF routing in IP networks

- Encoding:
  - A vector X of N random keys, where N is the number of links. The i-th random key corresponds to the i-th link weight.
- Decoding:

Encoding/decoding for OSPF routing in IP networks

- Encoding:
  - A vector X of N random keys, where N is the number of links. The i-th random key corresponds to the i-th link weight.
- Decoding:
  - For $i = 1, \ldots, N$: set $w(i) = \text{ceil}\,(X(i) \times w_{max})$

Encoding/decoding for OSPF routing in IP networks

- Encoding:
  - A vector X of N random keys, where N is the number of links. The i-th random key corresponds to the i-th link weight.
- Decoding:
  - For $i = 1, \ldots, N$: set $w(i) = \text{ceil}\,(X(i) \times w_{max})$
  - Compute shortest paths and route traffic according to OSPF.

Encoding/decoding for OSPF routing in IP networks

- Encoding:
  - A vector X of N random keys, where N is the number of links. The i-th random key corresponds to the i-th link weight.
- Decoding:
  - For i = 1, …, N: set w(i) = ceil (X(i) $\times$ w_{max})
  - Compute shortest paths and route traffic according to OSPF.
  - Compute load on each link, compute link congestion, add up all link congestions to compute network congestion.

Redundant content distribution

- Given:
  - A directed network $G = (V, E)$;
  - A set of nodes $B \in V$ where content-demanding users are located;
  - A set of nodes $M \in V$ where content warehouses can be located;
  - The set of all OSPF paths from m to b, for $m \in M$ and $b \in B$.

Redundant content distribution

- Compute:
  - The set of triples $\{ m_1, m_2, b \}^i$, $i = 1, 2, \ldots, T$, such that all paths from m_1 to b and from m_2 to b are disjoint, where $m_1, m_2 \in M$ and $b \in B$.

Redundant content distribution

- Solve the covering by pairs problem:
  - Find a smallest-cardinality set $M^* \subseteq M$ such that for all $b \in B$, there exists a triple $\{ m_1, m_2, b \}$ in the set of triples such that $m_1, m_2 \in M^*$.

Greedy algorithm for covering by pairs

- initialize partial cover M* = { }

Greedy algorithm for covering by pairs

- initialize partial cover M* = { }
- while M* is not a cover do:

Greedy algorithm for covering by pairs

- initialize partial cover $M^* = \{\}$
- while M^* is not a cover do:
  - find $m \in M \setminus M^*$ such that $M^* \cup \{m\}$ covers a maximum number of additional user nodes (break ties by vertex index) and set $M^* = M^* \cup \{m\}$

Greedy algorithm for covering by pairs

- initialize partial cover $M^* = \{\}$
- while M^* is not a cover do:
  - find $m \in M \setminus M^*$ such that $M^* \cup \{m\}$ covers a maximum number of additional user nodes (break ties by vertex index) and set $M^* = M^* \cup \{m\}$
  - if no $m \in M \setminus M^*$ yields an increase in coverage, then choose a pair $\{m_1, m_2\} \in M \setminus M^*$ that yields a maximum increase in coverage and set $M^* = M^* \cup \{m_1\} \cup \{m_2\}$

Greedy algorithm for covering by pairs

- initialize partial cover $M^* = \{\}$
- while M^* is not a cover do:
  - find $m \in M \setminus M^*$ such that $M^* \cup \{m\}$ covers a maximum number of additional user nodes (break ties by vertex index) and set $M^* = M^* \cup \{m\}$
  - if no $m \in M \setminus M^*$ yields an increase in coverage, then choose a pair $\{m_1, m_2\} \in M \setminus M^*$ that yields a maximum increase in coverage and set $M^* = M^* \cup \{m_1\} \cup \{m_2\}$
  - if no pair exists, then the problem is infeasible

Encoding / Decoding: covering by pairs

- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.

Encoding / Decoding covering by pairs

- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.
- Decoder:

Encoding / Decoding covering by pairs

- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.
- Decoder:
  - For i = 1, …, N: if X(i) > ½, add i-th data warehouse node to solution

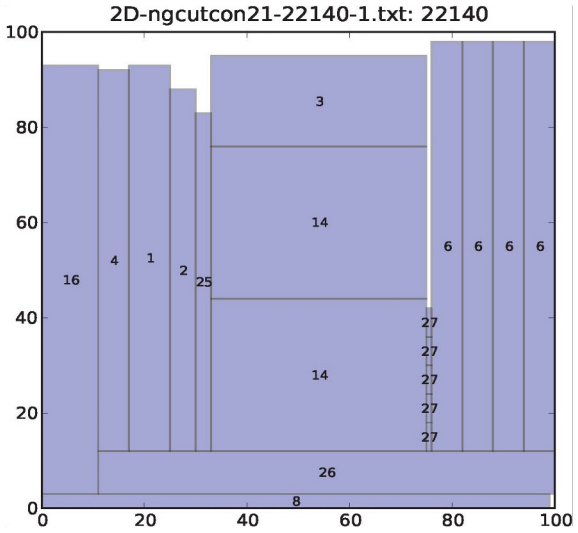
Encoding / Decoding covering by pairs

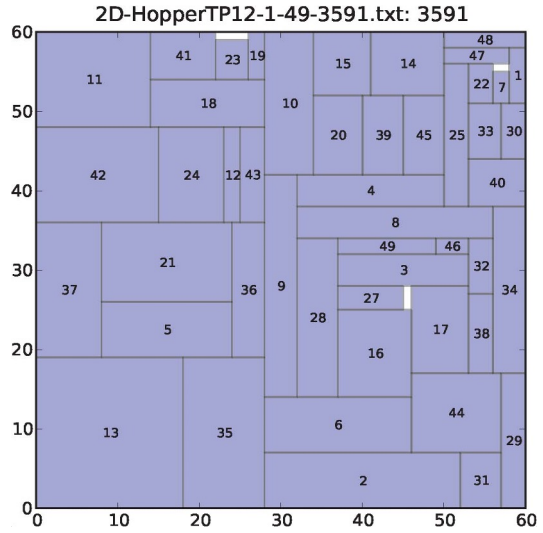
- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.
- Decoder:
  - For i = 1, …, N: if X(i) > ½, add i-th data warehouse node to solution
  - If solution is feasible, i.e. all users are covered: STOP

Encoding / Decoding covering by pairs

- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.
- Decoder:
  - For i = 1, …, N: if X(i) > ½, add i-th data warehouse node to solution
  - If solution is feasible, i.e. all users are covered: STOP
  - Else, apply greedy algorithm to cover uncovered user nodes.

Encoding & decoding: 2D packing





Among the many feasible packings,
we want to find one that maximizes
total value of packed rectangles:

$v[1]\, r[1] + v[2]\, r[2] + \cdots + v[N]\, r[N]$

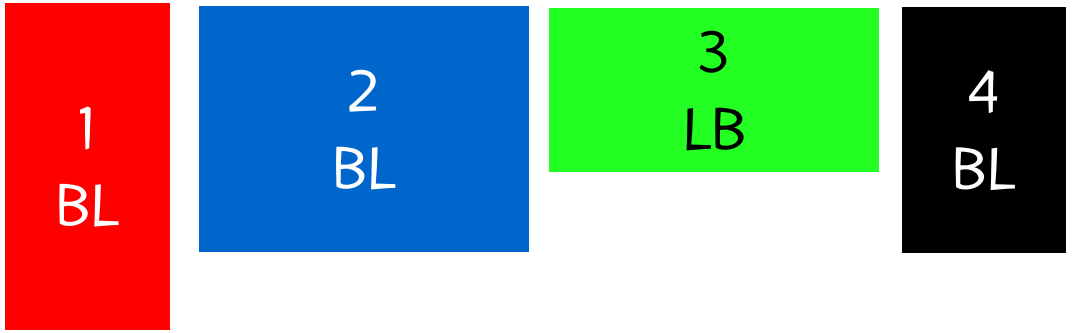
where $v[i]$ is the value of packing rectangle
of type i and $r[i]$ is the number of packed rectangles

Encoding & decoding: 2D packing

X = (.12, .34, .55, .88 | .63, .98, .21, .99)

Encoding

Solutions are encoded as vectors X of
2N' = 2 { Q[1] + Q[2] + … + Q[N] } random keys,
where Q[i] is the maximum number of rectangles of type i (for i = 1, …, N) that can be packed.



Decoder

- INPUT: *2N'*-vector of random keys
- Sort the first *N'* keys in increasing order. : Indices of sorted keys impose a sequencing for placement of rectangles. Use the last N' keys to determine the placement heuristic. If X(N'+ i) > 1/2, use bottom left placement, else use left bottom placement.
- OUTPUT: Feasible placement and its cost.

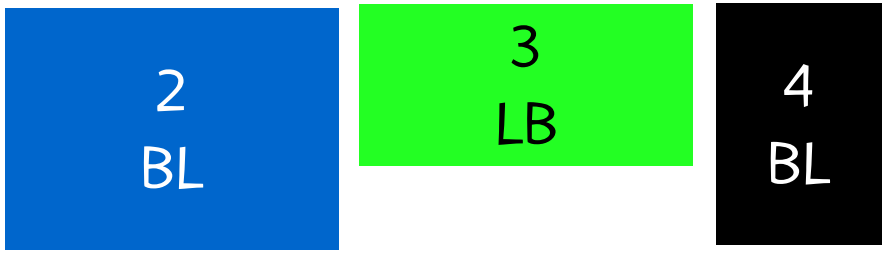


Encoding & decoding: 2D packing

X = (.12, .34, .55, .88 | .63, .98, .21, .99)

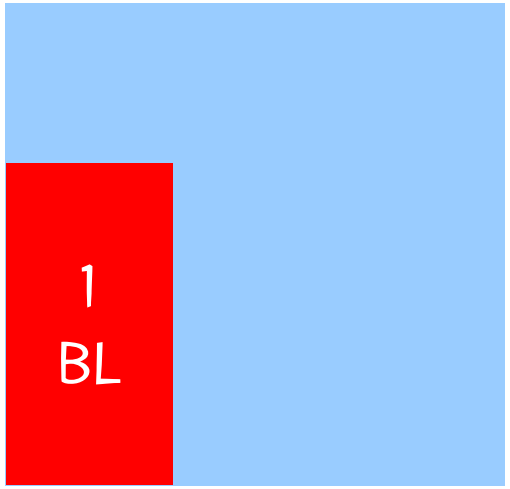
Encoding

Solutions are encoded as vectors X of
2N' = 2 { Q[1] + Q[2] + … + Q[N] } random keys,
where Q[i] is the maximum number of rectangles of type i (for i = 1, …, N) that can be packed.



Decoder

- INPUT: *2N'*-vector of random keys
- Sort the first *N'* keys in increasing order. : Indices of sorted keys impose a sequencing for placement of rectangles. Use the last N' keys to determine the placement heuristic. If X(N'+ i) > 1/2, use bottom left placement, else use left bottom placement.
- OUTPUT: Feasible placement and its cost.



Encoding & decoding: 2D packing

X = (.12, .34, .55, .88 | .63, .98, .21, .99)

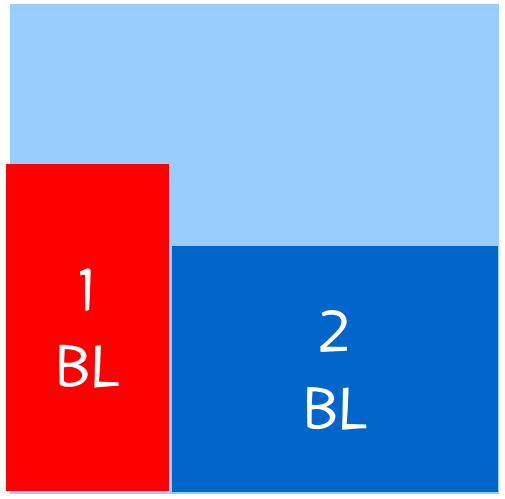
Encoding

Solutions are encoded as vectors X of
2N' = 2 { Q[1] + Q[2] + … + Q[N] } random keys,
where Q[i] is the maximum number of rectangles of type i (for i = 1, …, N) that can be packed.



Decoder

- INPUT: *2N'*-vector of random keys
- Sort the first *N'* keys in increasing order. : Indices of sorted keys impose a sequencing for placement of rectangles. Use the last N' keys to determine the placement heuristic. If X(N'+ i) > 1/2, use bottom left placement, else use left bottom placement.
- OUTPUT: Feasible placement and its cost.



Encoding & decoding: 2D packing

X = (.12, .34, .55, .88 | .63, .98, .21, .99)

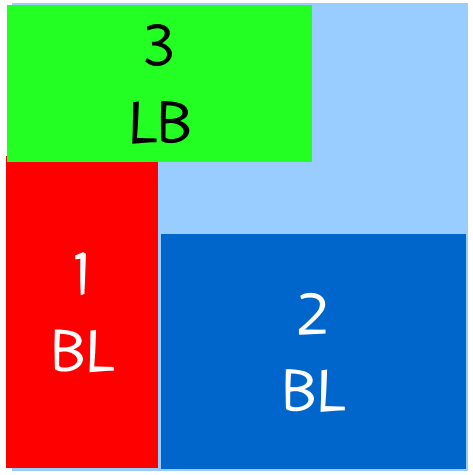
Encoding

Solutions are encoded as vectors X of
2N' = 2 { Q[1] + Q[2] + … + Q[N] } random keys,
where Q[i] is the maximum number of rectangles of type i (for i = 1, …, N) that can be packed.

4
BL

Decoder

- INPUT: *2N'*-vector of random keys
- Sort the first *N'* keys in increasing order. : Indices of sorted keys impose a sequencing for placement of rectangles. Use the last N' keys to determine the placement heuristic. If X(N'+ i) > 1/2, use bottom left placement, else use left bottom placement.
- OUTPUT: Feasible placement and its cost.



Encoding & decoding: 2D packing

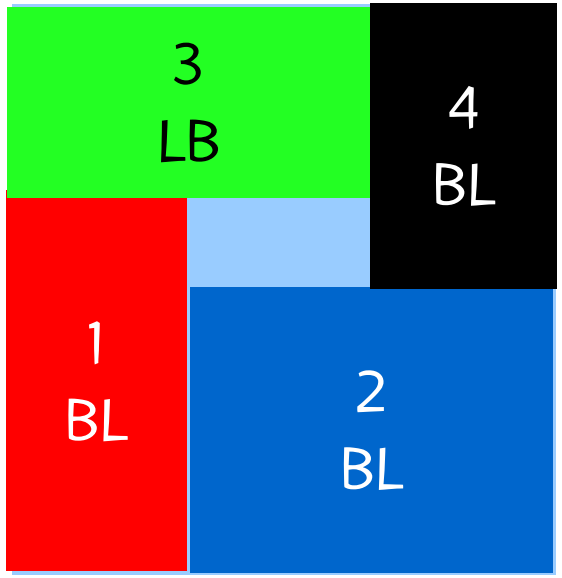
X = (.12, .34, .55, .88 | .63, .98, .21, .99)

Encoding

Solutions are encoded as vectors X of
2N' = 2 { Q[1] + Q[2] + … + Q[N] } random keys,
where Q[i] is the maximum number of rectangles of type i (for i = 1, …, N) that can be packed.

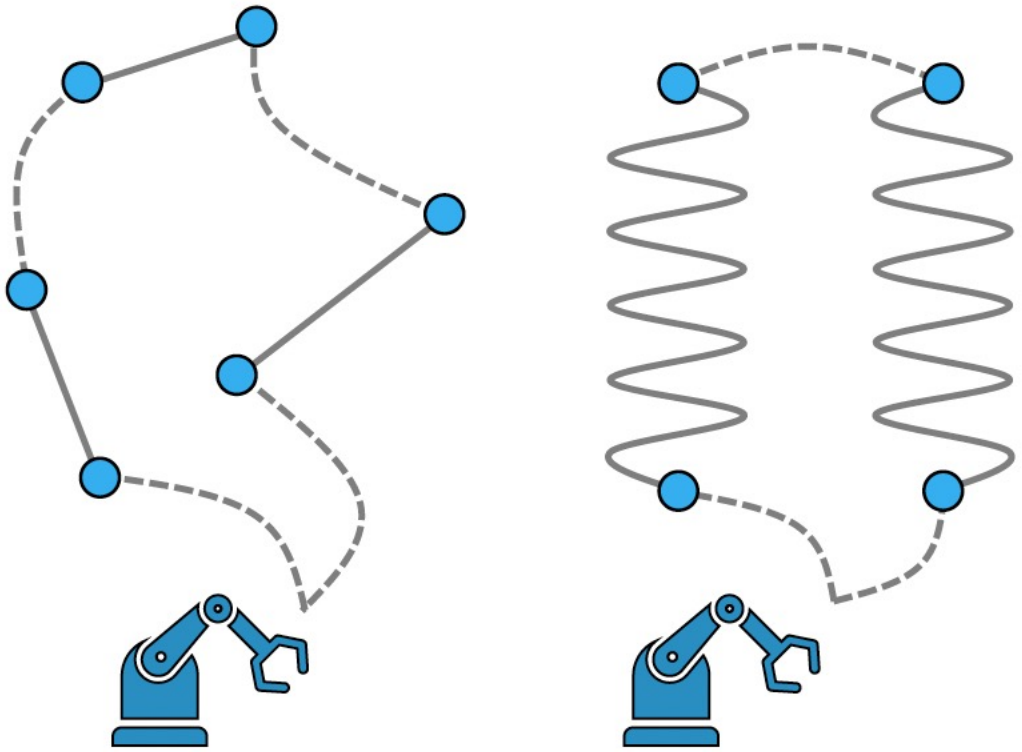
Decoder

- INPUT: *2N'*-vector of random keys
- Sort the first *N'* keys in increasing order. : Indices of sorted keys impose a sequencing for placement of rectangles. Use the last N' keys to determine the placement heuristic. If X(N'+ i) > 1/2, use bottom left placement, else use left bottom placement.
- OUTPUT: Feasible placement and its cost.



Optimal solution!

Robot motion planning



In automobile assembly PVC sealant is applied post-welding to every joint to ensure the car body's water-tightness.

Strips of PVC applied are called seams. Modern plants, like BMW, usually deploy a fleet of robots to apply the PVC sealant.

Motion planning involves determination of sequence of joints on which to apply seams and which of several available nozzles to use.

Travel time between $joint_{(i)}$ to $joint_{(j)}$ depends not only on the location of each joint but also on which nozzle is used to apply the sealant on each joint and other factors.

The goal is to identify collision-free trajectories such that all seams get processed within the minimum time span.

Robot motion planning

TABLE I. Example data set (cost matrix) for illustration. The problem is specified in terms of cost values (in seconds) for pairs of nodes, with every node described by a tuple $[s, d, t, c, p]$ that captures the seam index s, direction d, tool t, tool configuration c, and robot position p. Further details are provided in the main text.

node (from)					node (to)					cost
s	d	t	c	p	s	d	t	c	p	w [s]
0	0	0	0	0	18	1	1	1	1	0.877
11	2	1	0	1	12	1	2	0	1	0.473
11	2	1	0	1	12	0	3	0	0	0.541
32	2	1	2	1	25	2	1	2	1	0.558
⋮					⋮					⋮

Robot home position is [0,0,0,0,0]

In automobile assembly PVC sealant is applied post-welding to every joint to ensure the car body's water-tightness.

Strips of PVC applied are called seams. Modern plants, like BMW, usually deploy a fleet of robots to apply the PVC sealant.

Motion planning involves determination of sequence of joints on which to apply seams and which of several available nozzles to use.

Travel time between $\text{joint}_{(i)}$ to $\text{joint}_{(j)}$ depends not only on the location of each joint but also on which nozzle is used to apply the sealant on each joint and other factors.

The goal is to identify collision-free trajectories such that all seams get processed within the minimum time span.

Generalization of the TSP

Robot motion planning

Encoding

X = [x(1), x(2), …, x(nseams) | y(1), y(2), …, y(nseams) | … | z(1), z(2), …, z(nseams)]

Decoding

1. Sort the first x(1), x(2), …, x(nseams) to determine sequence of seams to be visited by robot
2. Use a threshold logic to set other configurations: e.g.

   If conf[y(i)] ∈ $\{1, 2, \dots, C\}$

   Then conf[y(i)] = k if y(i) ∈ ((k-1)/ C, k/C]

3. Route robot to visit seams in order determined by x(1), x(2), …, x(nseams) using configurations determined by y(1), y(2), …, y(nseams), …, z(1), z(2), …, z(nseams) and compute sum of travel times from travel time matrix.

Tree of hubs location

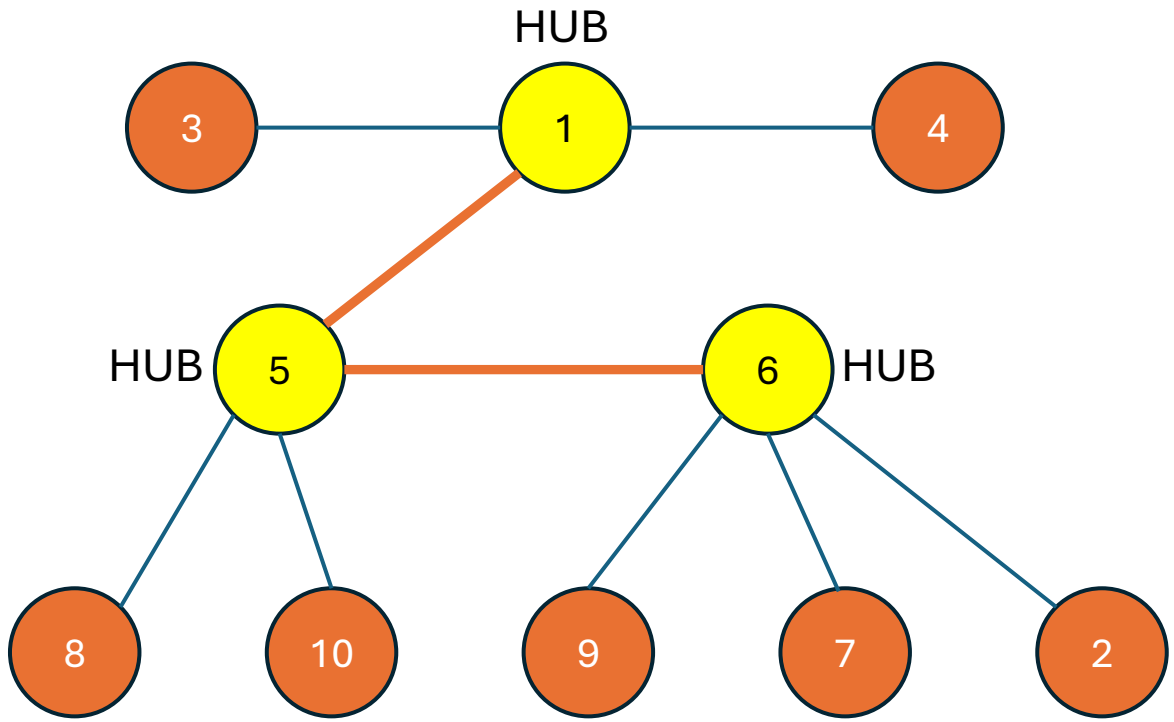
Consists of defining *p* hubs in a network that are connected by a tree with undirected edges.

From this tree, each non-hub node must be connected to a hub so that all the flow between the nodes must use the hubs to the network.

Each edge has a per-unit transportation cost, and there is a demand between each pair of nodes in the network.

The objective function to be minimized has two components:

- Transportation costs for fulfilling all demand from non-hub nodes to a designated hub
- Transportation costs of flow within the hub tree where economies of scale discount the unit cost.

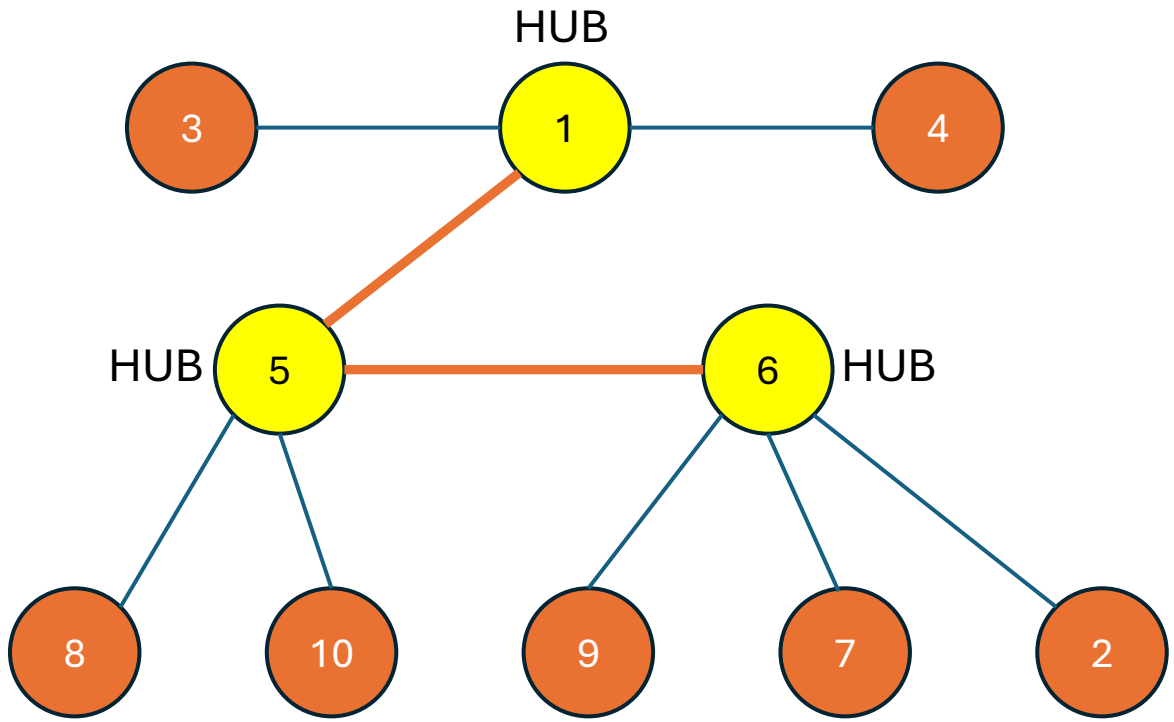


Tree of hubs location: encoding/decoding

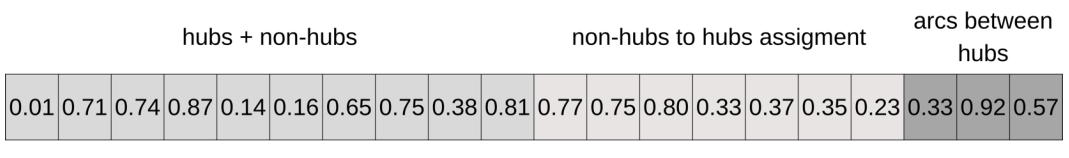
Encoding: $N+(N-p) + p(p-1)/2$ keys in three parts, first with N keys, second with $N-p$, and third with $p(p-1)/2$ keys, where N is the number of nodes in the network and p is the number of hubs.

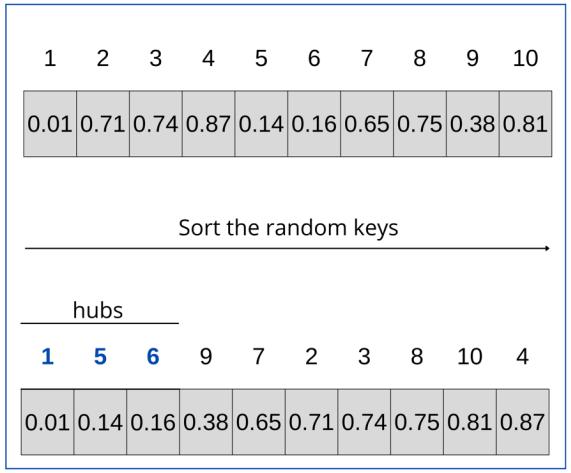
Decoding:

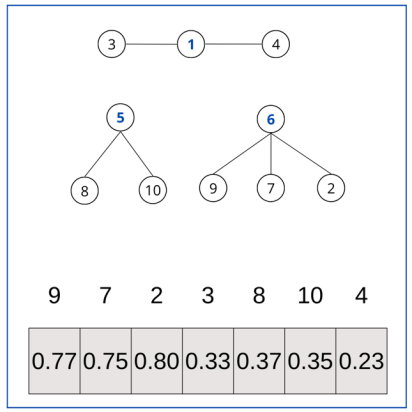
1. Sort the first N keys and assign the indices of the p smallest keys as hubs.
2. Use the second part of the keys with thresholding to assign non-hub nodes to one of the h hubs.
3. Sort the $p(p-1)/2$ keys of the third part and build the inter-hub tree with a Kruskal-like algorithm, i.e. assign edges in the order of sorted keys never creating a hub cycle.

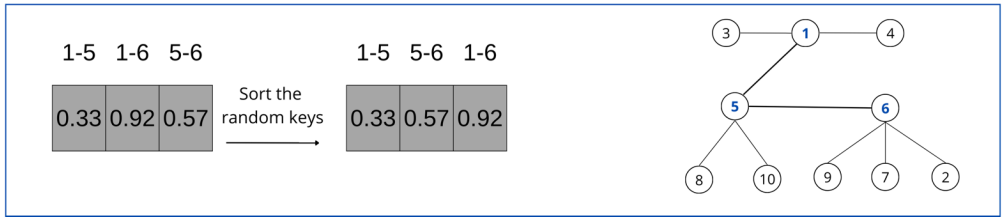


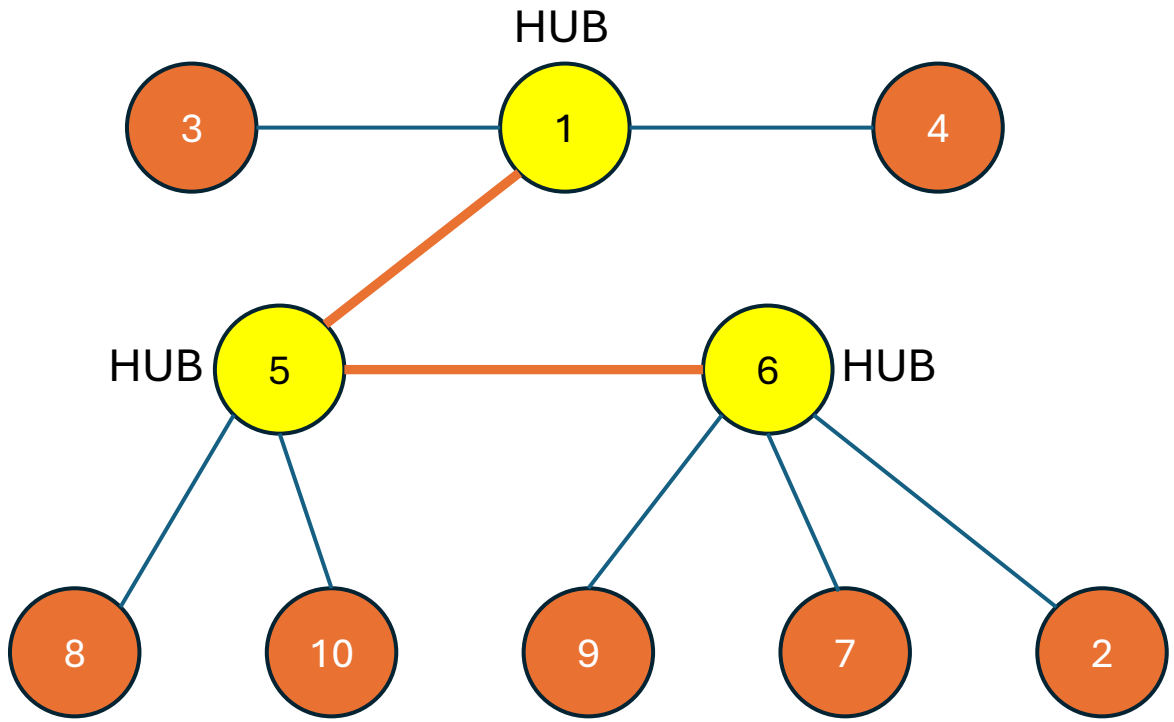
Tree of hubs location: encoding/decoding











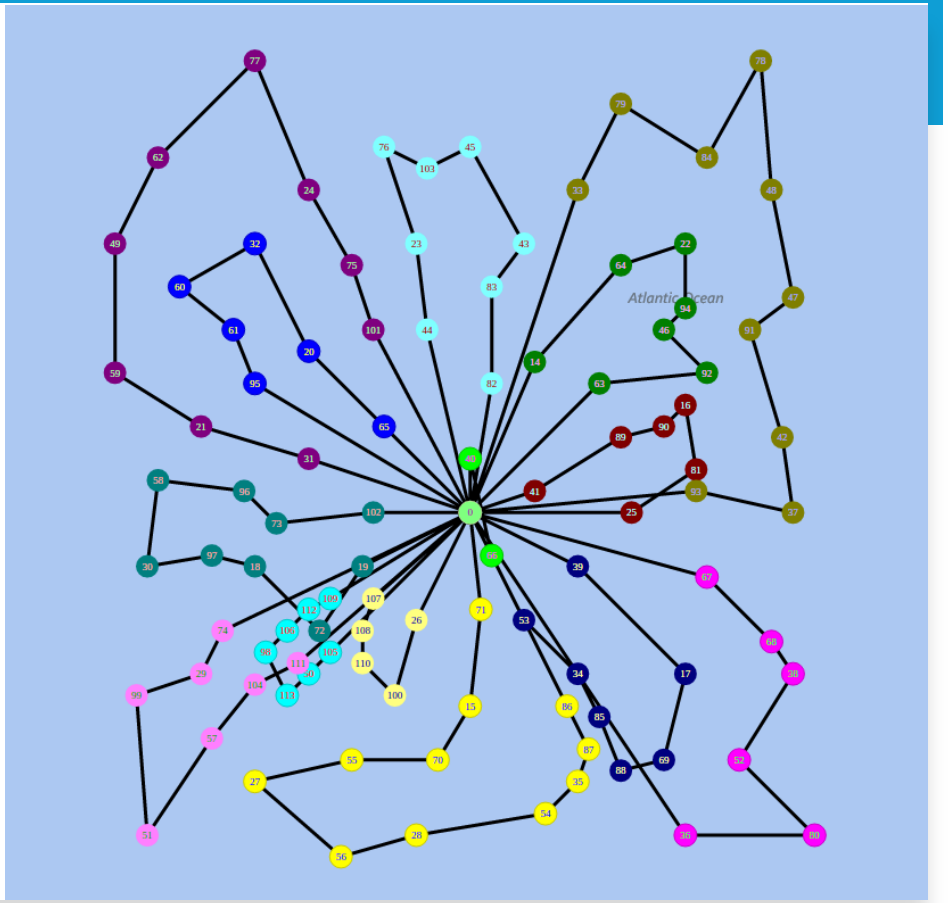
Capacitated vehicle routing with time windows

Given a set of transportation requests (customers) and a set of vehicles which must leave a depot (with given location), deliver the requests, and return to the depot. Each request has an associated location, demand, and time window in which delivery has to occur and each vehicle has a given capacity. Travel durations between all pairwise request locations and between the depot and all request locations are known.

Determine a subset of the vehicles with which to optimally deliver all the requests (feasibly), i.e. determine the sequence each vehicle will deliver the requests and their arrival and departure times at and from the depot and each request location.

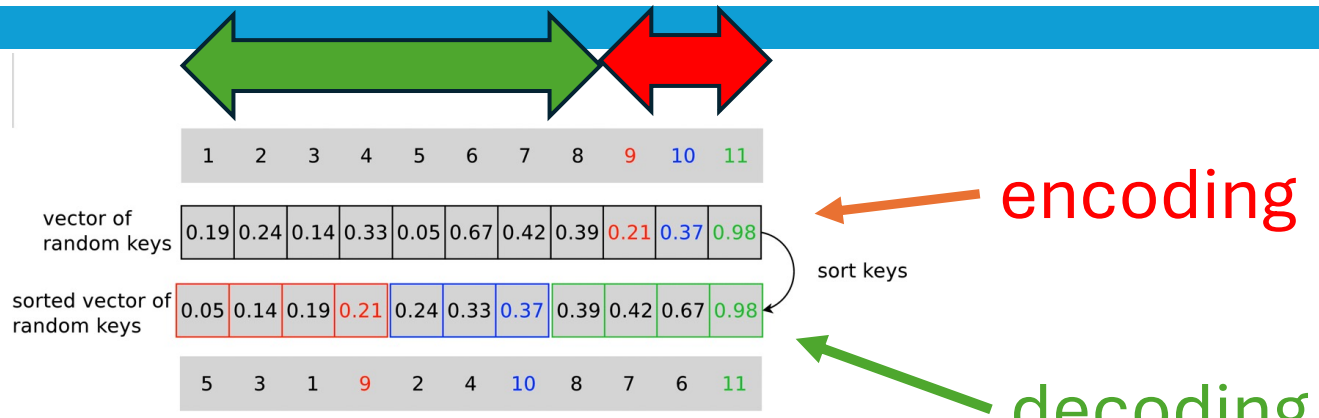
To be feasible, vehicle routes must satisfy a number of constraints:

1. A vehicle can never carry more than its capacity;
2. A vehicle cannot exceed a given total time on the road;
3. A vehicle must deliver each request within the time window of the request.
4. Optimization criterion can be, e.g. minimize number of vehicles; total travel time of vehicles.



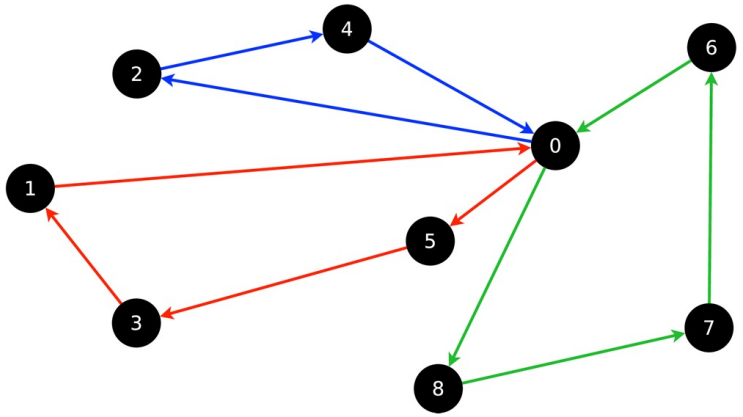
Capacitated vehicle routing with time windows Encoding / Decoding

8 customers 3 vehicles

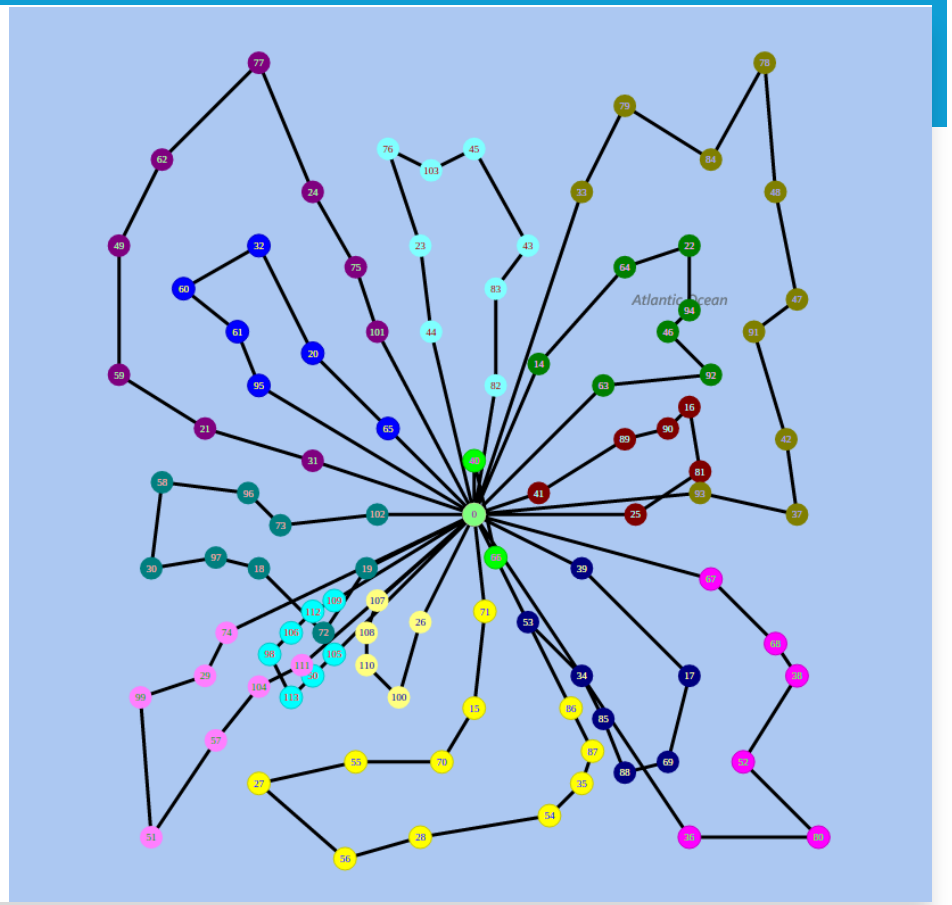


encoding

decoding



Use penalties to deal with infeasibilities



Prize collecting directed Steiner Forest

- Let $G = (V,E)$ be a given directed graph
- Let V_r, V_s, and V_d partition the node set, i.e.

$$V_r \cup V_s \cup V_d = V \quad \text{and} \quad V_r \cap V_s \cap V_d = \varnothing$$

  - V_r – set of root nodes
  - V_s – set of Steiner nodes
  - V_d – set of demand nodes

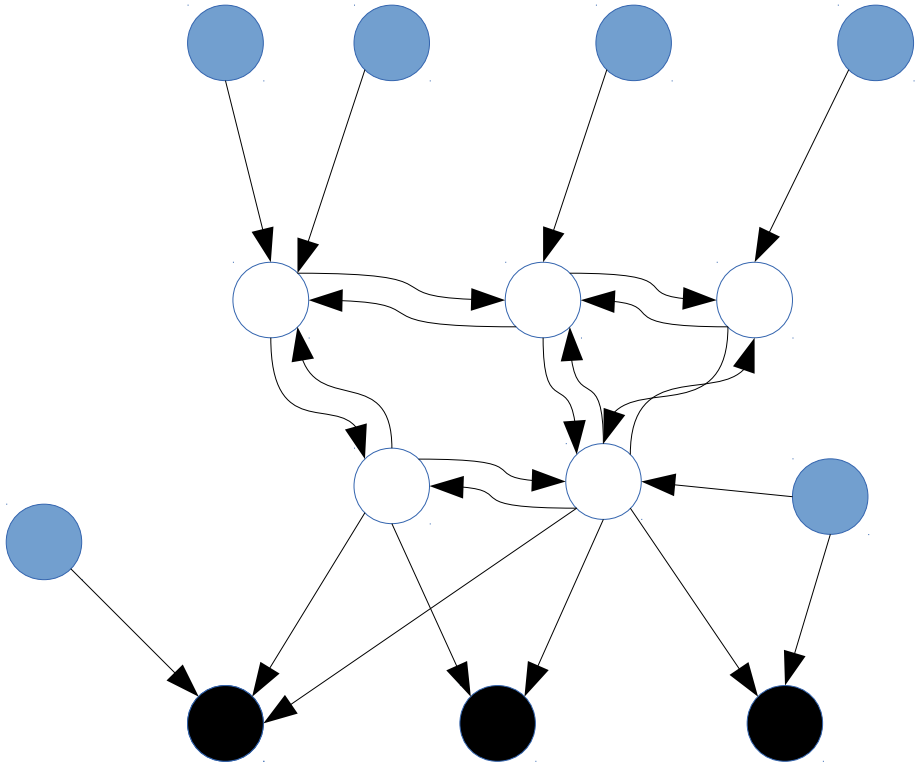
Rethink Possible
att.com/rethinkpossible

Prize collecting directed Steiner Forest

- Let $G = (V,E)$ be a given directed graph
- Arcs are directed
  - From demand nodes to Steiner nodes and root nodes
  - From Steiner nodes to root nodes
- For each pair of nodes $v, u \in V_s$
  - $(u,v) \in E$
  - $(v,u) \in E$

Rethink Possible
att.com/rethinkpossible

Prize collecting directed Steiner Forest



Demand node

Steiner node

Root node

Rethink Possible

att.com/rethinkpossible

Prize collecting directed Steiner Forest

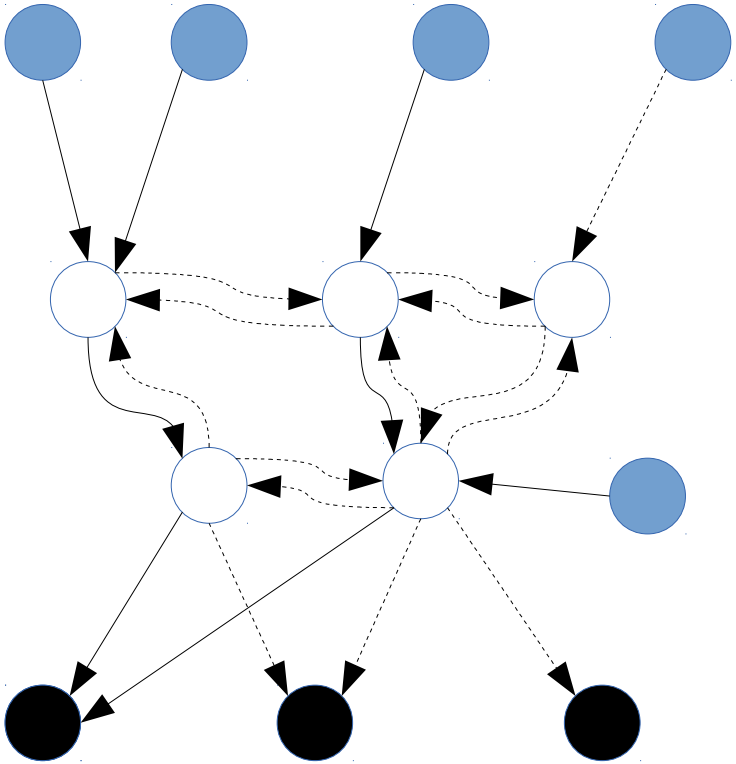
- A Directed Steiner Tree $T = (V[T] \subseteq V, E[T] \subseteq E)$ with root in $r \in V_r[T]$ is a loopless connected subgraph of G with a unique root r.
  - For each demand node $u \in V[T]$ there is a unique path in T from u to r.

Rethink Possible
att.com/rethinkpossible

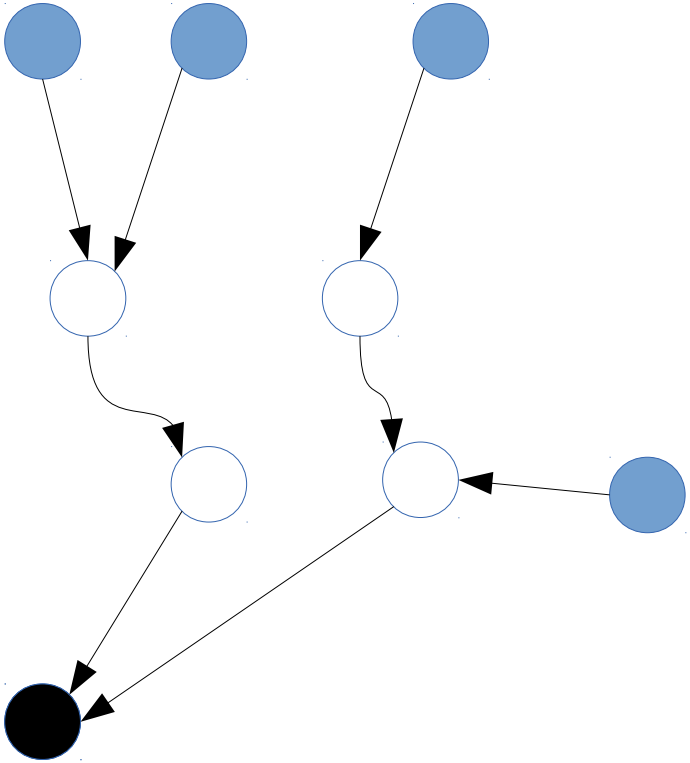
Prize collecting directed Steiner Forest

- A k-Hop Directed Steiner Tree is a directed Steiner tree (connected subgraph of G with a unique root r) such that:
  - Any path from a demand node u to the root r has no more than $k+2$ nodes, including u and r.
- A k-Hop Directed Steiner Forest is a collection of disjoint k-Hop Directed Steiner Trees

Rethink Possible
att.com/rethinkpossible



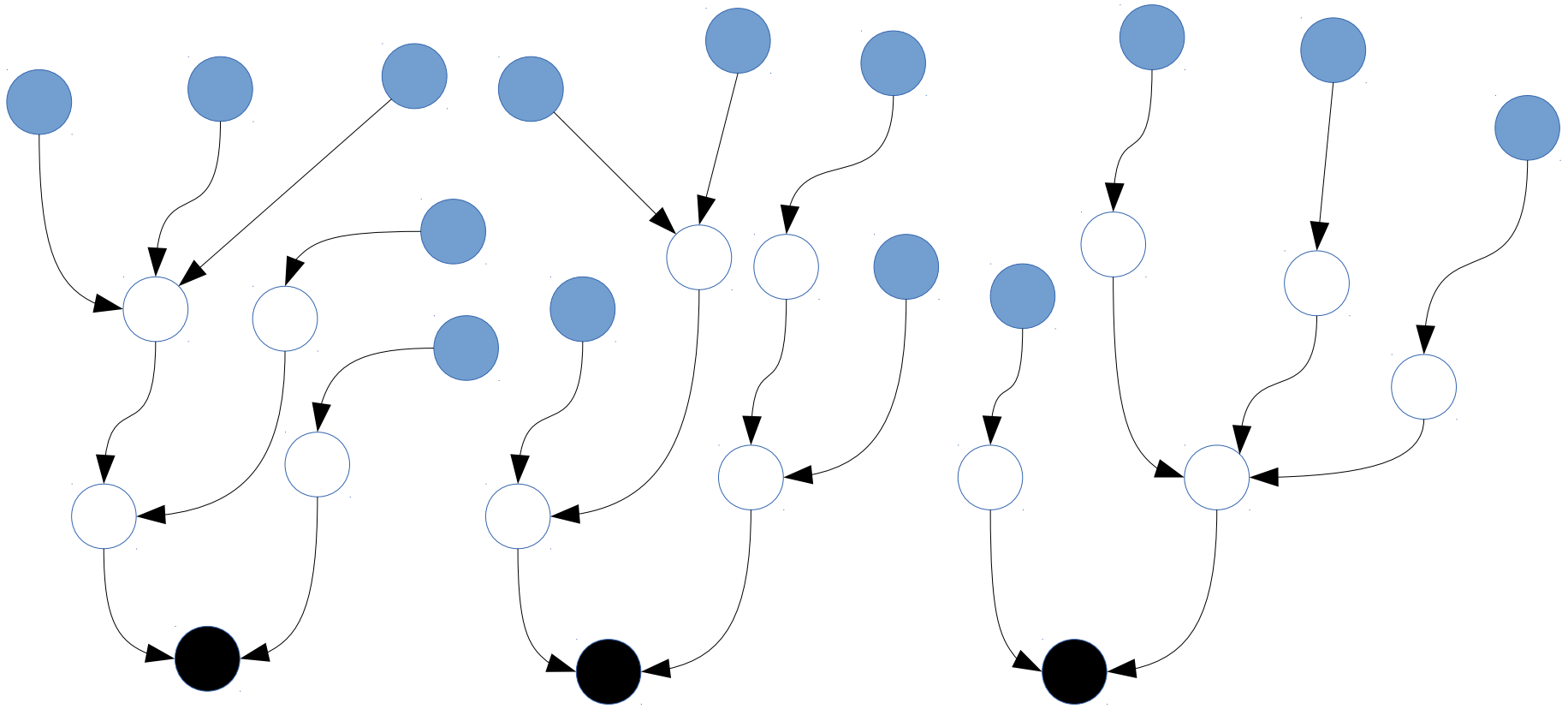
Acyclic directed graph



2-Hop Directed Steiner tree

Rethink Possible

att.com/rethinkpossible



2-Hop Directed Steiner Forest

Rethink Possible

att.com/rethinkpossible

Prize collecting directed Steiner Forest

- A demand function $d: V_d \rightarrow \mathbb{R}^+$ gives the prize to be collected from each demand node if it is a leaf on the tree.
- A cost function $c: V_s \rightarrow \mathbb{R}^+$ gives the cost of of each Steiner vertex.

Rethink Possible
att.com/rethinkpossible

Prize collecting directed Steiner Forest

- Given a directed graph $G = (V,E)$, vertex partition V_r , V_s , and V_d, hop parameter k, demand function d, and cost function c
- FIND: A k-hop directed Steiner forest F such that the profit

$$\text{sum}\,[\, v \in V_d[F] \,]\, d_v - \text{sum}\,[\, v \in V_s[F] \,]\, c_v$$

is maximized.

Rethink Possible
att.com/rethinkpossible

Wireless backhaul network planning

Given a geographical region where locations are represented as lat-long coordinates, where

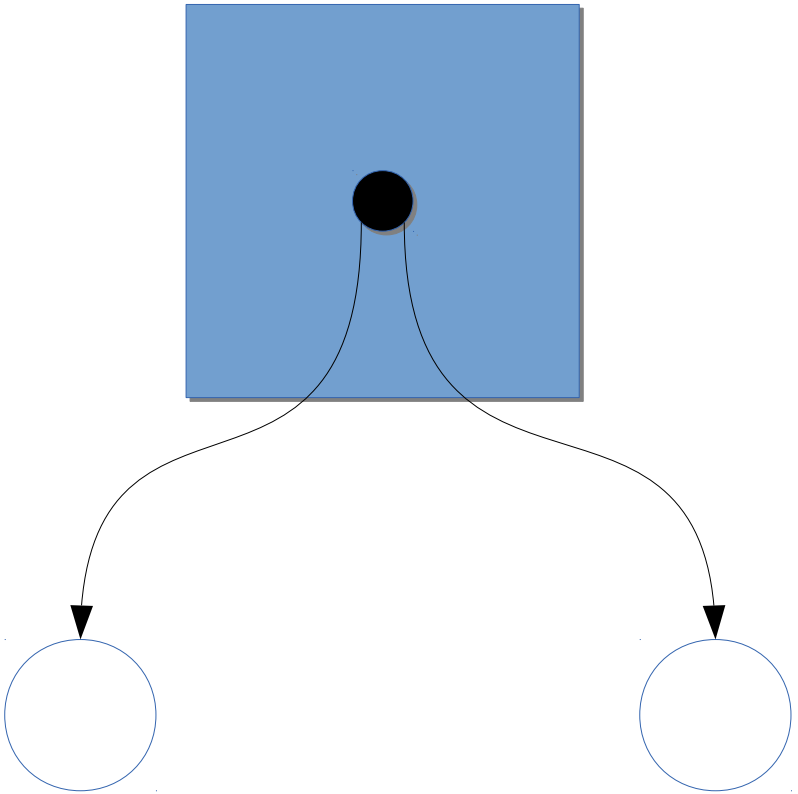
- V_d is the set of nodes representing traffic origination points (demand points) in the region
- V_s is the set of nodes representing locations where equipment for traffic collection and routing is located (e.g. utility poles)
- V_r is the set of nodes representing fibered access points (FAP), e.g. remote terminals (RT), macrocell (MC), or central offices.

Rethink Possible
att.com/rethinkpossible

Wireless backhaul network planning

Constraints: Demand splitting

- Estimate for each block total demand is placed in center of block
- Block can be served by one or more antennae so demand can be split among them
- Because of this undirected cycles can be introduced resulting in a directed acyclic graph (DAG)

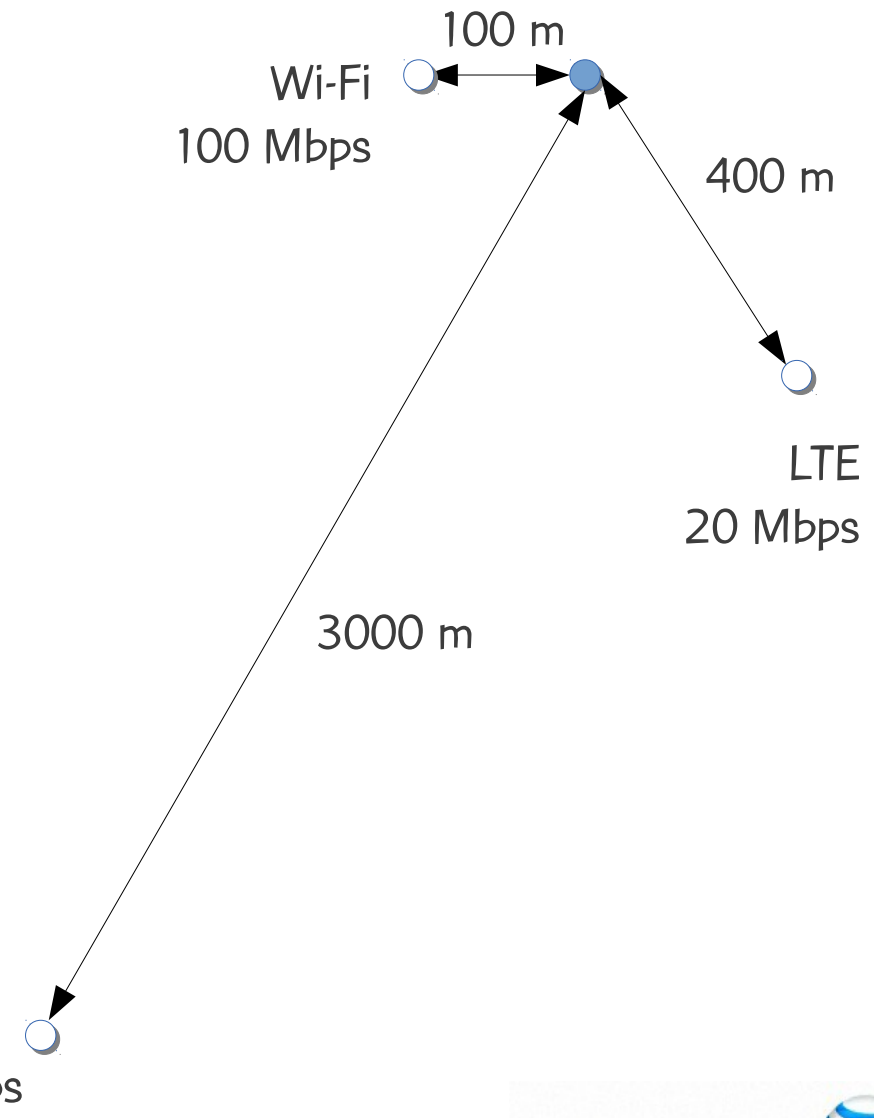


Rethink Possible
att.com/rethinkpossible

Wireless backhaul network planning

Constraints: Access equipment action radii & capacities

- Action radii
  - Wi-Fi: 100 m
  - LTE: 400 m
  - macrocell: 3 km
- Processing capacity
  - Wi-Fi: 100 Mbps
  - LTE: 20 Mbps
  - macrocell: 25 Mbps

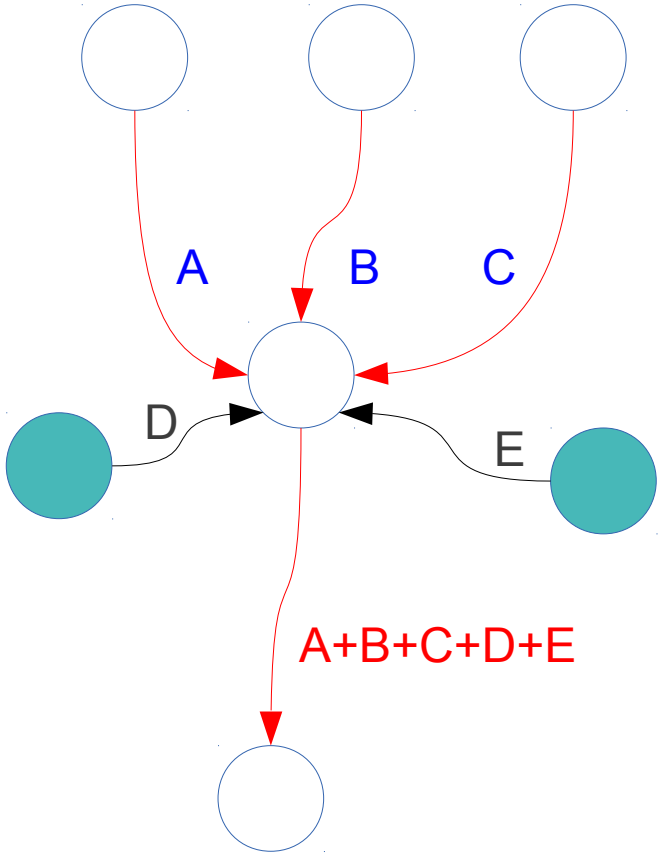


Rethink Possible
att.com/rethinkpossible

Wireless backhaul network planning

Constraints: Retransmitter equipment capacity

- Action radii: 1 km
- Processing capacity
  - Flow that equipment receives from other wireless retransmitters plus flow it sends to other retransmitters is limited to at most 100 Mbps



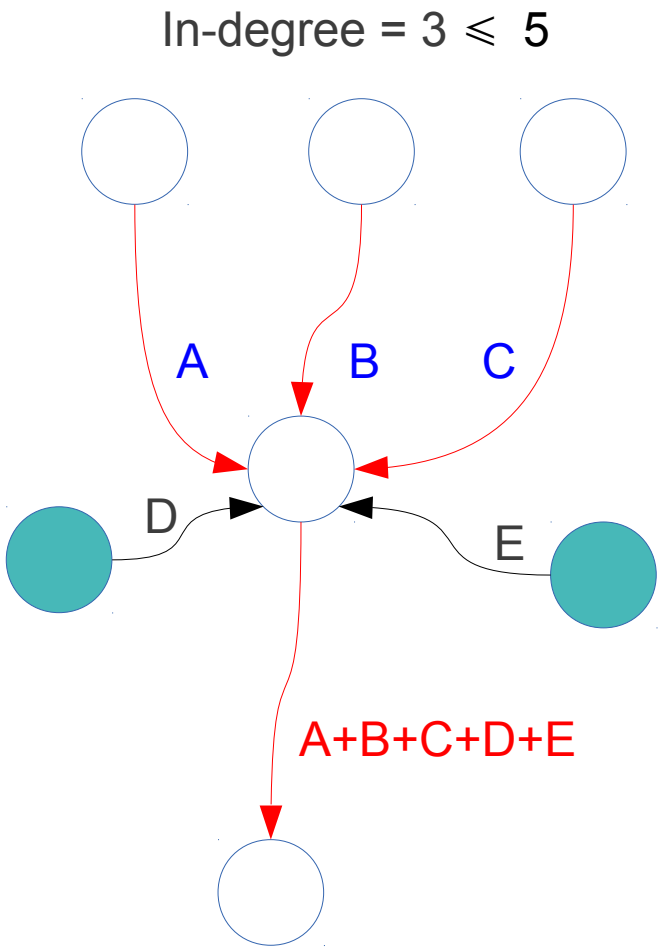
$$A+B+C+A+B+C+D+E \leqslant 100$$

Rethink Possible
att.com/rethinkpossible

Wireless backhaul network planning

Constraints: Retransmitter equipment capacity

- Fan-in constraint
  - A limited number (5) of neighboring transmission equipment can flow traffic into equipment



A+B+C+A+B+C+D+E ⩽100

Rethink Possible
att.com/rethinkpossible

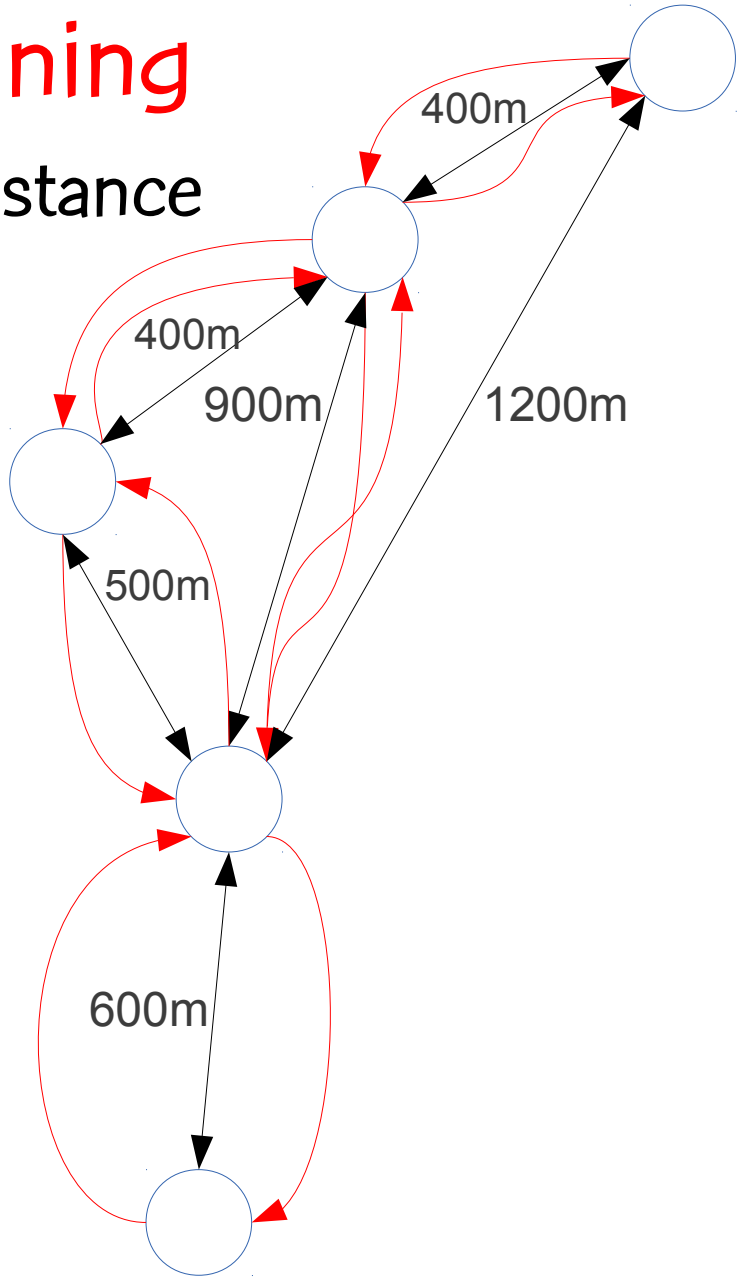
Wireless backhaul network planning

Constraints: Line of sight and minimum distance

PCSF Problem assumed, for each pair $u, v \in V_s$ that $(u,v) \in E$ and $(v,u) \in E$

Not so in wireless backhaul planning: $(u,v) \in E$ and $(v,u) \in E$ only if

- Poles u and v are within 1000 m of each other



Rethink Possible
att.com/rethinkpossible

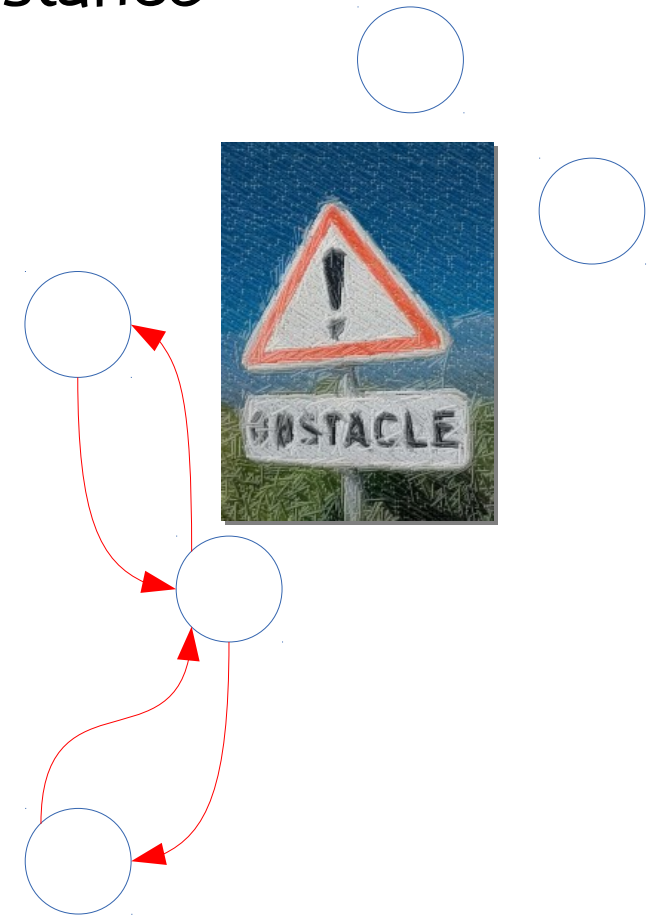
Wireless backhaul network planning

Constraints: Line of sight and minimum distance

PCSF Problem assumed, for each pair $u, v \in V_s$ that $(u,v) \in E$ and $(v,u) \in E$

Not so in wireless backhaul planning:
$(u,v) \in E$ and $(v,u) \in E$ only if

- Poles u and v are within 1000 m of each other
- Poles u and v are in each other's line of sight



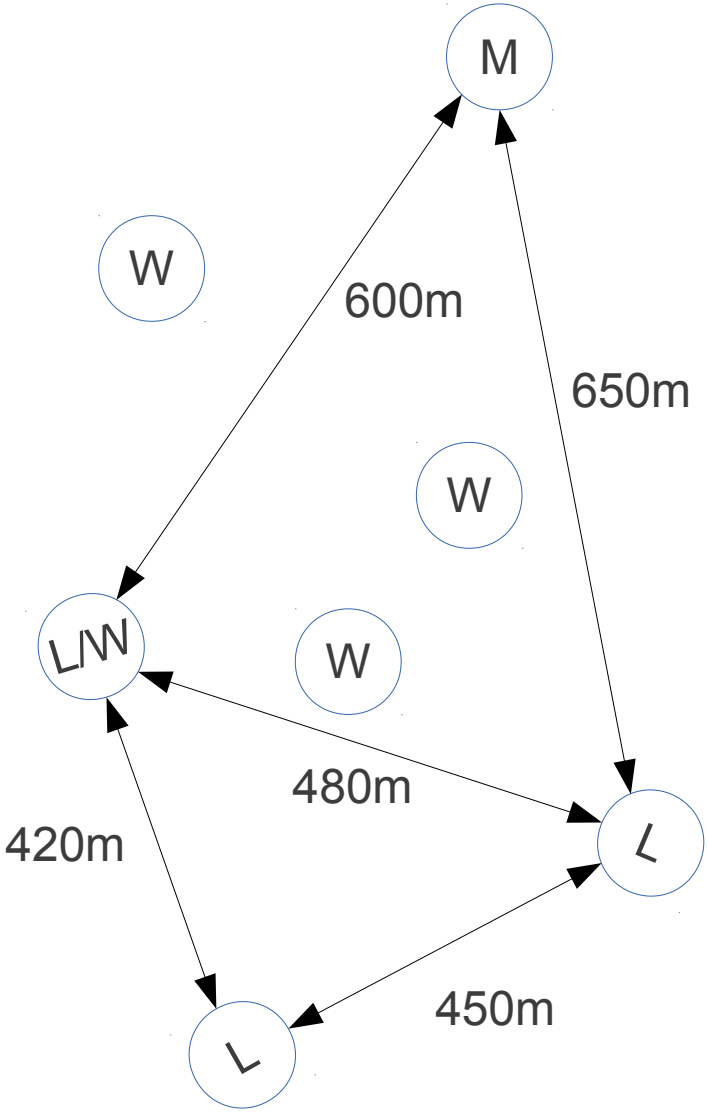
Rethink Possible
att.com/rethinkpossible

Wireless backhaul network planning

Constraints: Interference

LTE and MC use licensed spectrum and can interfere with each other.

- Pairs of LTE antennae must be separated by at least 400m
- LTE and macrocells by at least 500m
- No constraint on Wi-Fi exists since it uses non-licensed spectrum



Wireless backhaul network planning

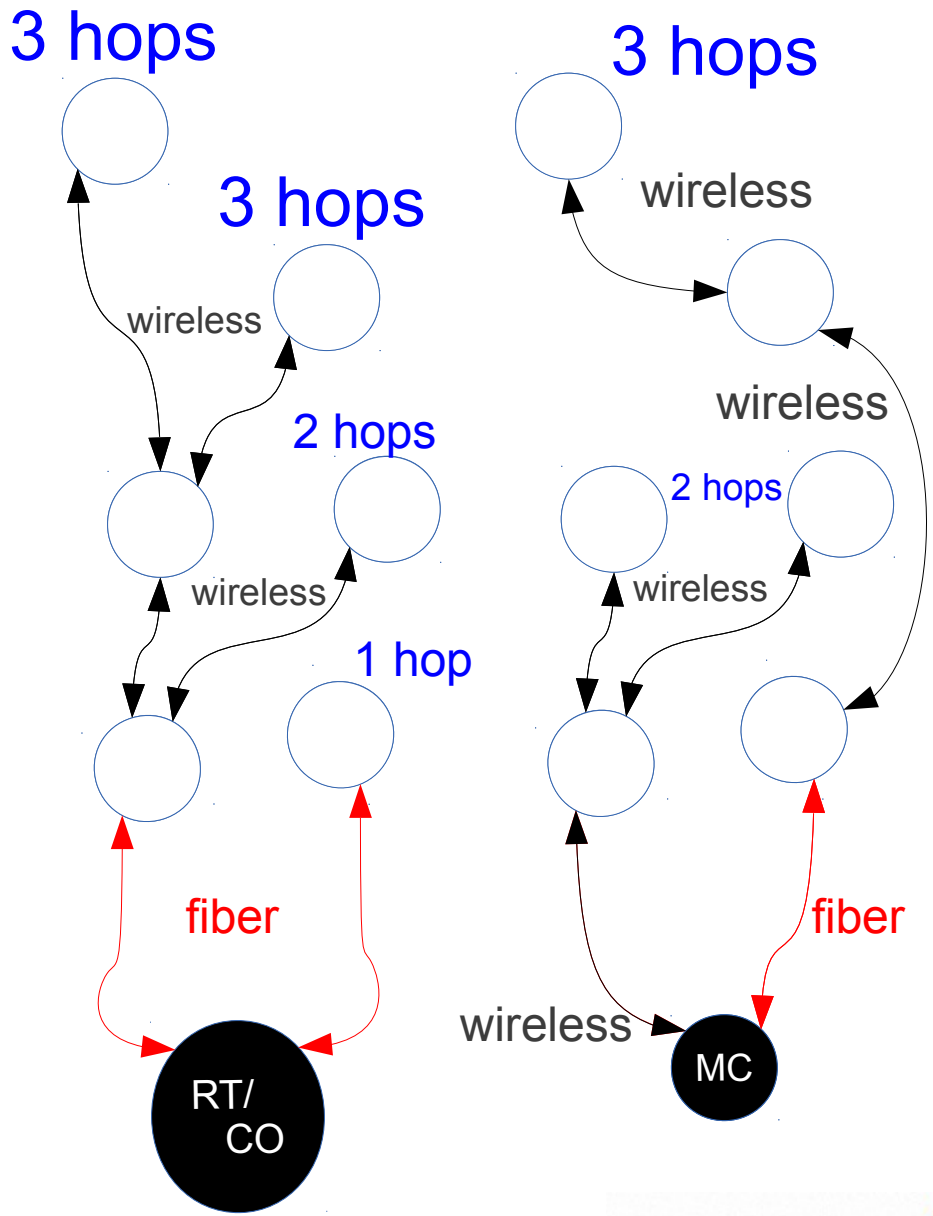
Constraints: k-Hops

First hop is from FAP (root) to pole

- If root is Central Office (CO) or RT link must use fiber
- If root is macrocell link can be fiber or wireless

All other links are wireless

Number of hops is limited to k = 2 or 3 (in case first link is fiber)

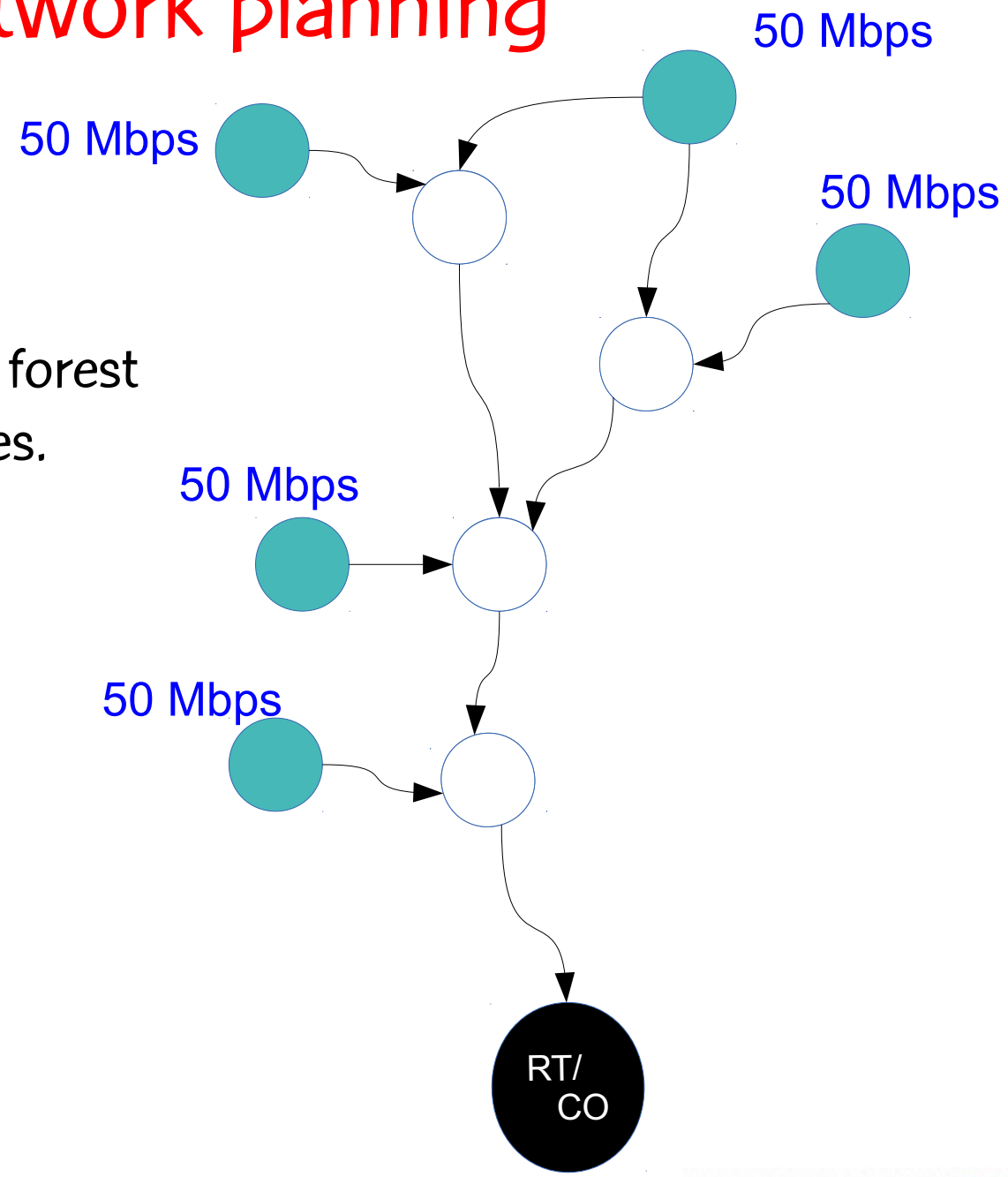


Wireless backhaul network planning

Constraints: Traffic flow

Traffic that is backhauled from demand points to root nodes of forest is limited by equipment capacities.

Only traffic that reaches roots is counted as revenue.

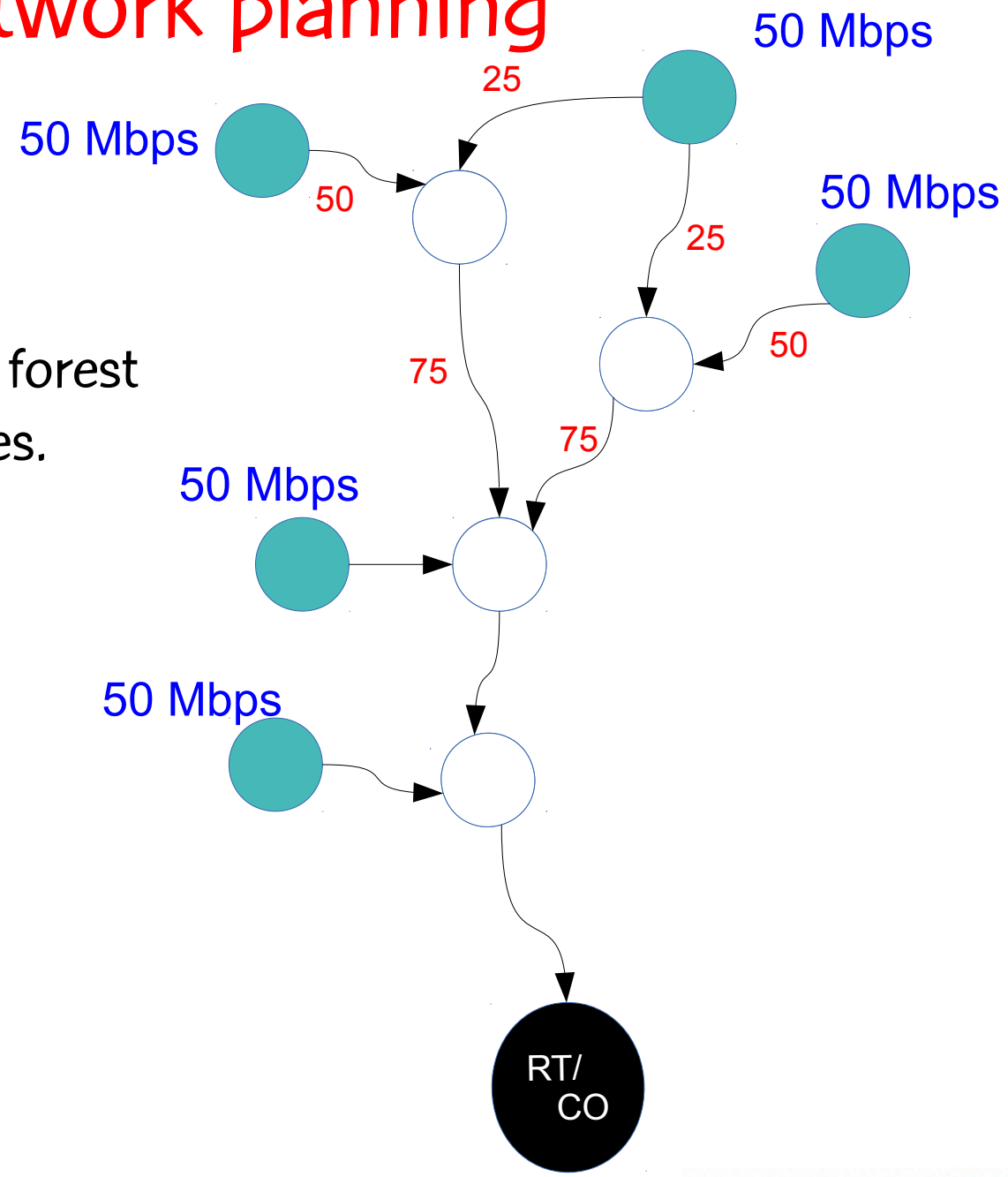


Wireless backhaul network planning

Constraints: Traffic flow

Traffic that is backhauled from demand points to root nodes of forest is limited by equipment capacities.

Only traffic that reaches roots is counted as revenue.

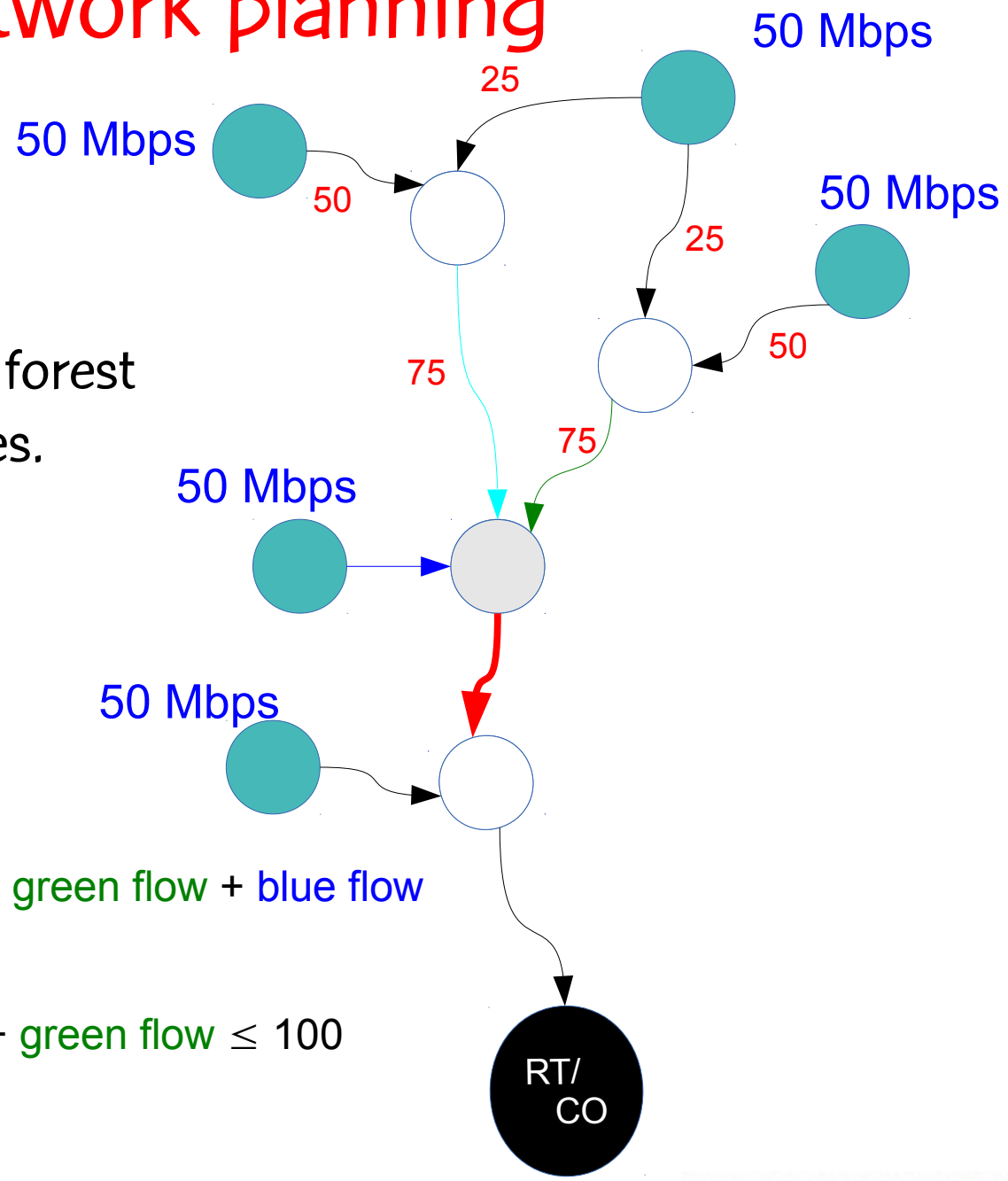


Wireless backhaul network planning

Constraints: Traffic flow

Traffic that is backhauled from demand points to root nodes of forest is limited by equipment capacities.

Only traffic that reaches roots is counted as revenue.



Flow conservation: red flow = cyan flow + green flow + blue flow

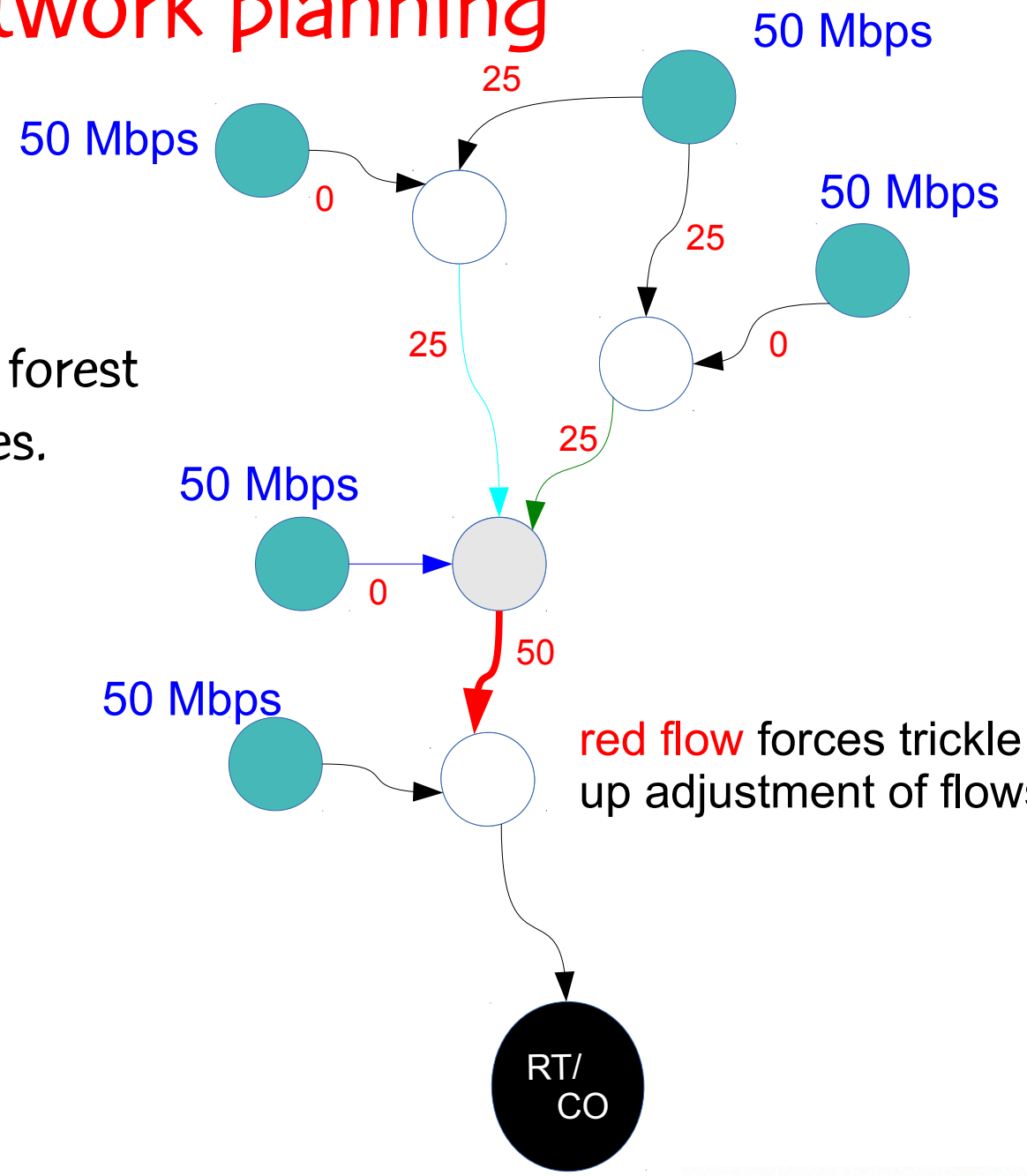
Capacity constraint: red flow + cyan flow + green flow ≤ 100

Wireless backhaul network planning

Constraints: Traffic flow

Traffic that is backhauled from demand points to root nodes of forest is limited by equipment capacities.

Only traffic that reaches roots is counted as revenue.

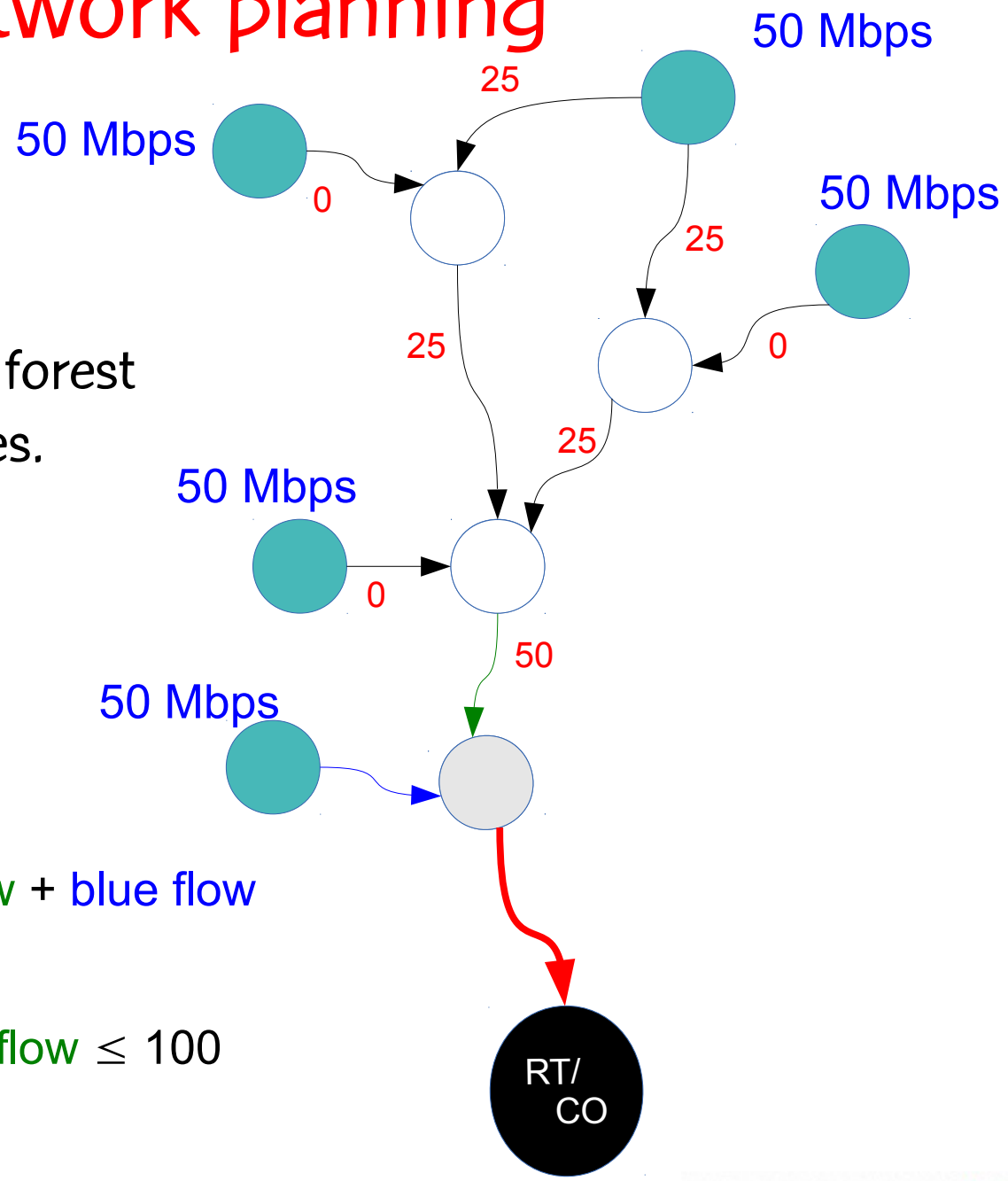


Wireless backhaul network planning

Constraints: Traffic flow

Traffic that is backhauled from demand points to root nodes of forest is limited by equipment capacities.

Only traffic that reaches roots is counted as revenue.



Flow conservation: red flow = green flow + blue flow

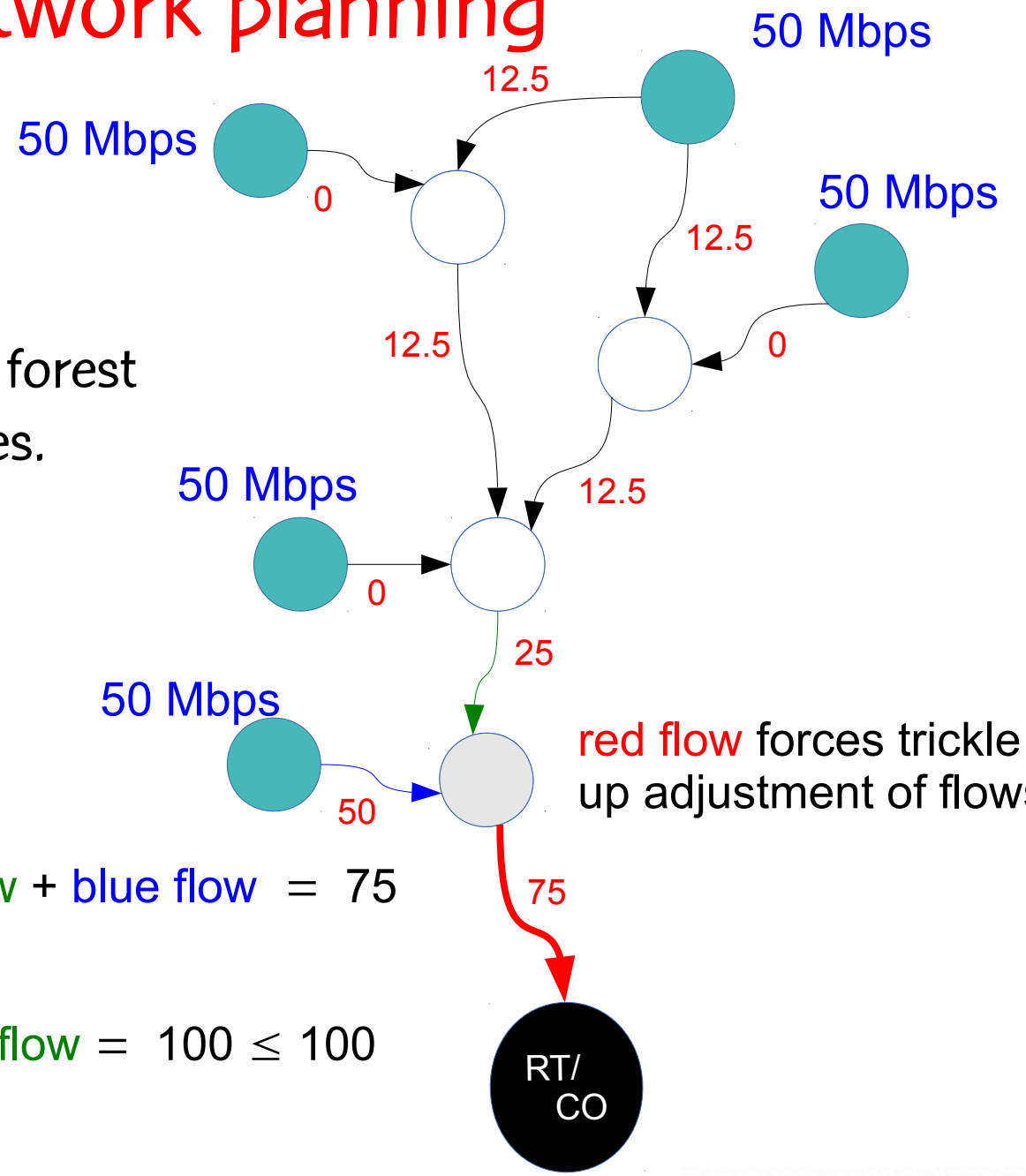
Capacity constraint: red flow + green flow $\leq$ 100

Wireless backhaul network planning

Constraints: Traffic flow

Traffic that is backhauled from demand points to root nodes of forest is limited by equipment capacities.

Only traffic that reaches roots is counted as revenue.



Flow conservation: red flow = green flow + blue flow = 75

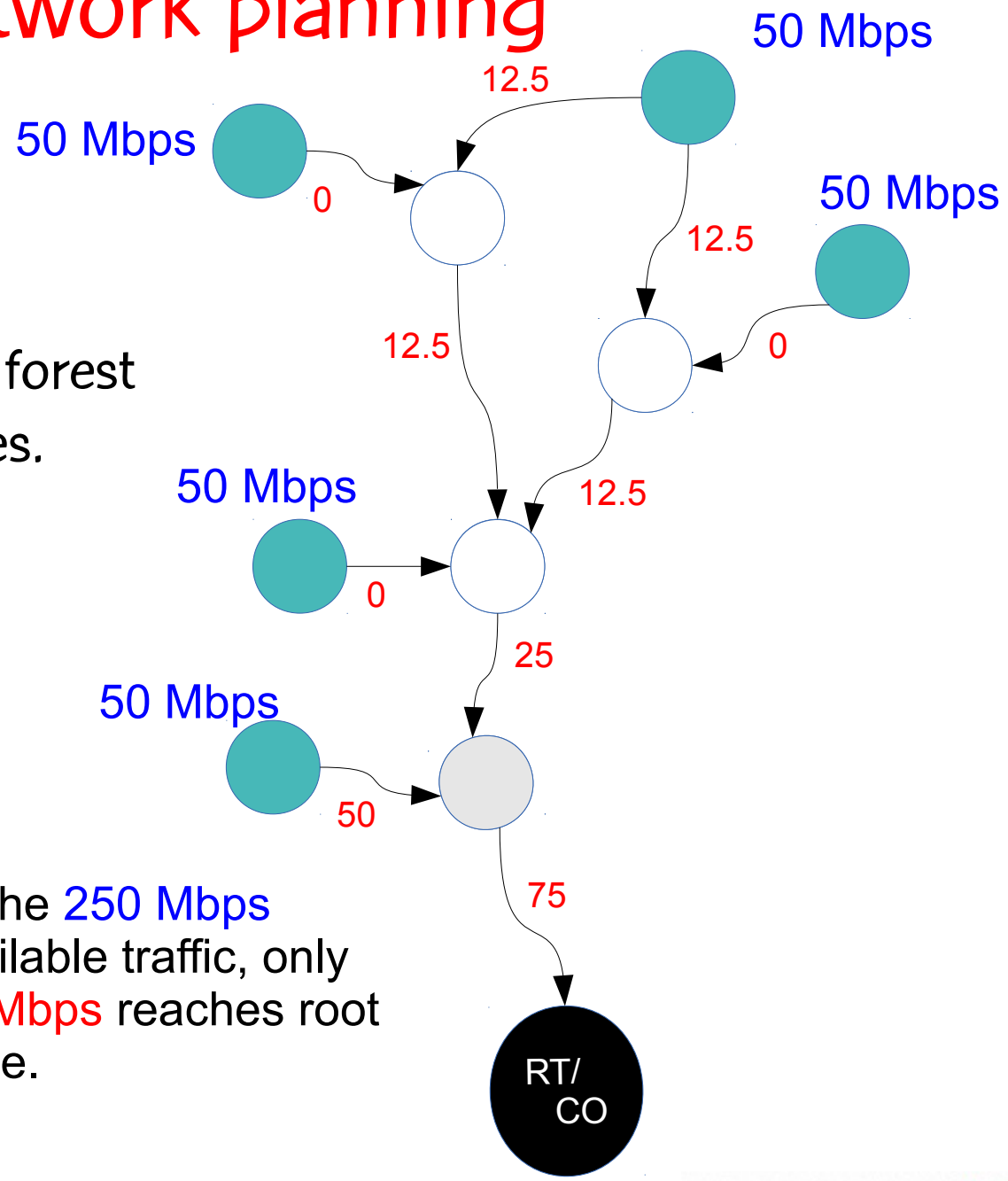
Capacity constraint: red flow + green flow = 100 ≤ 100

Wireless backhaul network planning

Constraints: Traffic flow

Traffic that is backhauled from demand points to root nodes of forest is limited by equipment capacities.

Only traffic that reaches roots is counted as revenue.



Of the 250 Mbps available traffic, only 75 Mbps reaches root node.

Decoder for wireless backhaul planning

- Biased Random-Key Genetic Algorithm
  - Learn the best network layout and equipment placement
- A solution is encoded by a vector $x \in [0,1]^n$
  - Where $n = 5 \times \#$ of poles



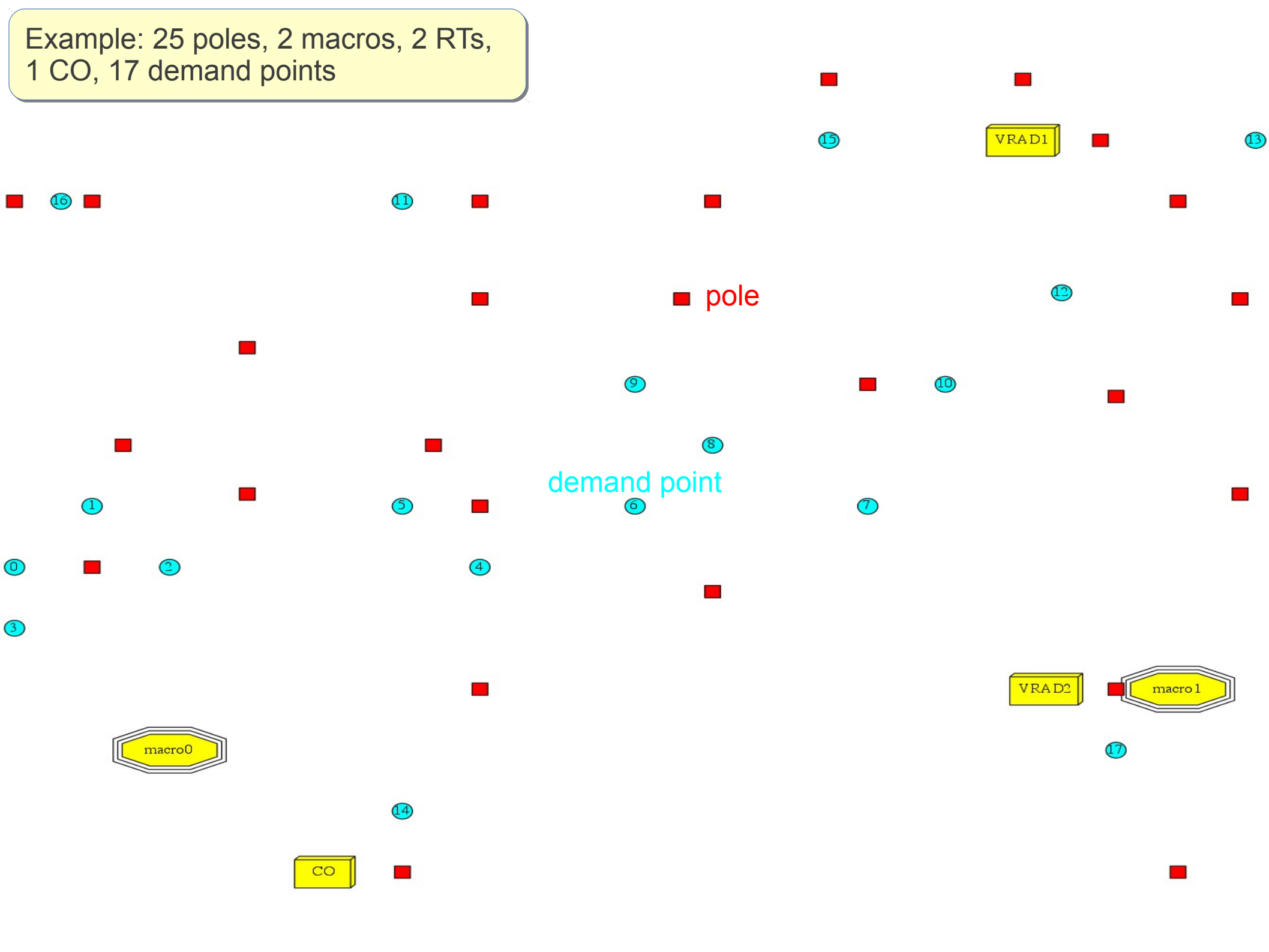
Rethink Possible
att.com/rethinkpossible

Decoder

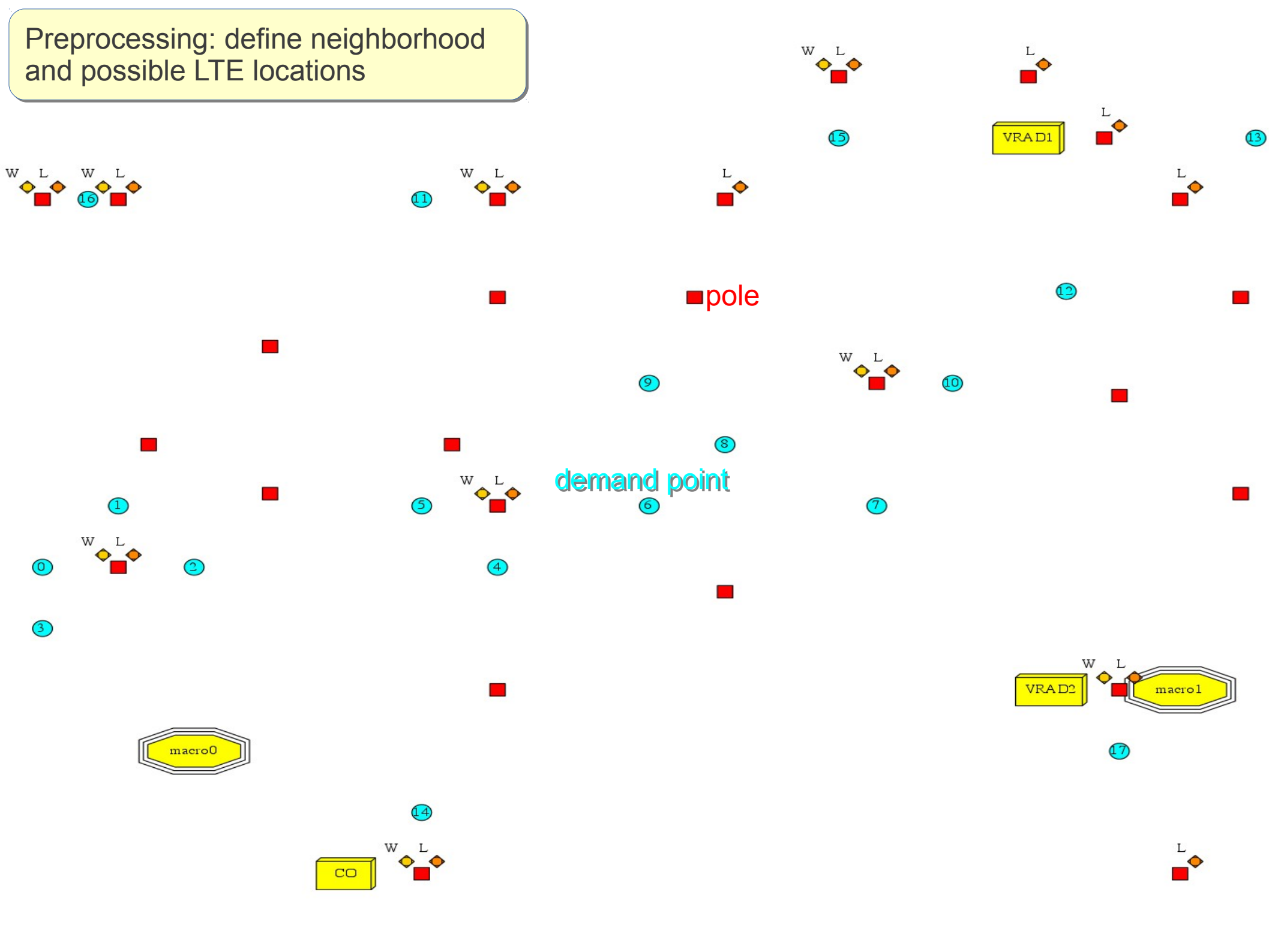
- Define activation order & install LTE on poles
- Build backhaul graph
- Remove unused equipment
- Compute maximum flow from demand points to FAPs
- Remove unused equipment & poles
- Compute cost and revenue and return objective function value

Rethink Possible
att.com/rethinkpossible

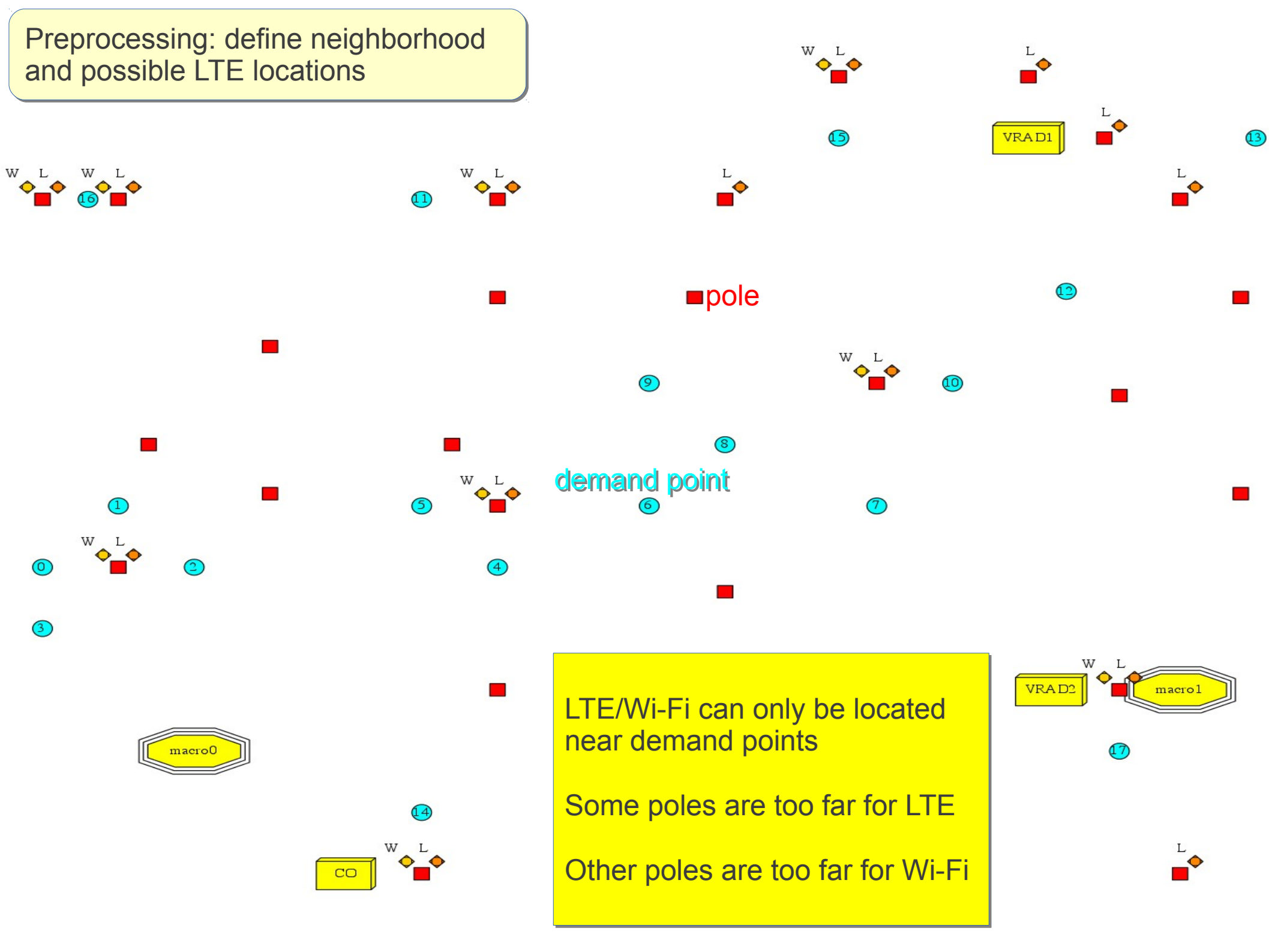
Example: 25 poles, 2 macros, 2 RTs, 1 CO, 17 demand points



Preprocessing: define neighborhood and possible LTE locations



Preprocessing: define neighborhood and possible LTE locations

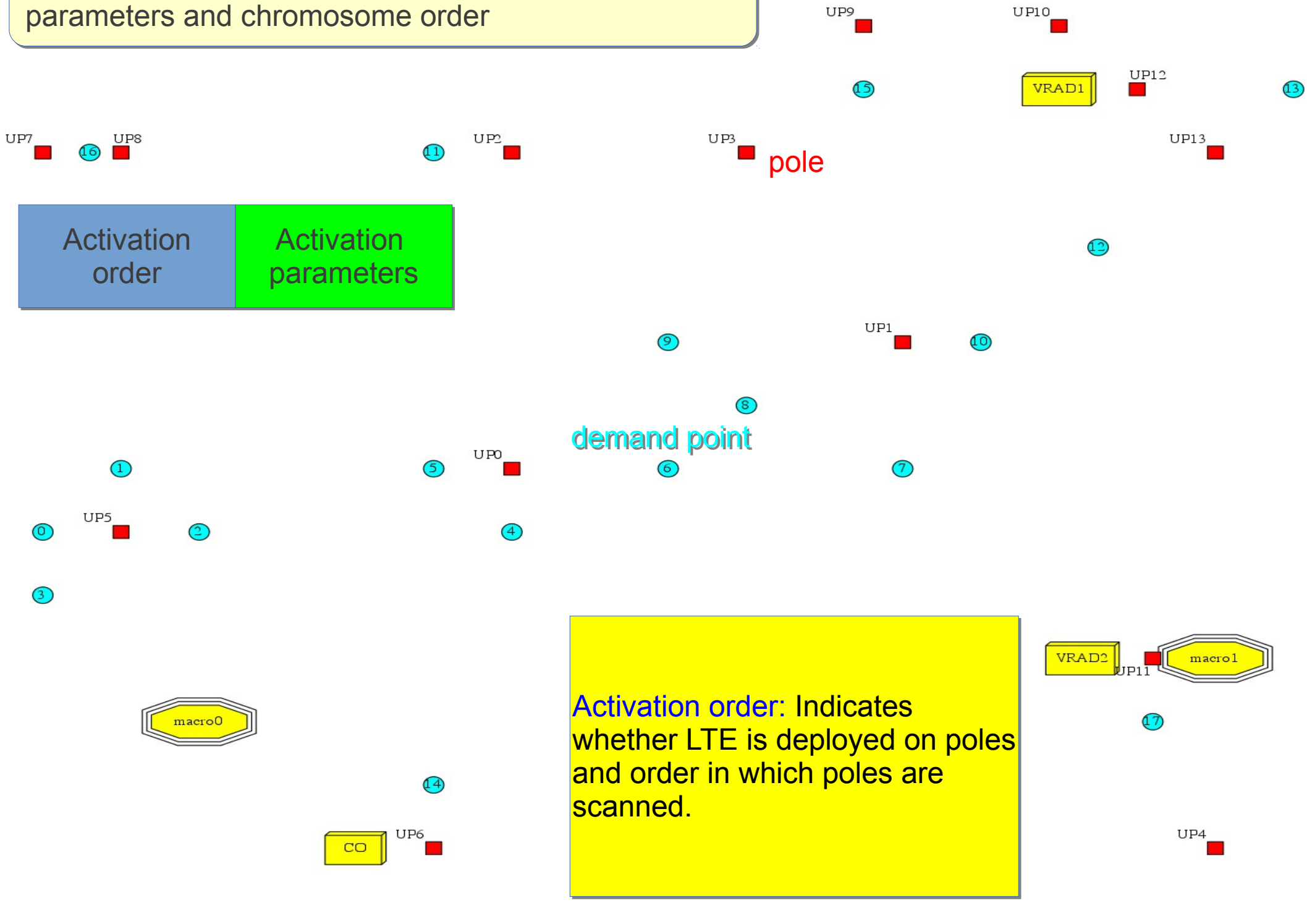


LTE/Wi-Fi can only be located near demand points

Some poles are too far for LTE

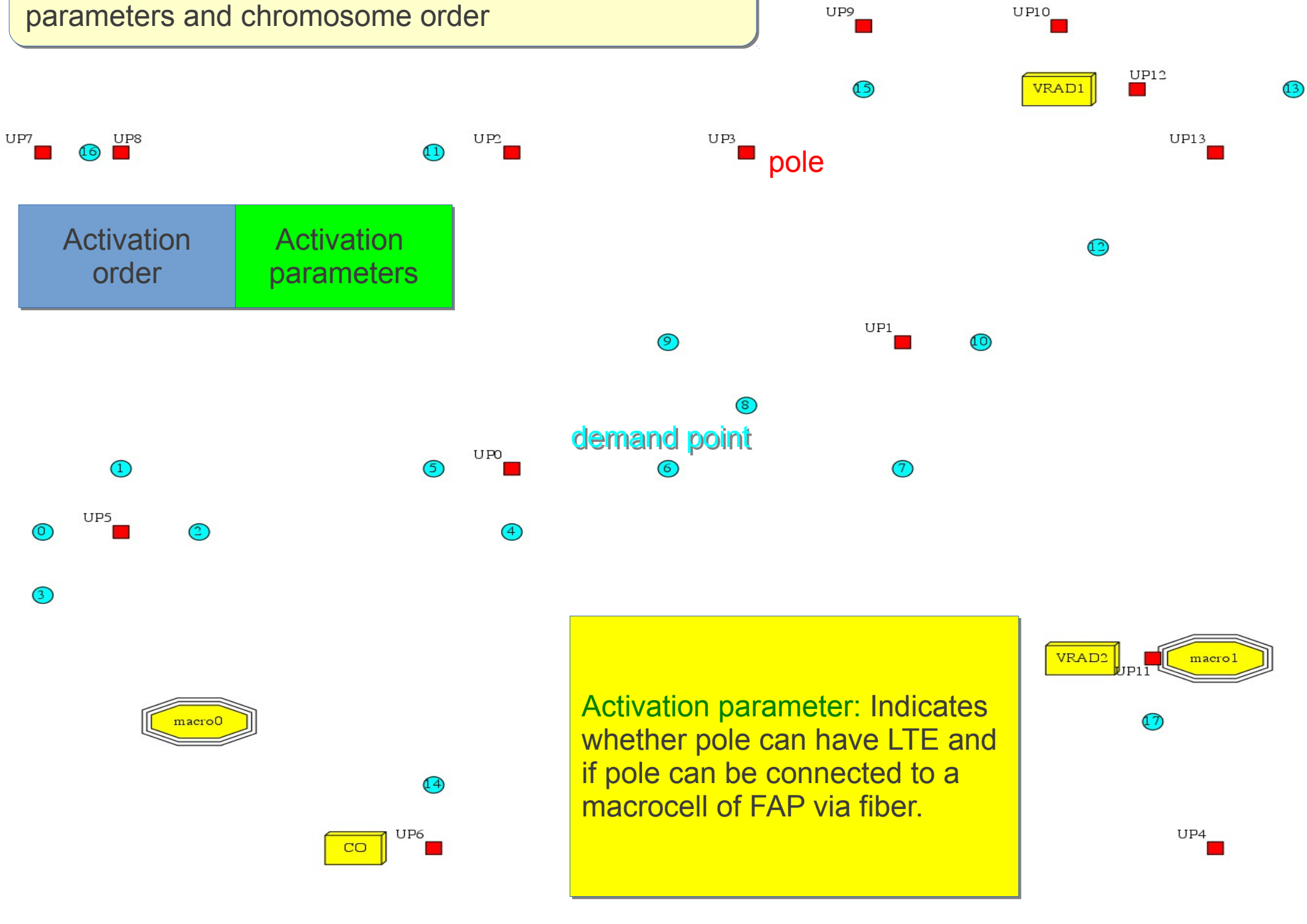
Other poles are too far for Wi-Fi

Activation: LTE deployment according to activation parameters and chromosome order

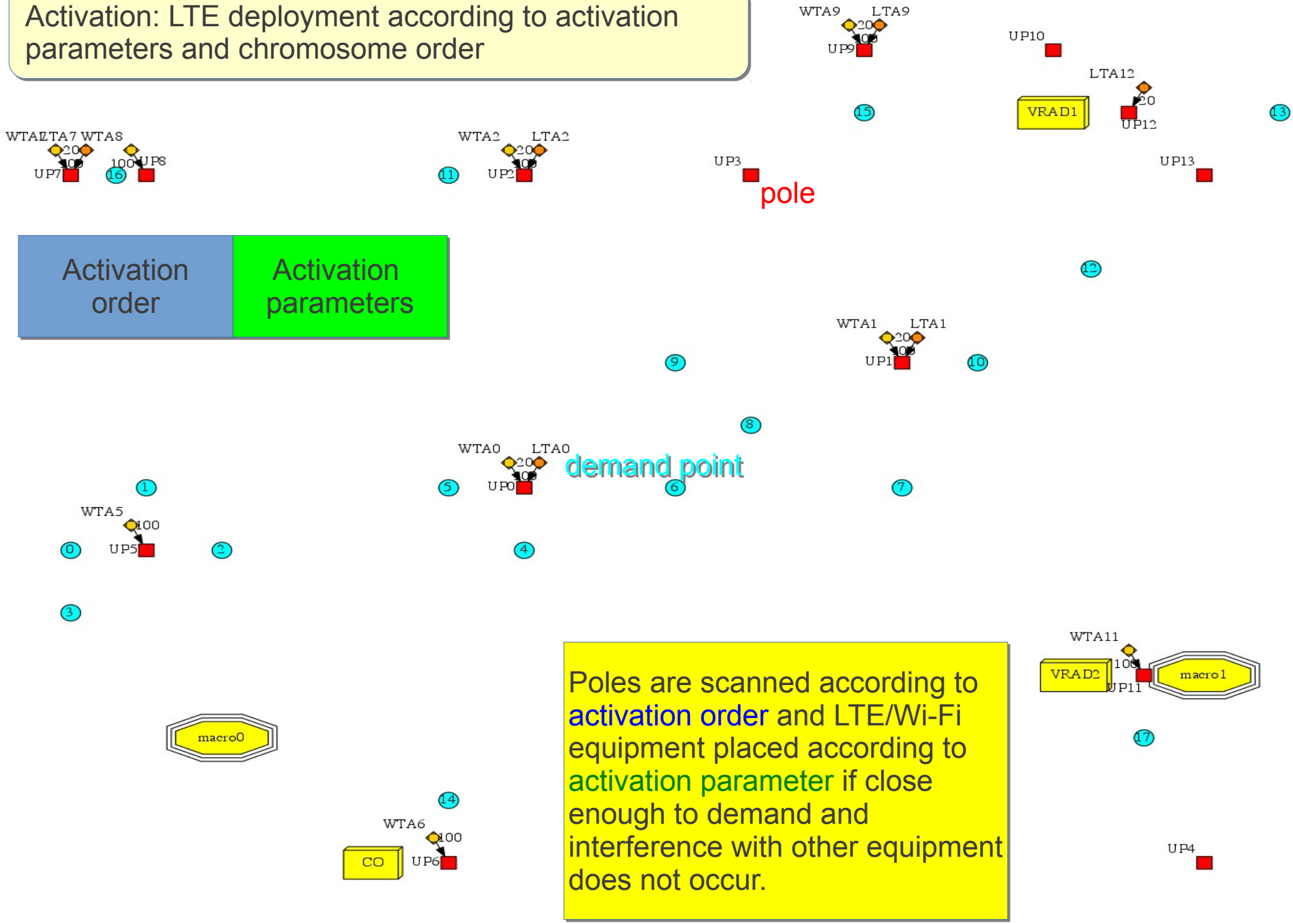


Activation order: Indicates whether LTE is deployed on poles and order in which poles are scanned.

Activation: LTE deployment according to activation parameters and chromosome order

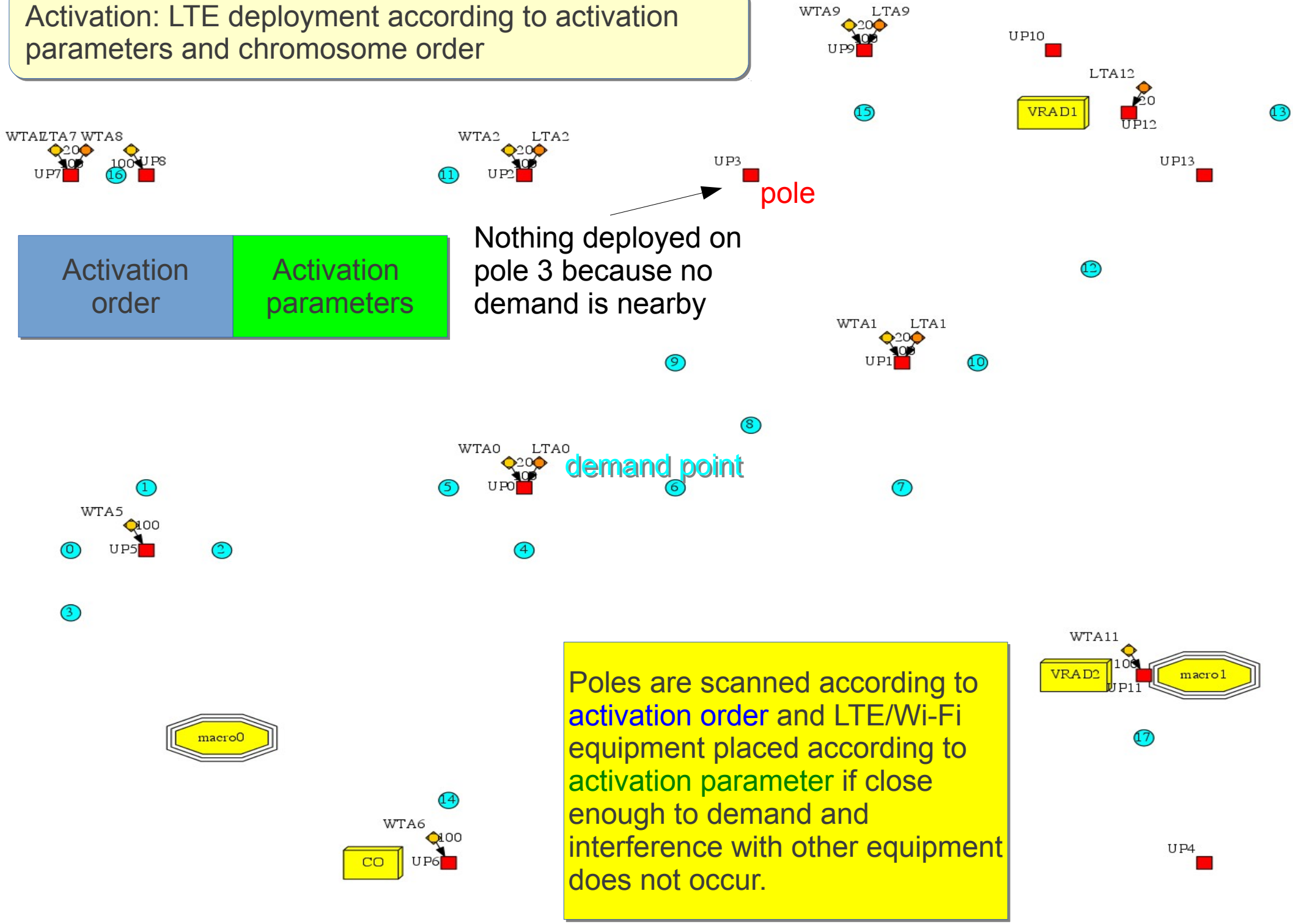


Activation: LTE deployment according to activation parameters and chromosome order

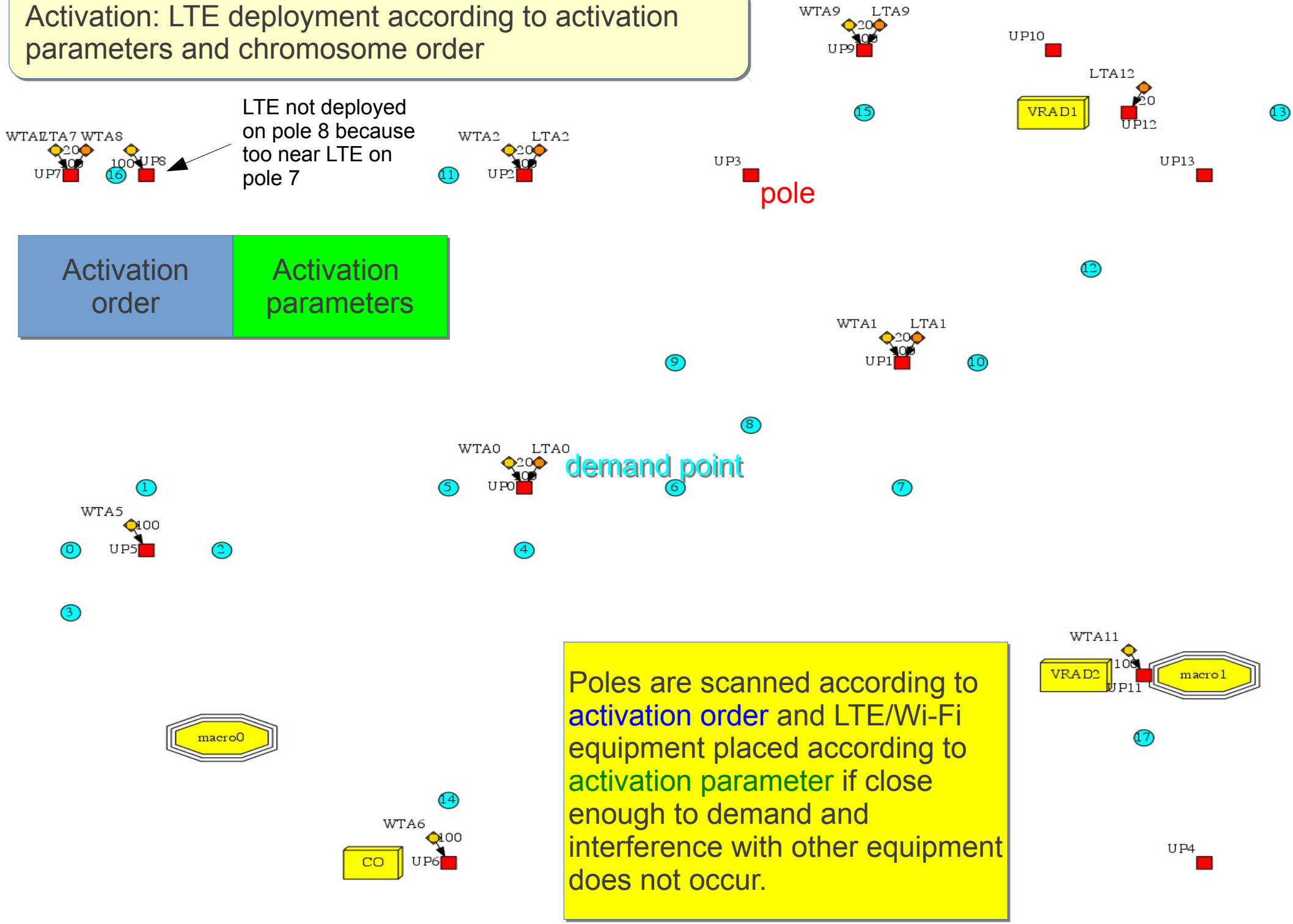


Poles are scanned according to activation order and LTE/Wi-Fi equipment placed according to activation parameter if close enough to demand and interference with other equipment does not occur.

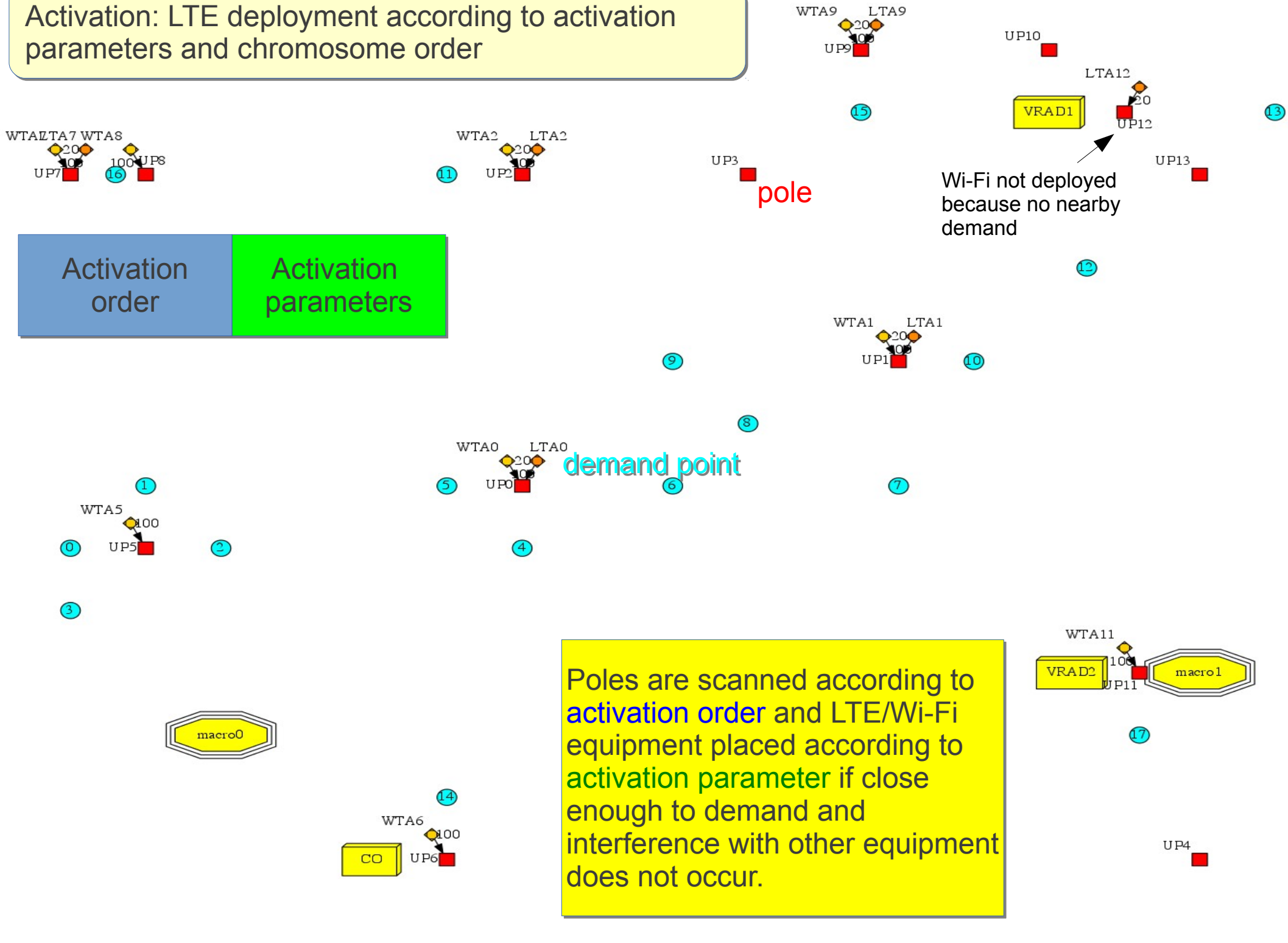
Activation: LTE deployment according to activation parameters and chromosome order



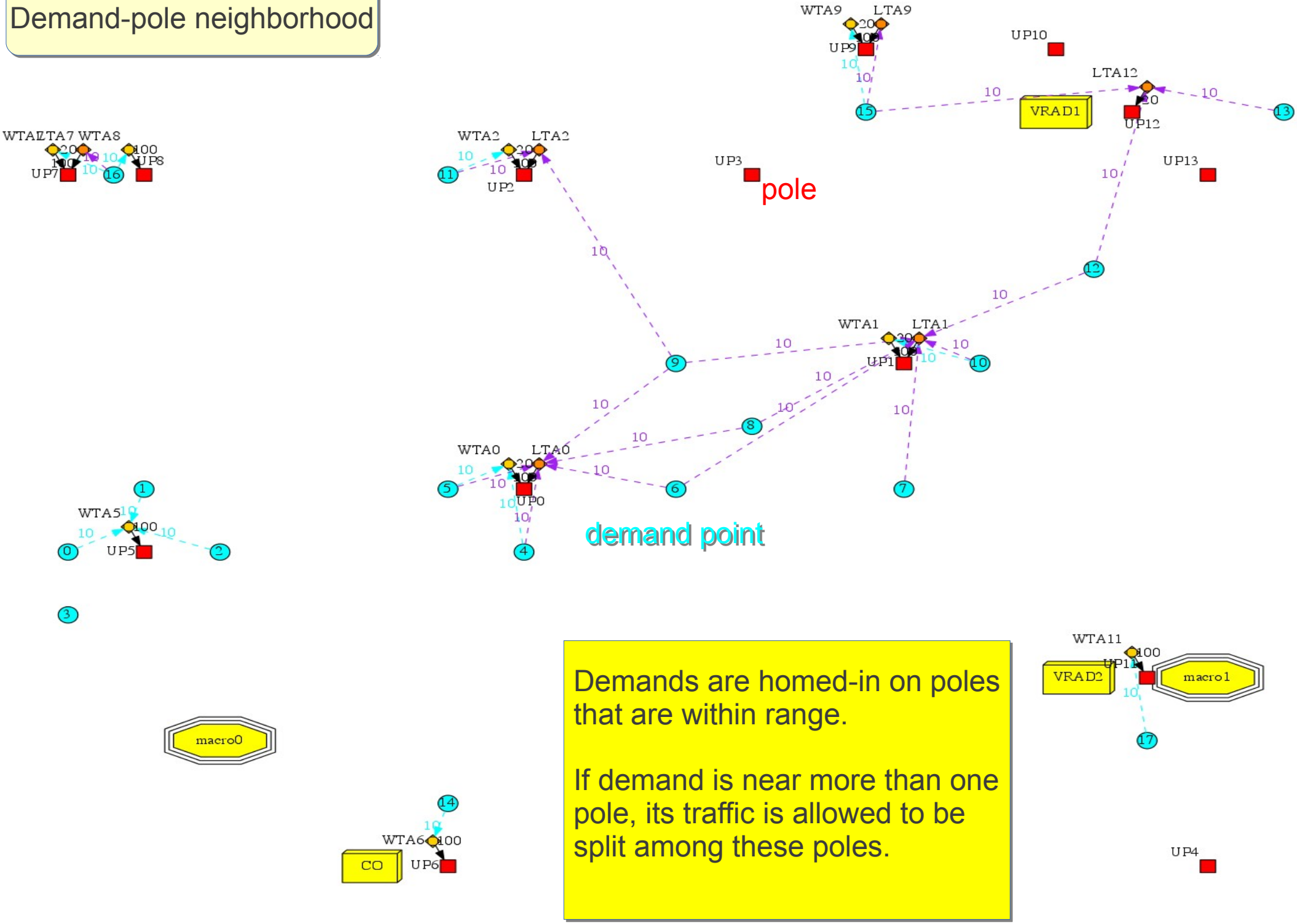
Activation: LTE deployment according to activation parameters and chromosome order



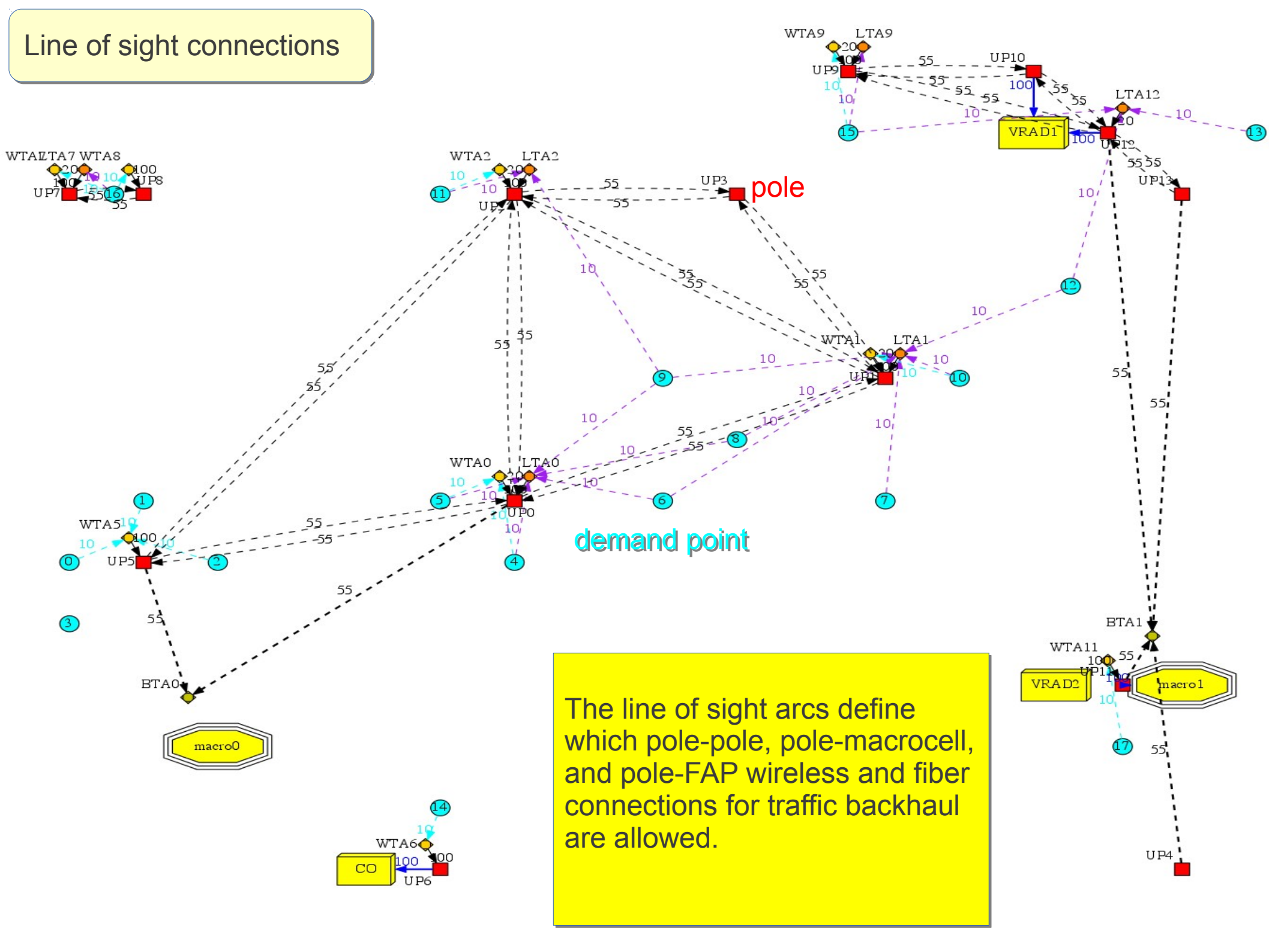
Activation: LTE deployment according to activation parameters and chromosome order



Demand-pole neighborhood



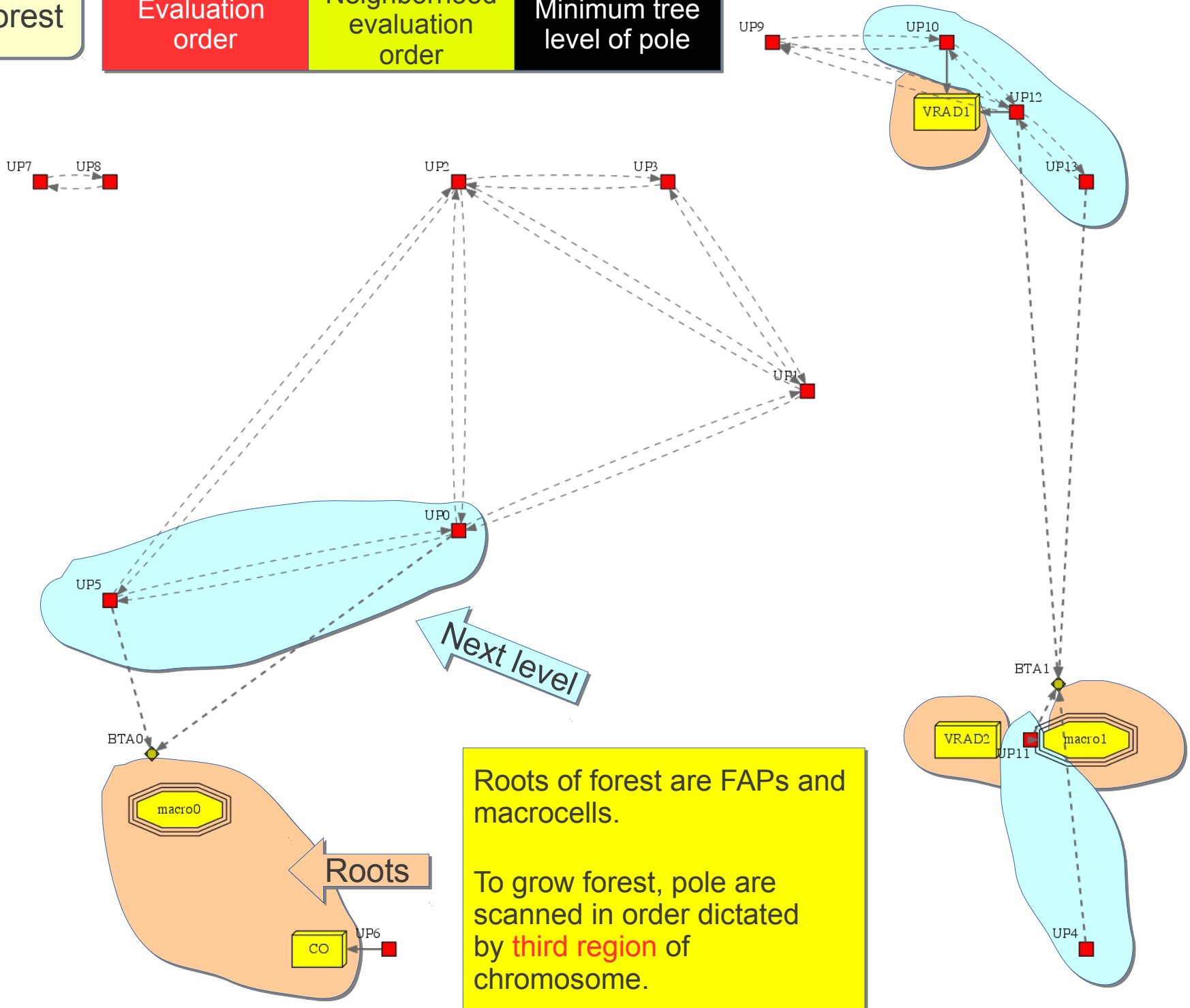
Line of sight connections



The line of sight arcs define which pole-pole, pole-macrocell, and pole-FAP wireless and fiber connections for traffic backhaul are allowed.

Growing forest

Evaluation order	Neighborhood evaluation order	Minimum tree level of pole

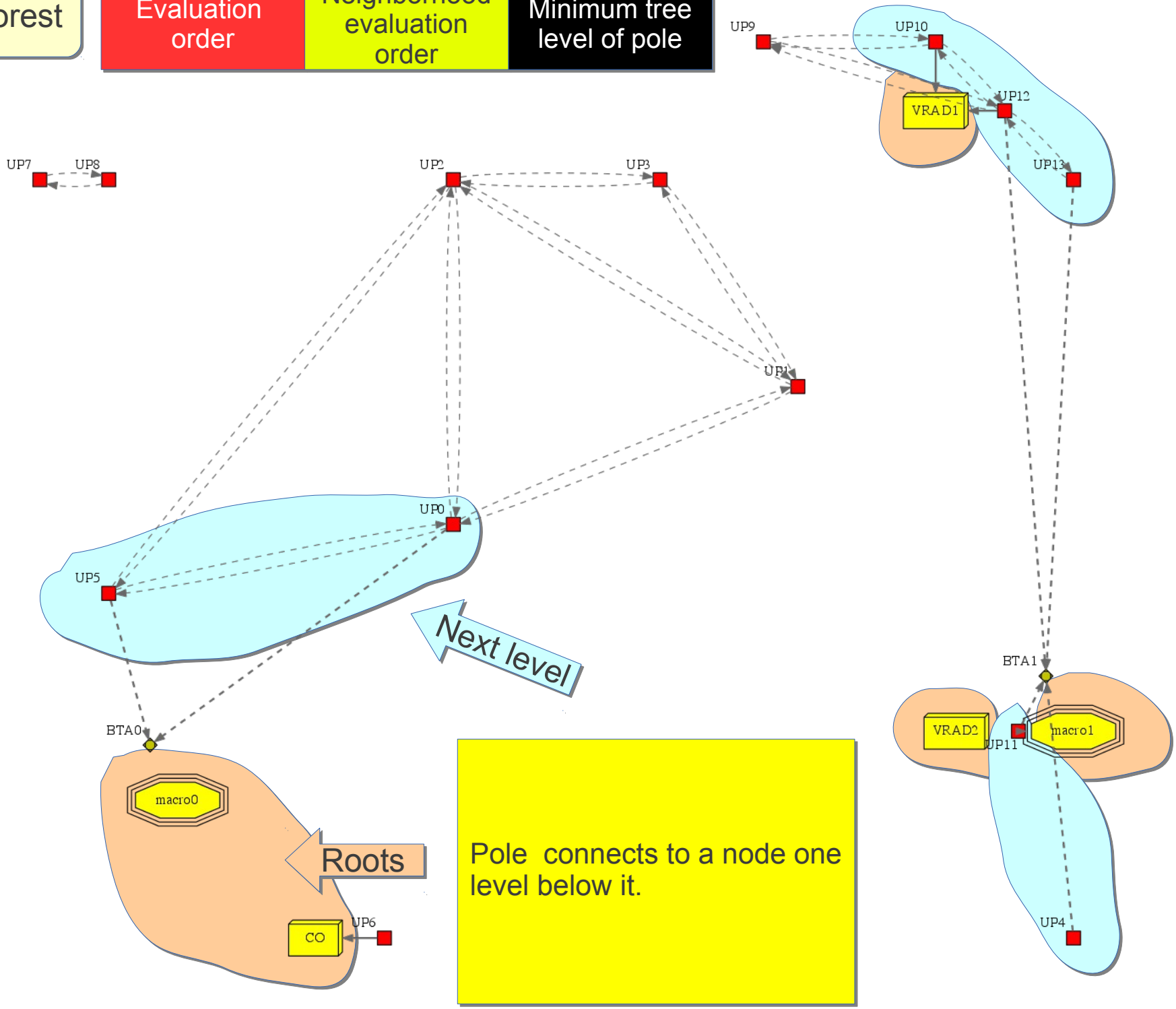


Roots of forest are FAPs and macrocells.

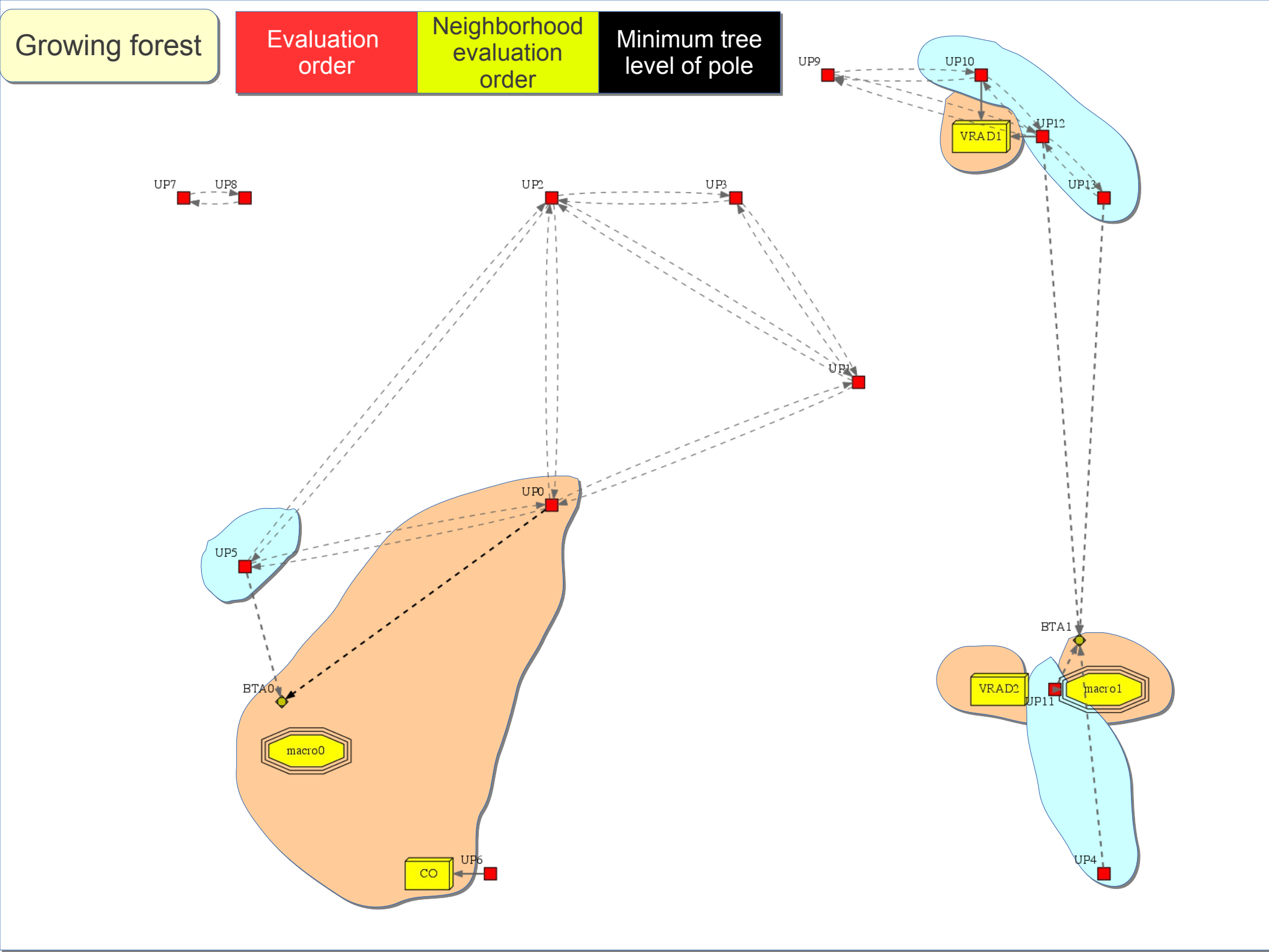
To grow forest, pole are scanned in order dictated by third region of chromosome.

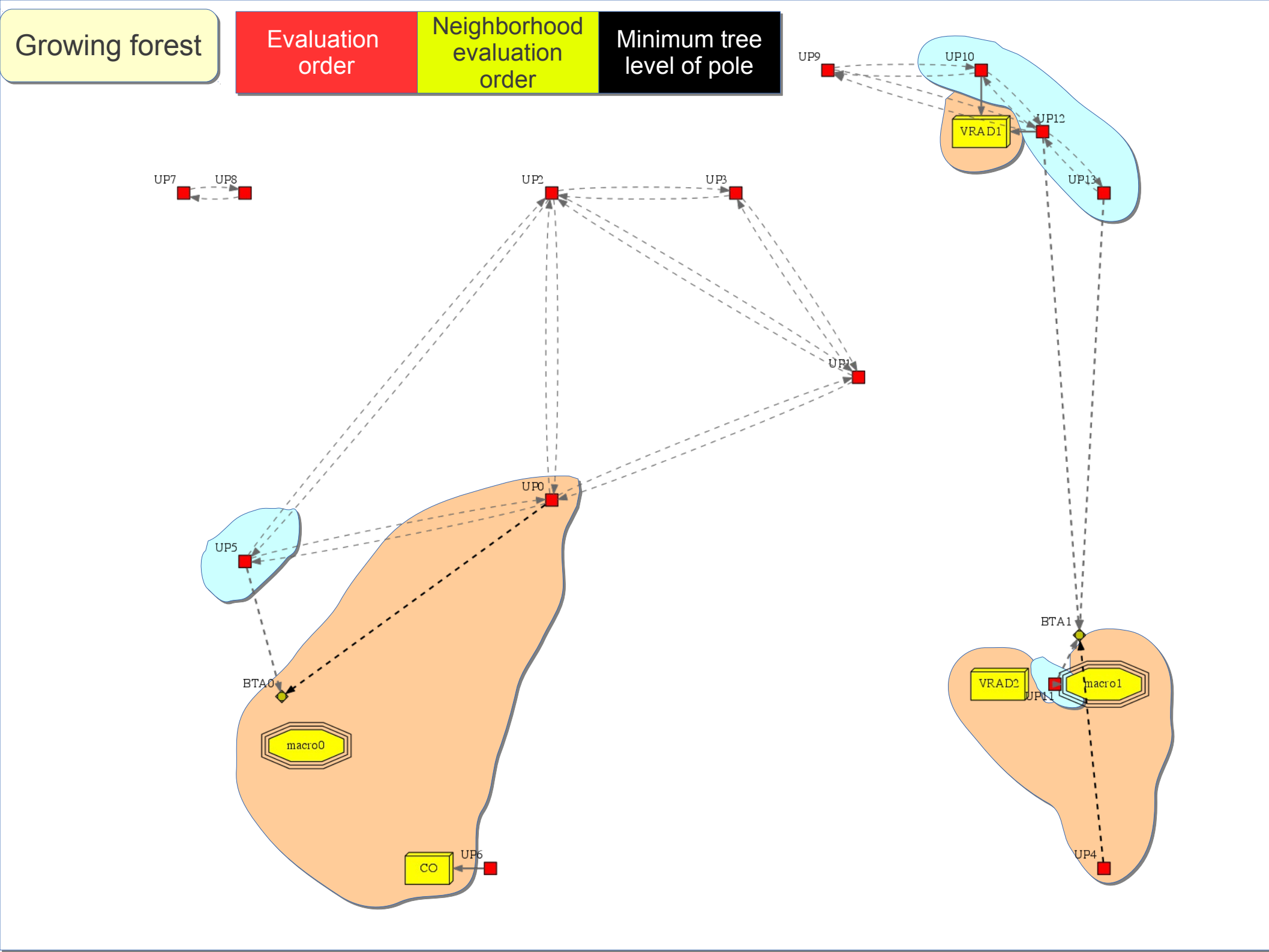
Growing forest

Evaluation order	Neighborhood evaluation order	Minimum tree level of pole



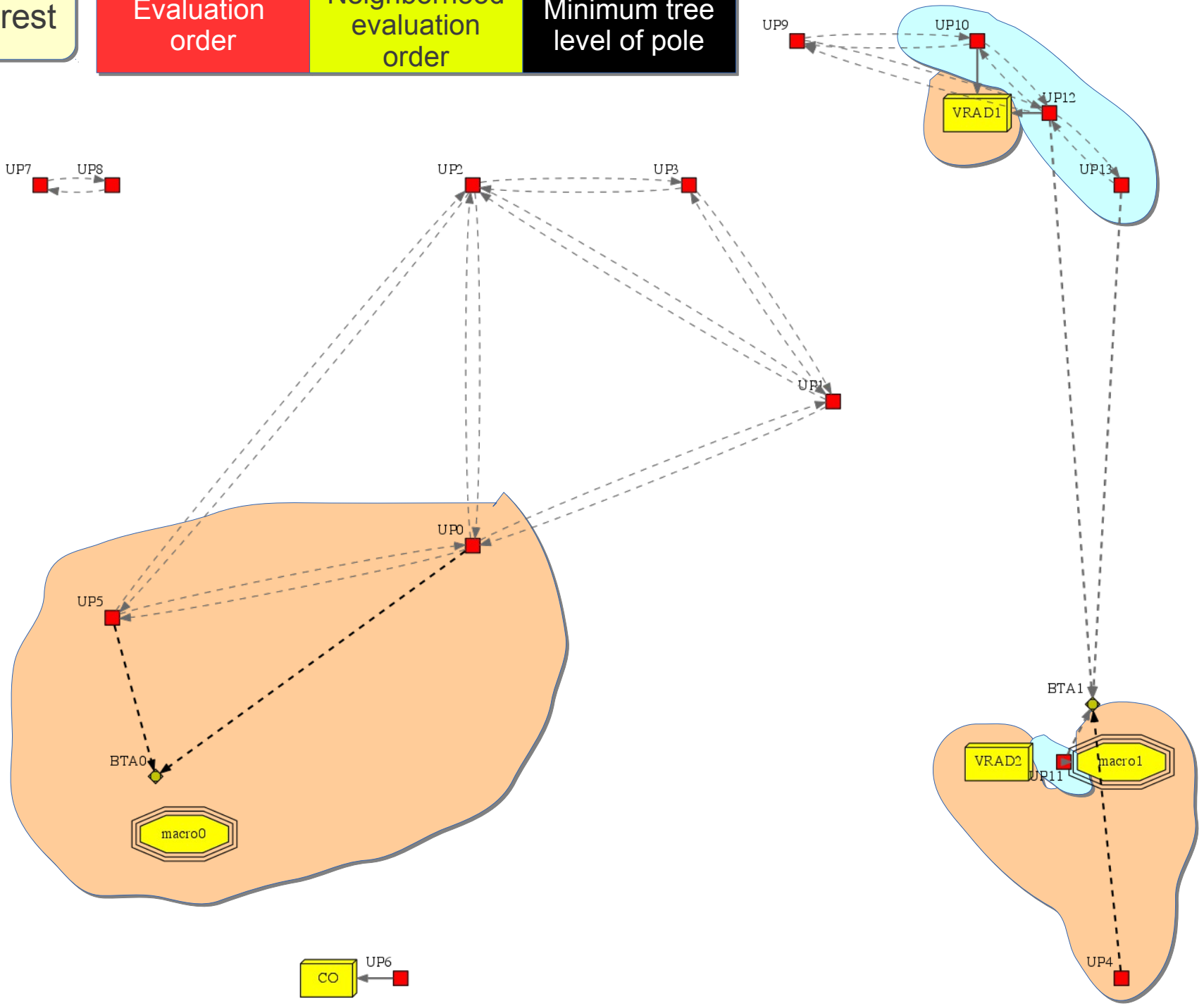
Pole connects to a node one level below it.

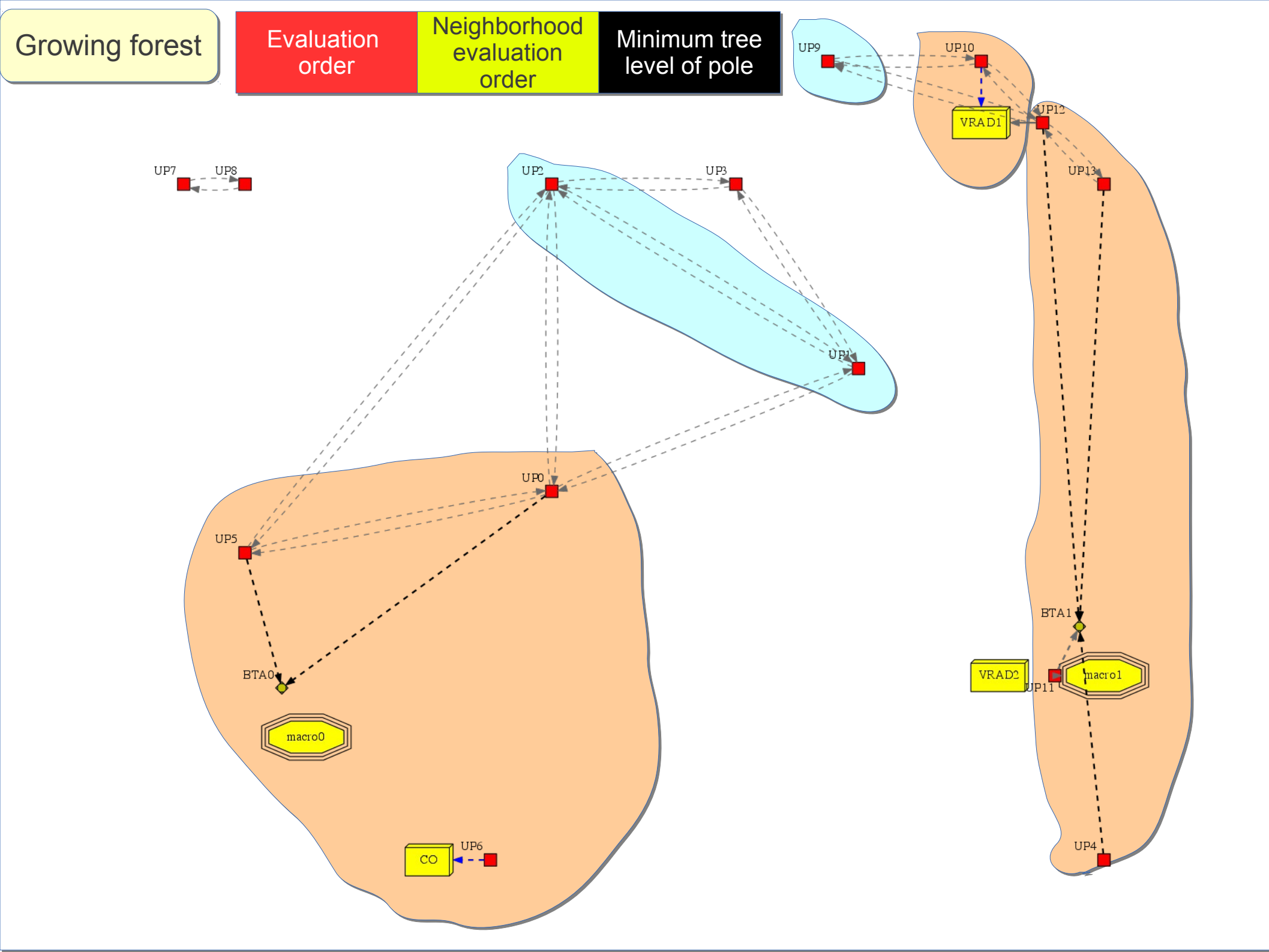

Growing forest
Evaluation order
Neighborhood evaluation order
Minimum tree level of pole
UP9
UP10
UP12
VRAD1
UP13
UP7
UP8
UP2
UP3
UP1
UP0
UP5
BTA0
macro0
CO
UP6
BTA1
VRAD2
UP11
macro1
UP4


Growing forest
Evaluation order
Neighborhood evaluation order
Minimum tree level of pole
UP9
UP10
UP12
VRAD1
UP13
UP7
UP8
UP2
UP3
UP1
UP0
UP5
BTA0
macro0
CO
UP6
BTA1
VRAD2
UP11
macro1
UP4

Growing forest

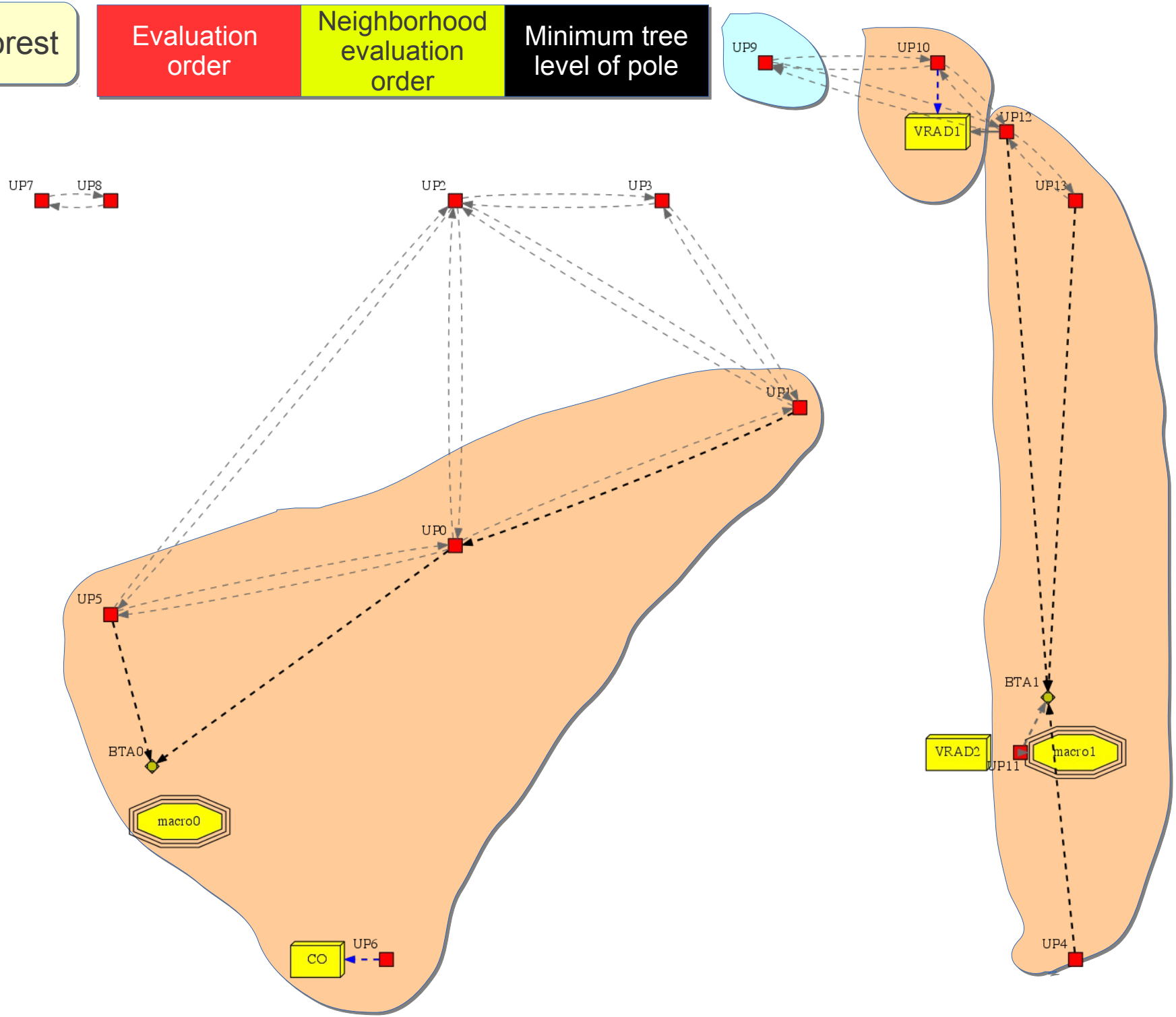
Evaluation order	Neighborhood evaluation order	Minimum tree level of pole



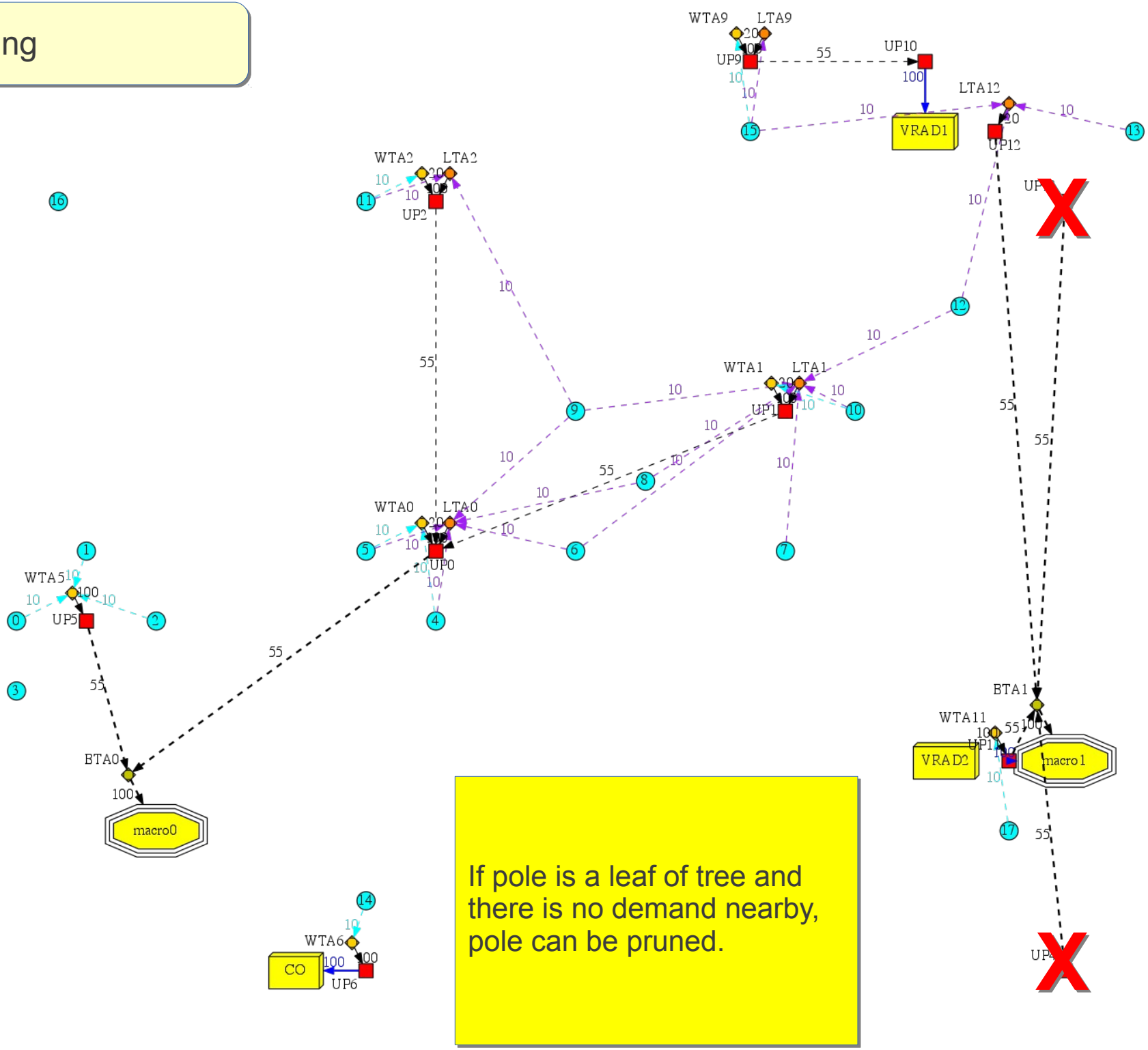

Growing forest
Evaluation order
Neighborhood evaluation order
Minimum tree level of pole
UP9
UP10
VRAD1
UP12
UP13
UP7
UP8
UP2
UP3
UP1
UP0
UP5
BTA0
macro0
CO
UP6
BTA1
VRAD2
UP11
macro1
UP4

Growing forest

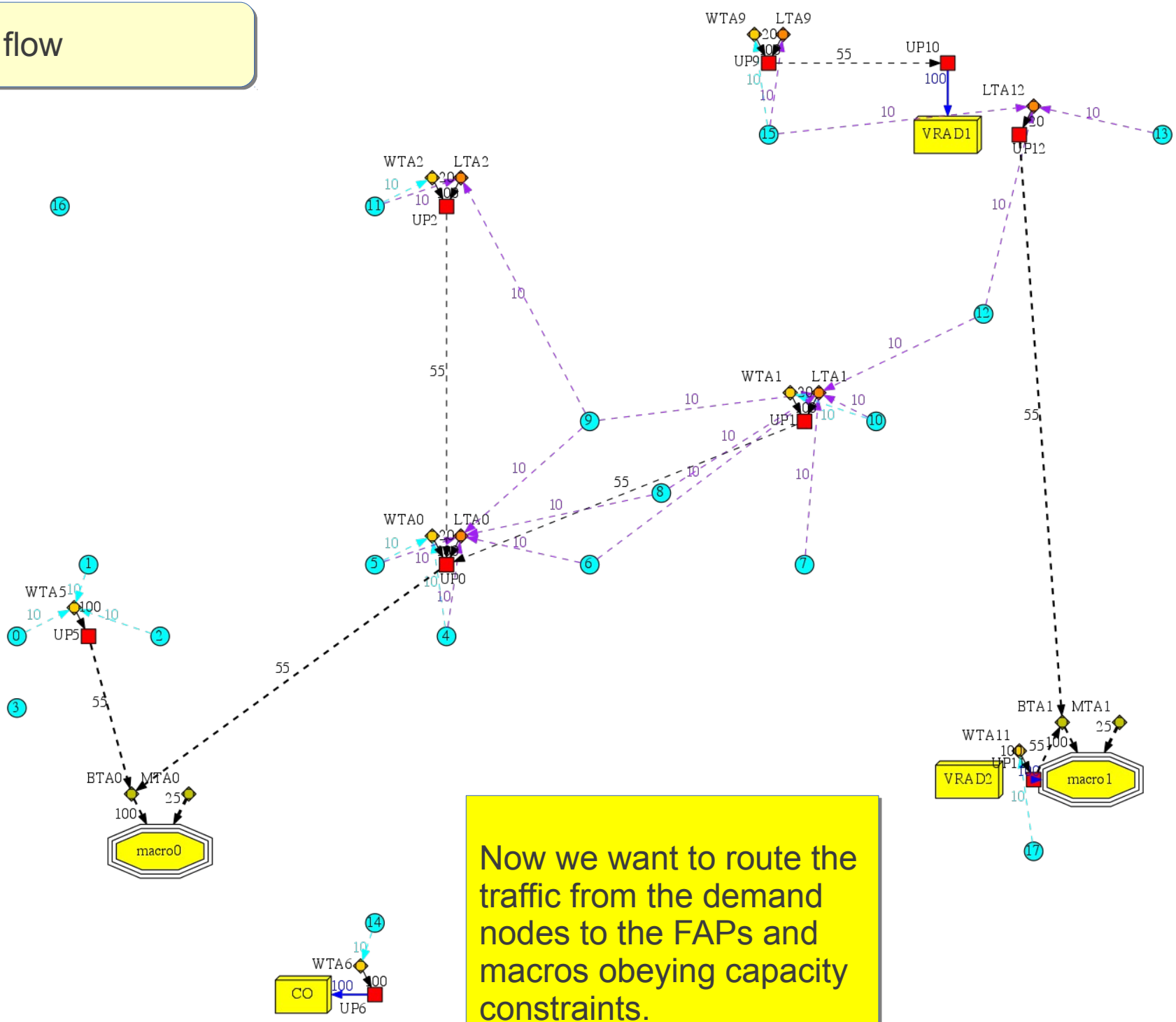
Evaluation order	Neighborhood evaluation order	Minimum tree level of pole



First Pruning

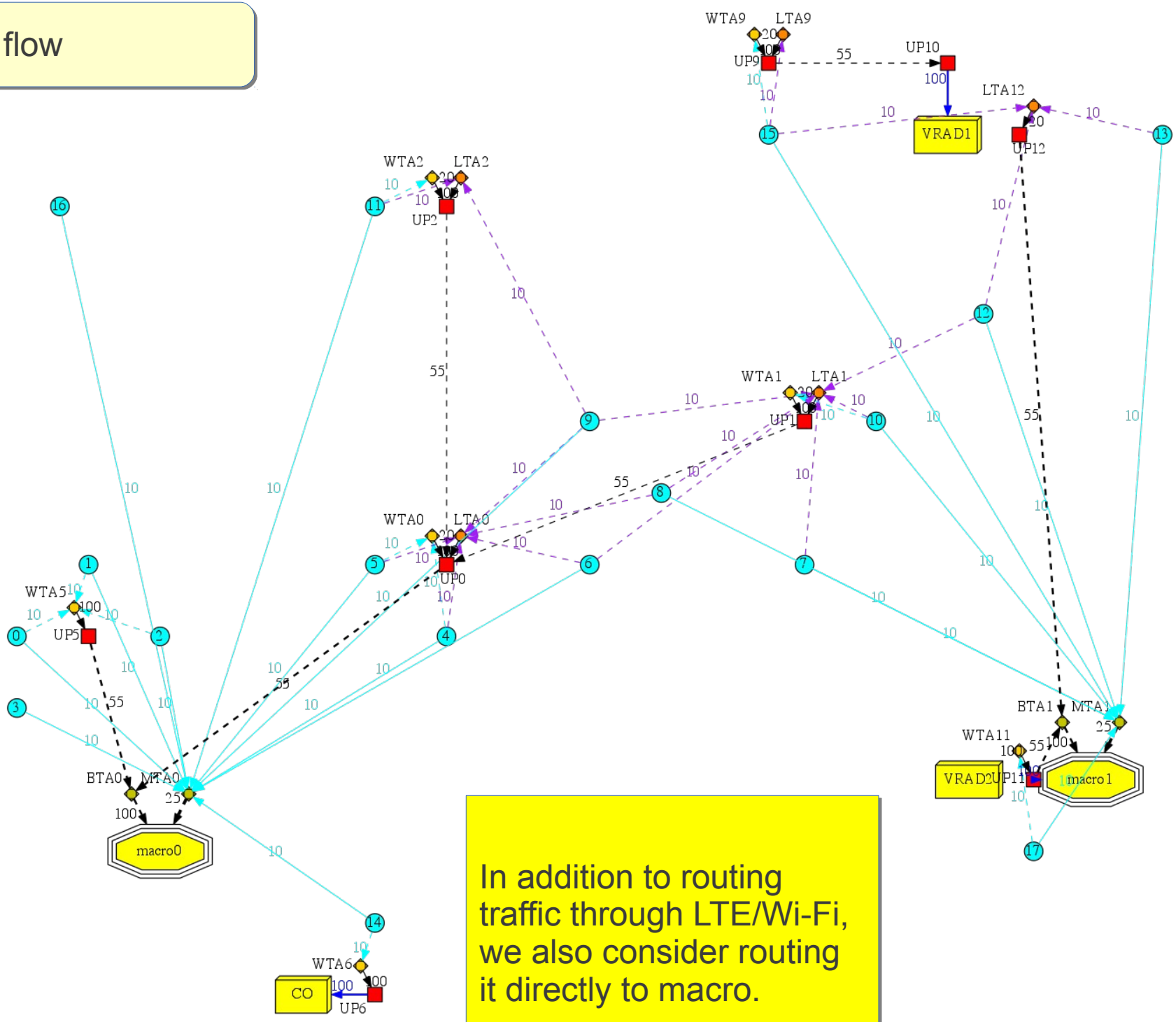


Maximum flow



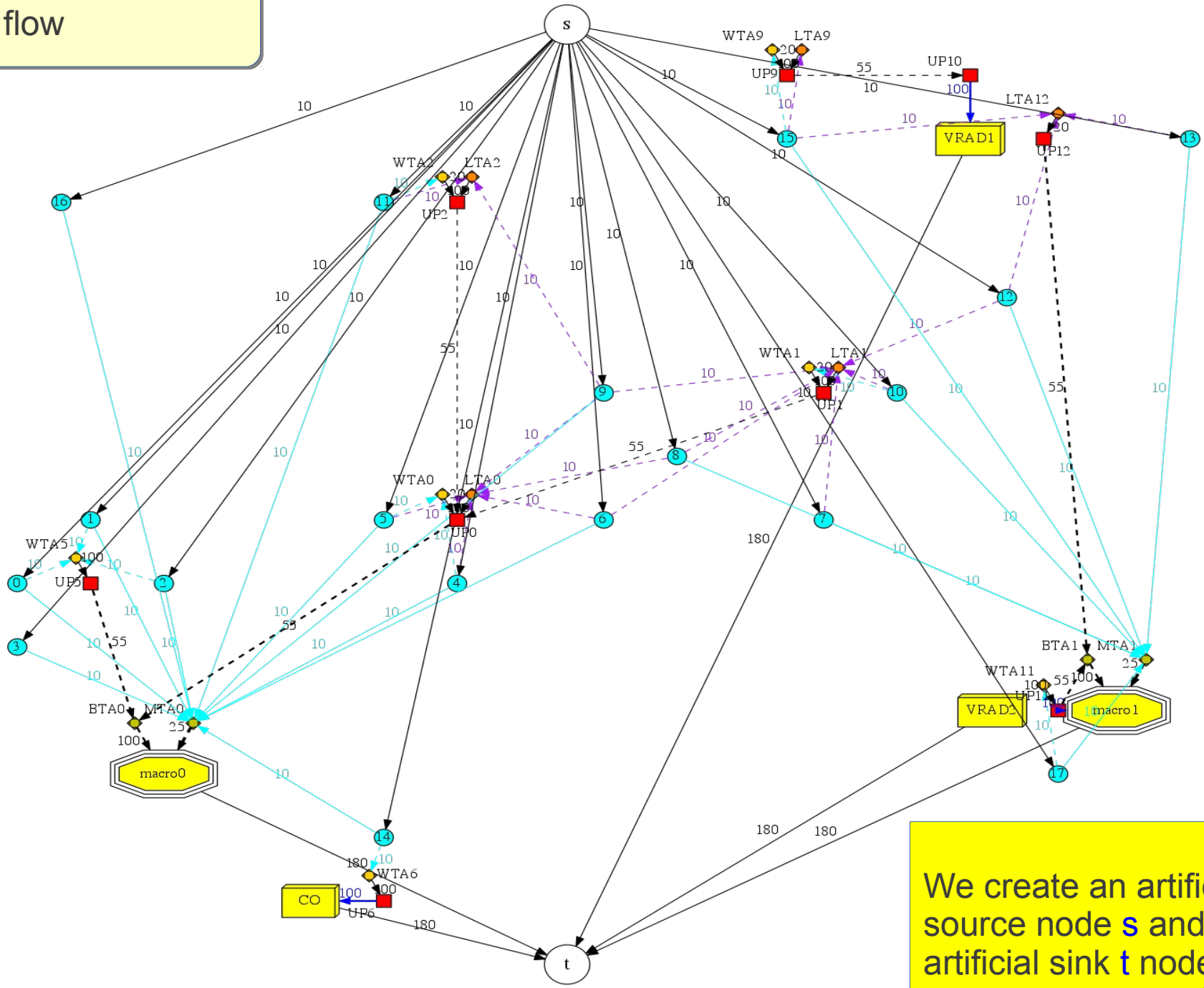
Now we want to route the traffic from the demand nodes to the FAPs and macros obeying capacity constraints.

Maximum flow



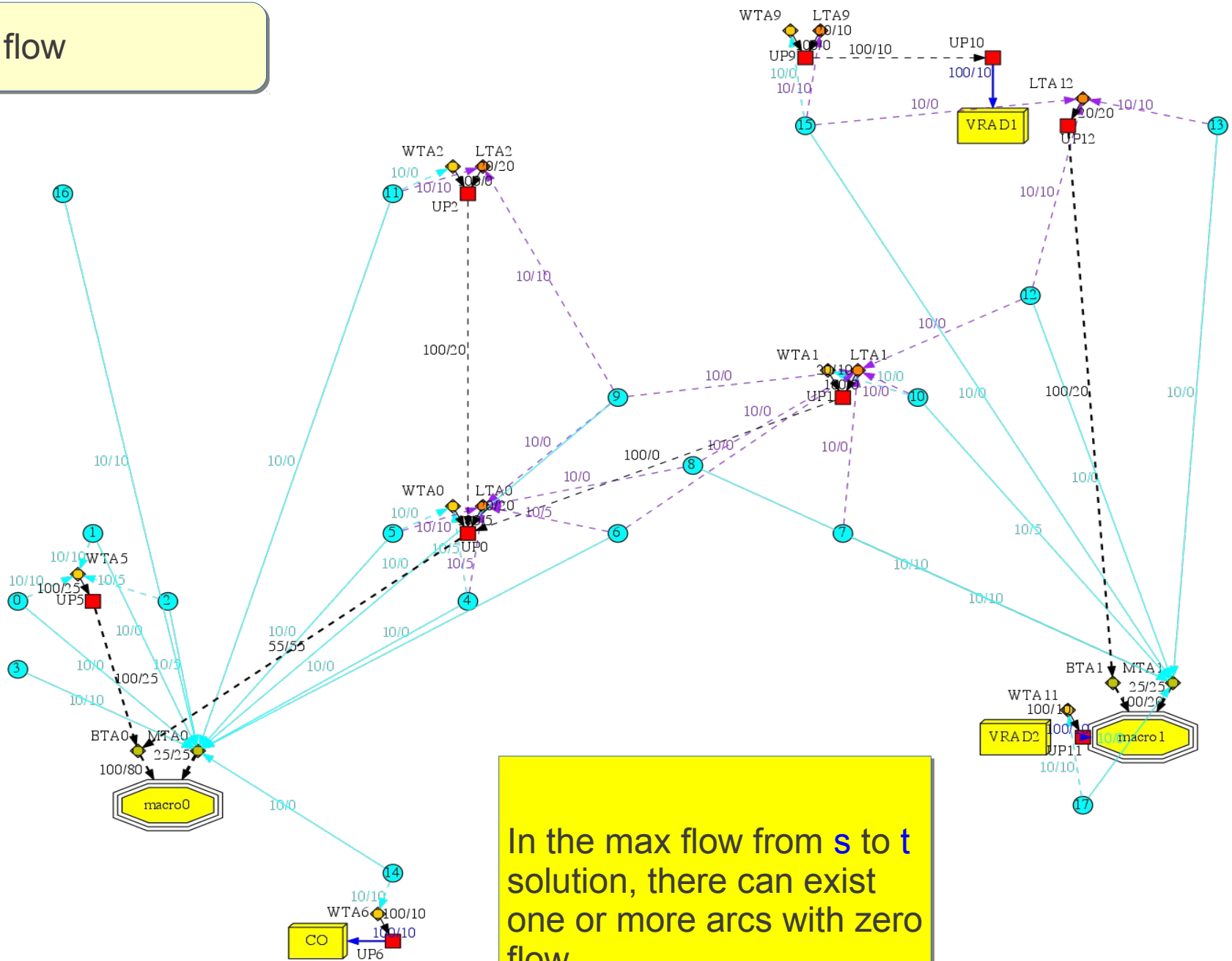
In addition to routing traffic through LTE/Wi-Fi, we also consider routing it directly to macro.

Maximum flow



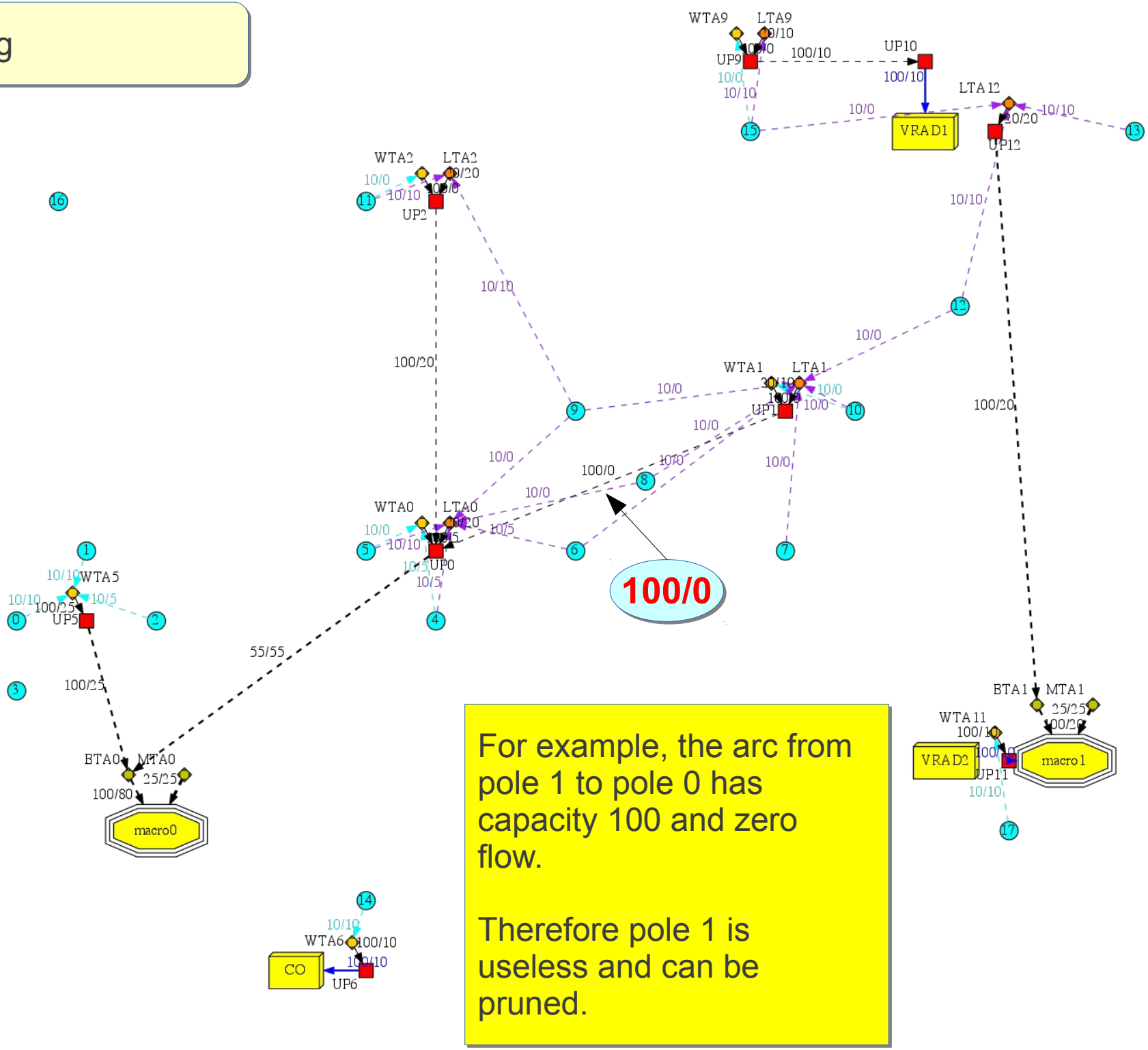
We create an artificial source node s and an artificial sink t node and compute the max flow from s to t.

Maximum flow



In the max flow from **s** to **t** solution, there can exist one or more arcs with zero flow.

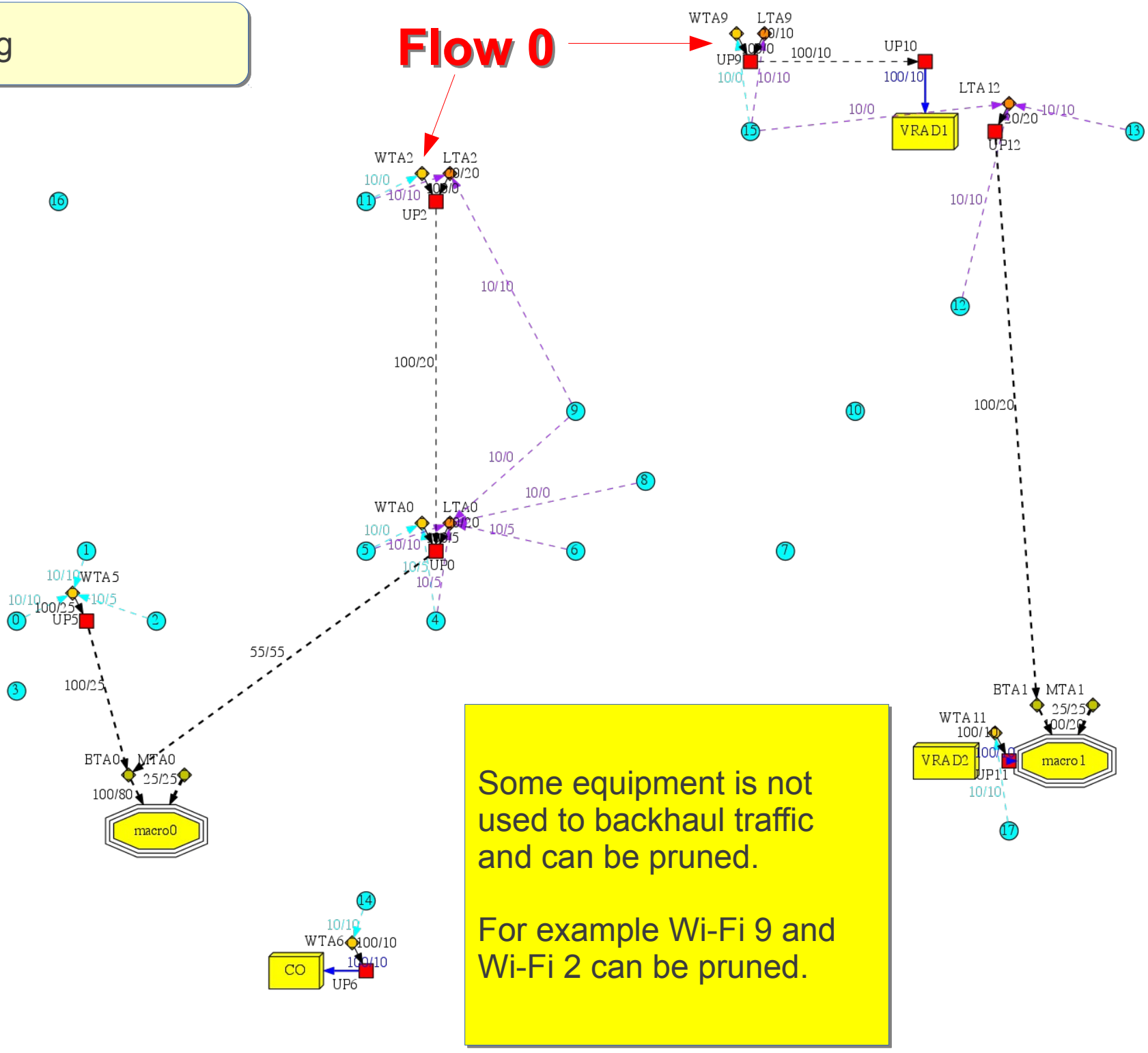
2nd Pruning



For example, the arc from pole 1 to pole 0 has capacity 100 and zero flow.

Therefore pole 1 is useless and can be pruned.

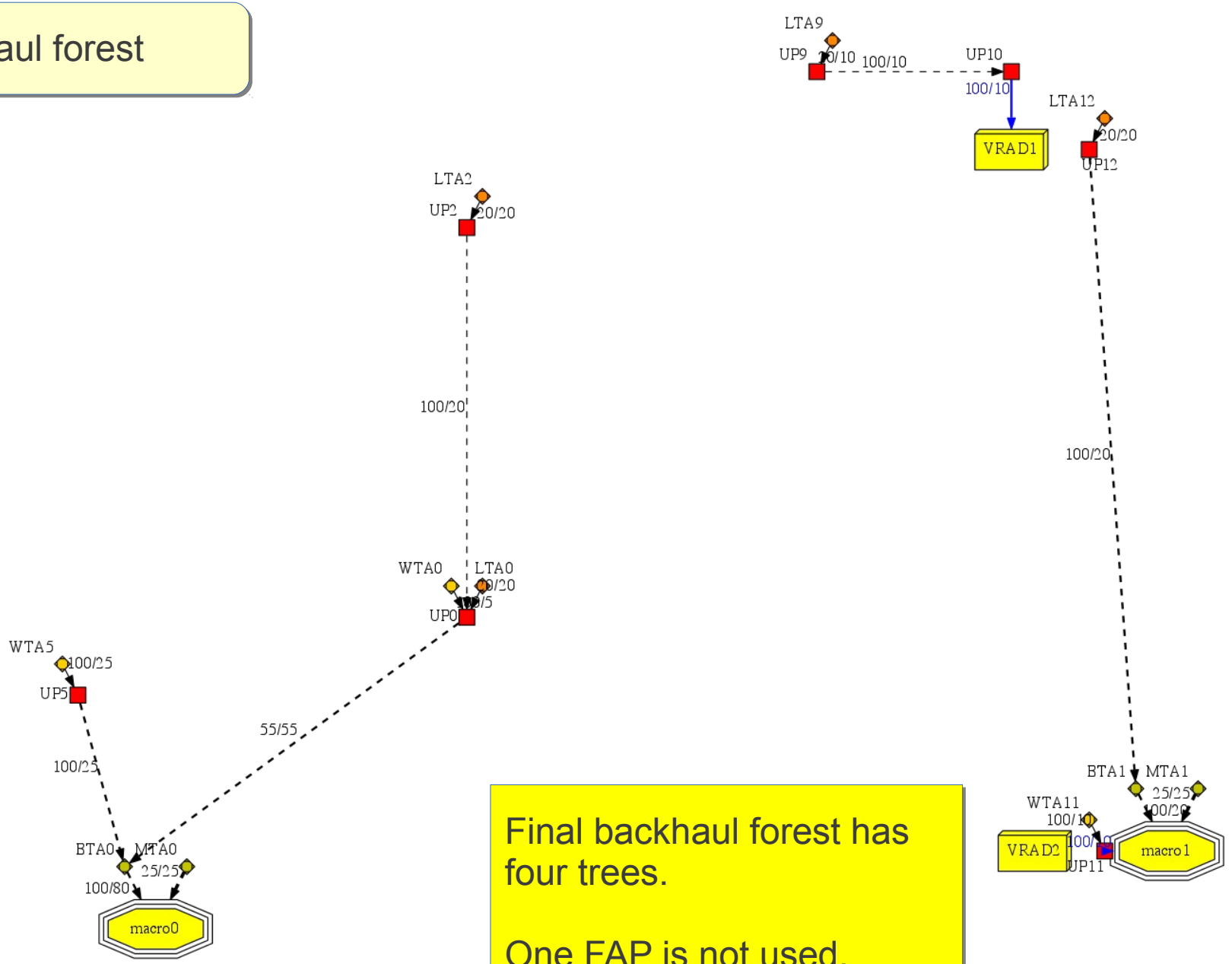
2nd Pruning



Some equipment is not used to backhaul traffic and can be pruned.

For example Wi-Fi 9 and Wi-Fi 2 can be pruned.

Final backhaul forest



Final backhaul forest has four trees.

One FAP is not used.

Revenues and costs can be computed to determine objective function value.

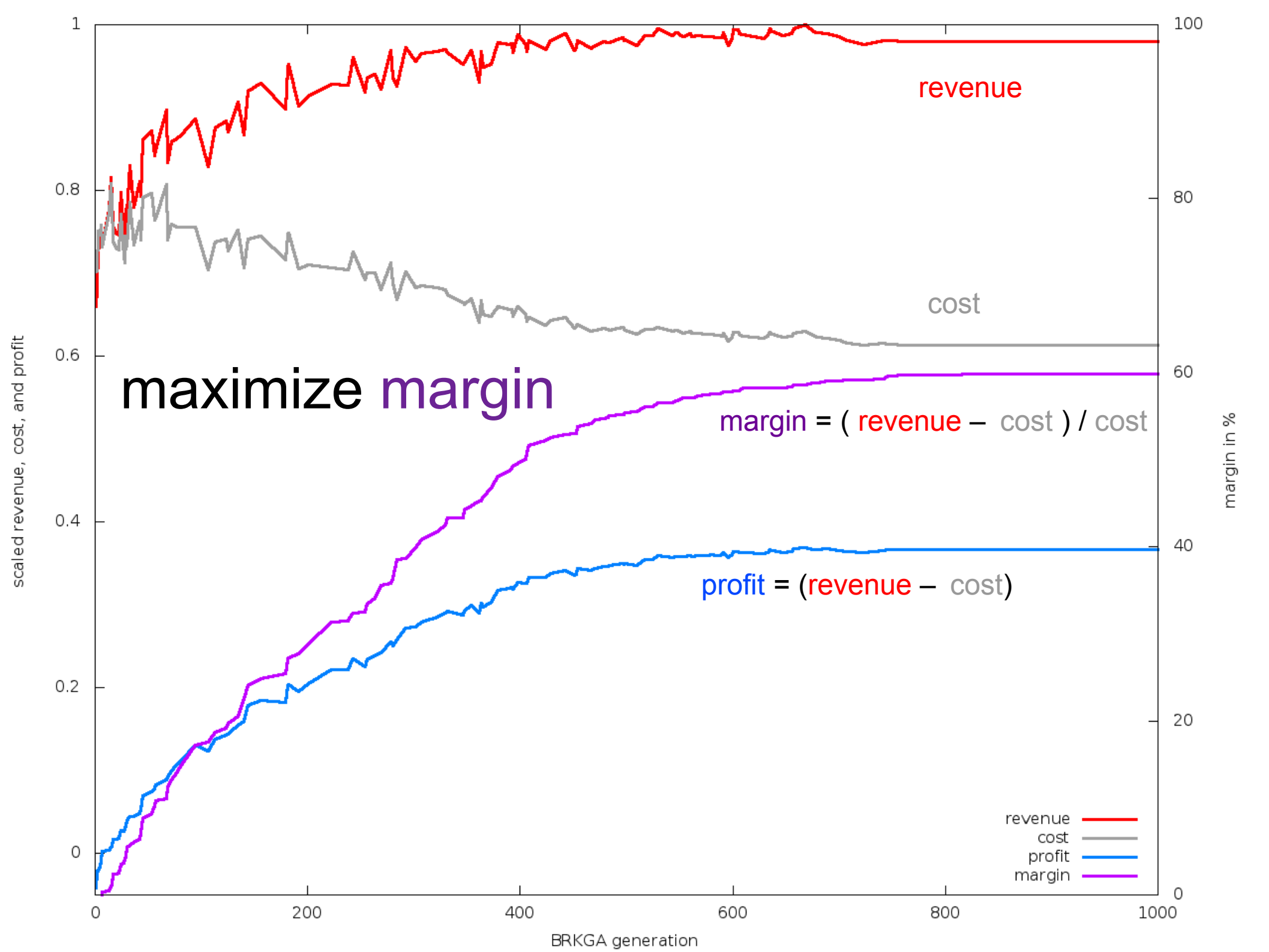
An example optimization

We ran the BRKGA on a "real" instance from a large Tier 1 Internet Service Provider

- Number of utility poles: 4395
- Number of demand points: 3236
- Number of fibered access points (FAP) excluding macrocells: 482
- Number of macrocells: 31

- Objective function: maximize margin = (revenue - cost)/cost

Rethink Possible
att.com/rethinkpossible


maximize margin
revenue
cost
margin = (revenue – cost) / cost
profit = (revenue – cost)
scaled revenue, cost, and profit
margin in %
BRKGA generation
0
0.2
0.4
0.6
0.8
1
0
20
40
60
80
100
0
200
400
600
800
1000
revenue
cost
profit
margin

Applied Soft Computing 33 (2015) 150–169

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

A biased random-key genetic algorithm for wireless backhaul network design

Carlos E. Andrade [a,*], Mauricio G.C. Resende [b,c], Weiyi Zhang [b], Rakesh K. Sinha [b], Kenneth C. Reichmann [b], Robert D. Doverspike [b], Flávio K. Miyazawa [a]

[a] Institute of Computing, University of Campinas, Avenida Albert Einstein 1251, Campinas, SP 13083-852, Brazil
[b] Network Evolution Research Department, AT&T Labs Research, 200 S. Laurel Avenue, Middletown, NJ 07748, USA
[c] Amazon.com, Inc., Mathematical Optimization and Planning Group, 333 Boren Avenue North, Seattle, WA 98109, USA

ARTICLE INFO

Article history:
Received 27 November 2014
Received in revised form 27 March 2015
Accepted 2 April 2015
Available online 16 April 2015

Keywords:
Wireless backhaul network design
Small cells
Genetic algorithm
Mixed integer programming model

ABSTRACT

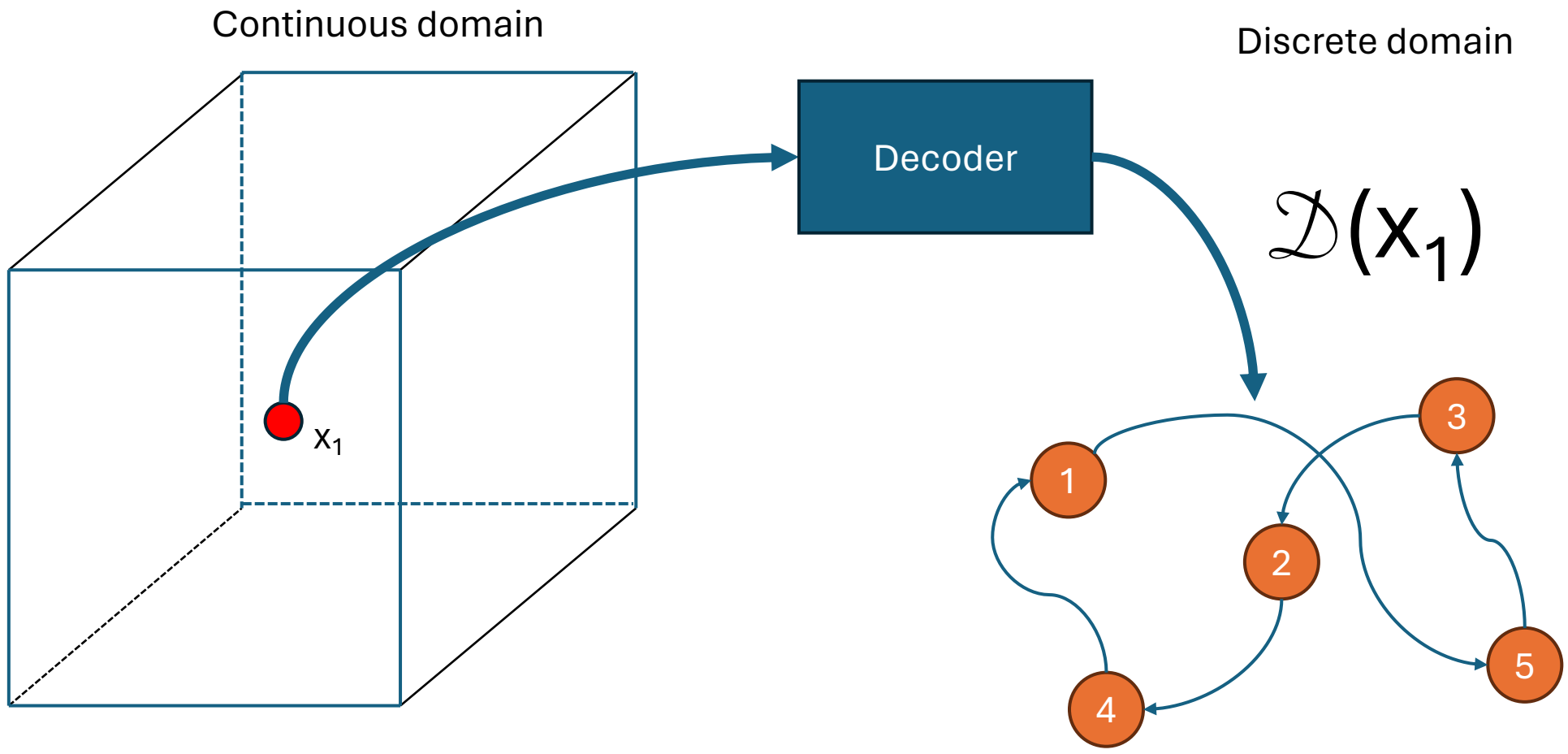
This paper describes a biased random-key genetic algorithm for a real-world wireless backhaul network design problem. This is a novel problem, closely related to variants of the Steiner tree problem and the facility location problem. Given a parameter h, we want to build a forest where each tree has at most h hops from the demand nodes, where traffic originates, to the root nodes where each tree is rooted. Candidate Steiner nodes do not have any demand but represent locations where we can install cellsites to cover the traffic and equipment to backhaul the traffic to the cellular core network. Each Steiner node can cover demand nodes within a given distance, subject to a capacity constraint. The aggregate set of constraints may make it impossible to cover or backhaul all demands. A revenue function computes the revenue associated with the total amount of traffic covered and backhauled to the root nodes. The objective of the problem is to build a forest that maximizes the difference between the total revenue and

Part II

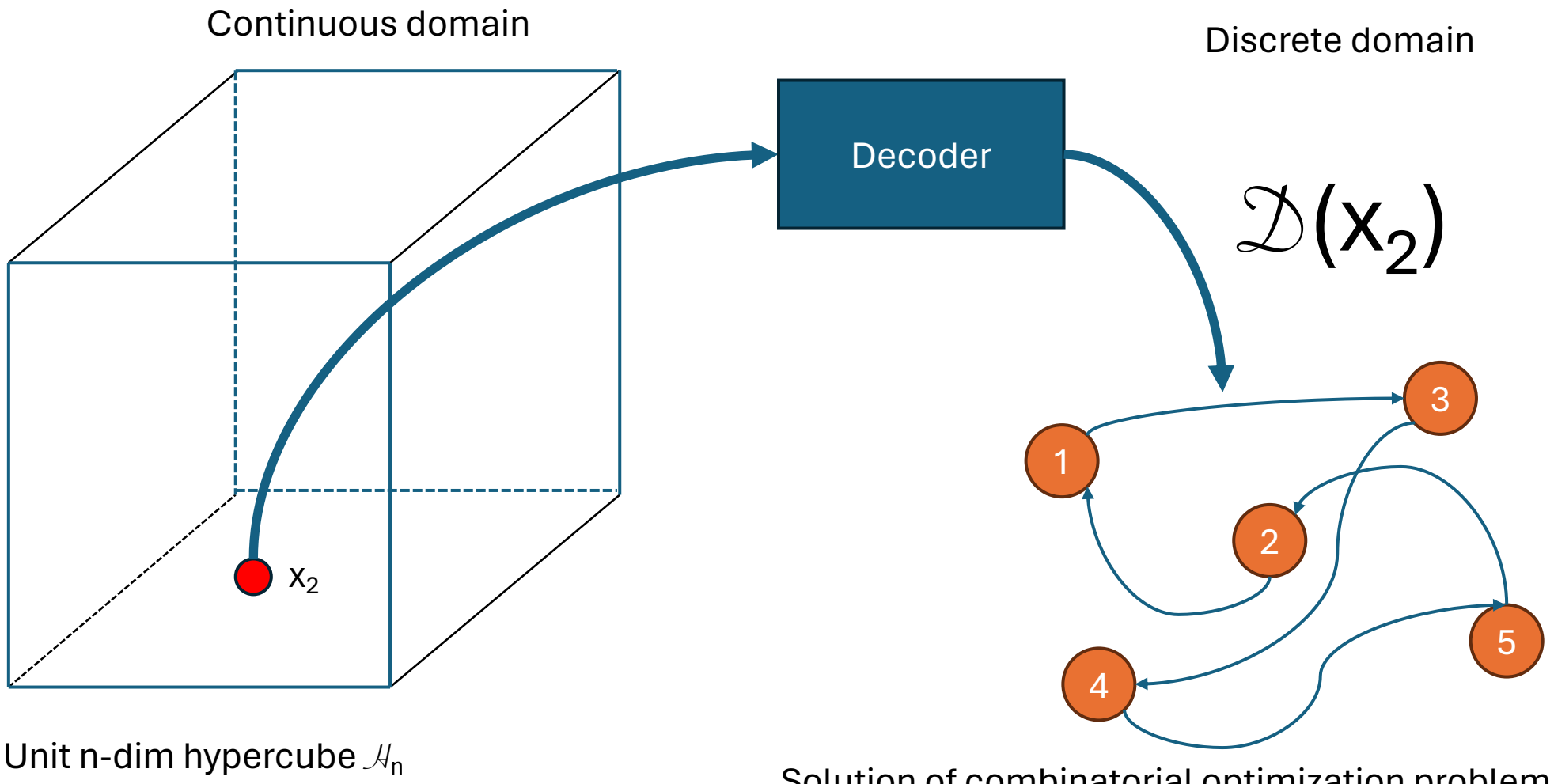
Random-Key Optimizers



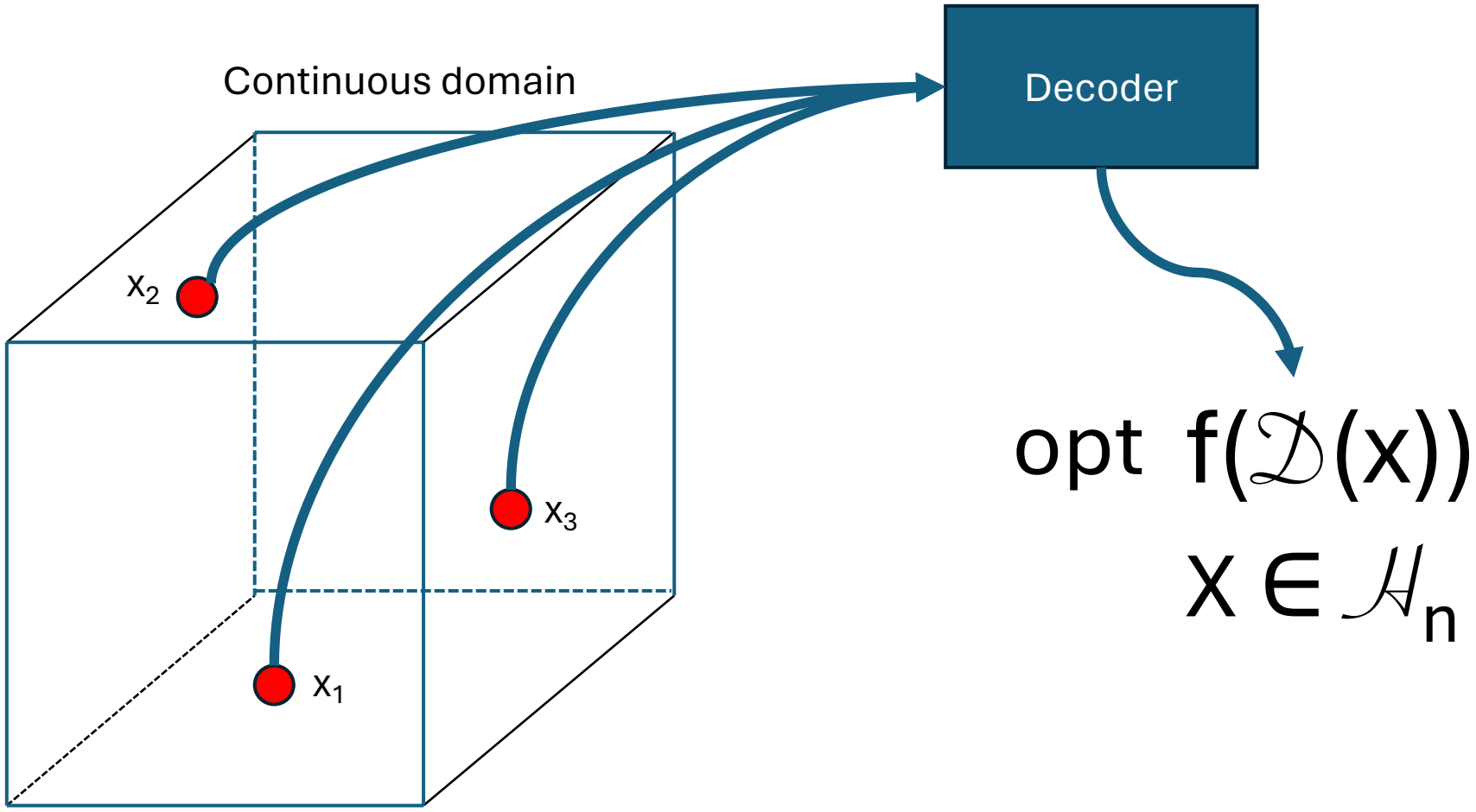
Random-key optimizers



Random-key optimizers

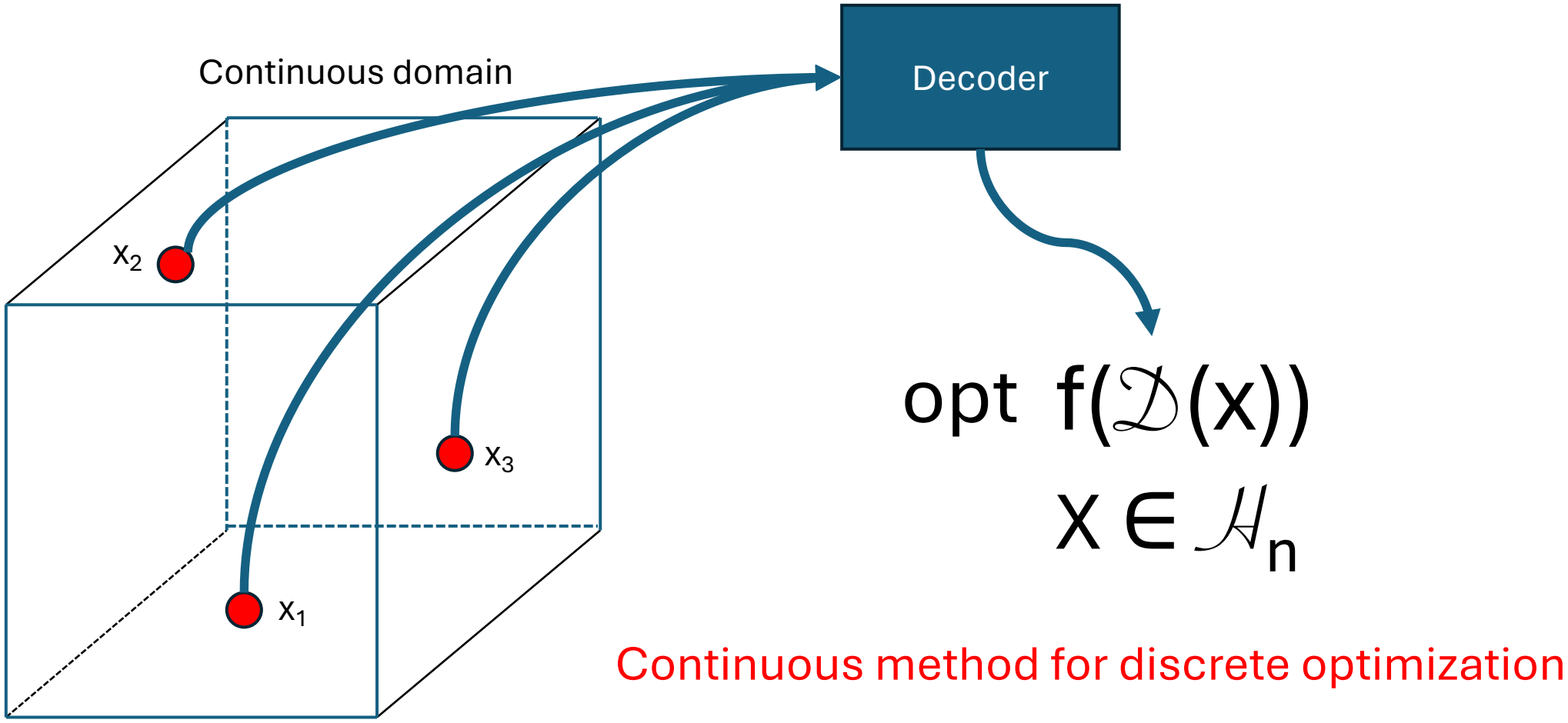


Random-key optimizers



Unit n-dim hypercube $\mathcal{H}_n$

Random-key optimizers

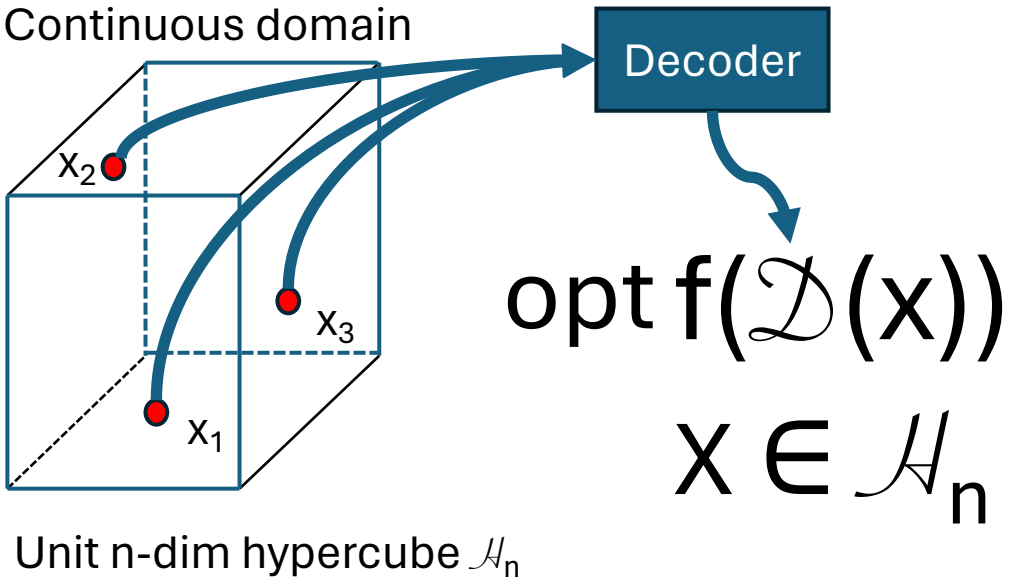


Unit n-dim hypercube $\mathcal{H}_n$

Continuous method for discrete optimization

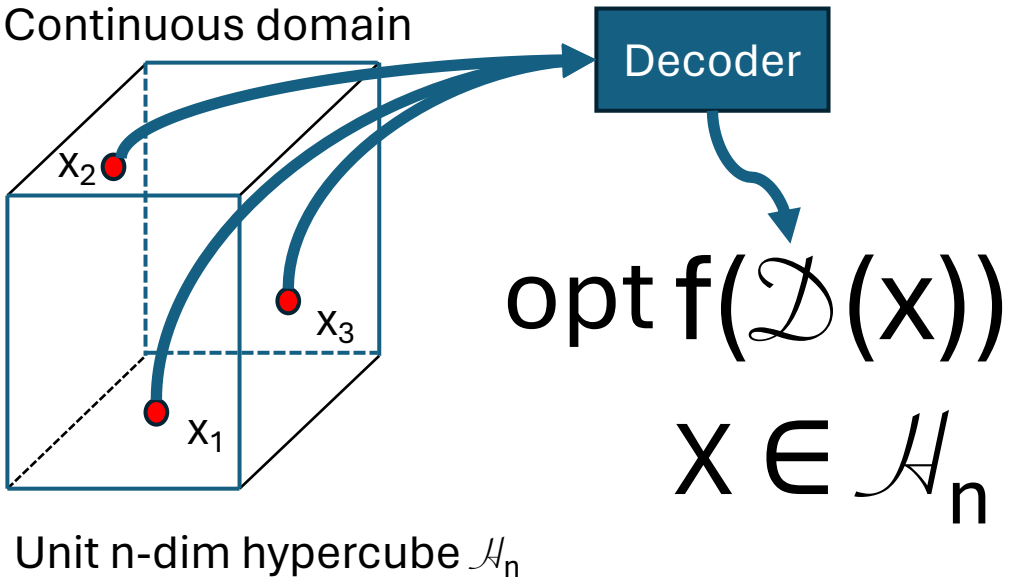
Random-key optimizers

- Random-Key Genetic Algorithms (Bean, 1994)



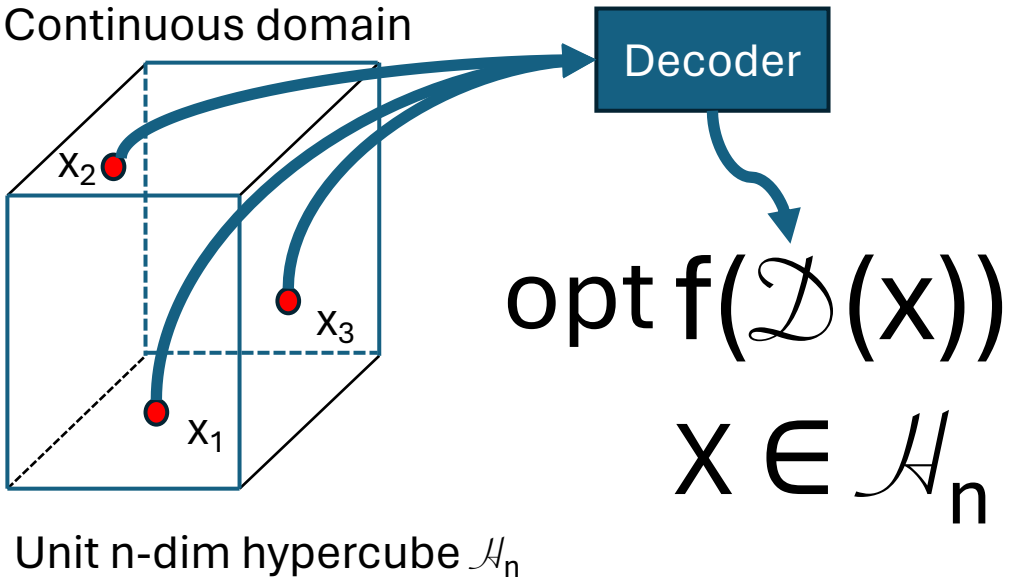
Random-key optimizers

- Random-Key Genetic Algorithms (Bean, 1994)
- Biased Random-Key Genetic Algorithms (Gonçalves & Resende, 2011)



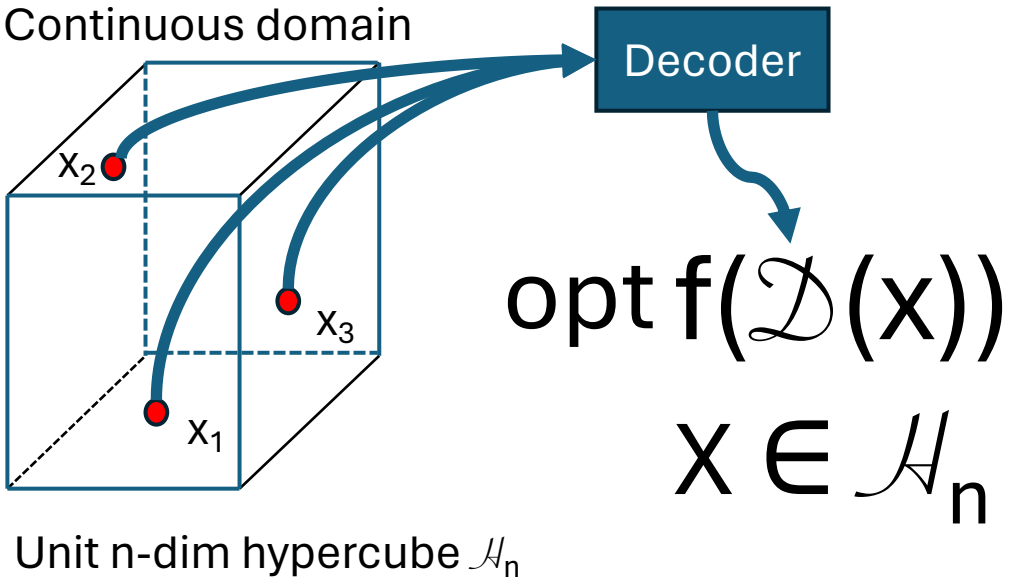
Random-key optimizers

- Random-Key Genetic Algorithms (Bean, 1994)
- Biased Random-Key Genetic Algorithms (Gonçalves & Resende, 2011)
- Dual Annealing RKO for robot motion planning at BMW (Schuetz et al., 2022)



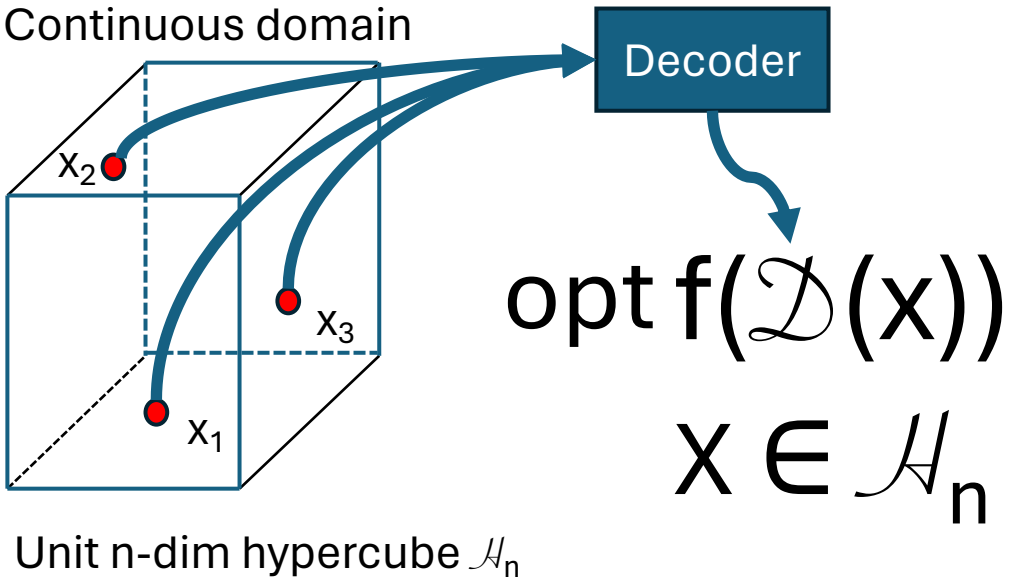
Random-key optimizers

- Random-Key Genetic Algorithms (Bean, 1994)
- Biased Random-Key Genetic Algorithms (Gonçalves & Resende, 2011)
- Dual Annealing RKO for robot motion planning at BMW (Schuetz et al., 2022)
- RKOs based on SA, ILS, and VNS for the tree hub location problem (Mangussi et al., 2023)



Random-key optimizers

- Random-Key Genetic Algorithms (Bean, 1994)
- Biased Random-Key Genetic Algorithms (Gonçalves & Resende, 2011)
- Dual Annealing RKO for robot motion planning at BMW (Schuetz et al., 2022)
- RKOs based on SA, ILS, and VNS for the tree hub location problem (Mangussi et al., 2023)
- C-GRASP RKO (This paper)



Genetic algorithms and random keys

GAs and random keys

- Introduced by Bean (1994) for sequencing problems.

GAs and random keys

- Introduced by Bean (1994) for sequencing problems.
- Individuals are strings of real-valued numbers (random keys) in the interval [0,1).

S = (0.25, 0.19, 0.67, 0.05, 0.89)
s(1) s(2) s(3) s(4) s(5)

GAs and random keys

- Introduced by Bean (1994) for sequencing problems.
- Individuals are strings of real-valued numbers (random keys) in the interval [0,1).
- Sorting random keys results in a sequencing order.

S = (0.25, 0.19, 0.67, 0.05, 0.89)
s(1) s(2) s(3) s(4) s(5)

S' = (0.05, 0.19, 0.25, 0.67, 0.89)
s(4) s(2) s(1) s(3) s(5)

Sequence: 4 – 2 – 1 – 3 – 5

GAs and random keys

- Mating is done using parametrized uniform crossover (Spears & DeJong , 1990)

a = (0.25, 0.19, 0.67, 0.05, 0.89)
b = (0.63, 0.90, 0.76, 0.93, 0.08)

GAs and random keys

- Mating is done using parametrized uniform crossover (Spears & DeJong , 1990)
- For each gene, flip a biased coin to choose which parent passes the allele (key, or value of gene) to the child.

a = (0.25, 0.19, 0.67, 0.05, 0.89)
b = (0.63, 0.90, 0.76, 0.93, 0.08)

GAs and random keys

- Mating is done using parametrized uniform crossover (Spears & DeJong , 1990)
- For each gene, flip a biased coin to choose which parent passes the allele (key, or value of gene) to the child.

a = (0.25, 0.19, 0.67, 0.05, 0.89)
b = (0.63, 0.90, 0.76, 0.93, 0.08)
c = ( )

GAs and random keys

- Mating is done using parametrized uniform crossover (Spears & DeJong , 1990)
- For each gene, flip a biased coin to choose which parent passes the allele (key, or value of gene) to the child.

a = (0.25, 0.19, 0.67, 0.05, 0.89)
b = (0.63, 0.90, 0.76, 0.93, 0.08)
c = (0.25)

GAs and random keys

- Mating is done using parametrized uniform crossover (Spears & DeJong , 1990)
- For each gene, flip a biased coin to choose which parent passes the allele (key, or value of gene) to the child.

a = (0.25, 0.19, 0.67, 0.05, 0.89)
b = (0.63, 0.90, 0.76, 0.93, 0.08)
c = (0.25, 0.90)

GAs and random keys

- Mating is done using parametrized uniform crossover (Spears & DeJong , 1990)
- For each gene, flip a biased coin to choose which parent passes the allele (key, or value of gene) to the child.

a = (0.25, 0.19, 0.67, 0.05, 0.89)
b = (0.63, 0.90, 0.76, 0.93, 0.08)
c = (0.25, 0.90, 0.76)

GAs and random keys

- Mating is done using parametrized uniform crossover (Spears & DeJong , 1990)
- For each gene, flip a biased coin to choose which parent passes the allele (key, or value of gene) to the child.

a = (0.25, 0.19, 0.67, 0.05, 0.89)
b = (0.63, 0.90, 0.76, 0.93, 0.08)
c = (0.25, 0.90, 0.76, 0.05)

GAs and random keys

- Mating is done using parametrized uniform crossover (Spears & DeJong , 1990)
- For each gene, flip a biased coin to choose which parent passes the allele (key, or value of gene) to the child.

a = (0.25, 0.19, 0.67, 0.05, 0.89)
b = (0.63, 0.90, 0.76, 0.93, 0.08)
c = (0.25, 0.90, 0.76, 0.05, 0.89)

GAs and random keys

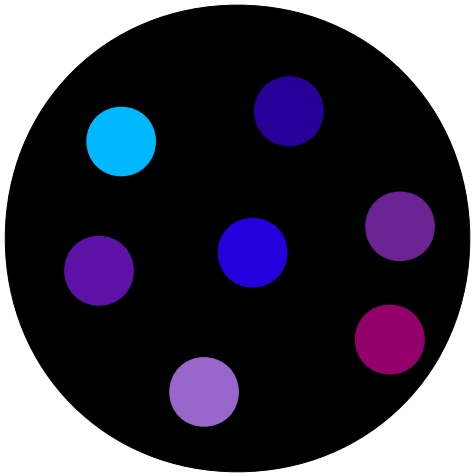
- Mating is done using parametrized uniform crossover (Spears & DeJong , 1990)
- For each gene, flip a biased coin to choose which parent passes the allele (key, or value of gene) to the child.

a = (0.25, 0.19, 0.67, 0.05, 0.89)
b = (0.63, 0.90, 0.76, 0.93, 0.08)
c = (0.25, 0.90, 0.76, 0.05, 0.89)

If every random-key array corresponds to a feasible solution: Mating always produces feasible offspring.

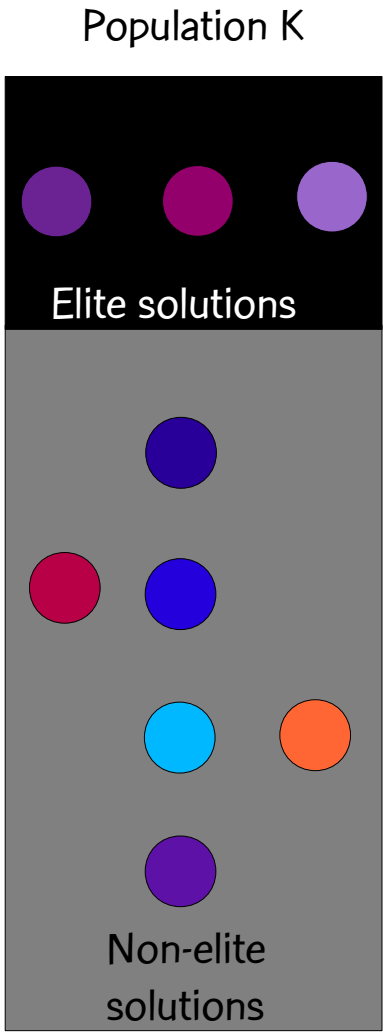
GAs and random keys

Initial population is made up of P random-key vectors, each with N keys, each having a value generated uniformly at random in the interval [0,1).



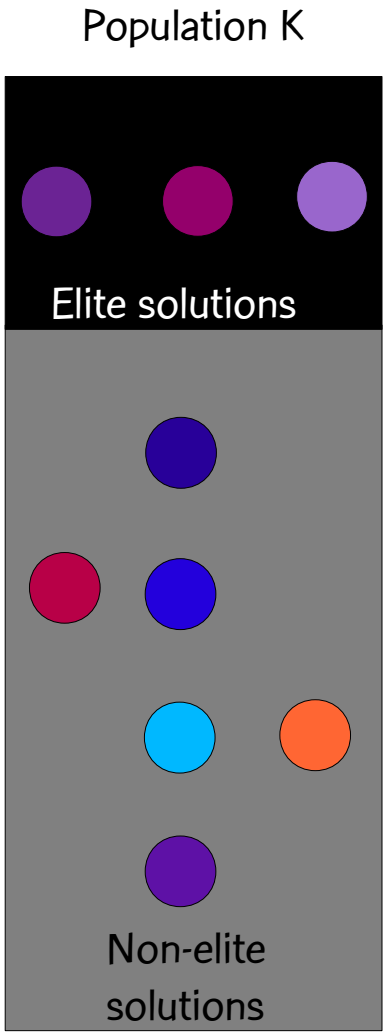
GAs and random keys

At the K-th generation, compute the cost of each solution ...



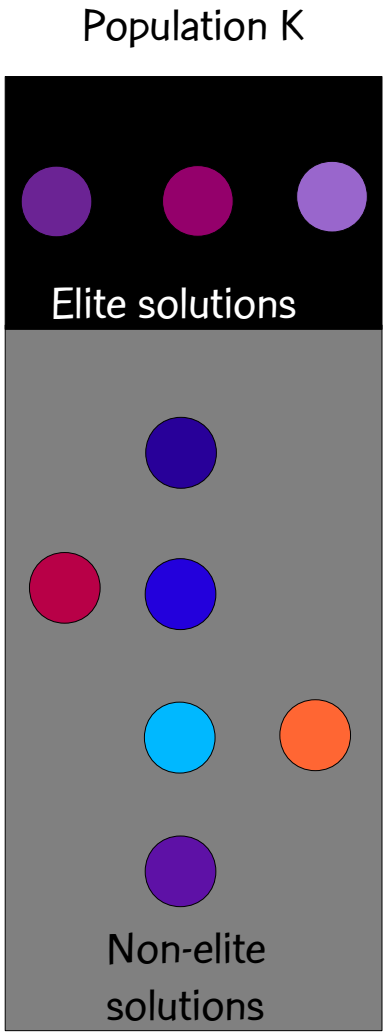
GAs and random keys

At the K-th generation, compute the cost of each solution and partition the solutions into two sets:



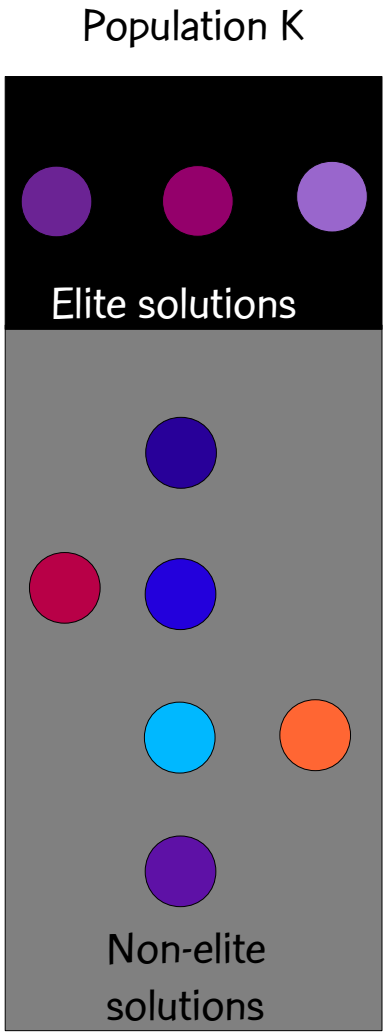
GAs and random keys

At the K-th generation, compute the cost of each solution and partition the solutions into two sets: elite solutions and non-elite solutions.



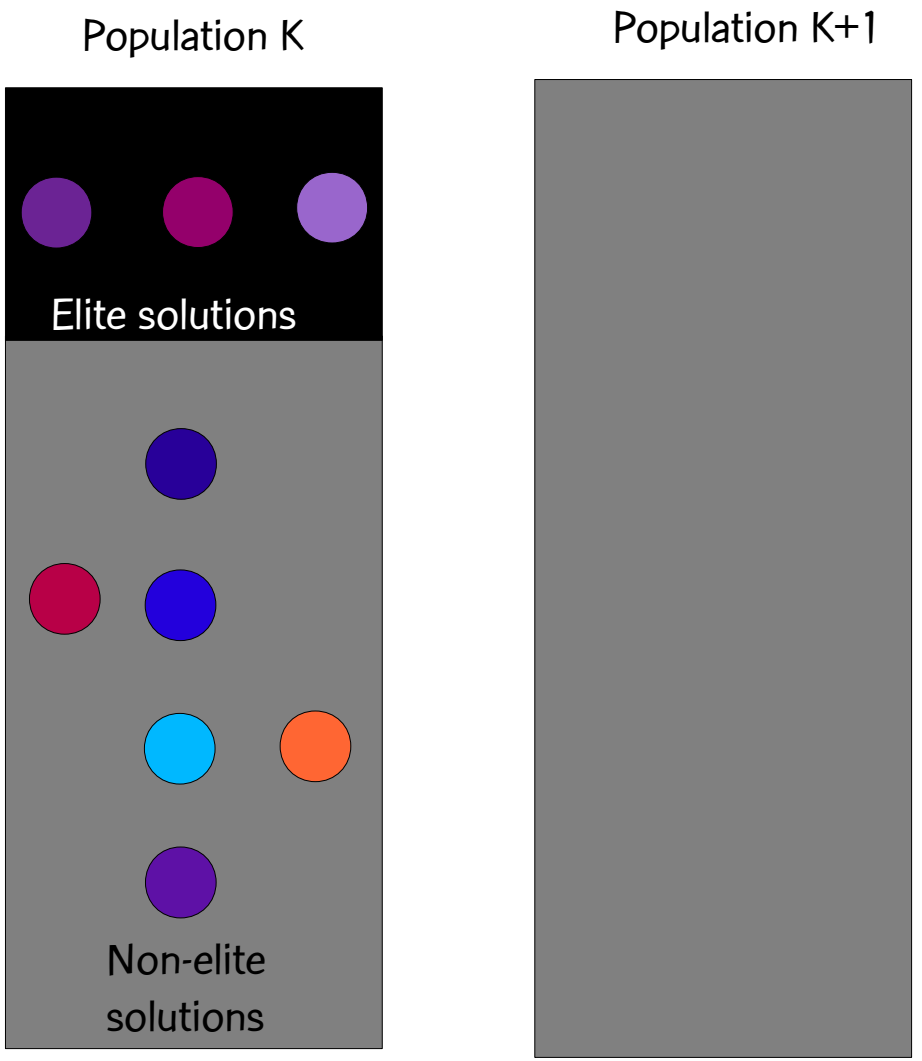
GAs and random keys

At the K-th generation, compute the cost of each solution and partition the solutions into two sets: elite solutions and non-elite solutions. Elite set should be smaller of the two sets and contain best solutions.



GAs and random keys

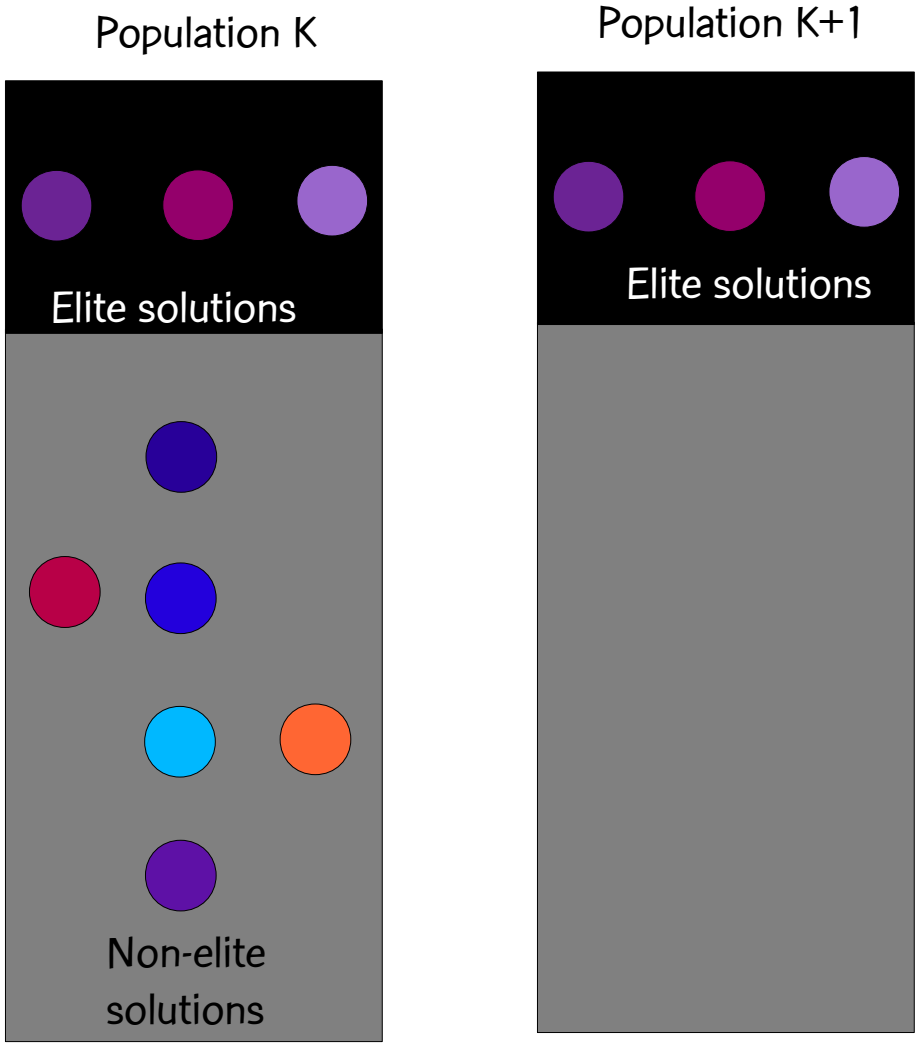
Evolutionary dynamics



GAs and random keys

Evolutionary dynamics

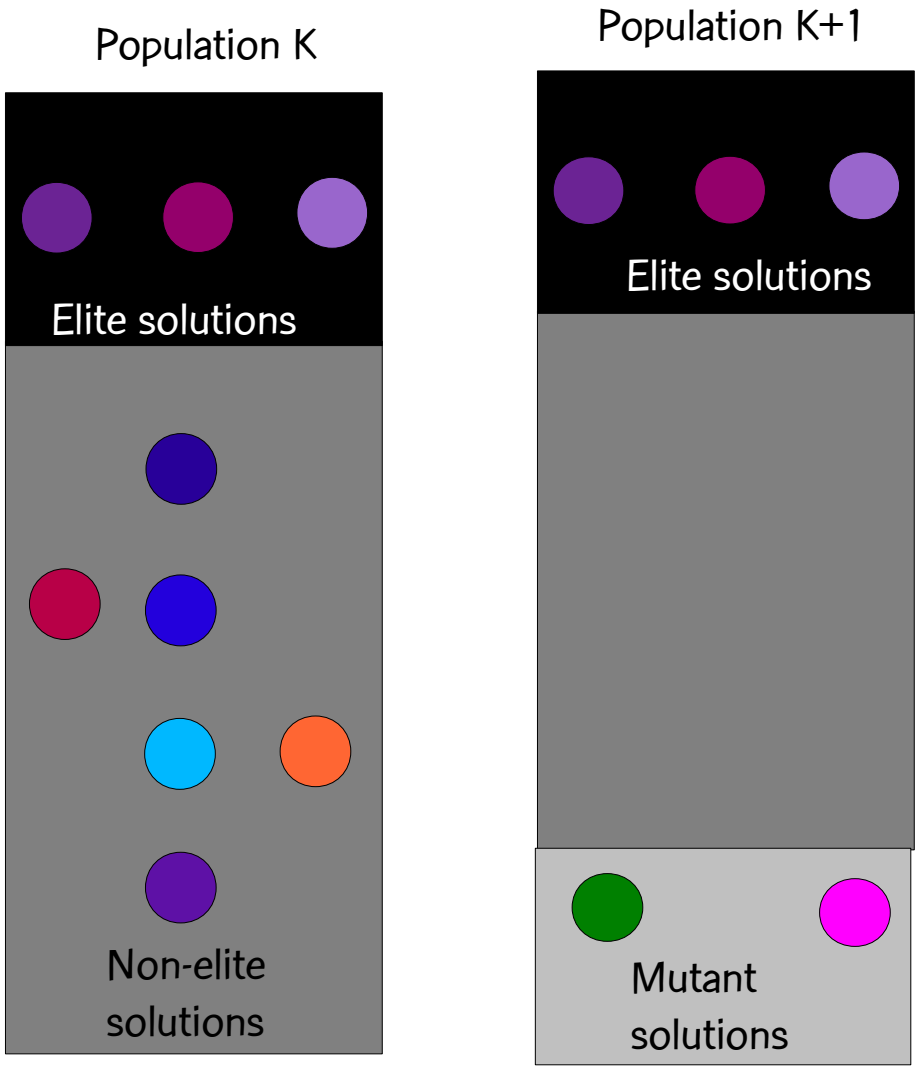
– Copy elite solutions from population K to population K+1



GAs and random keys

Evolutionary dynamics

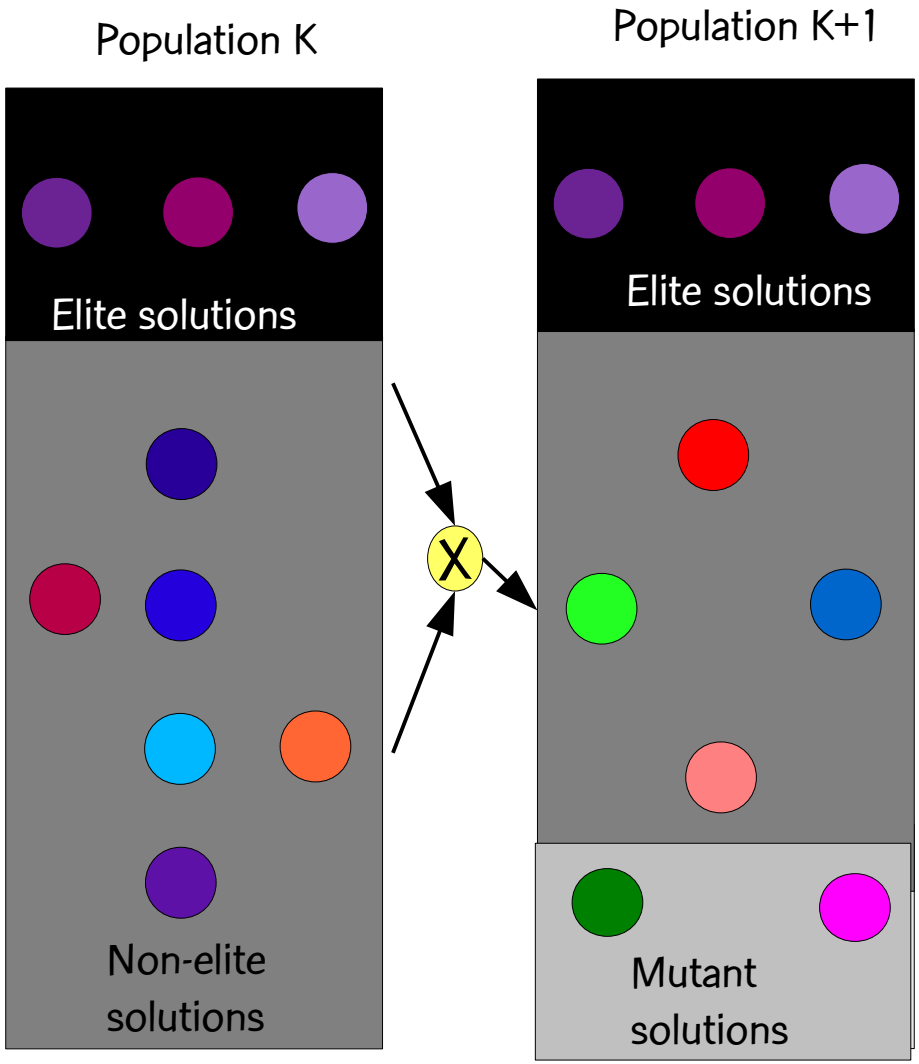
- Copy elite solutions from population K to population K+1
- Add R random solutions (mutants) to population K+1



GAs and random keys

Evolutionary dynamics

- Copy elite solutions from population K to population K+1
- Add R random solutions (mutants) to population K+1
- While K+1-th population < P
  - RANDOM-KEY GA: Use any two solutions in population K to produce child in population K+1. Mates are chosen at random.



Biased random key genetic algorithm

- A biased random key genetic algorithm (BRKGA) is a random key genetic algorithm (RKGA).

Biased random key genetic algorithm

- A biased random key genetic algorithm (BRKGA) is a random key genetic algorithm (RKGA).
- BRKGA and RKGA differ in how mates are chosen for crossover and how parametrized uniform crossover is applied.

How RKGA & BRKGA differ

RKGA

both parents chosen at random from entire population

BRKGA

How RKGA & BRKGA differ

RKGA

both parents chosen at random from entire population

BRKGA

both parents chosen at random but one parent chosen from population of elite solutions

How RKGA & BRKGA differ

RKGA

both parents chosen at random from entire population

either parent can be parent A in parametrized uniform crossover

BRKGA

both parents chosen at random but one parent chosen from population of elite solutions

How RKGA & BRKGA differ

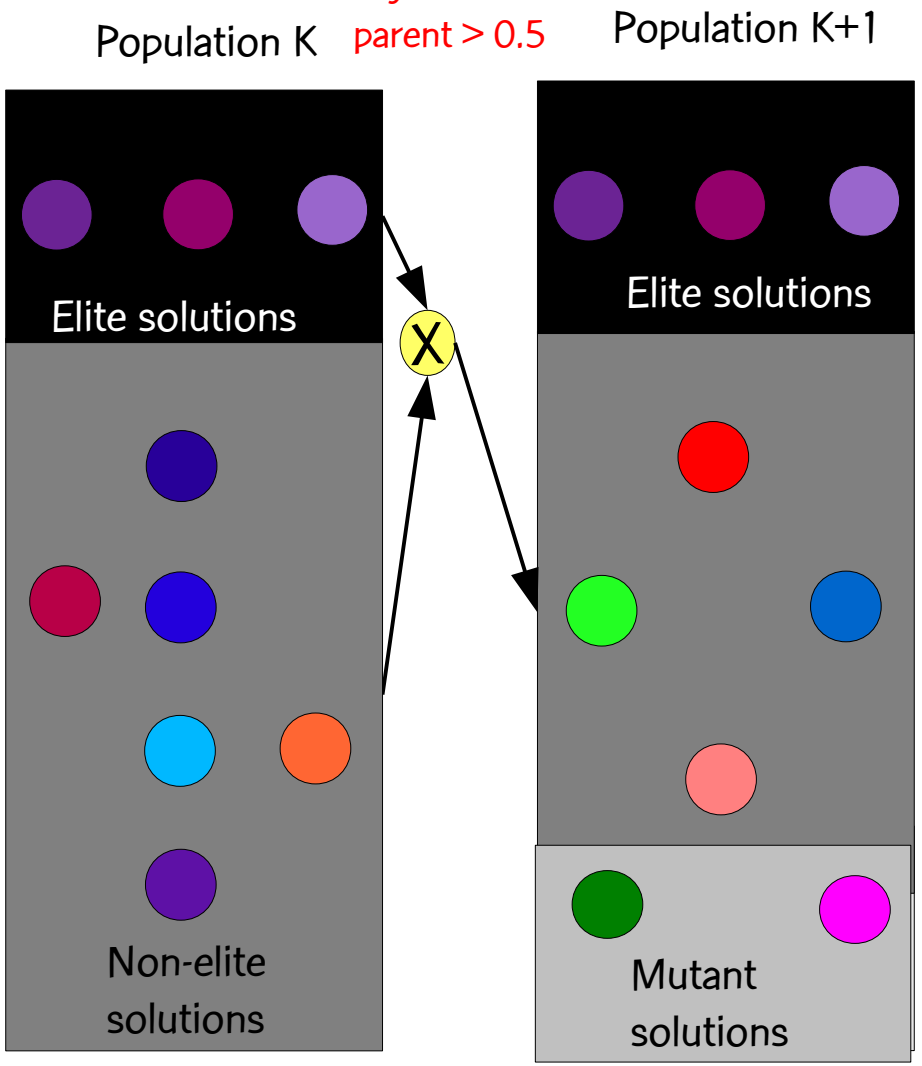
RKGA	BRKGA
both parents chosen at random from entire population	both parents chosen at random but one parent chosen from population of elite solutions
either parent can be parent A in parametrized uniform crossover	best fit parent is parent A in parametrized uniform crossover

Biased random key GA

BRKGA: Probability child inherits key of elite parent > 0.5

Evolutionary dynamics

- Copy elite solutions from population K to population K+1
- Add R random solutions (mutants) to population K+1
- While K+1-th population < P
  - RANDOM-KEY GA: Use any two solutions in population K to produce child in population K+1. Mates are chosen at random.
  - BIASED RANDOM-KEY GA: Mate elite solution with other solution of population K to produce child in population K+1. Mates are chosen at random.



Observations

- Random method: keys are randomly generated so solutions are always vectors of random keys

Observations

- Random method: keys are randomly generated so solutions are always vectors of random keys
- Elitist strategy: best solutions are passed without change from one generation to the next (incumbent is kept)

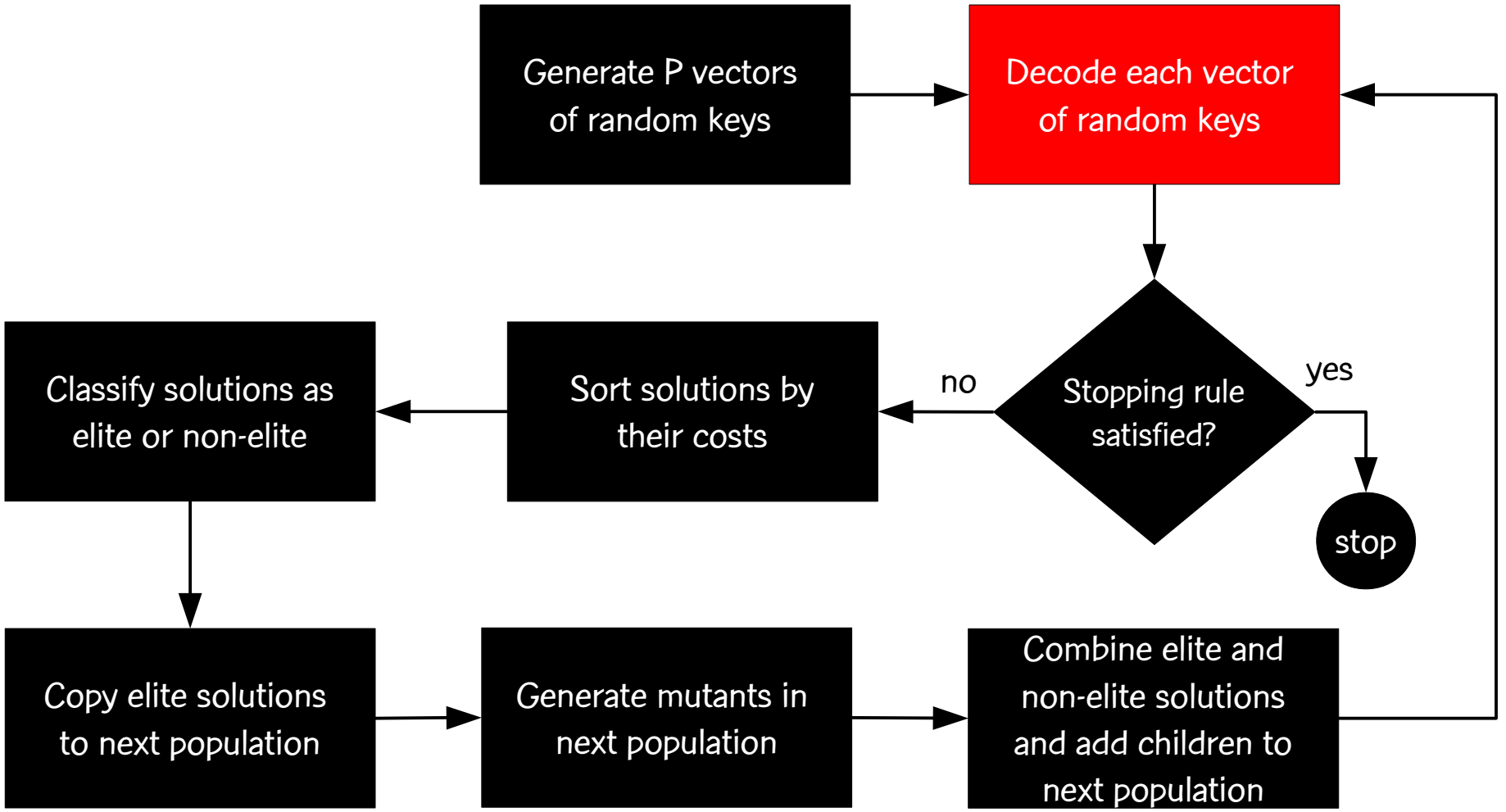
Observations

- Random method: keys are randomly generated so solutions are always vectors of random keys
- Elitist strategy: best solutions are passed without change from one generation to the next (incumbent is kept)
- Child inherits more characteristics of elite parent: one parent is always selected (with replacement) from the small elite set and probability that child inherits key of elite parent > 0.5 Not so in the RKGA of Bean.

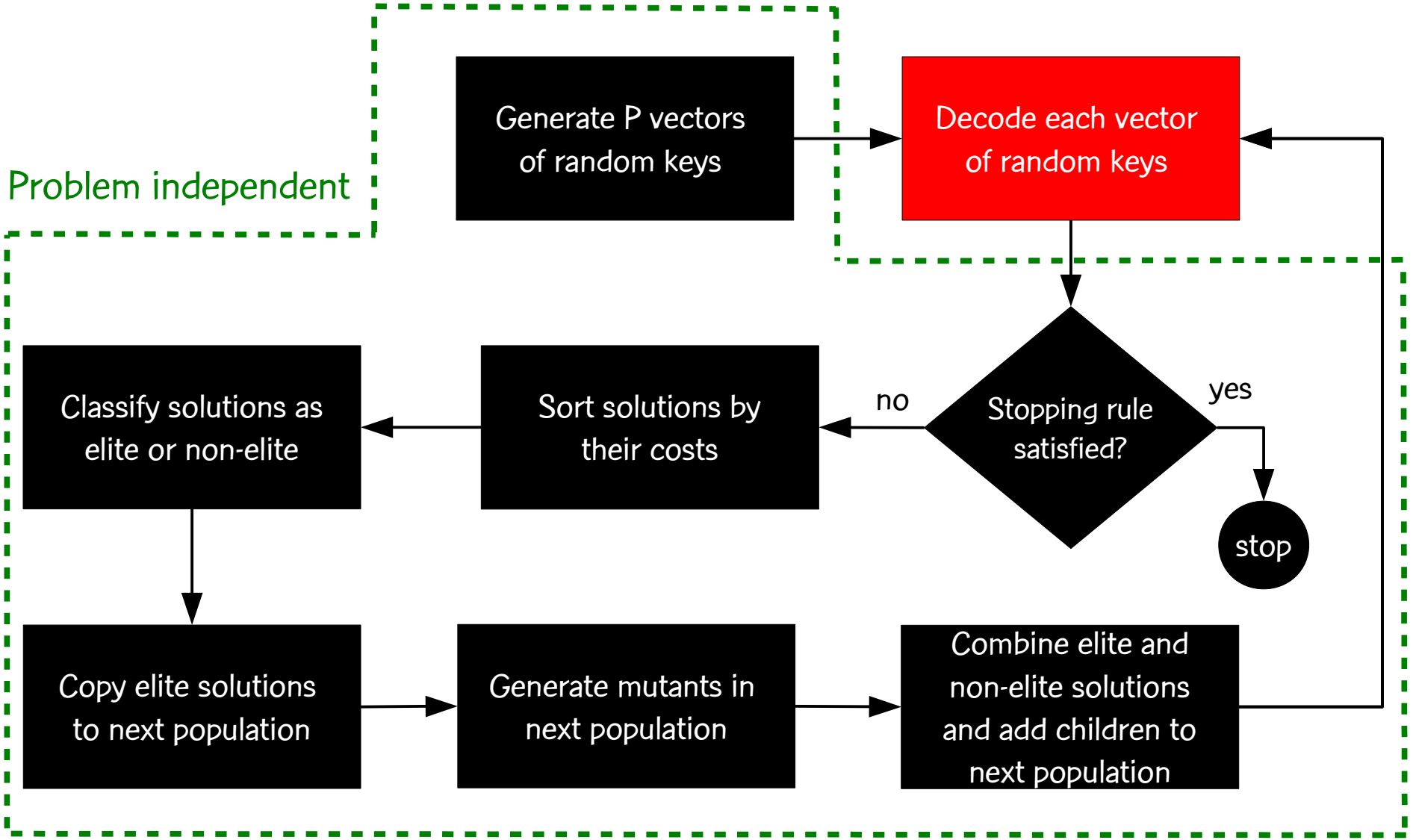
Observations

- Random method: keys are randomly generated so solutions are always vectors of random keys
- Elitist strategy: best solutions are passed without change from one generation to the next (incumbent is kept)
- Child inherits more characteristics of elite parent: one parent is always selected (with replacement) from the small elite set and probability that child inherits key of elite parent > 0.5 Not so in the RKGA of Bean.
- No mutation in crossover: mutants are used instead (they play same role as mutation in GAs ... help escape local optima)

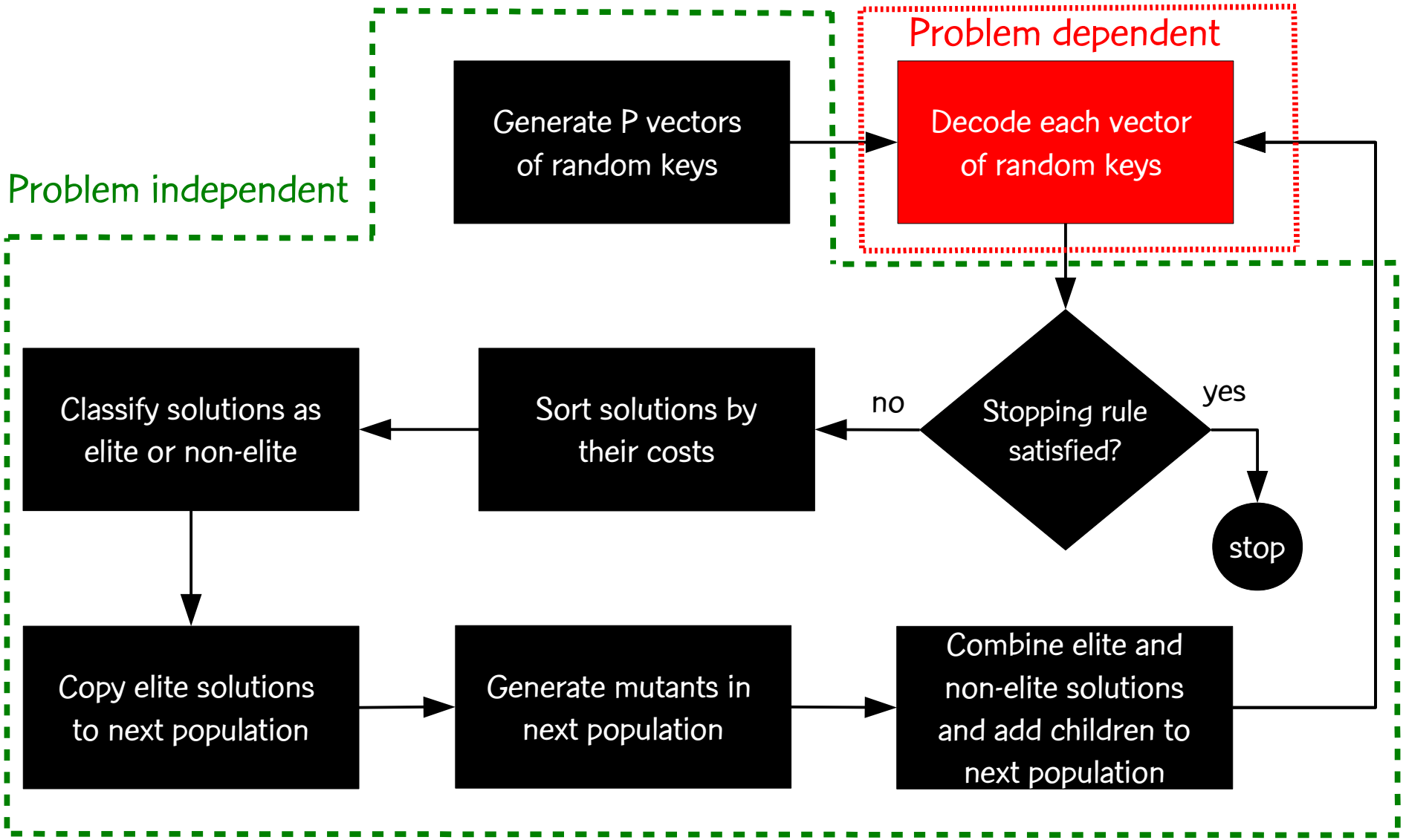
Framework for biased random-key genetic algorithms



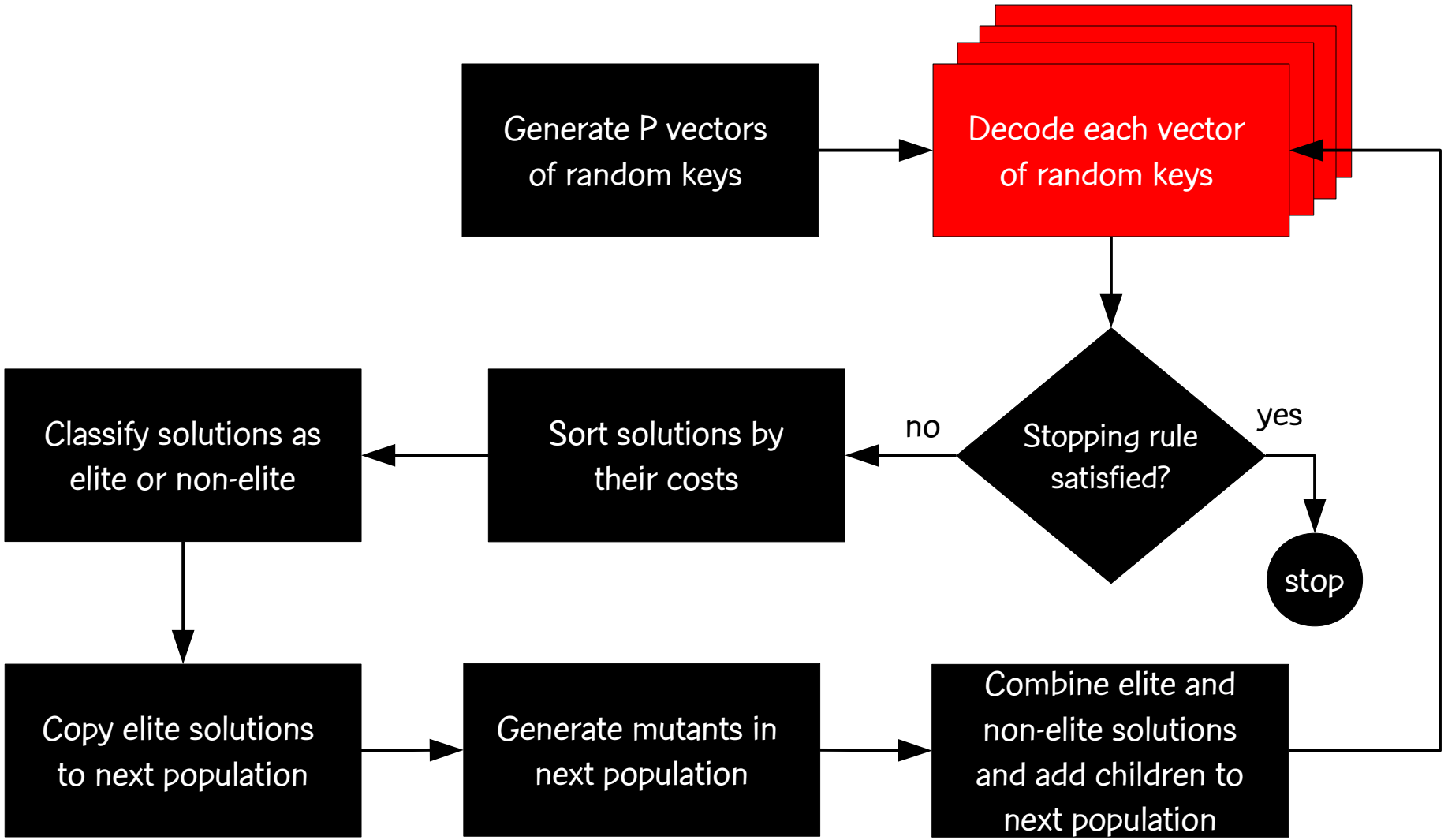
Framework for biased random-key genetic algorithms



Framework for biased random-key genetic algorithms



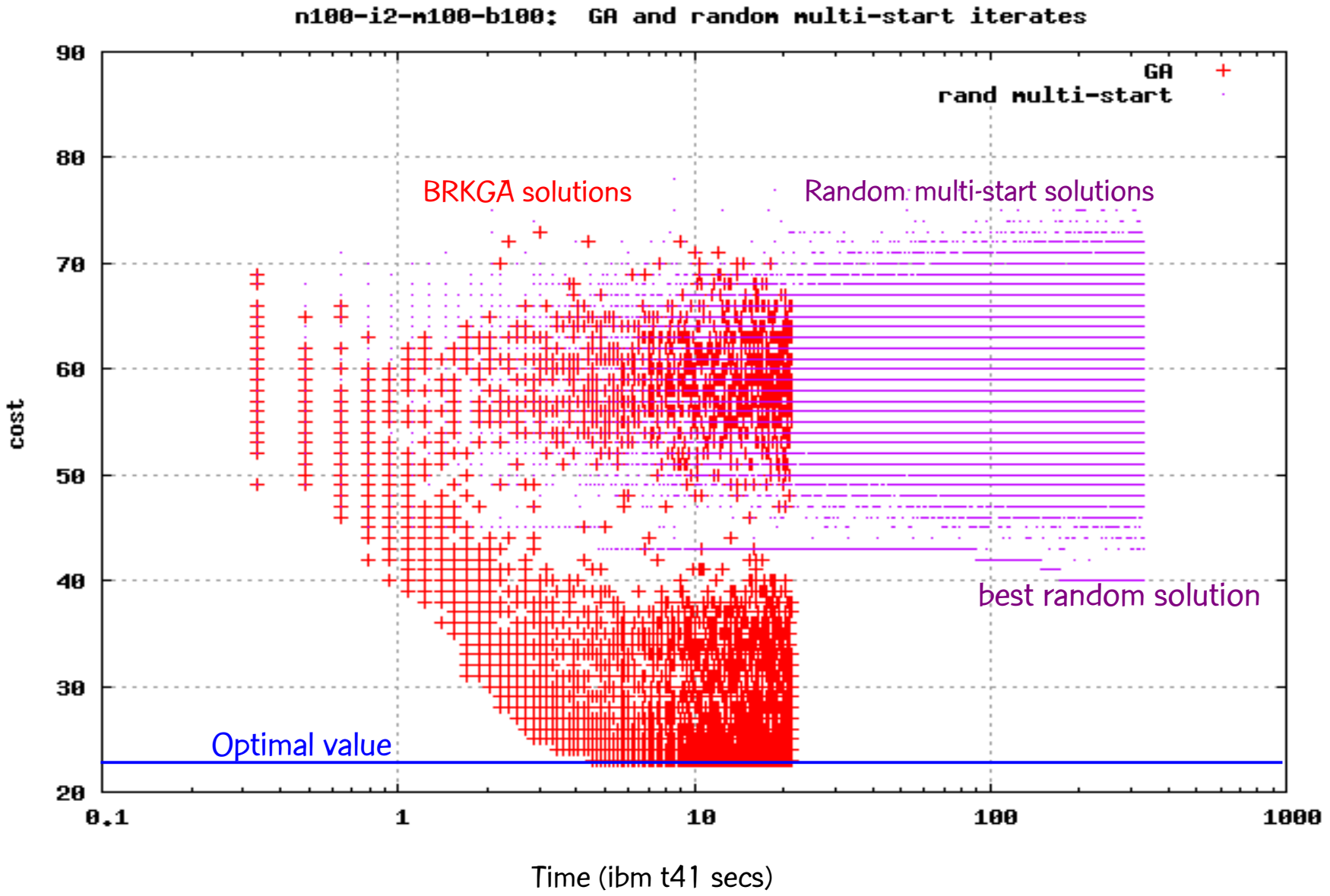
Decoding of random key vectors can be done in parallel



Is a BRKGA any different from applying the decoder to random keys?

- Simulate a random multi-start decoding method with a BRKGA by setting size of elite partition to 1 and number of mutants to P–1
- Each iteration, best solution is maintained in elite set and P–1 random key vectors are generated as mutants ... no mating is done since population already has P individuals

solution


n100-i2-m100-b100: GA and random multi-start iterates
GA
rand multi-start
BRKGA solutions
Random multi-start solutions
best random solution
Optimal value
cost
Time (ibm t41 secs)
90
80
70
60
50
40
30
20
0.1
1
10
100
1000

Specifying a BRKGA

Specifying a biased random-key GA

- Encoding is always done the same way, i.e. with a vector of N random-keys (parameter N must be specified)

Specifying a biased random-key GA

- Encoding is always done the same way, i.e. with a vector of N random-keys (parameter N must be specified)
- Decoder that takes as input a vector of N random-keys and outputs the corresponding solution of the combinatorial optimization problem and its cost (this is usually a heuristic)

Specifying a biased random-key GA

- Encoding is always done the same way, i.e. with a vector of N random-keys (parameter N must be specified)
- Decoder that takes as input a vector of N random-keys and outputs the corresponding solution of the combinatorial optimization problem and its cost (this is usually a heuristic)
- Parameters

Specifying a biased random-key GA

Parameters:

- Size of population
- Size of elite partition
- Size of mutant set
- Child inheritance probability
- Restart strategy parameter
- Stopping criterion

Specifying a biased random-key GA

Parameters:

- Size of population: a function of N, say N or 2N
- Size of elite partition
- Size of mutant set
- Child inheritance probability
- Restart strategy parameter
- Stopping criterion

Specifying a biased random-key GA

Parameters:

- Size of population: a function of N, say N or 2N
- Size of elite partition: 15-25% of population
- Size of mutant set
- Child inheritance probability
- Restart strategy parameter
- Stopping criterion

Specifying a biased random-key GA

Parameters:

- Size of population: a function of N, say N or 2N
- Size of elite partition: 15-25% of population
- Size of mutant set: 5-15% of population
- Child inheritance probability
- Restart strategy parameter
- Stopping criterion

Specifying a biased random-key GA

Parameters:

- Size of population: a function of N, say N or 2N
- Size of elite partition: 15-25% of population
- Size of mutant set: 5-15% of population
- Child inheritance probability: > 0.5, say 0.7
- Restart strategy parameter
- Stopping criterion

Specifying a biased random-key GA

Parameters:

- Size of population: a function of N, say N or 2N
- Size of elite partition: 15-25% of population
- Size of mutant set: 5-15% of population
- Child inheritance probability: > 0.5, say 0.7
- Restart strategy parameter: a function of N, say 2N or 10N
- Stopping criterion

Specifying a biased random-key GA

Parameters:

- Size of population: a function of N, say N or 2N
- Size of elite partition: 15-25% of population
- Size of mutant set: 5-15% of population
- Child inheritance probability: > 0.5, say 0.7
- Restart strategy parameter: a function of N, say 2N or 10N
- Stopping criterion: e.g. time, # generations, solution quality, # generations without improvement

brkgaAPI: A C++ API for BRKGA

- Efficient and easy-to-use object oriented application programming interface (API) for the algorithmic framework of BRKGA.

brkgaAPI: A C++ API for BRKGA

- Efficient and easy-to-use object oriented application programming interface (API) for the algorithmic framework of BRKGA.
- Cross-platform library handles large portion of problem independent modules that make up the framework, e.g.
  - population management
  - evolutionary dynamics

brkgaAPI: A C++ API for BRKGA

- Efficient and easy-to-use object oriented application programming interface (API) for the algorithmic framework of BRKGA.
- Cross-platform library handles large portion of problem independent modules that make up the framework, e.g.
  - population management
  - evolutionary dynamics
- Implemented in C++ and may benefit from shared-memory parallelism if available.

brkgaAPI: A C++ API for BRKGA

- Efficient and easy-to-use object oriented application programming interface (API) for the algorithmic framework of BRKGA.
- Cross-platform library handles large portion of problem independent modules that make up the framework, e.g.
  - population management
  - evolutionary dynamics
- Implemented in C++ and may benefit from shared-memory parallelism if available.
- User only needs to implement problem-dependent decoder.

brkgaAPI: A C++ API for BRKGA

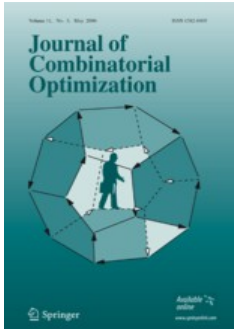


Paper: Rodrigo F. Toso and M.G.C.R.,
"A C++ Application Programming Interface for Biased Random-Key Genetic Algorithms,"
Optimization Methods & Software, vol. 30, pp. 81-93, 2015.

Software: http://mauricio.resende.info/src/brkgaAPI

An example BRKGA: Packing weighted rectangles

Reference



J.F. Gonçalves and R., "A parallel multi-population genetic algorithm for a constrained two-dimensional orthogonal packing problem," Journal of Combinatorial Optimization, vol. 22, pp. 180-201, 2011.

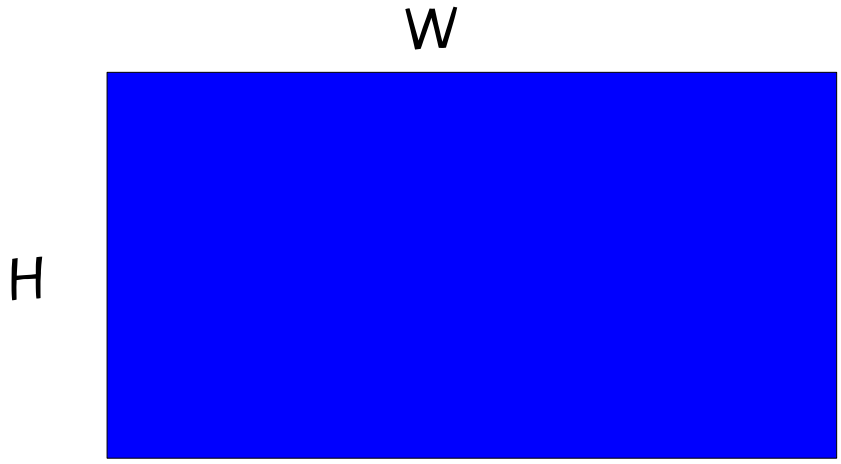
Tech report:
http://mauricio.resende.info/doc/pack2d.pdf

Constrained orthogonal packing

- Given a large planar stock rectangle (W, H) of width W and height H;

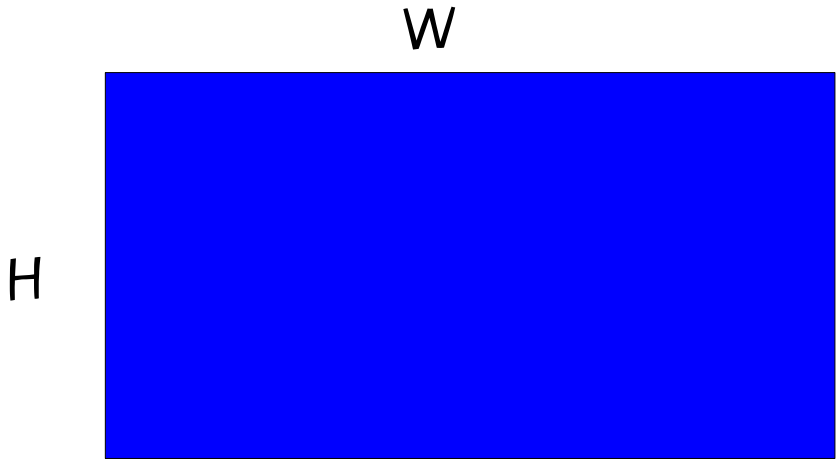
Constrained orthogonal packing

- Given a large planar stock rectangle (W, H) of width W and height H;



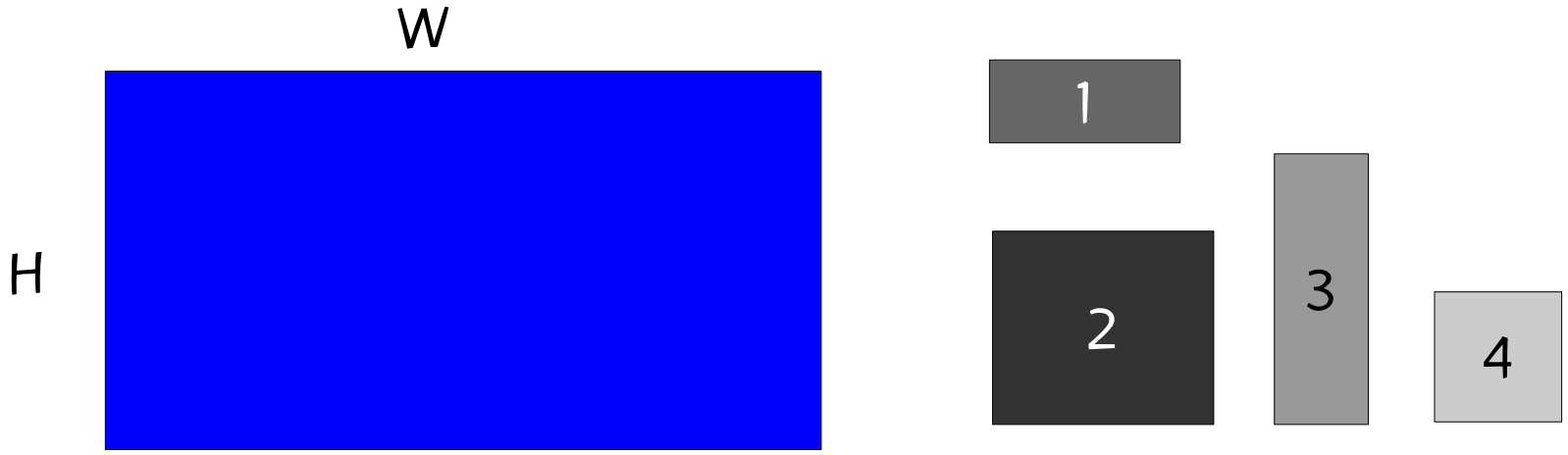
Constrained orthogonal packing

- Given a large planar stock rectangle (W, H) of width W and height H;
- Given N smaller rectangle types (w[i], h[i]), i = 1,...,N, each of width w[i], height h[i], and value v[i];



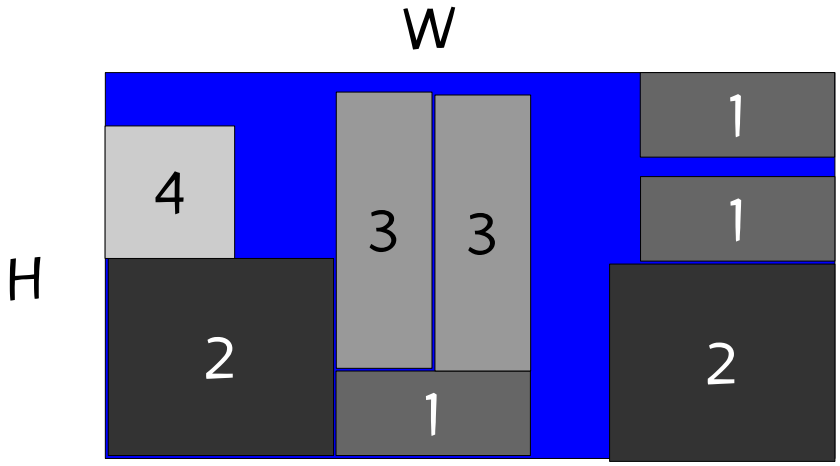
Constrained orthogonal packing

- Given a large planar stock rectangle (W, H) of width W and height H;
- Given N smaller rectangle types (w[i], h[i]), i = 1,...,N, each of width w[i], height h[i], and value v[i];



Constrained orthogonal packing

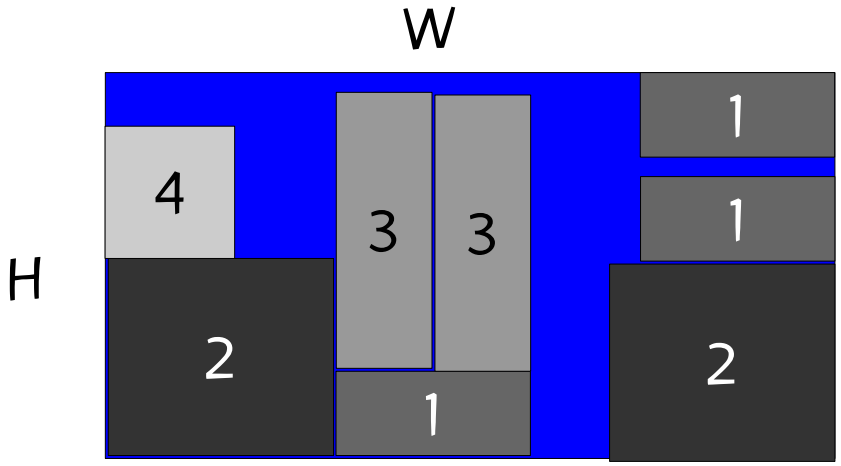
- r[i] rectangles of type i = 1, ..., N are to be packed in the large rectangle without overlap and such that their edges are parallel to the edges of the large rectangle;



Constrained orthogonal packing

- r[i] rectangles of type i = 1, ..., N are to be packed in the large rectangle without overlap and such that their edges are parallel to the edges of the large rectangle;
- For i = 1, ..., N, we require that:

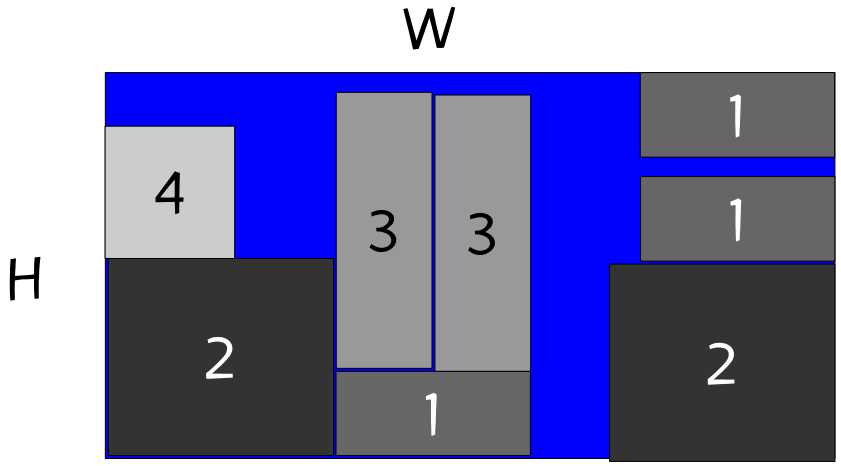
$$0 \leq P[i] \leq r[i] \leq Q[i]$$



Constrained orthogonal packing

- r[i] rectangles of type i = 1, ..., N are to be packed in the large rectangle without overlap and such that their edges are parallel to the edges of the large rectangle;
- For i = 1, ..., N, we require that:

$$0 \leq P[i] \leq r[i] \leq Q[i]$$

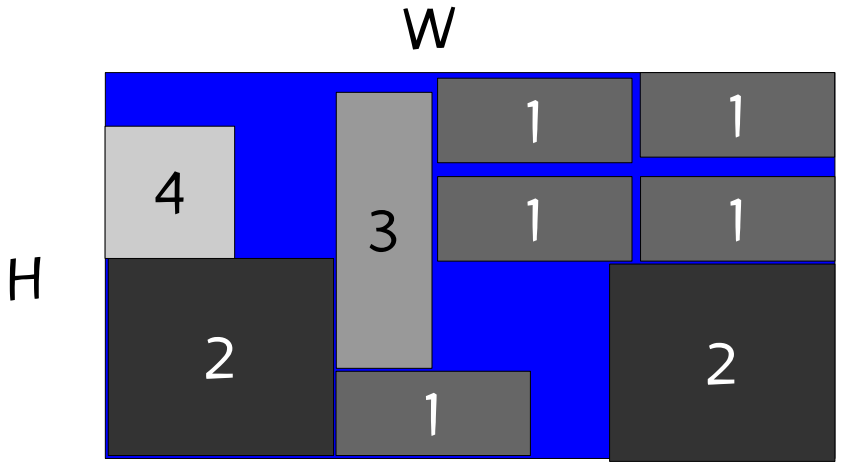


Suppose $5 \leq r[1] \leq 12$

Constrained orthogonal packing

- r[i] rectangles of type i = 1, ..., N are to be packed in the large rectangle without overlap and such that their edges are parallel to the edges of the large rectangle;
- For i = 1, ..., N, we require that:

$$0 \leq P[i] \leq r[i] \leq Q[i]$$

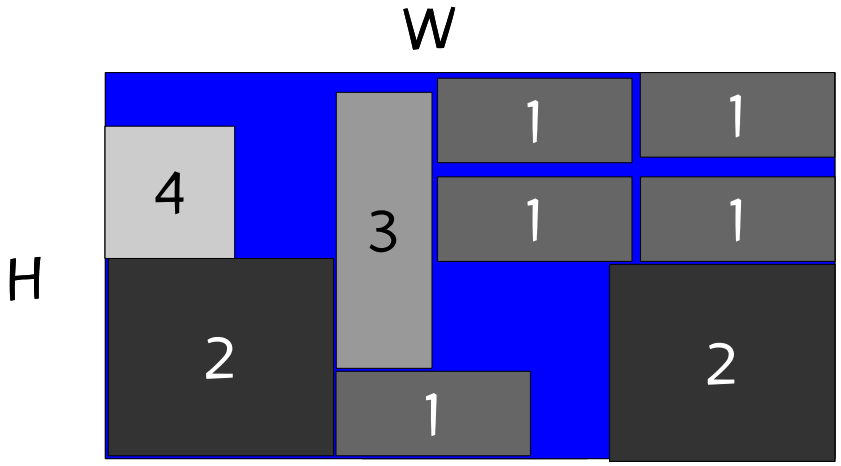


Suppose $5 \leq r[1] \leq 12$

Objective

Among the many feasible packings, we want to find one that maximizes total value of packed rectangles:

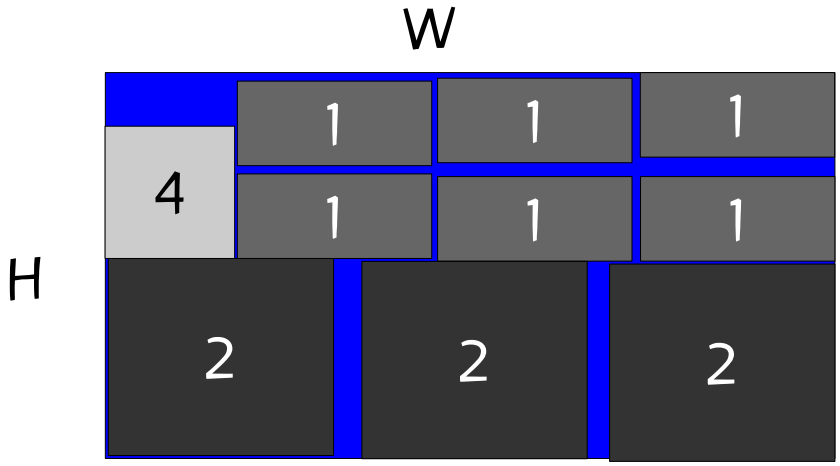
$$v[1]\, r[1] + v[2]\, r[2] + \cdots + v[N]\, r[N]$$



Objective

Among the many feasible packings, we want to find one that maximizes total value of packed rectangles:

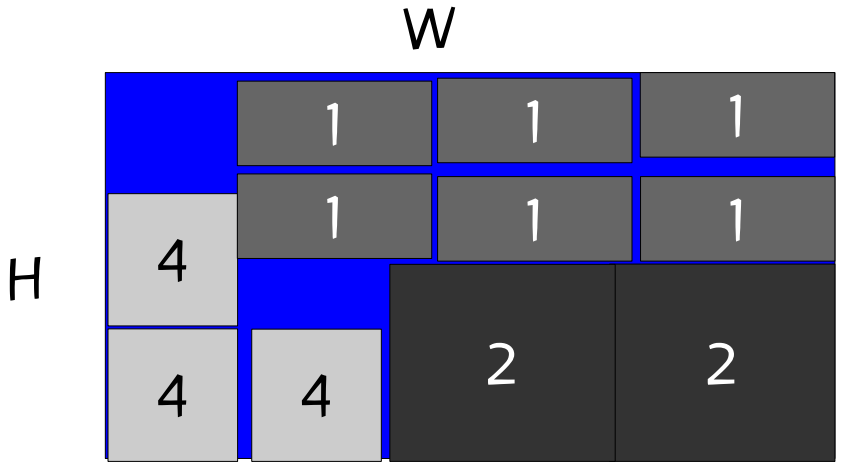
$$v[1]\,r[1] + v[2]\,r[2] + \cdots + v[N]\,r[N]$$



Objective

Among the many feasible packings, we want to find one that maximizes total value of packed rectangles:

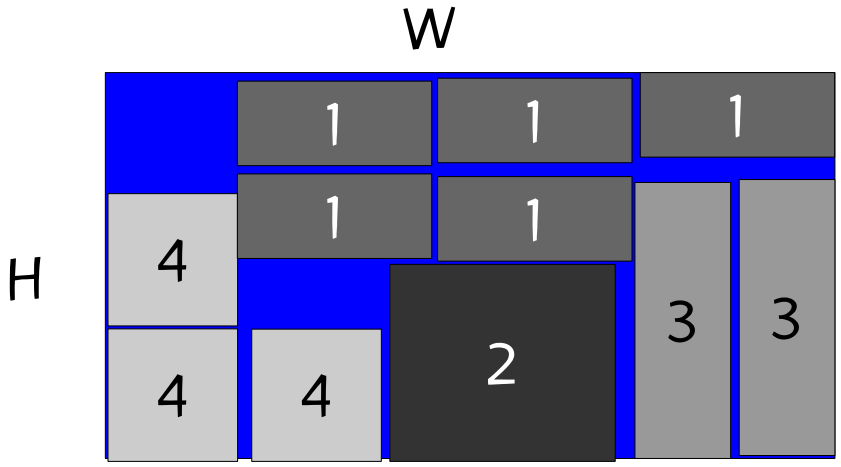
$$v[1]\,r[1] + v[2]\,r[2] + \cdots + v[N]\,r[N]$$



Objective

Among the many feasible packings, we want to find one that maximizes total value of packed rectangles:

$$v[1]\,r[1] + v[2]\,r[2] + \cdots + v[N]\,r[N]$$

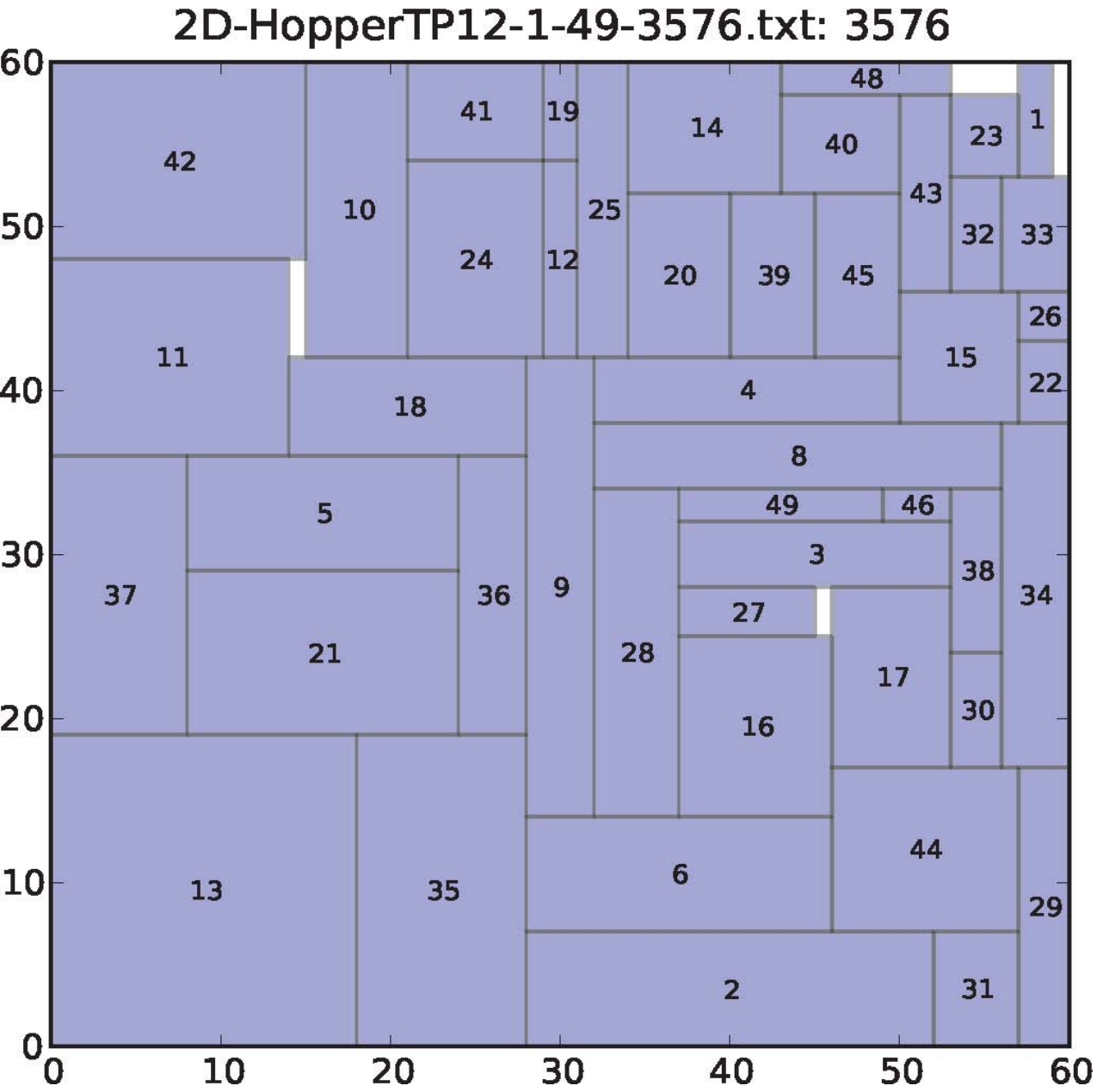


Applications

Problem arises in several production processes, e.g.

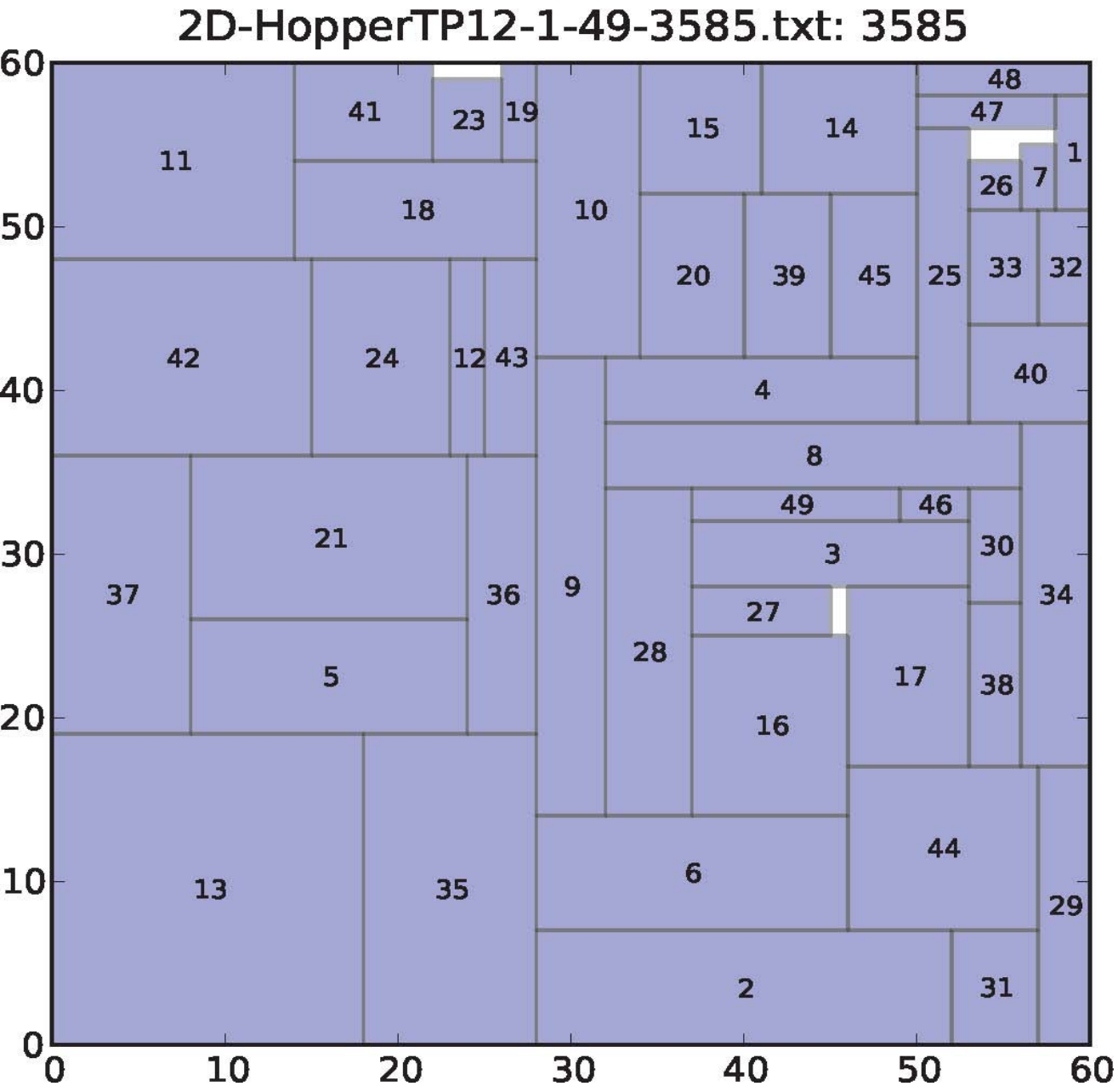
- Textile
- Glass
- Wood
- Paper

where rectangular figures are cut from large rectangular sheets of materials.



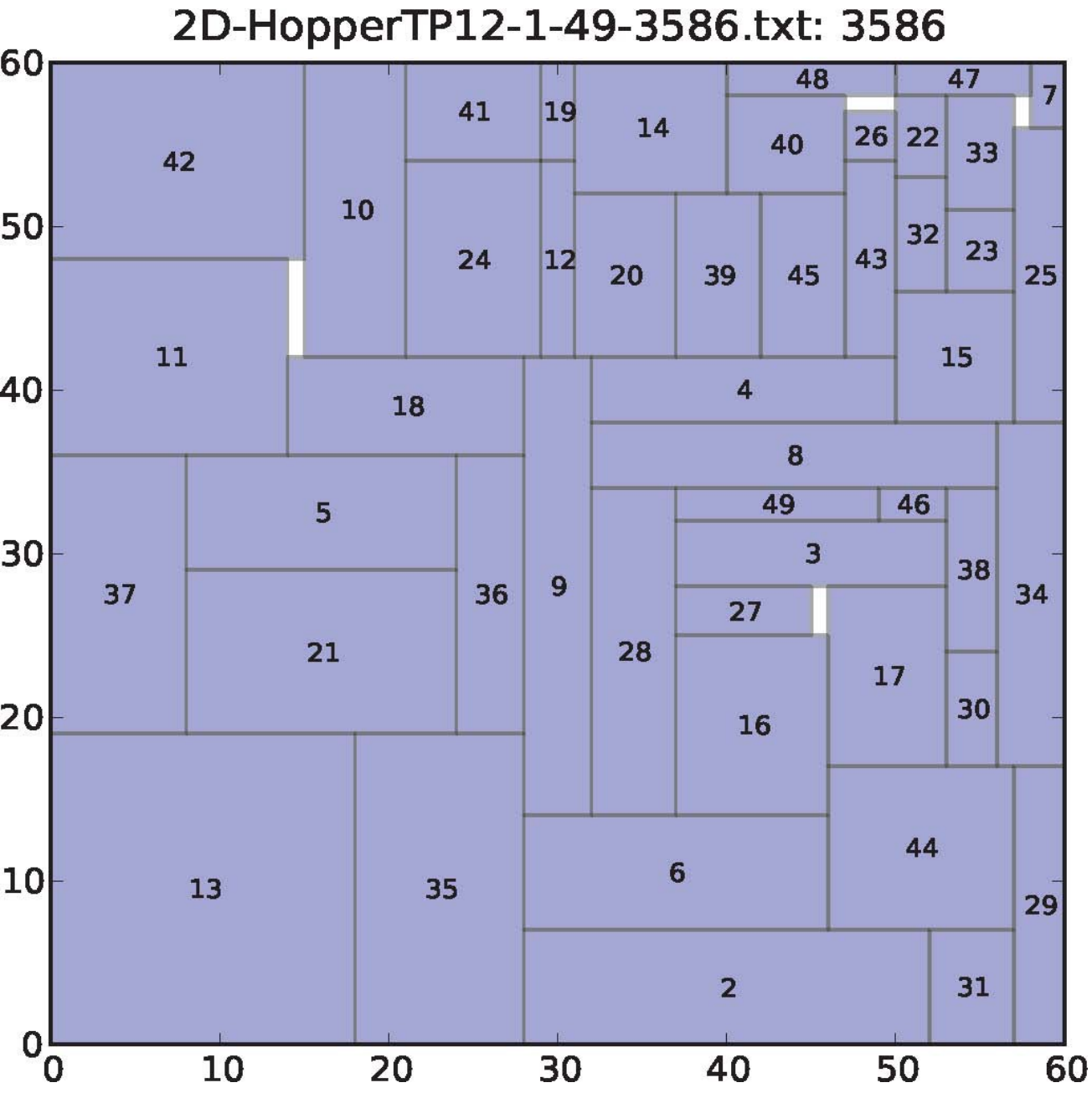
Hopper & Turton, 2001
Instance 4-1 60 x 60
Value: 3576

Previous best: 3580 by a Tabu Search heuristic (Alvarez-Valdes et al., 2007)



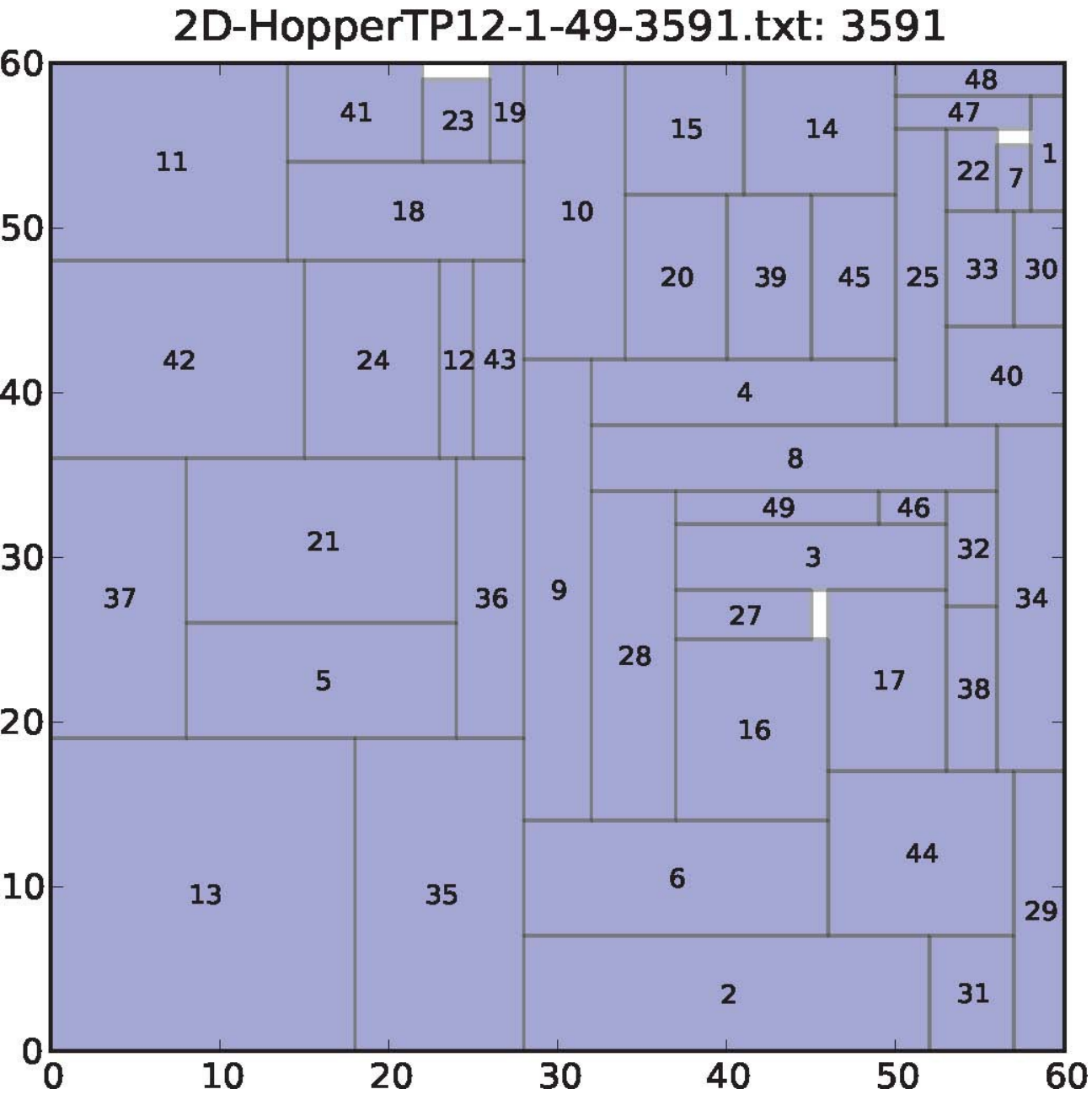
Hopper & Turton, 2001
Instance 4-2 60 x 60
Value: 3585

Previous best: 3580 by a Tabu Search heuristic (Alvarez-Valdes et al., 2007)



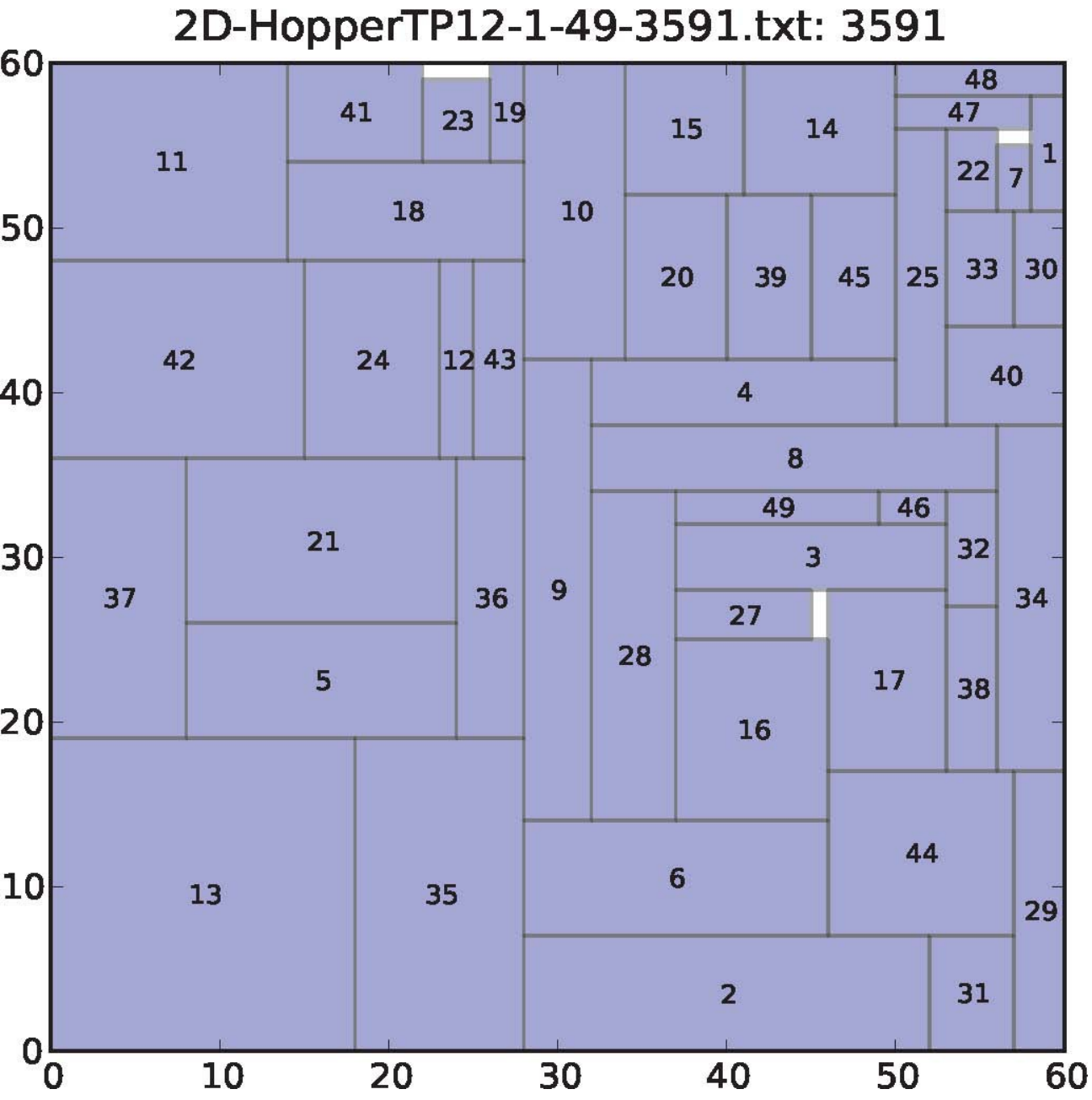
Hopper & Turton, 2001
Instance 4-2 60 x 60
Value: 3586

Previous best: 3580 by a Tabu Search heuristic (Alvarez-Valdes et al., 2007)



Hopper & Turton, 2001
Instance 4-2 60 x 60
Value: 3591

Previous best: 3580 by a Tabu Search heuristic (Alvarez-Valdes et al., 2007)



Hopper & Turton, 2001
Instance 4-2 60 x 60
Value: 3591
New best known solution!
Previous best: 3580 by a Tabu Search heuristic (Alvarez-Valdes et al., 2007)

BRKGA for constrained 2-dim orthogonal packing

Encoding

- Solutions are encoded as vectors X of

  $$2N' = 2 \{ Q[1] + Q[2] + \cdots + Q[N] \}$$

  random keys, where Q[i] is the maximum number of rectangles of type i (for i = 1, ..., N) that can be packed.

- X = (X[1], ..., X[N'], X[N'+1], ..., X[2N'])

Encoding

- Solutions are encoded as vectors X of

$$2N' = 2 \{ Q[1] + Q[2] + \cdots + Q[N] \}$$

random keys, where Q[i] is the maximum number of rectangles of type i (for i = 1, ..., N) that can be packed.

- X = (X[1], ..., X[N'], X[N'+1], ..., X[2N'])

Rectangle type packing sequence (RTPS)

Encoding

- Solutions are encoded as vectors X of

$$2N' = 2 \{ Q[1] + Q[2] + \cdots + Q[N] \}$$

random keys, where Q[i] is the maximum number of rectangles of type i (for i = 1, ..., N) that can be packed.

- X = (X[1], ..., X[N'], X[N'+1], ..., X[2N'])

Rectangle type packing sequence (RTPS)

Vector of placement procedures (VPP)

Decoding

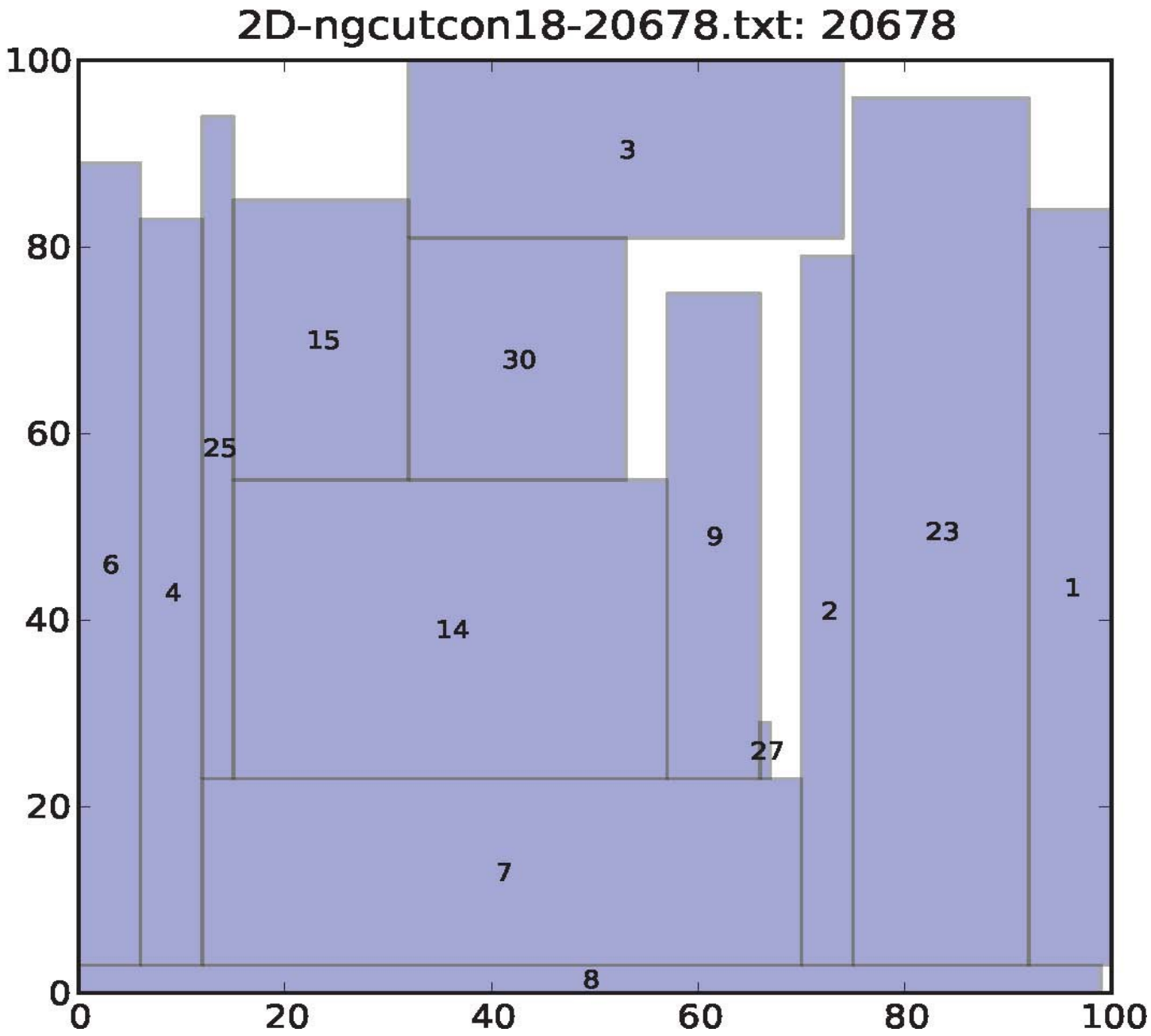
- Simple heuristic to pack rectangles:
  - Make Q[i] copies of rectangle i, for i = 1, ..., N.
  - Order the N' = Q[1] + Q[2] + ··· + Q[N] rectangles in some way.
  - Process the rectangles in the above order. Place the rectangle in the stock rectangle according to one of the following heuristics: bottom-left (BL) or left-bottom (LB). If rectangle cannot be positioned, discard it and go on to the next rectangle in the order.

Decoding

- Simple heuristic to pack rectangles:
  - Make Q[i] copies of rectangle i, for i = 1, ..., N.
  - Order the N' = Q[1] + Q[2] + ⋯ + Q[N] rectangles in some way. **Sort first N' keys of X to obtain order.**
  - Process the rectangles in the above order. Place the rectangle in the stock rectangle according to one of the following heuristics: bottom-left (BL) or left-bottom (LB). If rectangle cannot be positioned, discard it and go on to the next rectangle in the order.

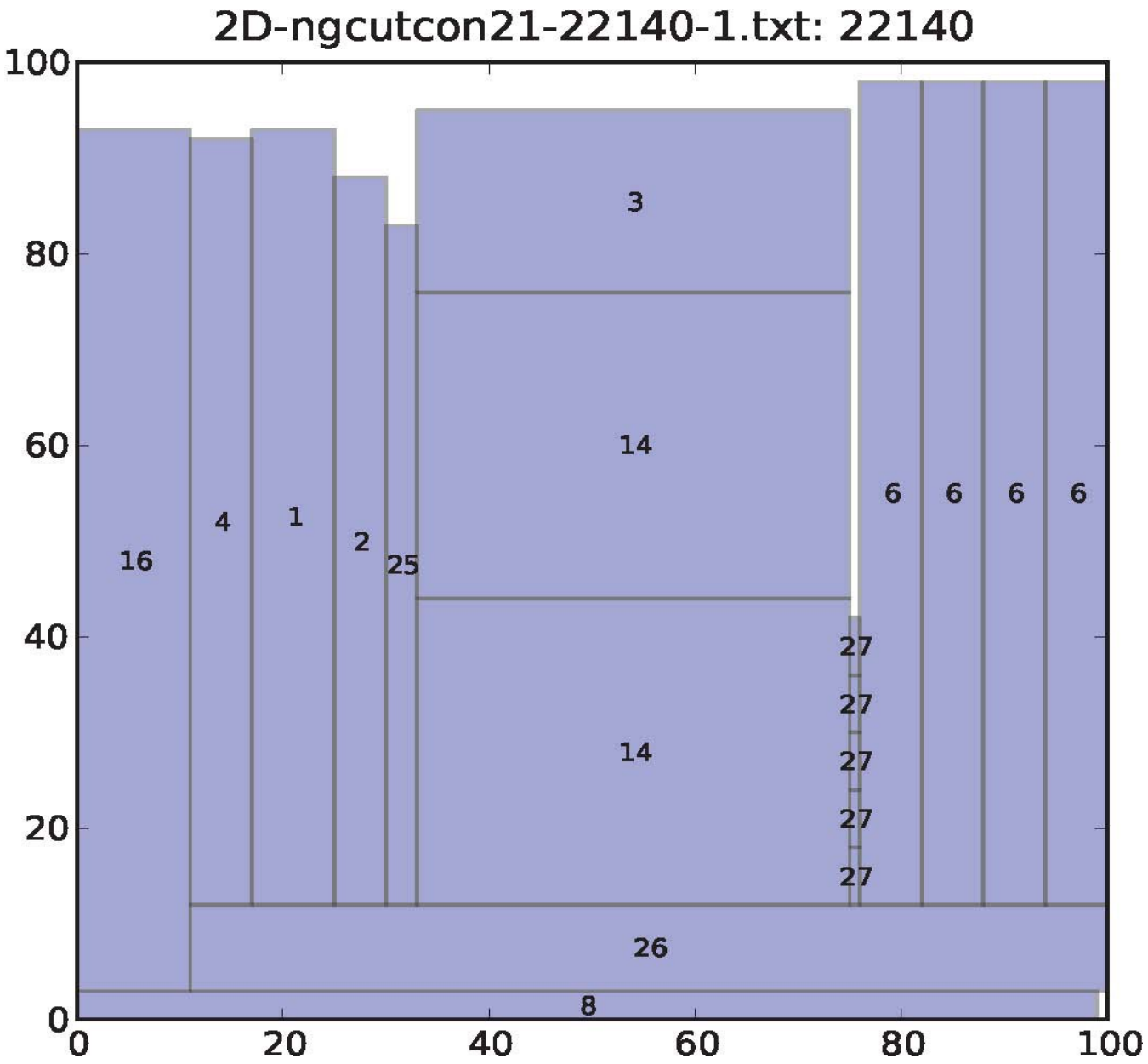
Decoding

- Simple heuristic to pack rectangles:
  - Make Q[i] copies of rectangle i, for i = 1, ..., N.
  - Order the N' = Q[1] + Q[2] + ··· + Q[N] rectangles in some way. **Sort first N' keys of X to obtain order.**
  - Process the rectangles in the above order. Place the rectangle in the stock rectangle according to one of the following heuristics: bottom-left (BL) or left-bottom (LB). If rectangle cannot be positioned, discard it and go on to the next rectangle in the order. **Use the last N' keys of X to determine which heuristic to use. If k[N'+i] > 0.5 use LB, else use BL.**



New BKS for a 100 x100 doubly constrained instance of Fekete & Schepers (1997) of value **20678.** Previous best was **19657** by tabu search of Alvarez-Valdes et al., (2007).

30 types
30 rectangles



New BKS for a 100 x 100 doubly constrained instance Fekete & Schepers (1997) of value **22140**.

Previous BKS was **22011** by tabu search of Alvarez-Valdes et al. (2007).

29 types
97 rectangles

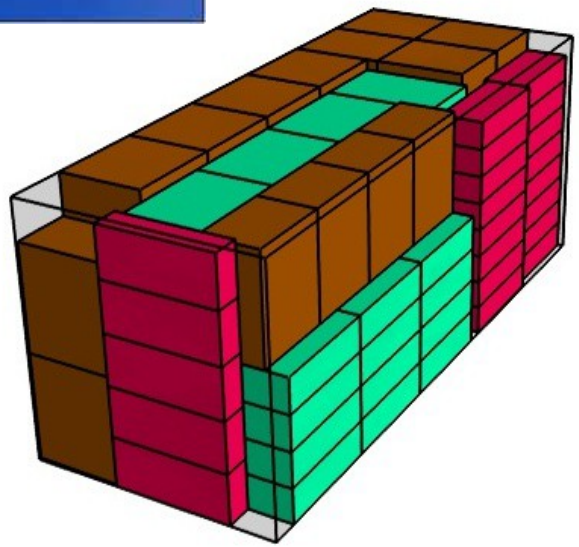
Some remarks

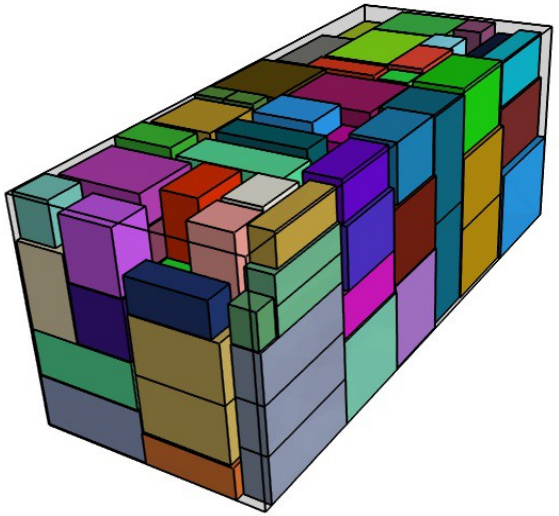


We have extended this to 3D packing:

J.F. Gonçalves and M.G.C.R., "A parallel multi-population biased random-key genetic algorithm for a container loading problem," Computers & Operations Research, vol. 29, pp. 179-190, 2012.

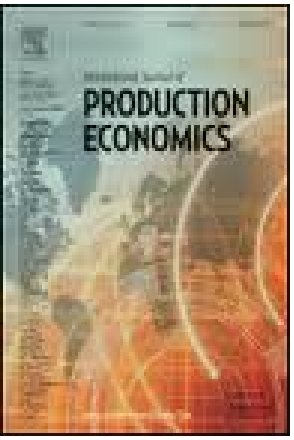
Tech report: http://mauricio.resende.info/doc/brkga-pack3d.pdf





BRKGA

3D bin packing

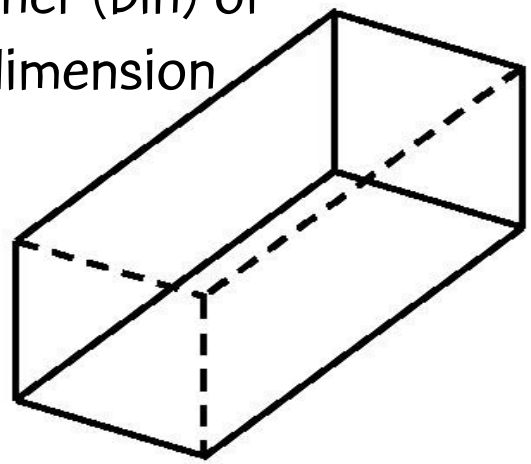


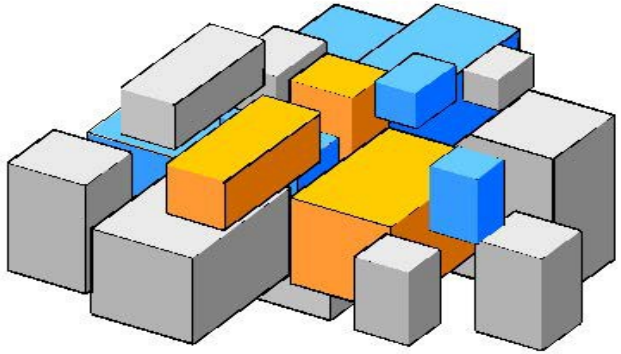
J.F. Gonçalves and R., "A biased random-key genetic algorithm for 2D and 3D bin packing problems," International J. of Production Economics, vol. 15, pp. 500–510, 2013.

http://mauricio.resende.info/doc/brkga-binpacking.pdf

3D bin packing problem

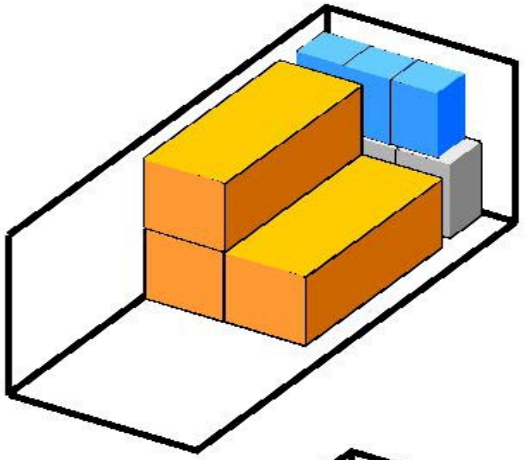
Container (bin) of fixed dimension

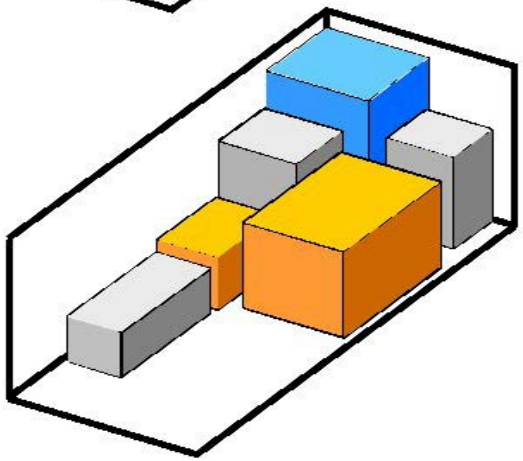




Boxes of different dimensions

Minimize number of containers (bins) needed to pack all boxes

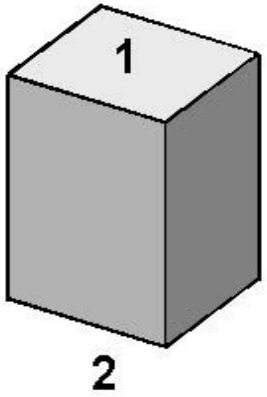


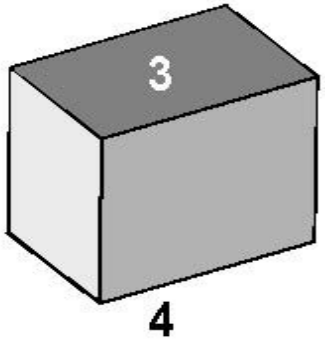


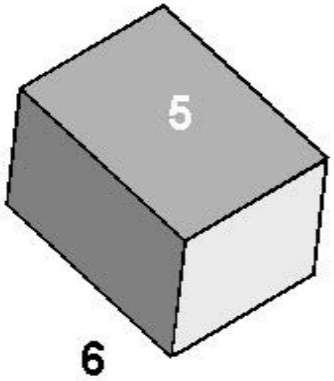
3D bin packing constraints

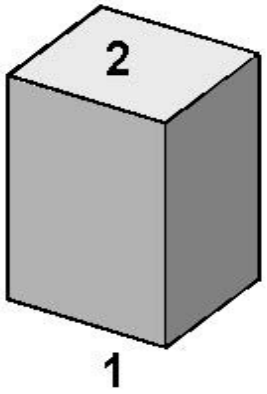
- Each box is placed completely within container
- Boxes do not overlap with each other
- Each box is placed parallel to the side walls of bin
- In some instances, only certain box orientations are allowed (there are at most six possible orientations)

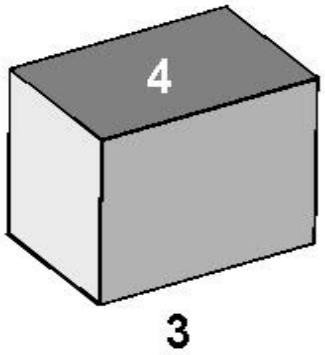
Six possible orientations for each box

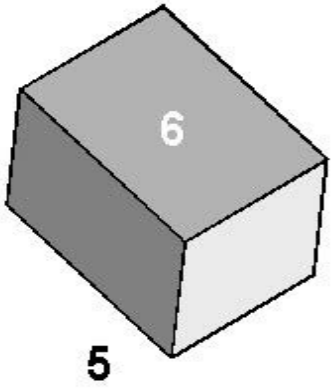






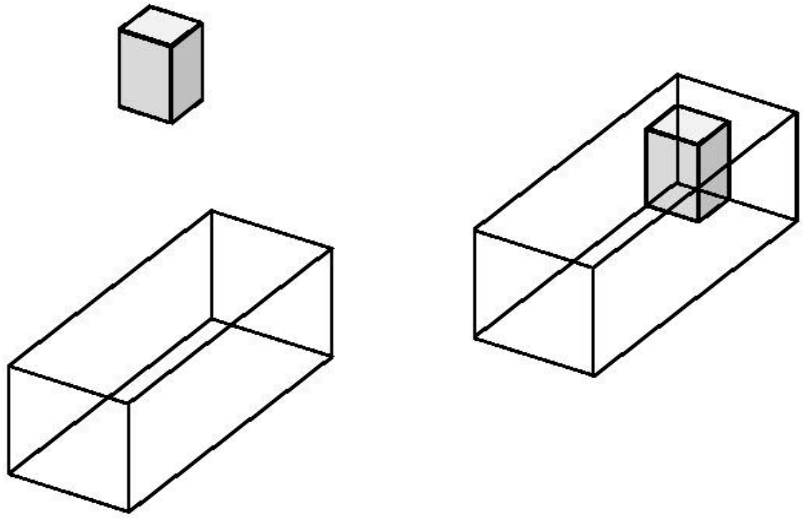


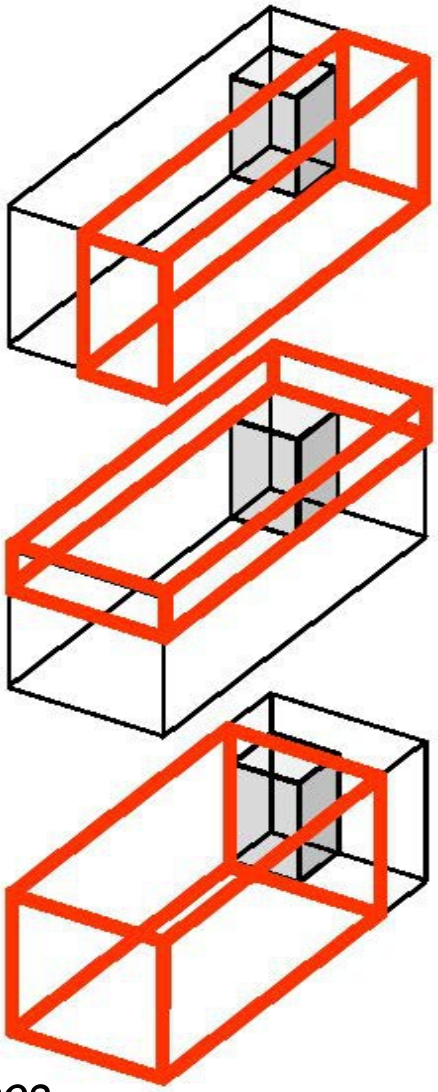




Difference process - DP

(Lai & Chan, 1997)





When box is placed in container ...

use DP to keep track of maximal free spaces

Encoding

Solutions are encoded as vectors of 3n random keys, where n is the number of boxed to be packed.

$$X = (x_1 , x_2 , \ldots, x_n , x_{n+1}, x_{n+2}, \ldots, x_{2n}, x_{2n+1}, x_{2n+2}, \ldots, x_{3n})$$

Box packing sequence — Placement heuristic — Box orientation

Decoding

1) Sort first n keys of X to produce sequence boxes will be packed;
2) Use second n keys of X to determine which placement heuristic to use (back-bottom-left or back-left-bottom):
   - if $x_{n+i} < ½$ then use back-bottom-left to pack i-th box
   - if $x_{n+i} \geq ½$ then use back-left-bottom to pack i-th box
3) Use third n keys of X to determine which of six orientations to use when packing box:
   - $x_{2n+i} \in [0,1/6)$: orientation 1;
   - $x_{2n+i} \in [1/6,2/6)$: orientation 2; ...
   - $x_{2n+i} \in [5/6,1]$: orientation 6.

Decoding

For each box

- scan containers in order they were opened
- use placement heuristic to place box in first container in which box fits with its specified orientation
- if box does not fit in any open container, open new container and place box using placement heuristic with its specified orientation

Fitness function

Instead of using as fitness measure the number of bins (NB)

- use adjusted fitness: aNB
- aNB = NB + (LeastLoad / BinVolume), where
  - LeastLoad is load on least loaded bin
  - BinVolume is volume of bin: H x W x L

Experiment

- Parameters:
  - population size: $p = 30n$
  - size of elite partition: $p_e = .10p$
  - number of of mutans: $p_m = .15p$
  - crossover probability: 0.7
  - stopping criterion: 300 generations

Experiment

- Instances:
  - 320 instances of Martello et al. (2000)
  - generator is available at http://www.diku.dk/~pisinger/codes/html
  - 8 classes
  - 40 instances per class
  - 10 instances for each value of n ∈ {50, 100, 150, 200)

Experiment

- We compare BRKGA with:
  - TS3, the tabu search of Lodi et al. (2002)
  - GLS, the guided local search of Faroe et al. (2003)
  - TS2PACK, the tabu search of Crainic et al. (2009)
  - GRASP, the greedy randomized adaptive search procedure of Parreno et al. (2010)

Summary

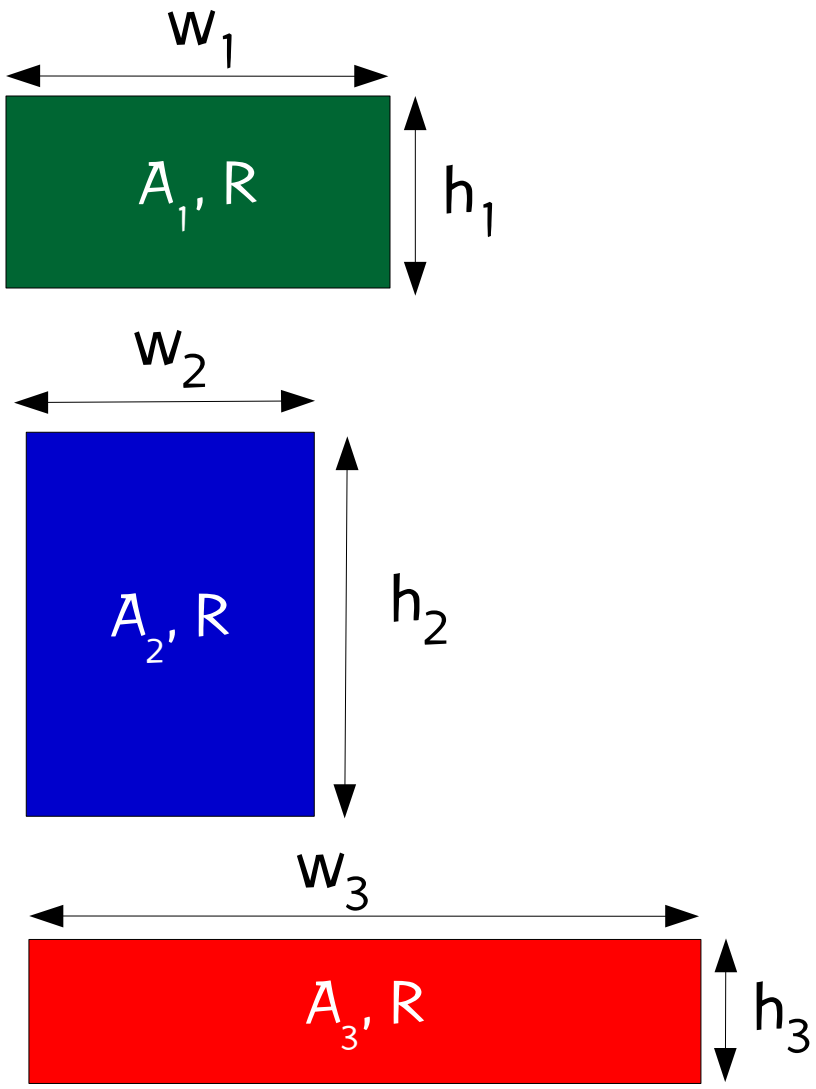
Class	Bin size	BRKGA	GRASP	TS3	TS2PACK	GLS
1	100^3	**127.3**	**127.3**	127.9	128.2	128.3
2	100^3	**125.5**	125.8	126.8		
3	100^3	**126.5**	126.9	127.5		
4	100^3	294.0	294.0	294.0	**293.9**	294.2
5	100^3	**70.4**	70.5	71.4	71.0	70.8
6	10^3	**95.0**	95.4	96.1	95.8	96.0
7	40^3	**58.2**	59.4	60.0	59.0	59.0
8	100^3	**80.9**	82.0	82.6	81.9	81.9
Sum(rows 1, 4-8):		**725.8**	728.6	732.0	729.8	730.2
Sum(rows 1-8):		**977.8**	981.3	986.3		

The unequal area facility layout problem



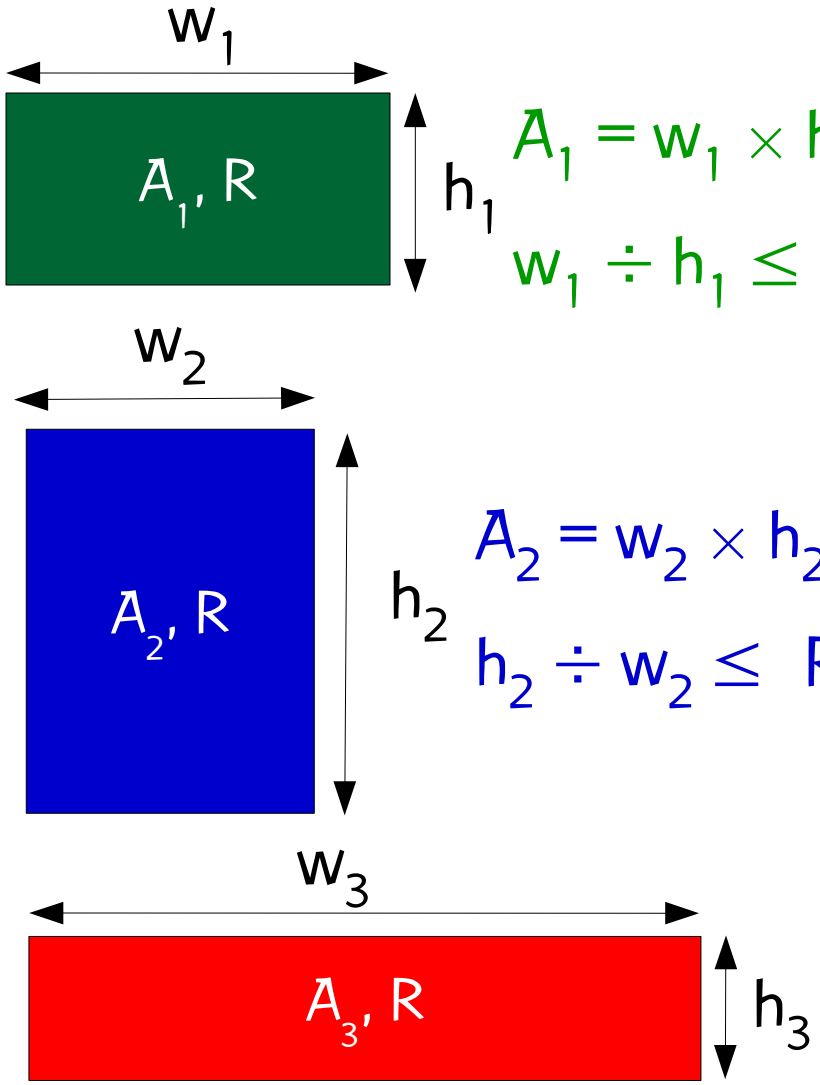
J.F. Gonçalves & R., "A biased random-key genetic algorithm for the unequal area facility layout problem," European J. of Operational Research, 2015

Unequal area facility layout



Given N rectangular facilities, i = 1, 2, ..., N, each having given area $A_i = w_i \times h_i$ all of maximum aspect ratio (between longest & shortest dimensions) R

Unequal area facility layout



$A_1 = w_1 \times h_1$

$w_1 \div h_1 \leq R$

$A_2 = w_2 \times h_2$

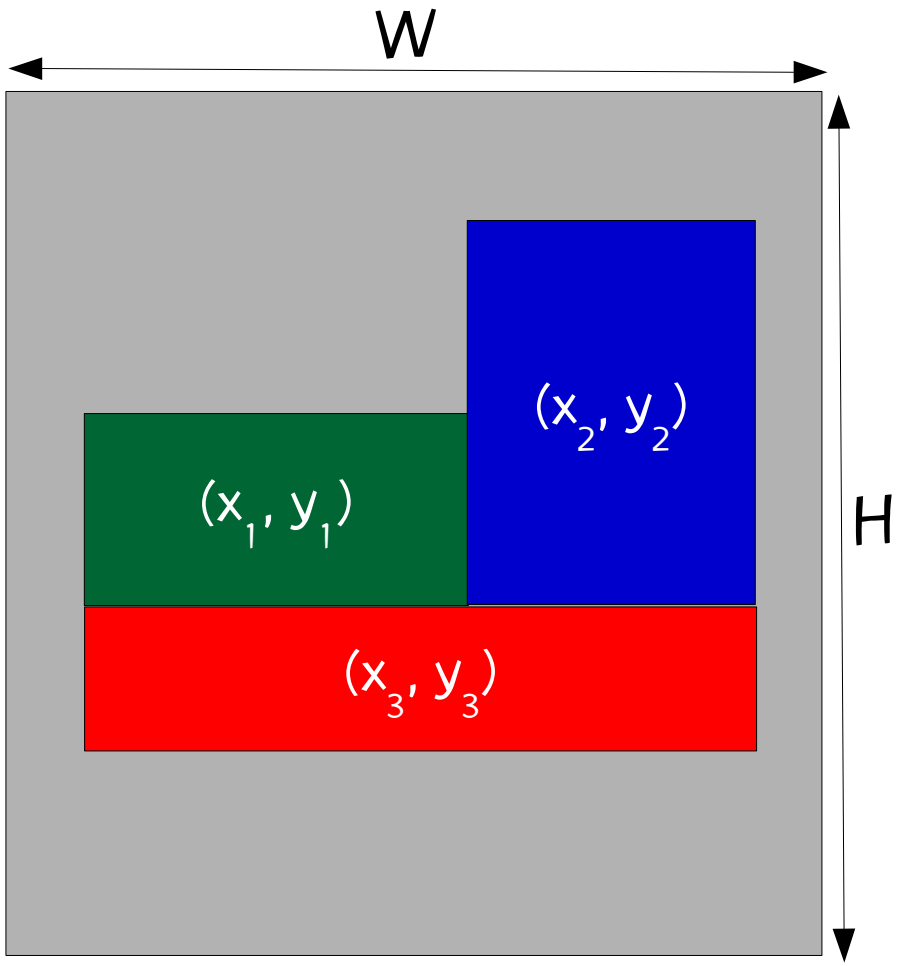
$h_2 \div w_2 \leq R$

$A_3 = w_3 \times h_3$

$w_3 \div h_3 \leq R$

Given N rectangular facilities, i = 1, 2, ..., N, each having given area $A_i = w_i \times h_i$ all of maximum aspect ratio (between longest & shortest dimensions) R (Note that w_i and h_i are not given, only A_i and R are)

Unequal area facility layout



Layout the facilities, without overlap or rotation, on a rectangular floor of area $W \times H$ with centroids at coordinates $(x_1,y_1), (x_2,y_2), \ldots, (x_N,y_N)$ and dimensions $w_1 \times h_1, w_2 \times h_2, \ldots, w_N \times h_N$.

Unequal area facility layout

We consider two types of problems

- In the constrained type, we are given the rectangular floor dimensions $W \times H$.
- In the unconstrained type, we assume the floor space can include all the facilities laid out horizontally or vertically at their maximum horizontal or vertical dimensions, i.e.

$$(W, H) = \left(\sum_{i=1}^{N} (A_i \times R)^{1/2}, \sum_{i=1}^{N} (A_i \times R)^{1/2} \right)$$

Unequal area facility layout

Of all feasible layouts, find one that minimizes

$$\sum_{i=1}^{N} \sum_{j=1}^{N} f_{i,j} \times c_{i,j} \times d_{i,j}$$

where

- $f_{i,j}$ is the flow between facilities i and j ($f_{i,i} = 0$)
- $c_{i,j}$ is the cost per unit distance between i and j
- $d_{i,j} = |x_i - x_j| + |y_i - y_j|$ is the rectilinear distance between (x_i,y_i) and (x_j,y_j)

Unequal area facility layout

Of all feasible layouts, find one that minimizes

$$\sum_{i=1}^{N}\sum_{j=1}^{N} f_{i,j} \times c_{i,j} \times d_{i,j}$$

quadratic assignment problem (QAP)

where

- $f_{i,j}$ is the flow between facilities i and j ($f_{i,i} = 0$)
- $c_{i,j}$ is the cost per unit distance between i and j
- $d_{i,j} = |x_i - x_j| + |y_i - y_j|$ is the rectilinear distance between (x_i,y_i) and (x_j,y_j)

Unequal area facility layout

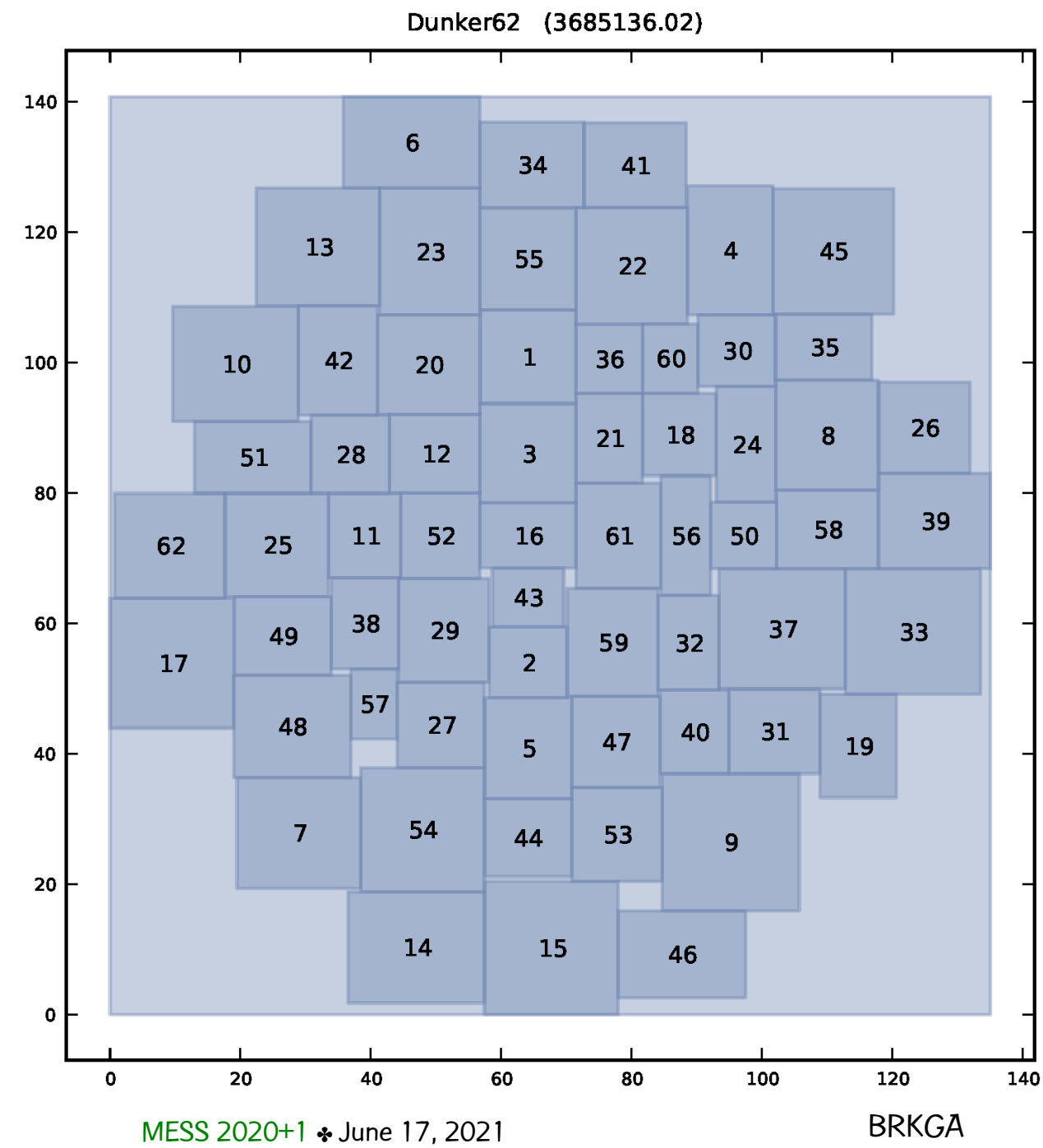
Of all feasible layouts, find one that minimizes

$$\sum_{i=1}^{N}\sum_{j=1}^{N} f_{i,j} \times c_{i,j} \times d_{i,j}$$

Besides rectilinear (R) distance metric, we also deal with Euclidean (E), and Squared Euclidean (SE) in paper.

where

- $f_{i,j}$ is the flow between facilities i and j ($f_{i,i} = 0$)
- $c_{i,j}$ is the cost per unit distance between i and j
- $d_{i,j} = |x_i - x_j| + |y_i - y_j|$ is the rectilinear distance between (x_i,y_i) and (x_j,y_j)

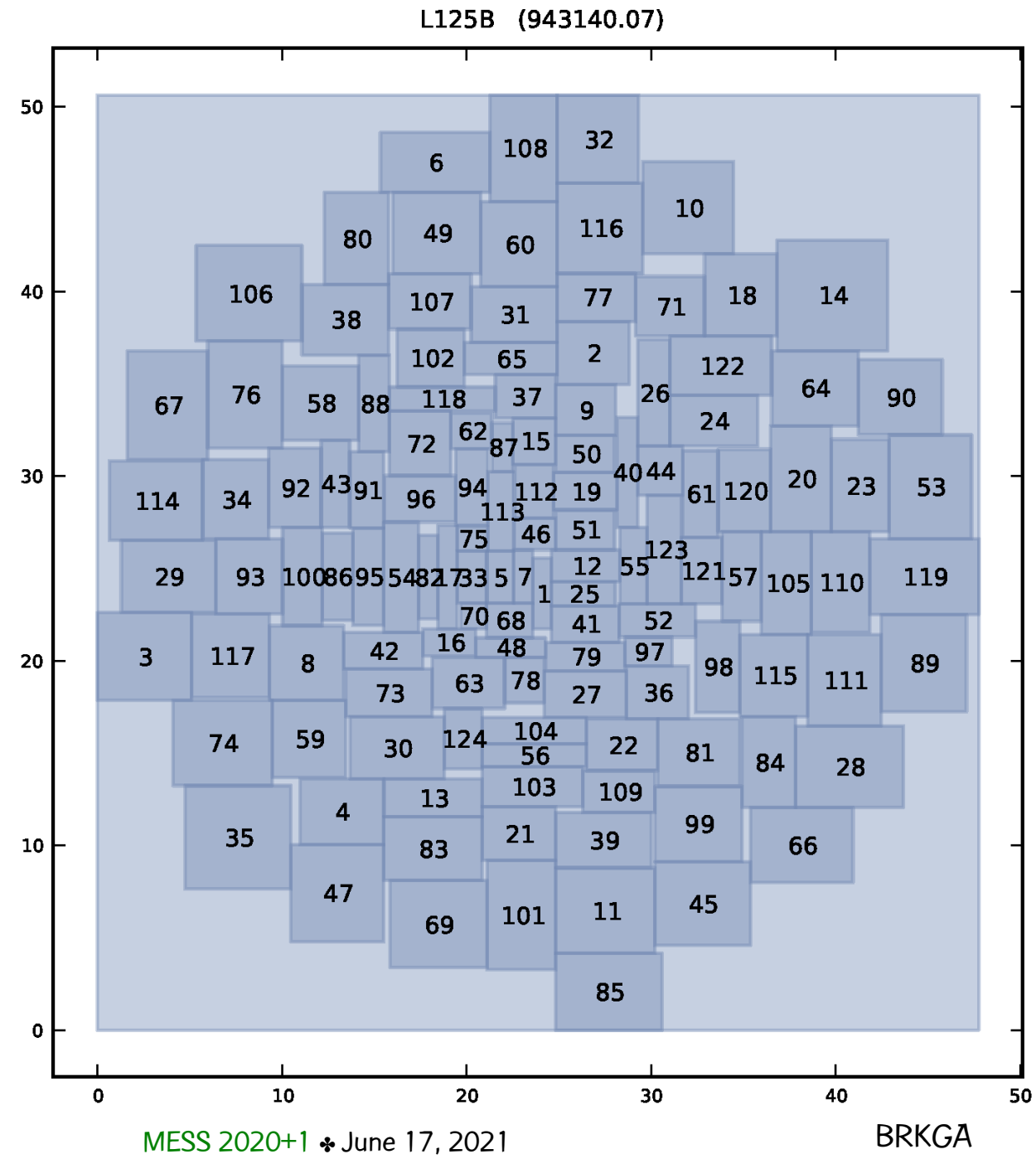


Dunker62

New best known solution: 3.68E6

Previous best known solution: 3.81E6

TS-BST (McKendall Jr. & Hajobyan, 2010)



L125B

New best known solution: 9.43E5

Previous best known solution: 1.01E6

TS-BST (McKendall Jr. & Hajobyan, 2010)

BRKGA for the unequal area facility layout problem

Encoding

Solutions are encoded with a vector of random keys of length 2N+2

$$X = (X_1, \ldots, X_N, X_{N+1}, \ldots, X_{2N}, X_{2N+1}, X_{2N+2})$$

Encoding

Solutions are encoded with a vector of random keys of length 2N+2

$$X = (X_1, \ldots, X_N, X_{N+1}, \ldots, X_{2N}, X_{2N+1}, X_{2N+2})$$

Facility placement sequence

Encoding

Solutions are encoded with a vector of random keys of length 2N+2

$$X = (\underbrace{X_1, \ldots, X_N}_{\text{Facility placement sequence}}, \underbrace{X_{N+1}, \ldots, X_{2N}}_{\text{Facility aspect ratios}}, X_{2N+1}, X_{2N+2})$$

Encoding

Solutions are encoded with a vector of random keys of length 2N+2

$$X = (\underbrace{X_1, \ldots, X_N}, \underbrace{X_{N+1}, \ldots, X_{2N}}, \underbrace{X_{2N+1}, X_{2N+2}})$$

Facility placement sequence

Facility aspect ratios

(x, y) coordinates of the first facility to be placed

Decoding

1. Use X_1, ..., X_N to determine the sequence in which the facilities are placed on the floor space
2. Use X_{N+1}, ..., X_{2N} to determine the aspect ratio of each facility
3. Use X_{2N+1}, X_{2N+2} to determine the (x, y) coordinates of the first facility to be placed on the floor space
4. Use results of (1)-(3) with placement heuristic to place all the facilities on the floor space
5. Evaluate fitness of solution

Decoder: Step 1

Use $X_1, \ldots, X_N$ to determine the sequence in which the facilities are placed on the floor space:

Simply sort the key values $X_1, \ldots, X_N$ to determine the indices of the permutation of the facilities.

Decoder: Step 2

Use X_{N+1}, ..., X_{2N} to determine the aspect ratio of each facility:

Aspect ratio of facility i is

$$FAR_i = (1/R) + X_{N+i} \times (R - (1/R)),$$

where R is the given maximum facility aspect ratio.

Decoder: Step 2

Use $X_{N+1}, \ldots, X_{2N}$ to determine the aspect ratio of each facility:

Aspect ratio of facility i is

$$FAR_i = (1/R) + X_{N+i} \times (R - (1/R)),$$

where R is the given maximum facility aspect ratio.

$$w_i = (A_i \times FAR_i)^{1/2} \text{ and}$$

$$h_i = A_i / w_i$$

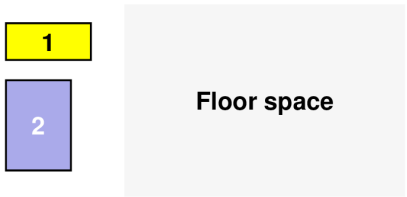
Decoder: Step 3

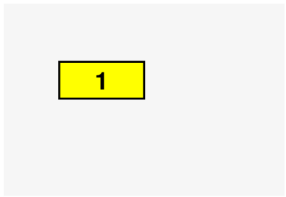
Use X_{2N+1}, X_{2N+2} to determine the (x, y) coordinates of the first facility to be placed on the floor space.

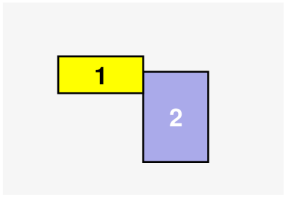
$$x = (w_i/2) + X_{2N+1} \times (W - w_i)$$

$$y = (h_i/2) + X_{2N+2} \times (H - h_i)$$

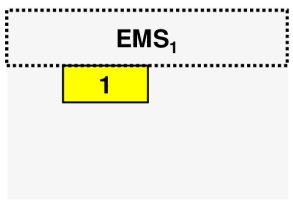
Decoder: Step 4 Makes use of empty maximal-spaces (EMS)

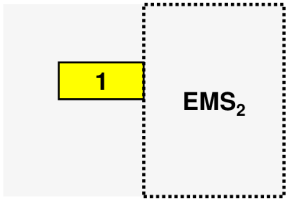


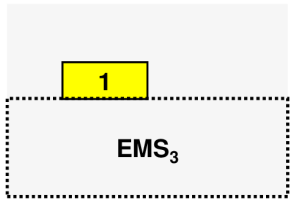


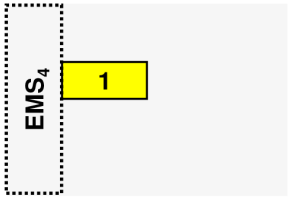


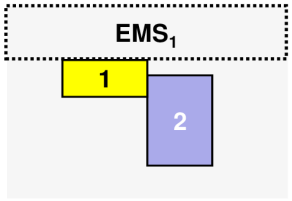


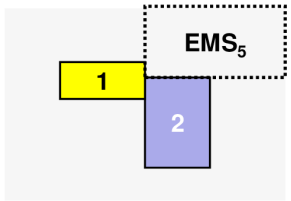


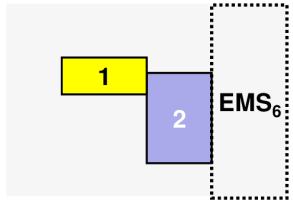


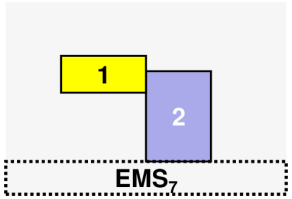


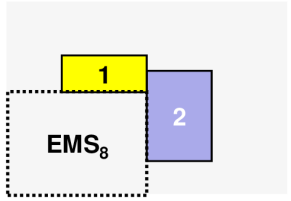


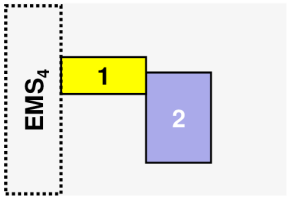








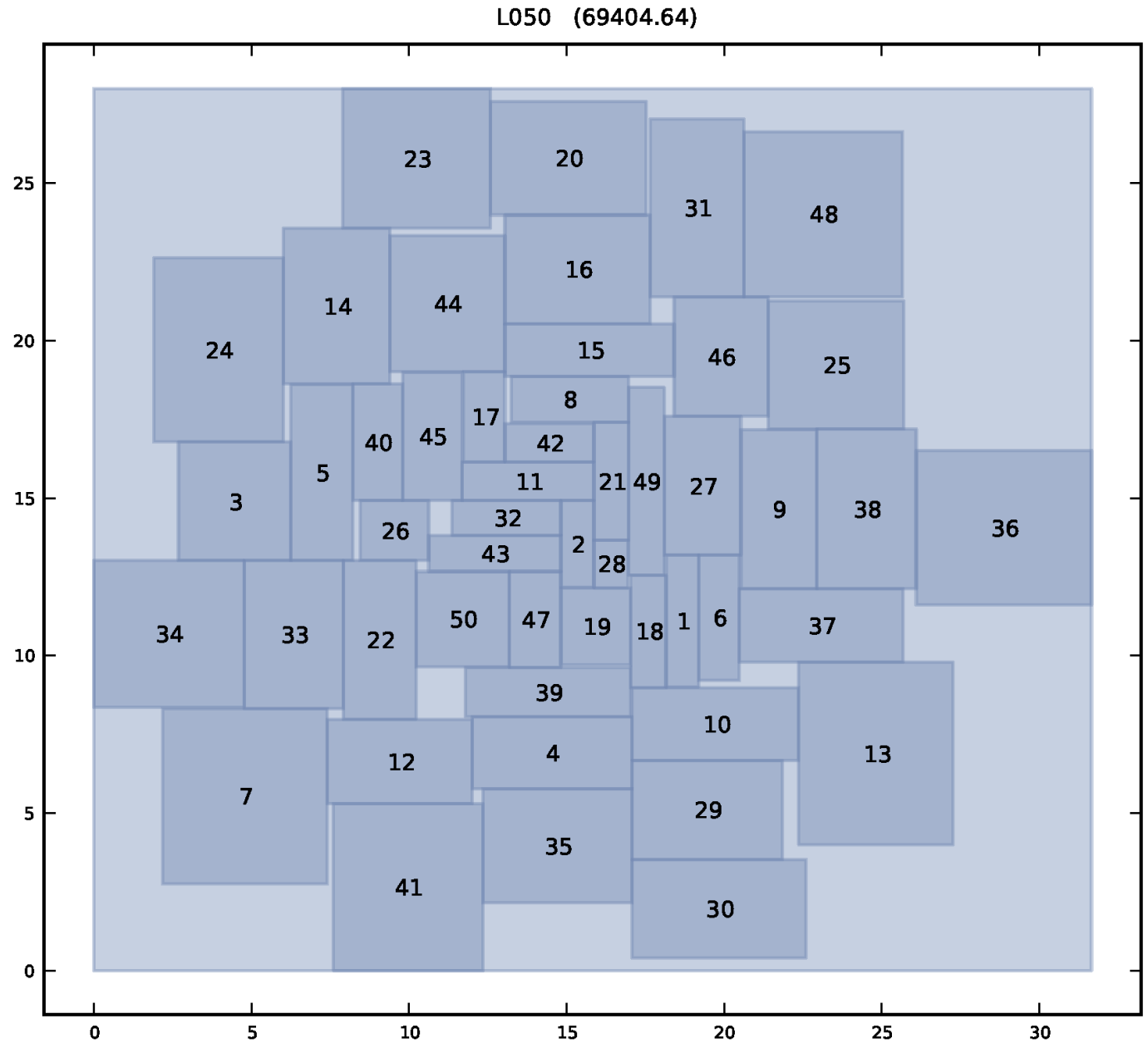




a) Facilities to be placed and the initial empty maximal-space (the floor space)

b) Empty maximal-spaces after placing facility 1.

c) Empty maximal-spaces after placing facility 2.

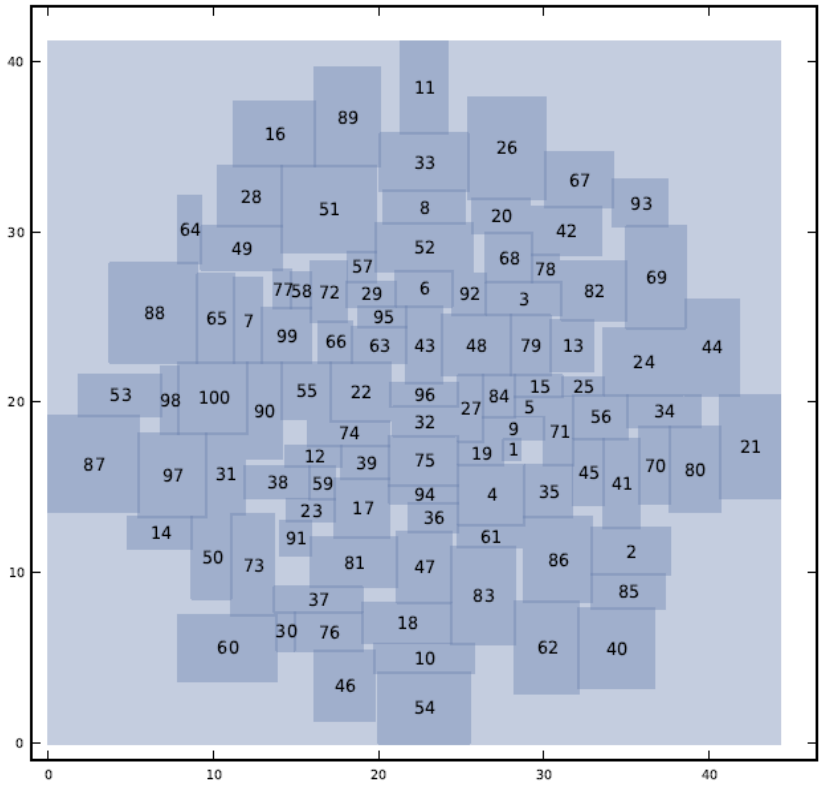


L050

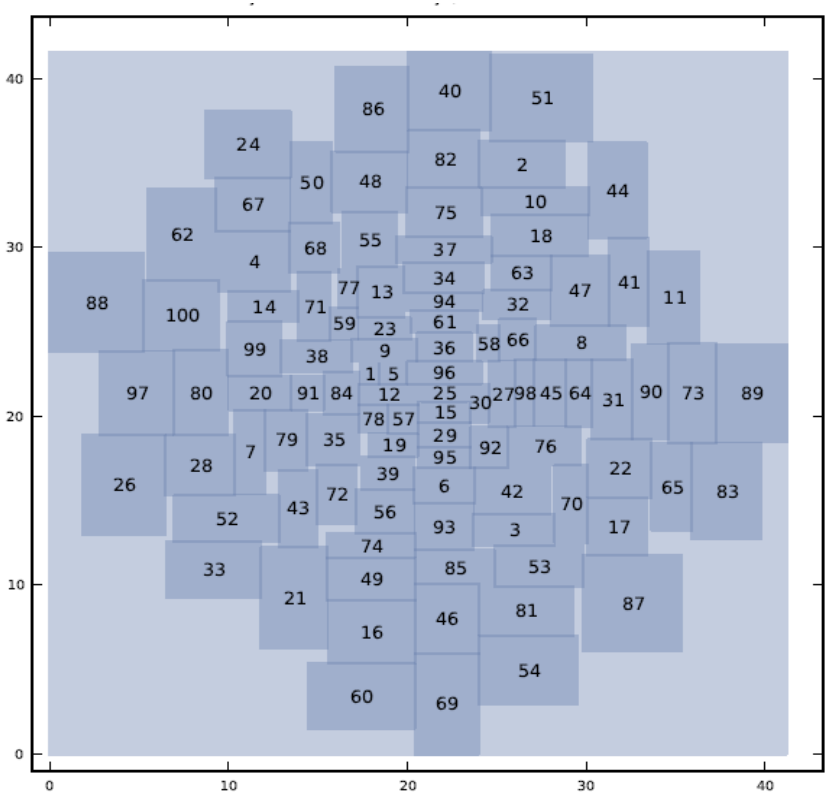
New best known Solution: 6.94E4

Previous best known Solution: 7.13E4 TS-BST (McKendall Jr. & Hajobyan, 2010)

1st generation: 530404.76



50th generation: 478910.09

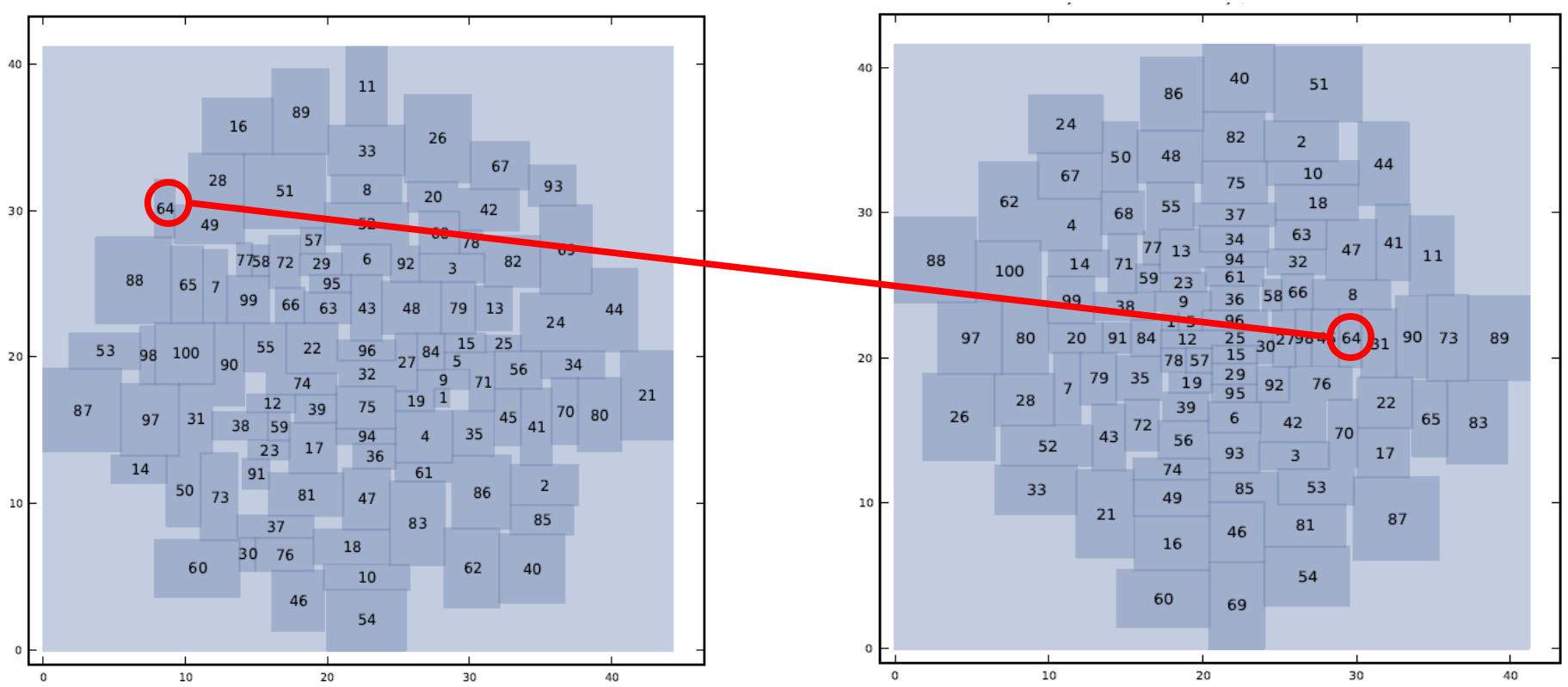


L100

BRKGA

1st generation: 530404.76

50th generation: 478910.09

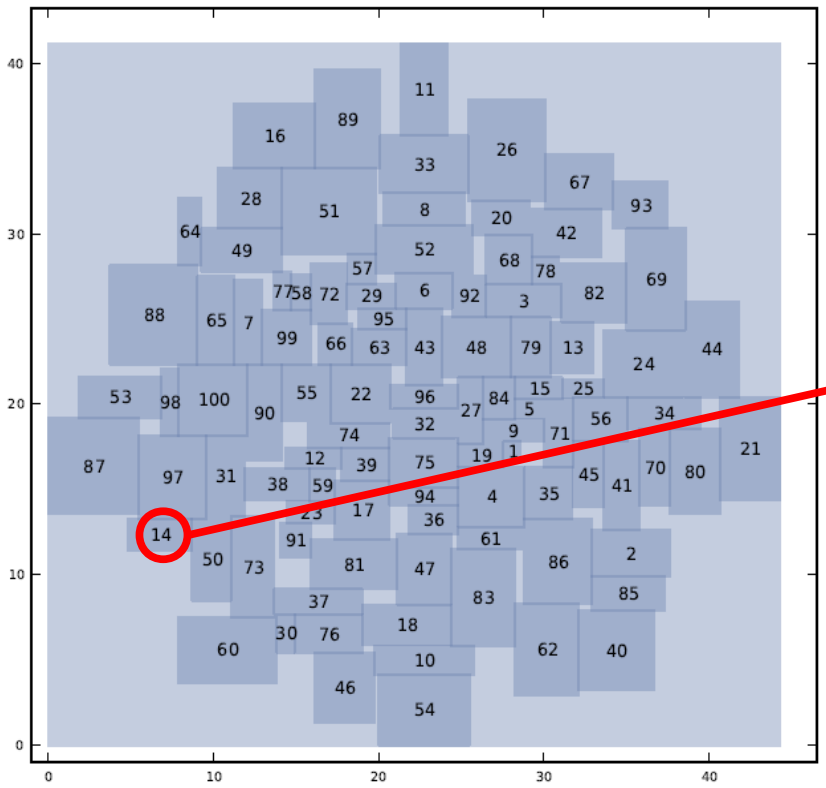


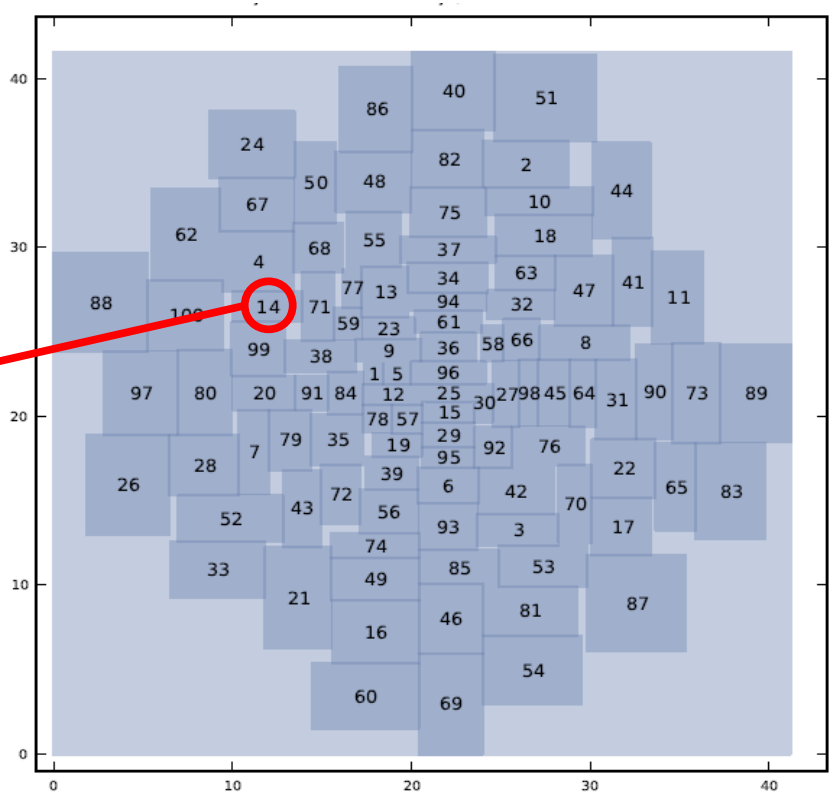
L100

BRKGA

1st generation: 530404.76

50th generation: 478910.09



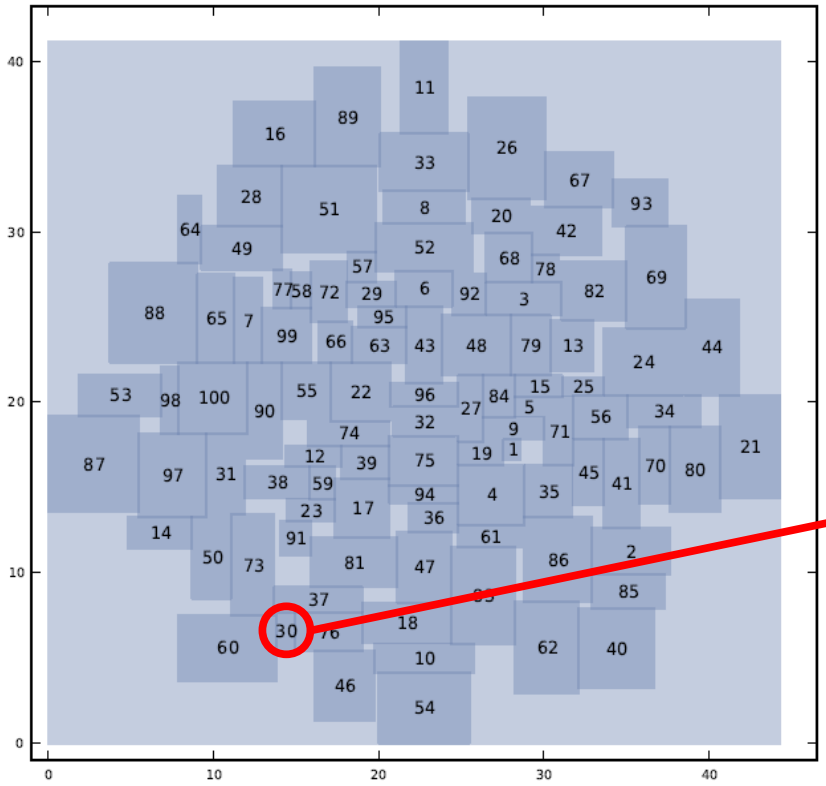


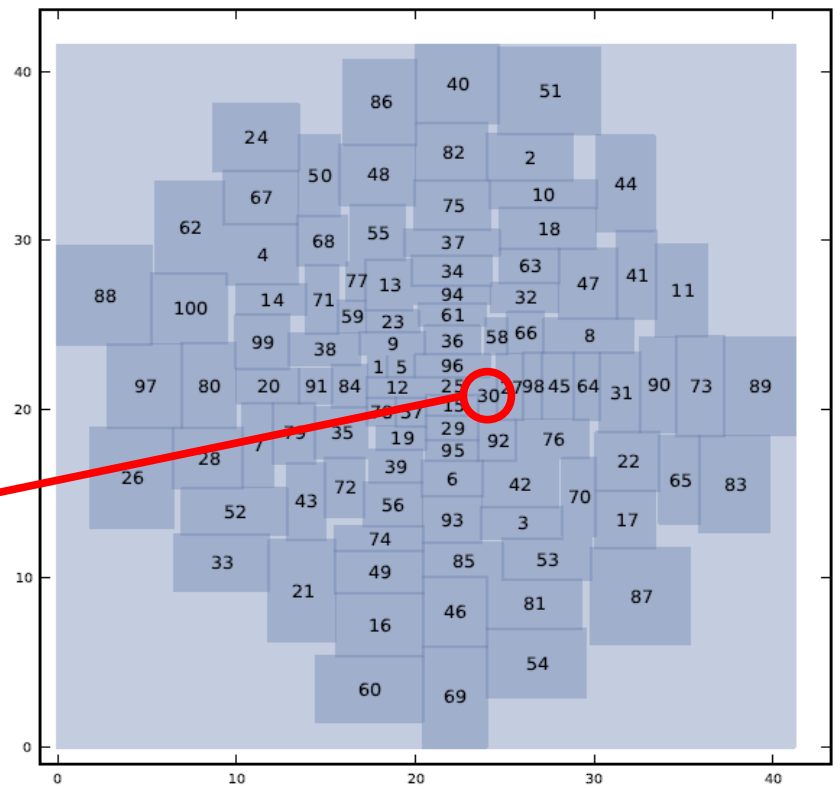
L100

BRKGA

1st generation: 530404.76

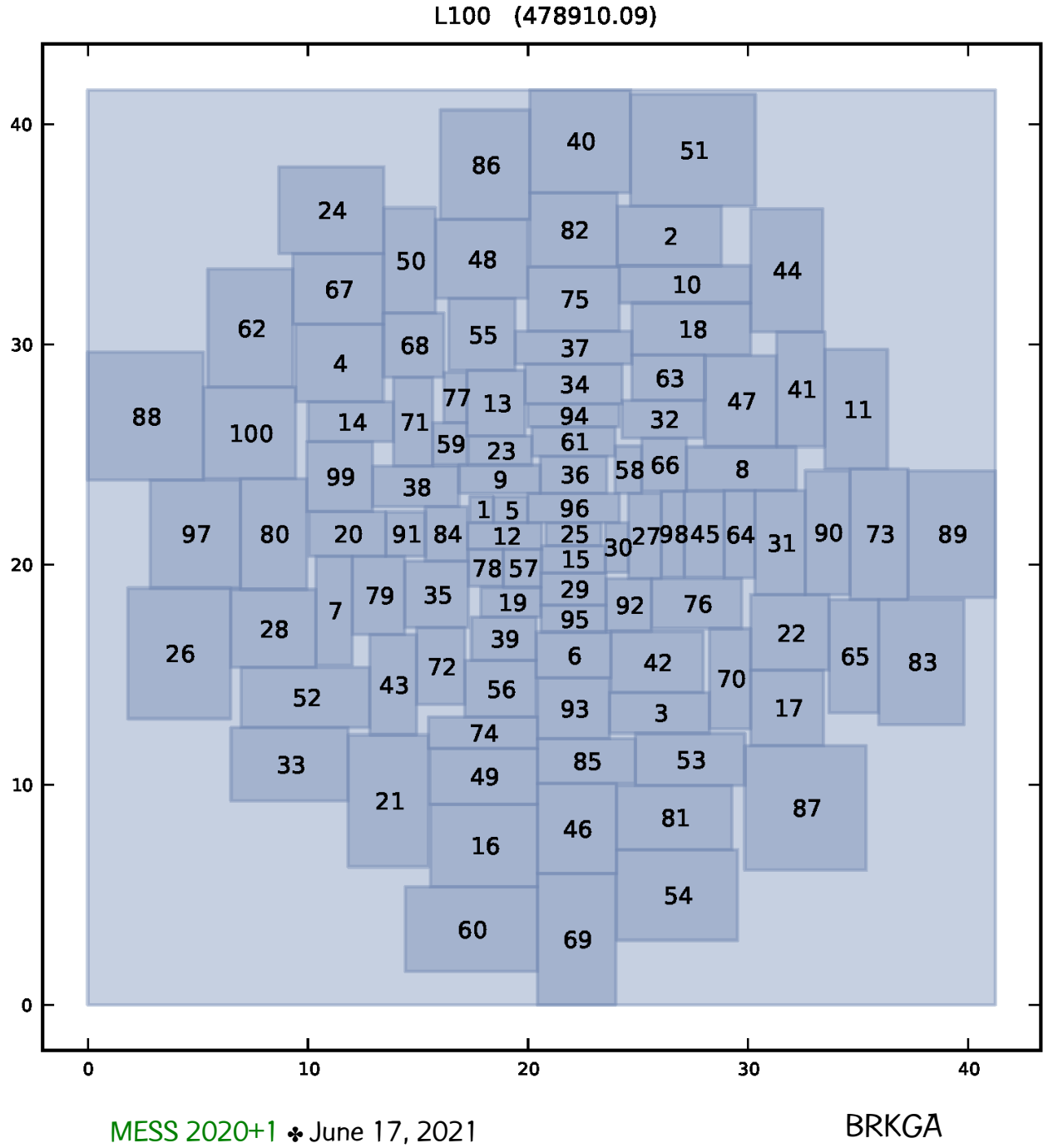
50th generation: 478910.09





L100

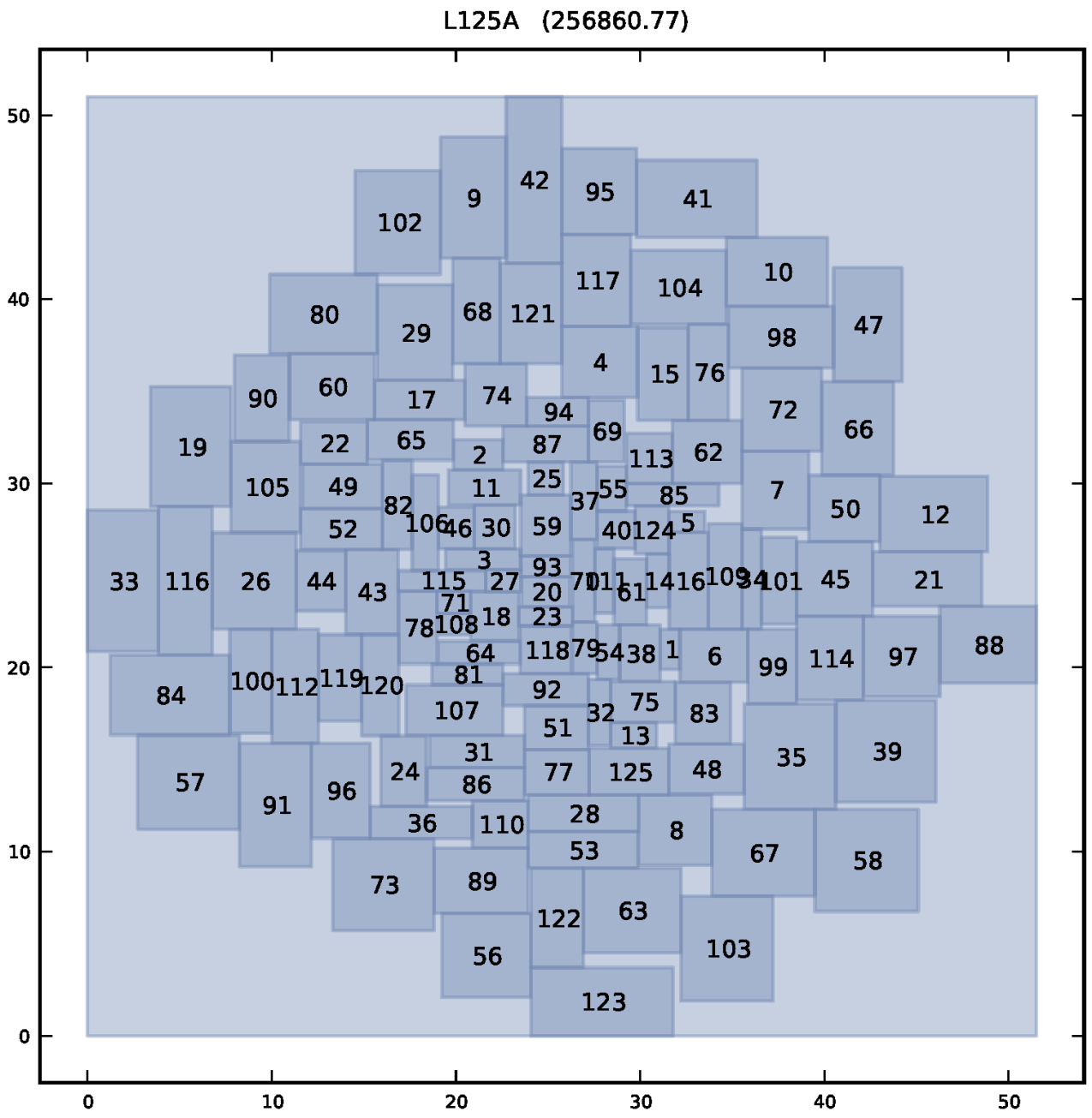
BRKGA



L100

New best known
Solution: 4.79E5

Previous best known
Solution: 4.97E5
TS-BST (McKendall Jr. & Hajobyan, 2010)



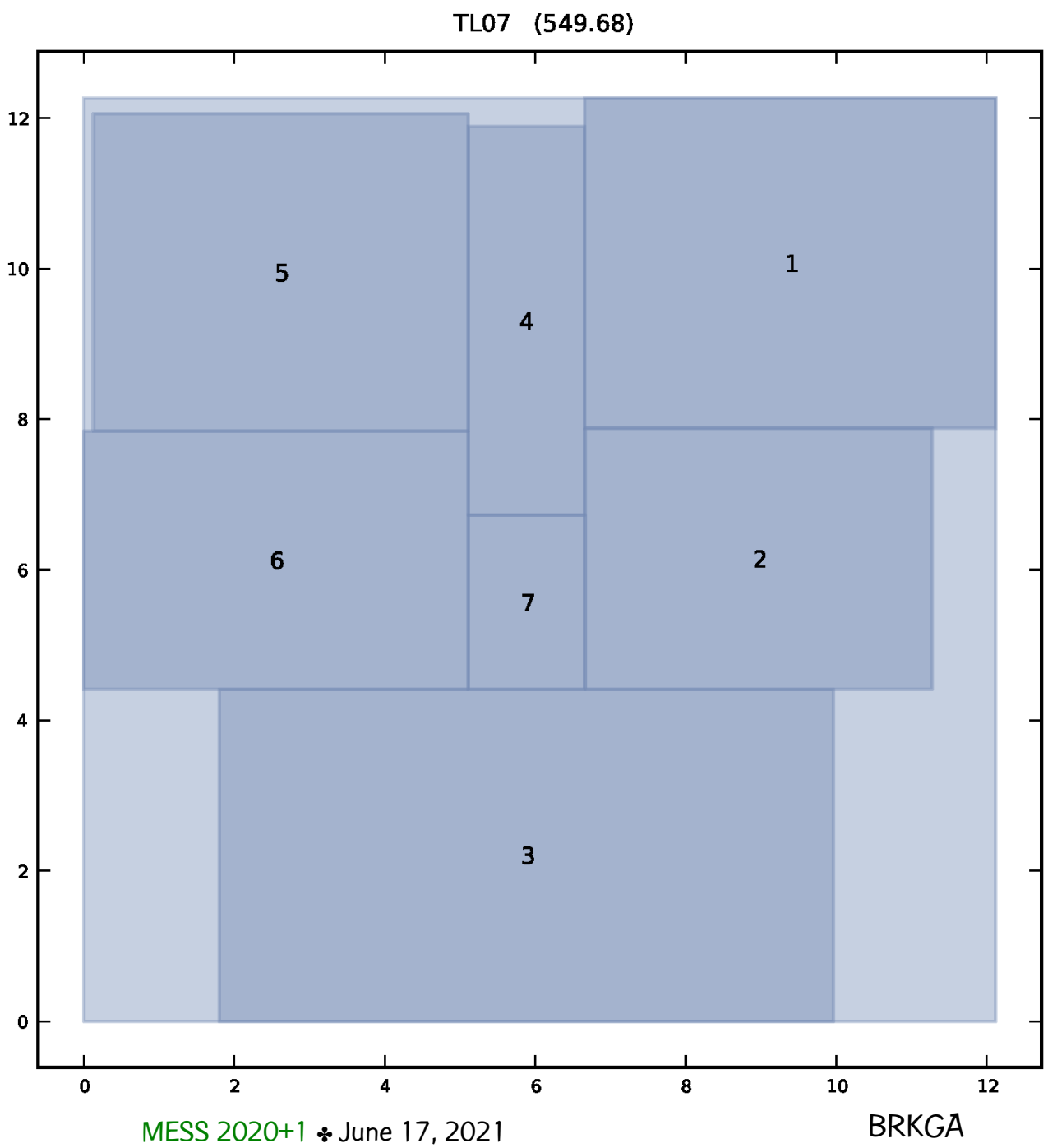
L125A

New best known
Solution: 2.57E5

Previous best known
Solution: 2.89E5
VIP-PLANOPT (2010)

MESS 2020+1 ✤ June 17, 2021

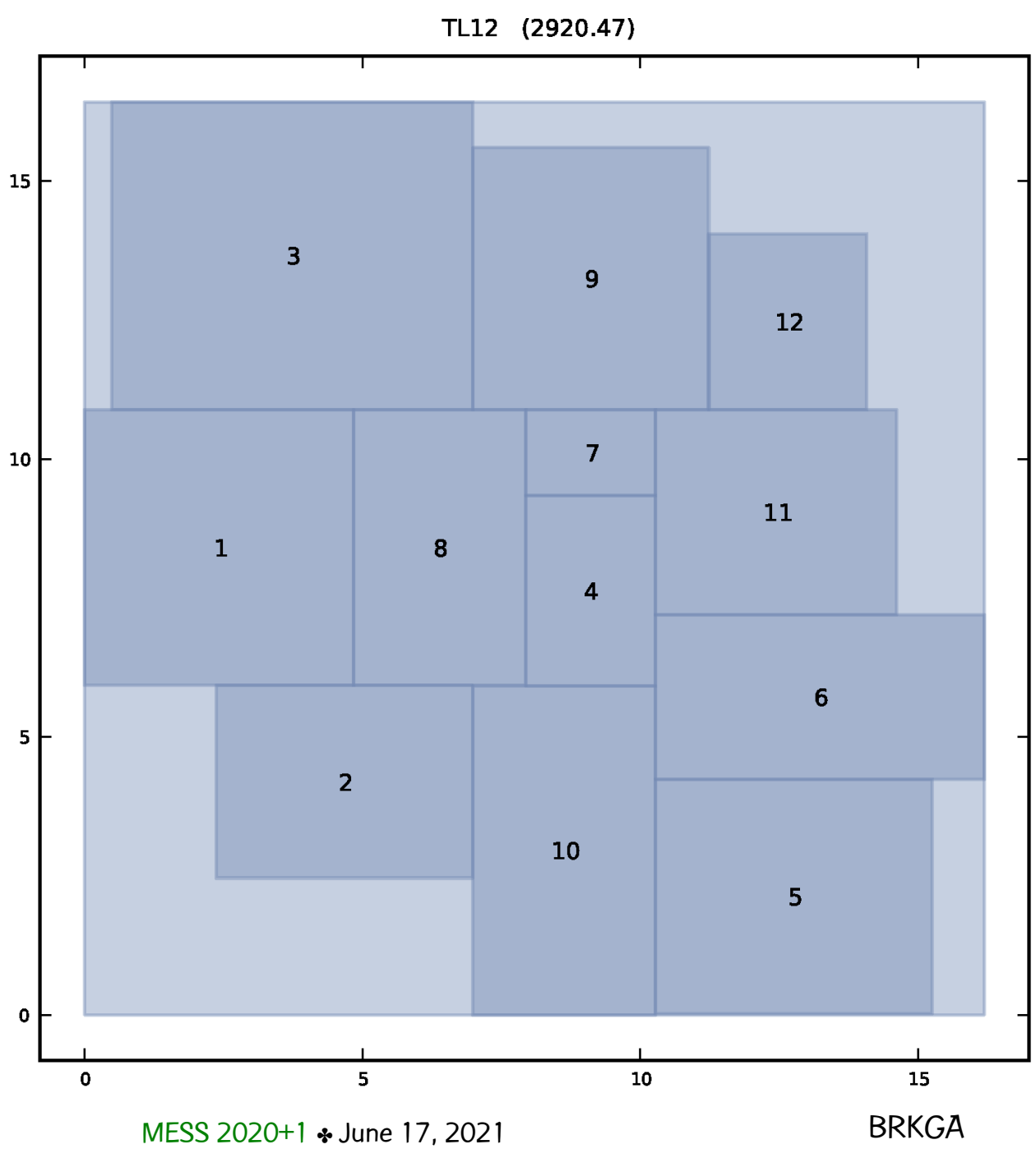
BRKGA



TL07

New best known
Solution: 549.7

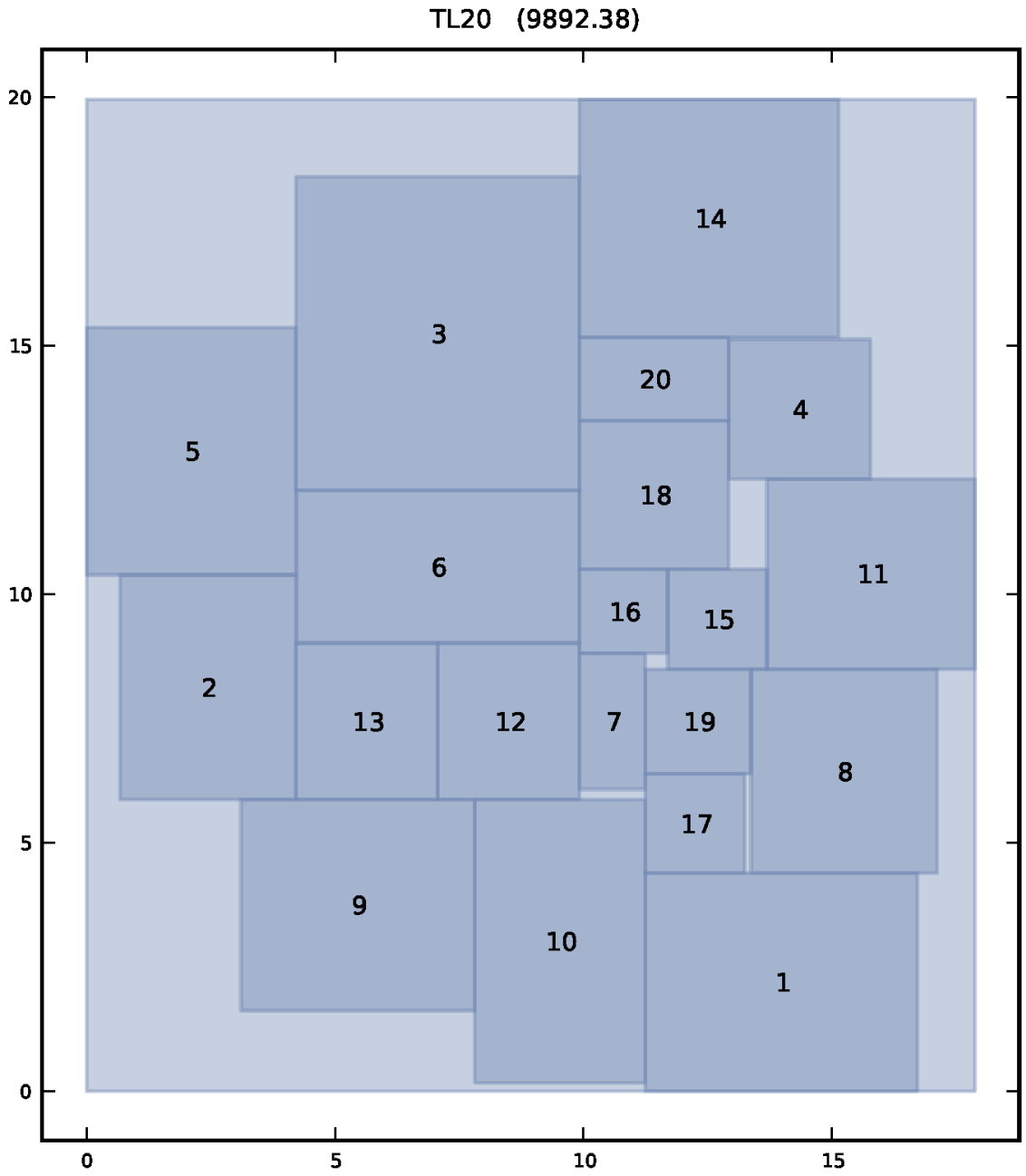
Previous best known
Solution: 559.0
HA-C (Tam and Li, 1991)



TL12

New best known
Solution: 2920.5

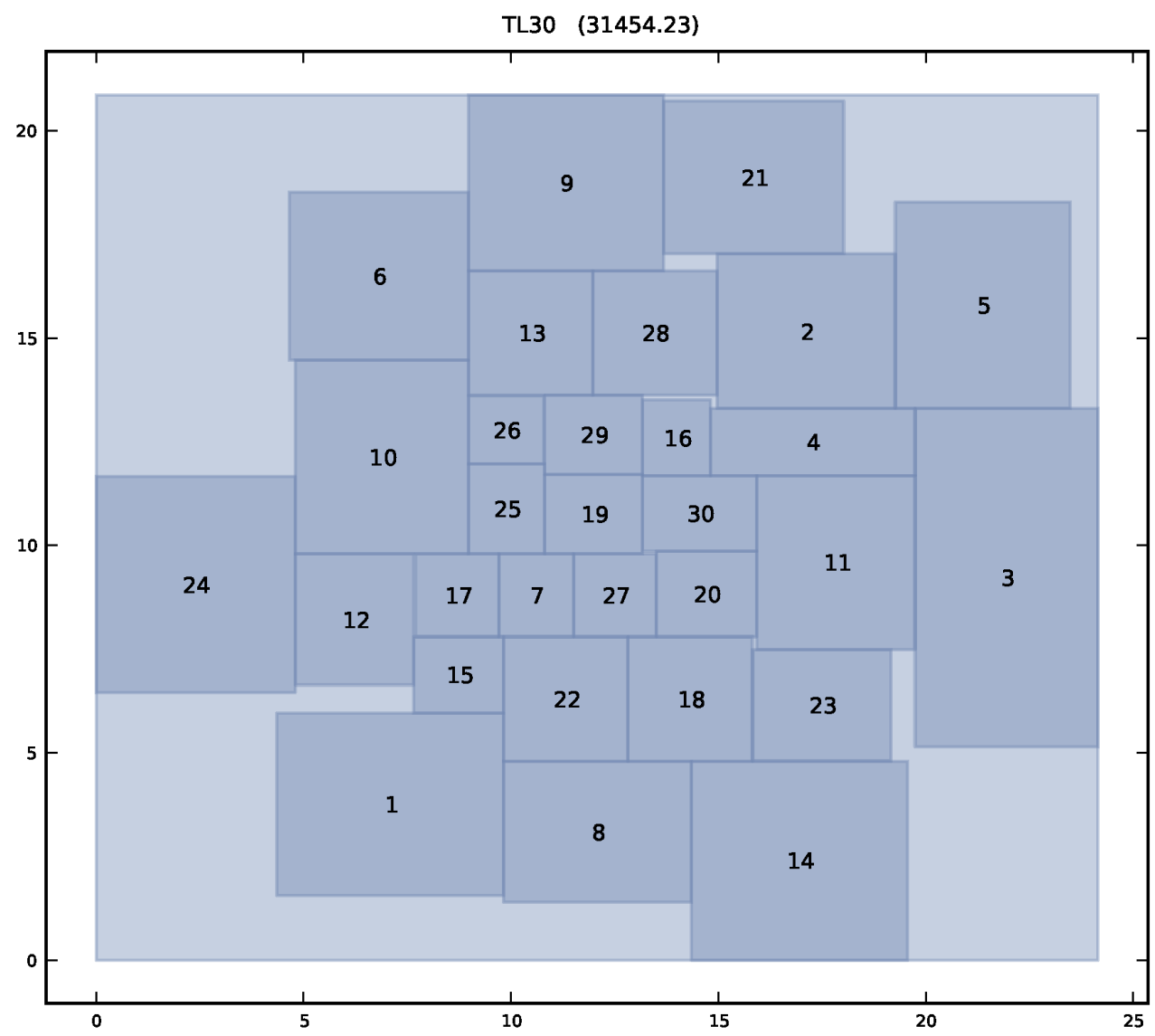
Previous best known
Solution: 3054.2
TSaST (Scholtz et al., 2009)



TL20

New best known
Solution: 9892.4

Previous best known
Solution: 13190.0
GA-TSG (Schnecke and Vornberger, 1997)



TL30

New best known
Solution: 31454.2

Previous best known
Solution: 33721.5
TSaST (Scholtz et al., 2009)

Redundant content distribution

Reference:



L. Breslau, I. Diakonikolas, N. Duffield, Y. Gu, M. Hajiaghayi, D.S. Johnson, H. Karloff, M.G.C.R., and S. Sen, "Disjoint-path facility location: Theory and practice," Proceedings of the Thirteenth Workshop on Algorithm Engineering and Experiments (ALENEX11), SIAM, San Francisco, pp. 60–74, January 22, 2011

Tech report version:

http://mauricio.resende.info/doc/monitoring-alenex.pdf

Redundant content distribution (RCD)

- Suppose a number of users located at nodes in a network demand content.

Redundant content distribution

- Suppose a number of users located at nodes in a network demand content.
- Copies of content are stored throughout the network in data warehouses.

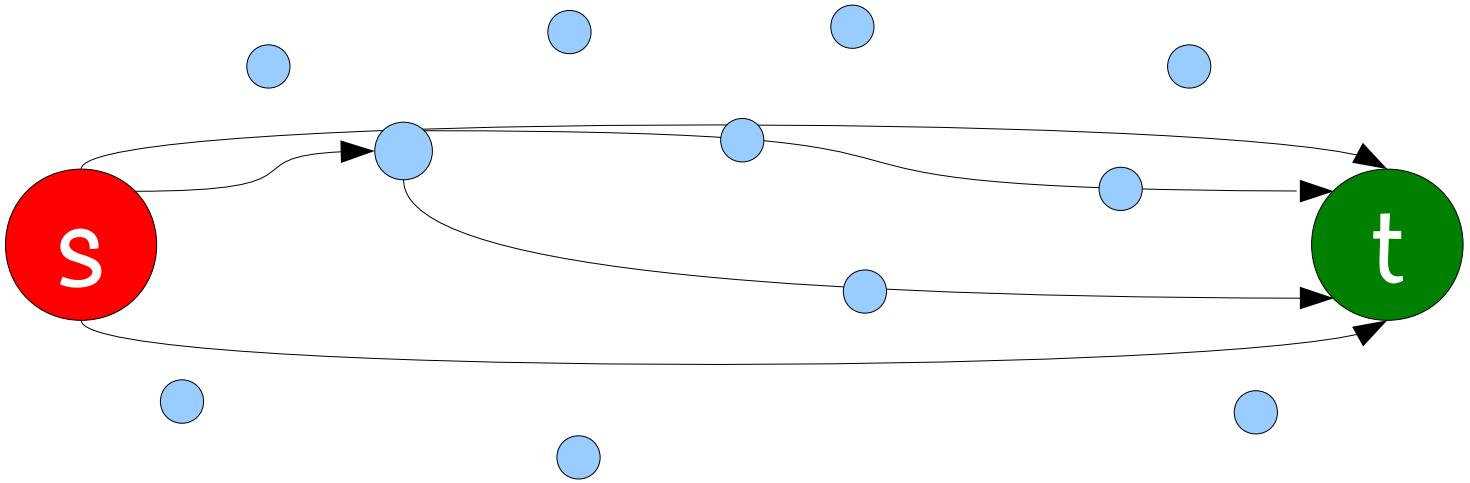
Redundant content distribution

- Suppose a number of users located at nodes in a network demand content.
- Copies of content are stored throughout the network in data warehouses.
- Content is sent from data warehouse to user on routes determined by OSPF.

Redundant content distribution

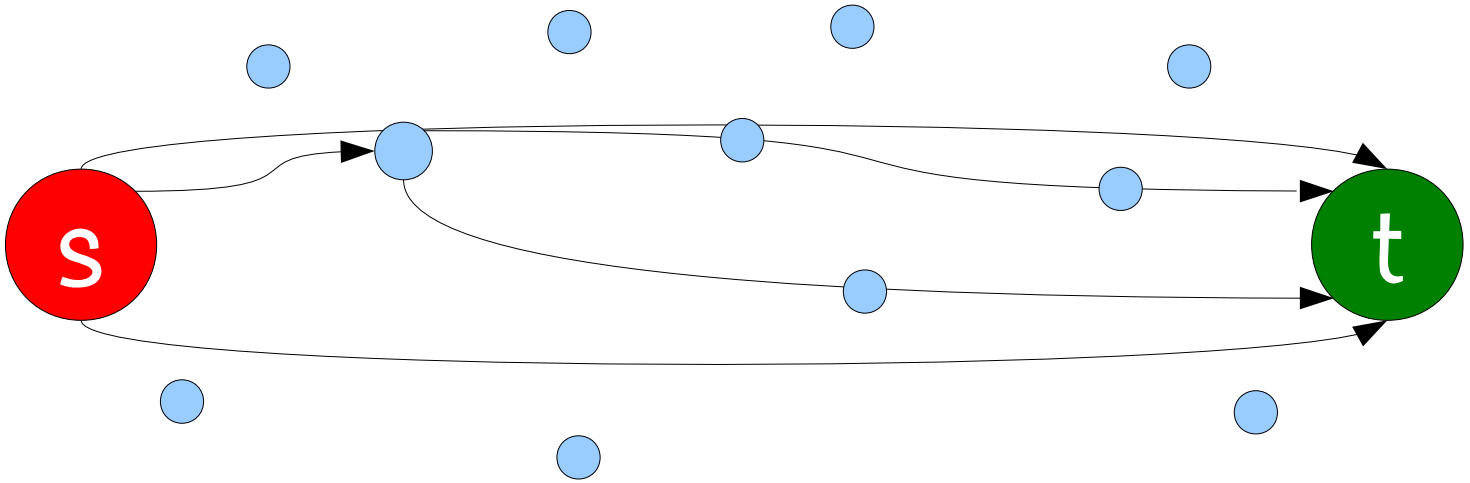
- Suppose a number of users located at nodes in a network demand content.
- Copies of content are stored throughout the network in data warehouses.
- Content is sent from data warehouse to user on routes determined by OSPF.
- Problem: Locate minimum number of warehouses in network such all users get their content even in presence of edge failures.

Redundant content distribution



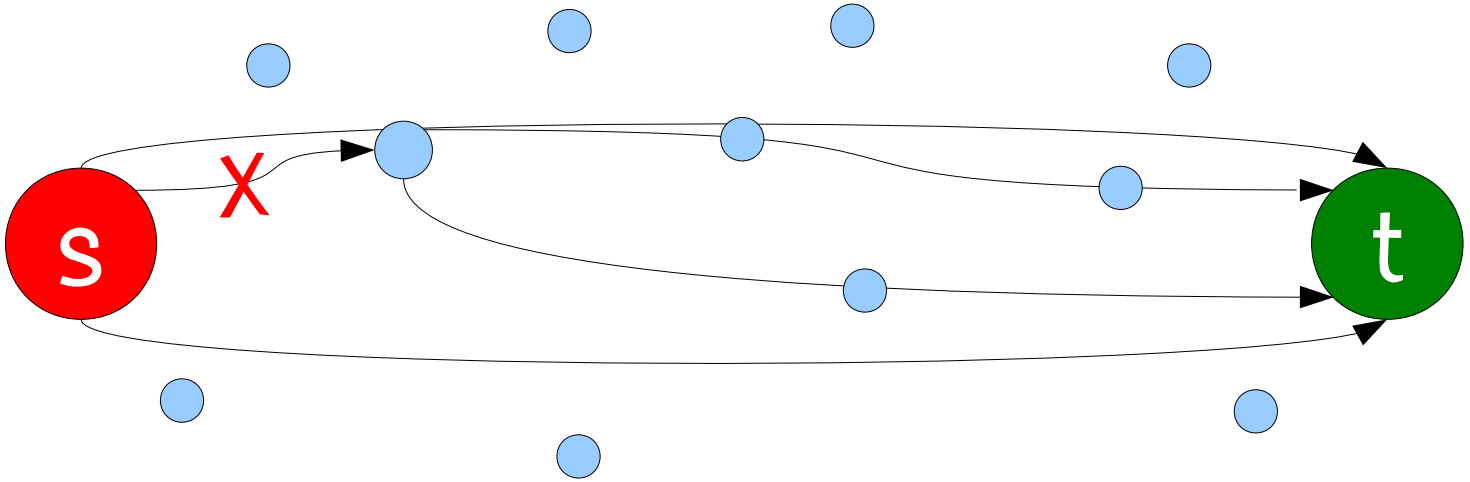
Traffic from node s to node t flows on paths defined by OSPF.

Redundant content distribution

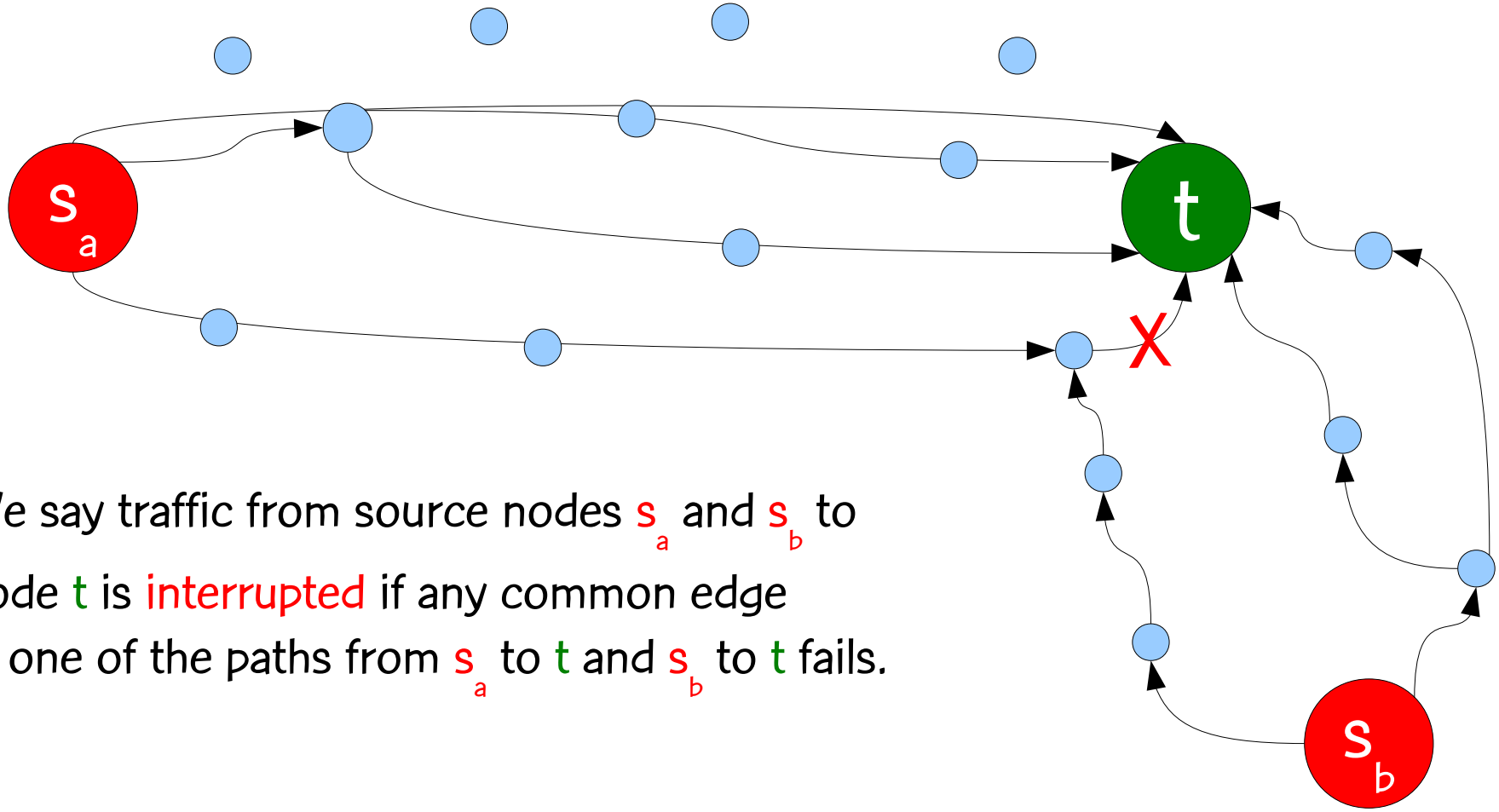


We don't know on which path a particular packet will flow.

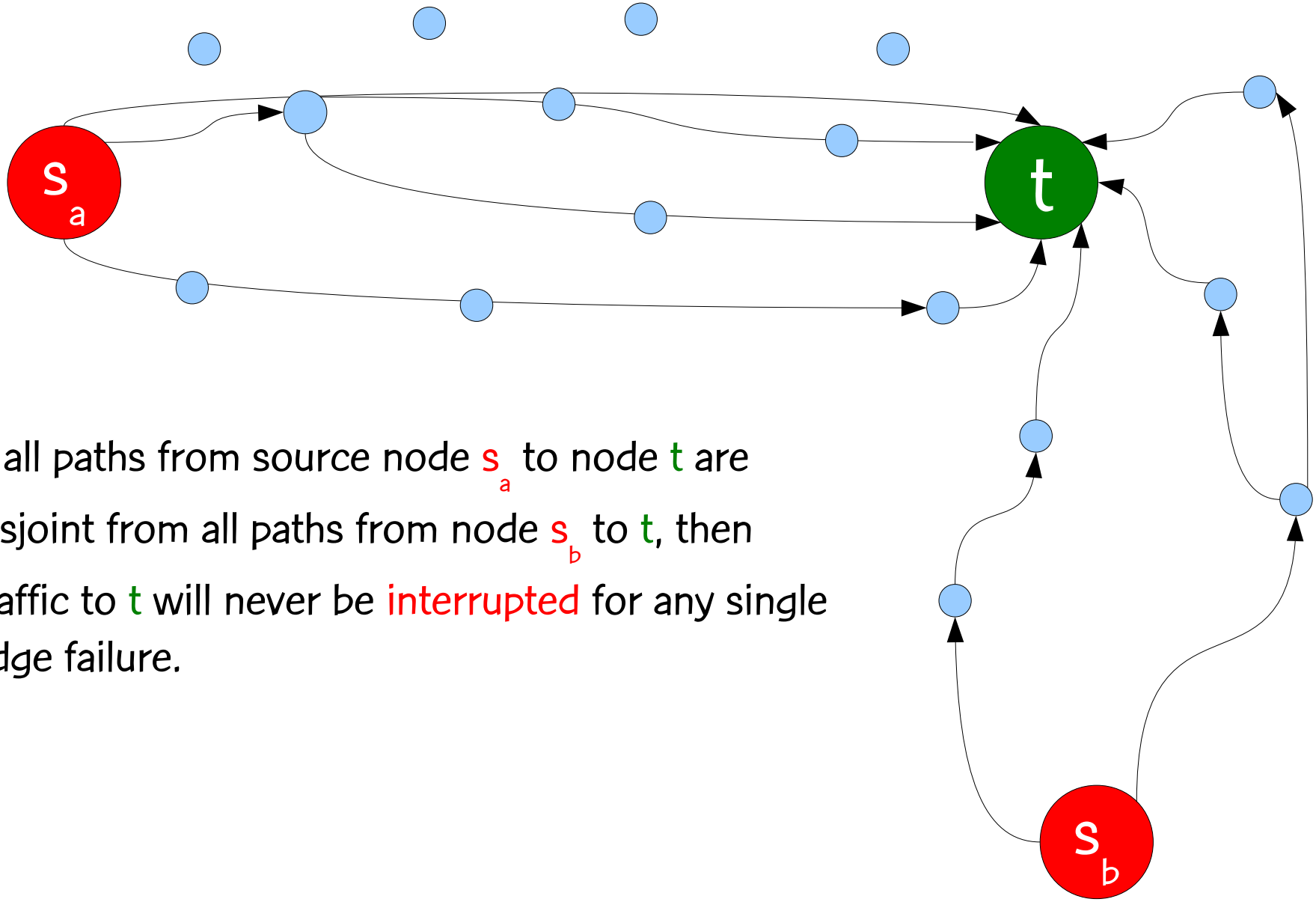
Redundant content distribution



We say traffic from node s to node t is interrupted if any edge in one of the paths from s to t fails.



We say traffic from source nodes s_a and s_b to node t is interrupted if any common edge in one of the paths from s_a to t and s_b to t fails.



If all paths from source node s_a to node t are disjoint from all paths from node s_b to t, then traffic to t will never be interrupted for any single edge failure.

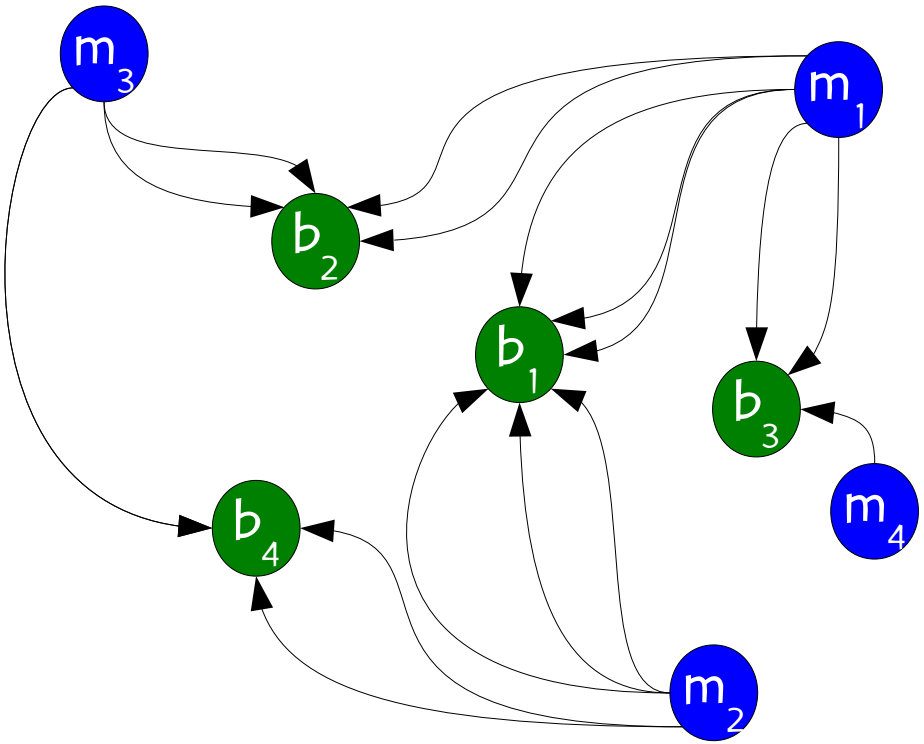
Redundant content distribution

Suppose nodes b_1, b_2, ... want some content (e.g. video).

We want the smallest set **S** of servers such that:

for every b_i there exist m_1, m_2 ! **S** both of which can provide content to b_i

and all paths $m_1 \rightarrow b$ are disjoint with all paths $m_2 \rightarrow b$



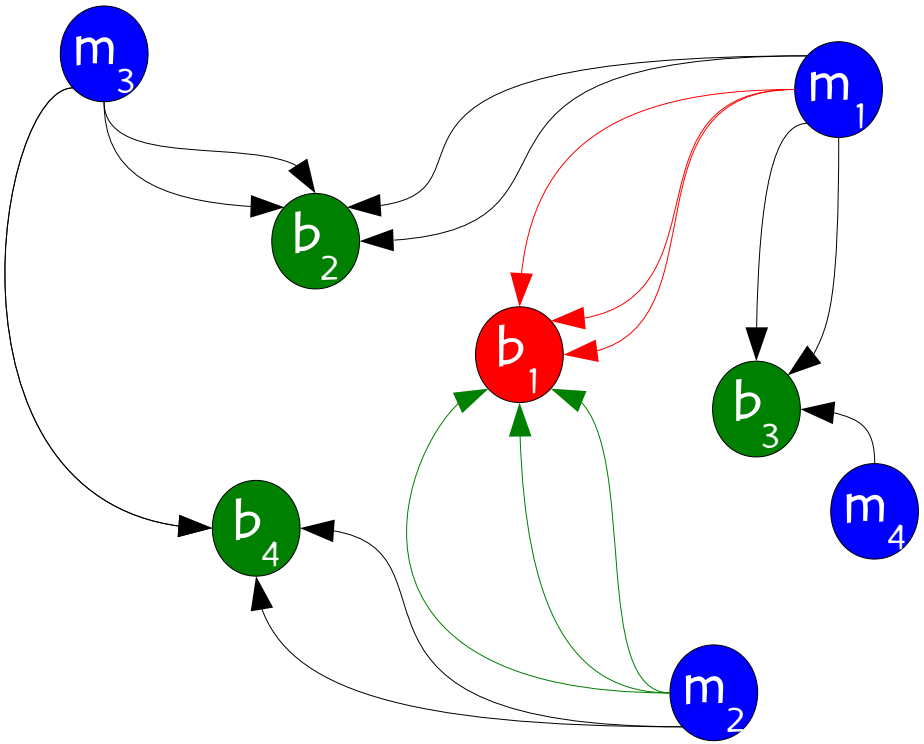
Redundant content distribution

Suppose nodes b_1, b_2, ... want some content (e.g. video).

We want the smallest set **S** of servers such that:

for every b_i there exist m_1, m_2 ! **S** both of which can provide content to b_i

and all paths $m_1 \rightarrow b$ are disjoint with all paths $m_2 \rightarrow b$



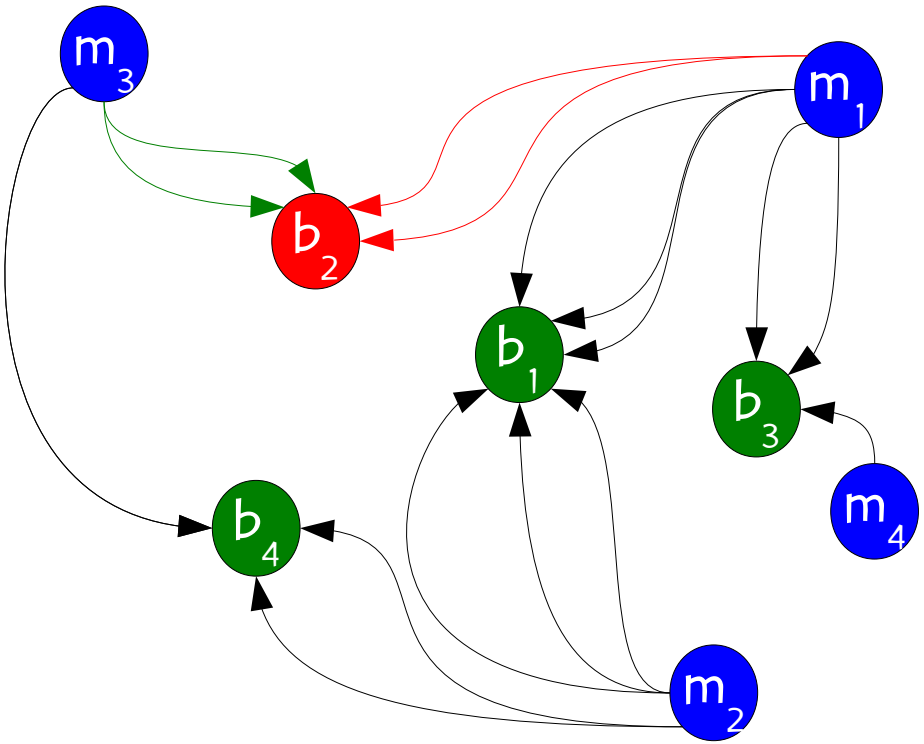
Redundant content distribution

Suppose nodes b_1, b_2, ... want some content (e.g. video).

We want the smallest set **S** of servers such that:

for every b_i there exist m_1, m_2 ! **S** both of which can provide content to b_i

and all paths $m_1 \rightarrow b$ are disjoint with all paths $m_2 \rightarrow b$



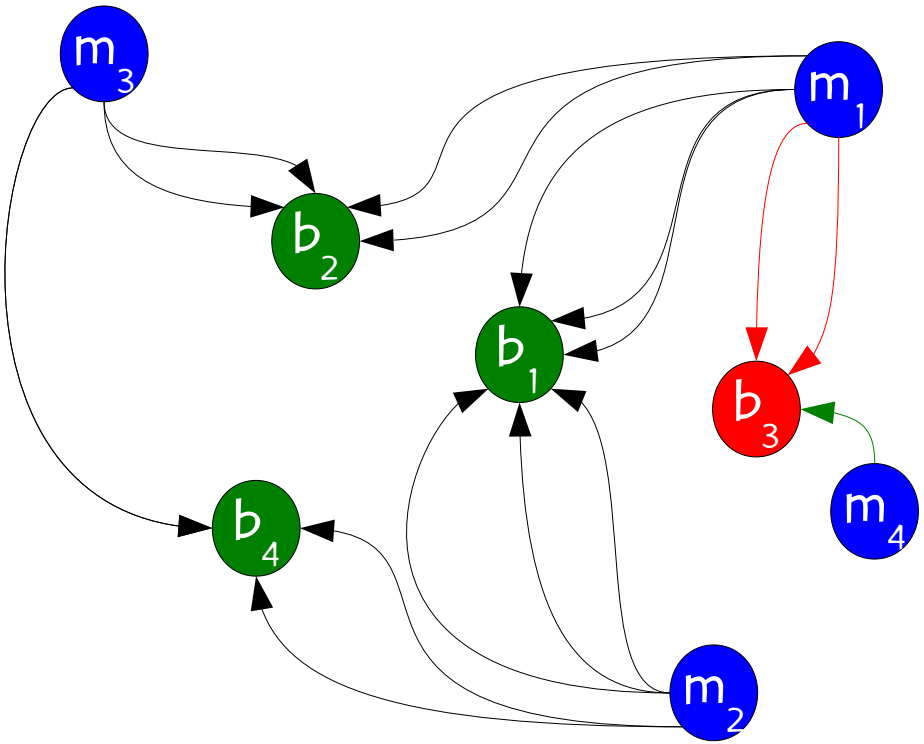
Redundant content distribution

Suppose nodes b_1, b_2, ... want some content (e.g. video).

We want the smallest set **S** of servers such that:

for every b_i there exist m_1, m_2 ! **S** both of which can provide content to b_i

and all paths $m_1 \rightarrow b$ are disjoint with all paths $m_2 \rightarrow b$



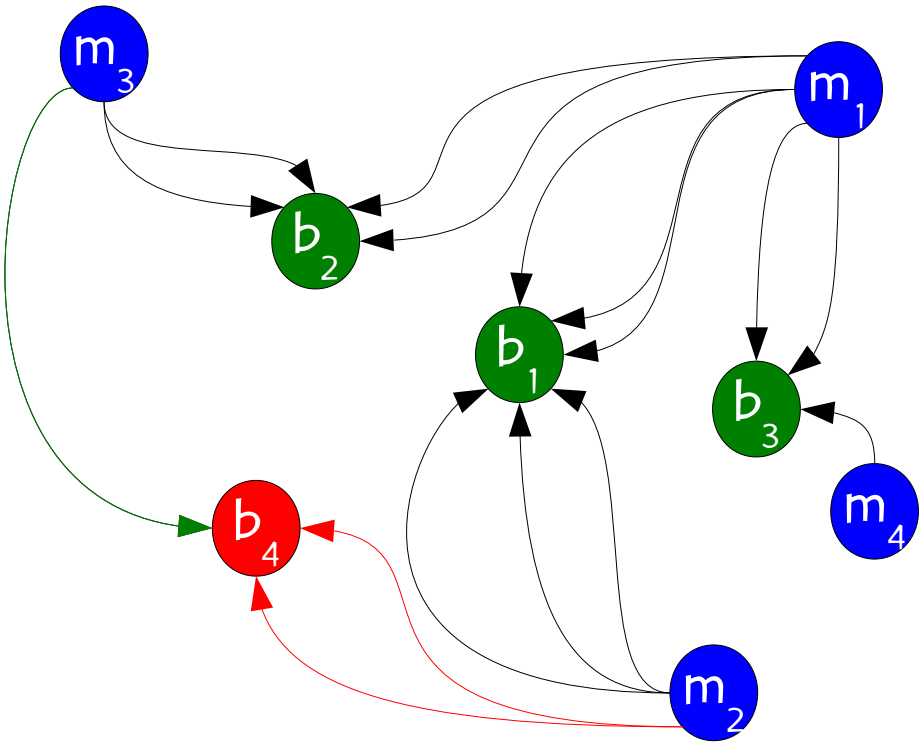
Redundant content distribution

Suppose nodes b_1, b_2, ... want some content (e.g. video).

We want the smallest set **S** of servers such that:

for every b_i there exist m_1, m_2 ! **S** both of which can provide content to b_i

and all paths $m_1 \rightarrow b$ are disjoint with all paths $m_2 \rightarrow b$



Redundant content distribution

- Given:
  - A directed network $G = (V, E)$;
  - A set of nodes $B \subseteq V$ where content-demanding users are located;
  - A set of nodes $M \subseteq V$ where content warehouses can be located;
  - The set of all OSPF paths from m to b, for $m \in M$ and $b \in B$.

Redundant content distribution

- Compute:
  - The set of triples $\{ m_1, m_2, b \}^i$, $i = 1, 2, \ldots, T$, such that all paths from m_1 to b and from m_2 to b are disjoint, where $m_1, m_2 \in M$ and $b \in B$.
  - Note that if $B \cap M \neq \emptyset$, then some triples will be of the type $\{ b, b, b \}$, where $b \in B \cap M$, i.e. a data warehouse that is co-located with a user can provide content to the user by itself.

Redundant content distribution

- Solve the covering by pairs problem:
  - Find a smallest-cardinality set $M^* \subseteq M$ such that for all $b \in B$, there exists a triple $\{ m_1, m_2, b \}$ in the set of triples such that $m_1, m_2 \in M^*$.

Greedy algorithm for covering by pairs

- initialize partial cover $M^* = \{\}$

Greedy algorithm for covering by pairs

- initialize partial cover $M^* = \{\}$
- while M^* is not a cover do:

Greedy algorithm for covering by pairs

- initialize partial cover $M^* = \{\}$
- while M^* is not a cover do:
  - find $m \in M \setminus M^*$ such that $M^* \cup \{m\}$ covers a maximum number of additional user nodes (break ties by vertex index) and set $M^* = M^* \cup \{m\}$

Greedy algorithm for covering by pairs

- initialize partial cover $M^* = \{\}$
- while M^* is not a cover do:
  - find $m \in M \setminus M^*$ such that $M^* \cup \{m\}$ covers a maximum number of additional user nodes (break ties by vertex index) and set $M^* = M^* \cup \{m\}$
  - if no $m \in M \setminus M^*$ yields an increase in coverage, then choose a pair $\{m_1, m_2\} \in M \setminus M^*$ that yields a maximum increase in coverage and set $M^* = M^* \cup \{m_1\} \cup \{m_2\}$

Greedy algorithm for covering by pairs

- initialize partial cover $M^* = \{\}$
- while M^* is not a cover do:
  - find $m \in M \setminus M^*$ such that $M^* \cup \{m\}$ covers a maximum number of additional user nodes (break ties by vertex index) and set $M^* = M^* \cup \{m\}$
  - if no $m \in M \setminus M^*$ yields an increase in coverage, then choose a pair $\{m_1, m_2\} \in M \setminus M^*$ that yields a maximum increase in coverage and set $M^* = M^* \cup \{m_1\} \cup \{m_2\}$
  - if no pair exists, then the problem is infeasible

BRKGA for redundant content distribution

BRKGA for the RCD problem

- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.

BRKGA for the RCD problem

- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.
- Decoder:

BRKGA for the RCD problem

- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.
- Decoder:
  - For i = 1, ..., N: if X(i) > ½ , add i-th data warehouse node to solution

BRKGA for the RCD problem

- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.
- Decoder:
  - For i = 1, ..., N: if X(i) > ½ , add i-th data warehouse node to solution
  - If solution is feasible, i.e. all users are covered: STOP

BRKGA for the RCD problem

- Encoding:
  - A vector X of N keys randomly generated in the real interval (0,1], where N = |M| is the number of potential data warehouse nodes. The i-th random key corresponds to the i-th potential data warehouse node.
- Decoder:
  - For i = 1, ..., N: if X(i) > ½ , add i-th data warehouse node to solution
  - If solution is feasible, i.e. all users are covered: STOP
  - Else, apply greedy algorithm to cover uncovered user nodes.

BRKGA for the RCD problem

- Size of population: N (number of monitoring nodes)
- Size of elite set: 15% of N
- Size of mutant set: 10% of N
- Biased coin probability: 70%
- Stop after N generations without improvement of best found solution

Another application: Host placement for end-to-end monitoring

- Internet service provider (ISP) delivers virtual private network (VPN) service to customers.

Another application: Host placement for end-to-end monitoring

- Internet service provider (ISP) delivers virtual private network (VPN) service to customers.
- The ISP agrees to send traffic between locations specified by the customer and promises to provide certain level of service on the connections.

Another application: Host placement for end-to-end monitoring

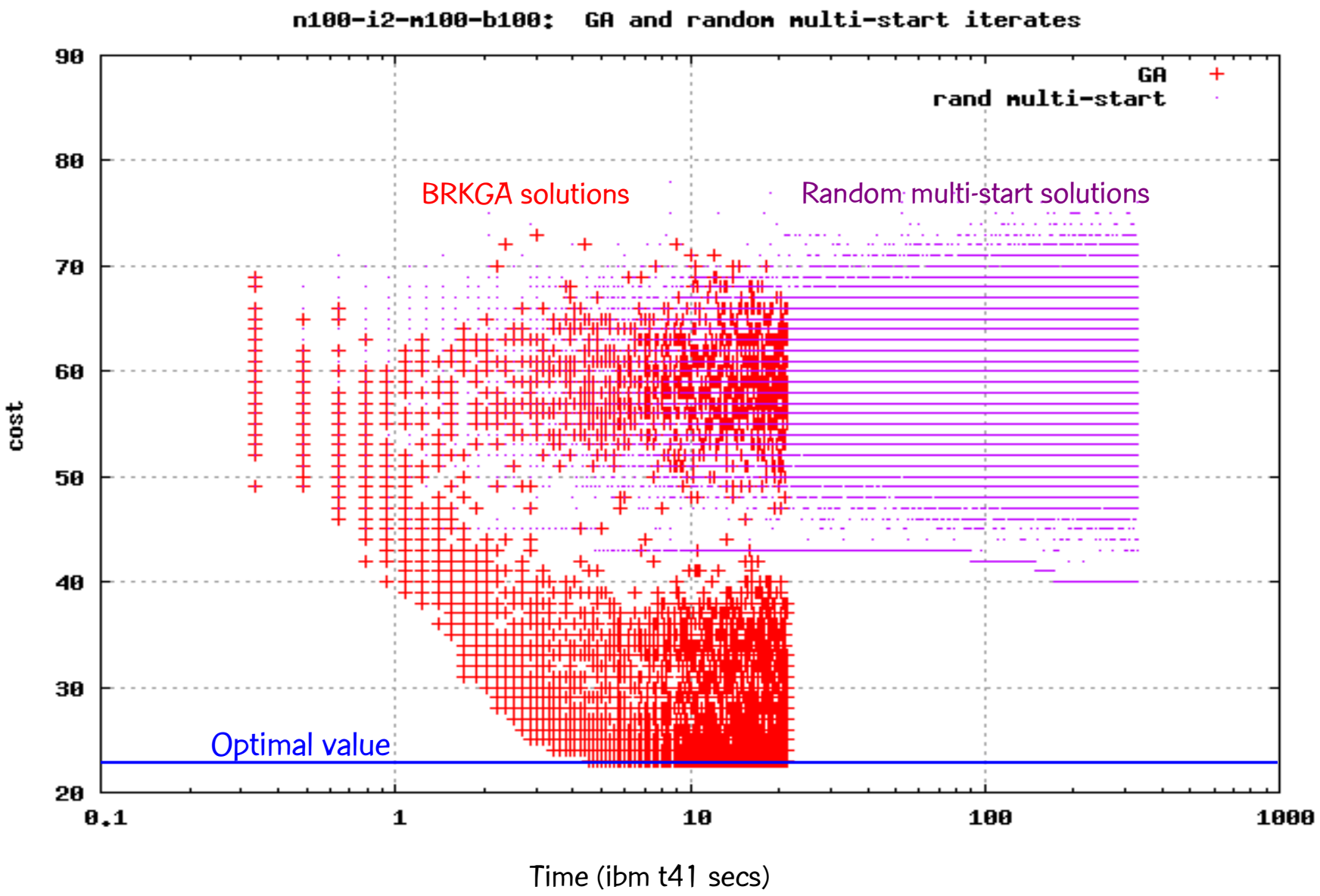
- Internet service provider (ISP) delivers virtual private network (VPN) service to customers.
- The ISP agrees to send traffic between locations specified by the customer and promises to provide certain level of service on the connections.
- A key service quality metric is packet loss rate.

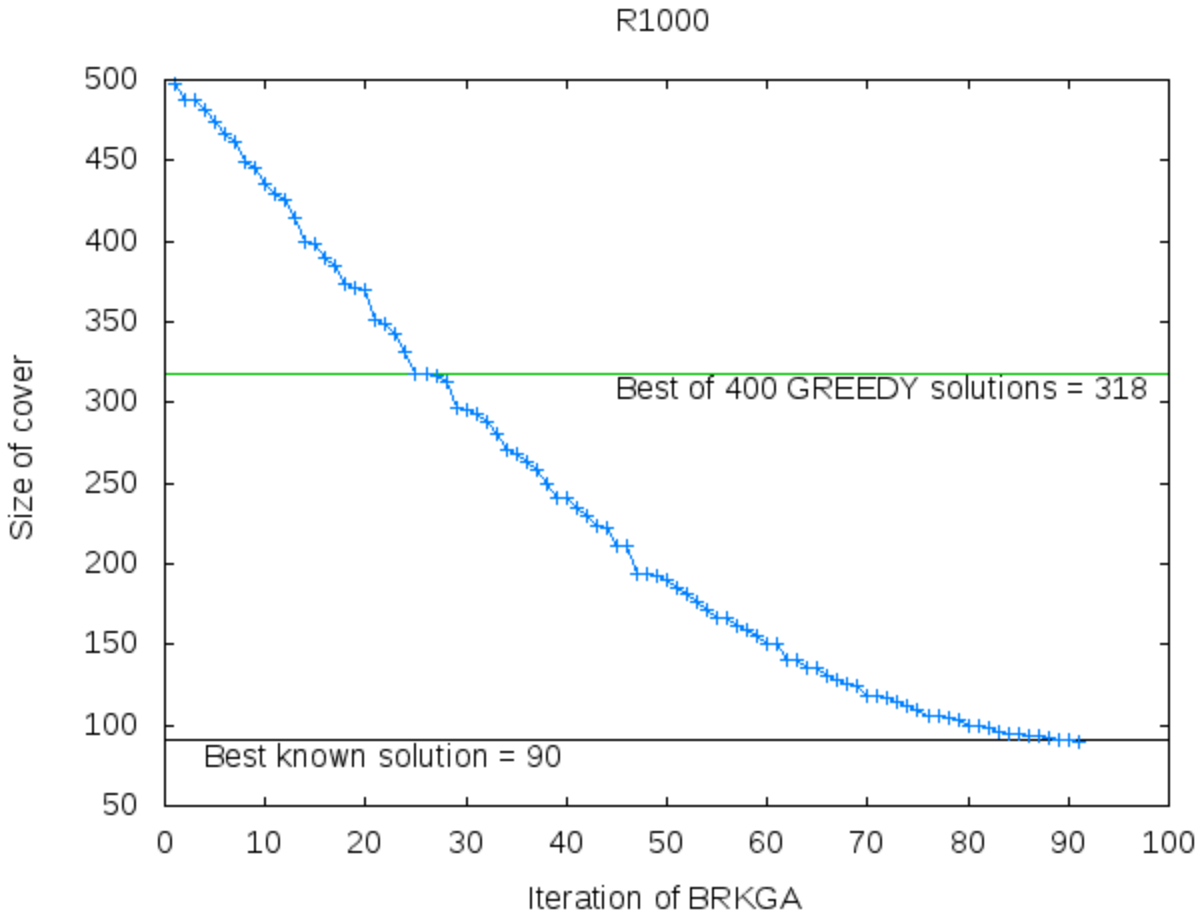
Another application: Host placement for end-to-end monitoring

- Internet service provider (ISP) delivers virtual private network (VPN) service to customers.
- The ISP agrees to send traffic between locations specified by the customer and promises to provide certain level of service on the connections.
- A key service quality metric is packet loss rate.
- We want to minimize the number of monitoring equipment placed in the network to measure packet loss rate: This is a type of covering by pairs problem.

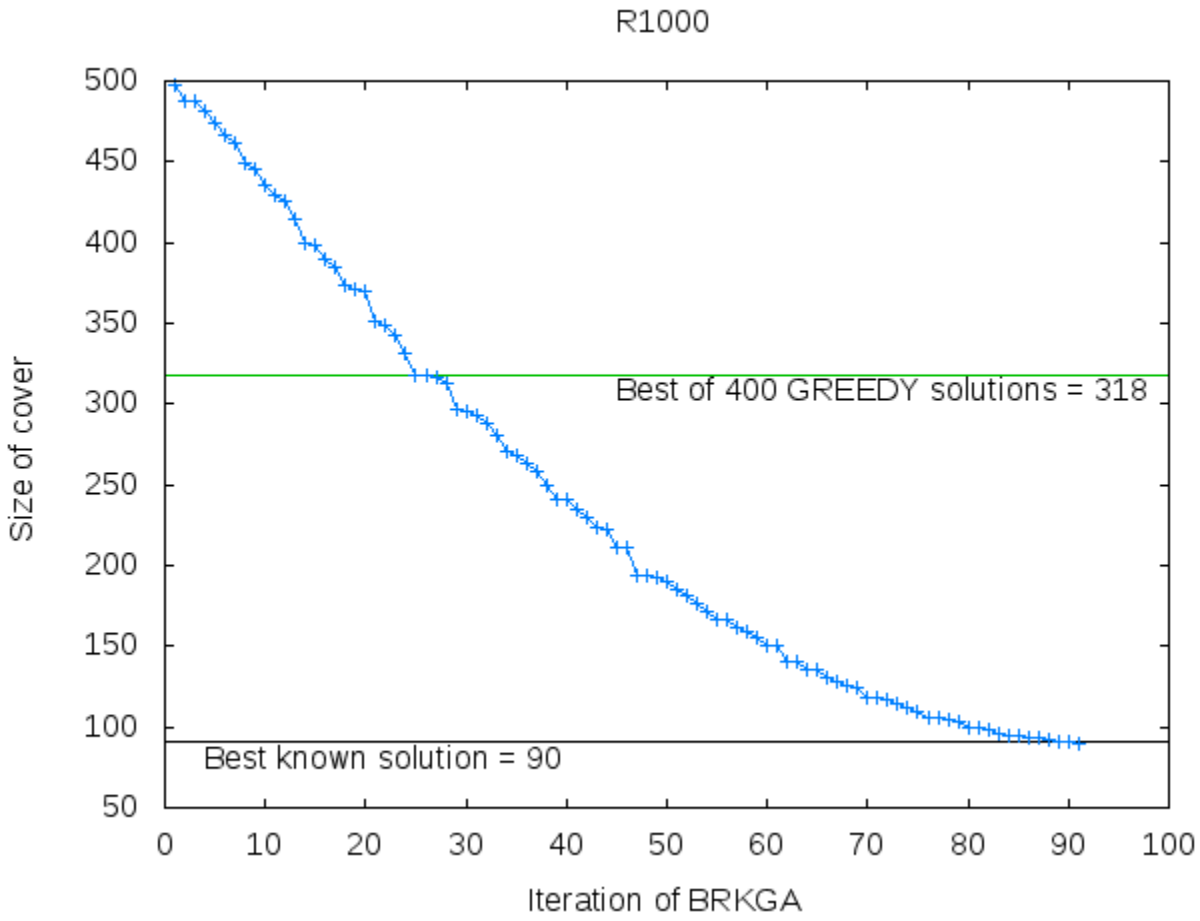
n100-i2-m100-b100 (opt = 23)

solution





Real-world instance derived from a proprietary Tier-1 Internet Service Provider (ISP) backbone network using OSPF for routing.



Size of network: about 1000 nodes, where almost all can store content and about 90% have content-demanding users. Over 45 million triples.

C-GRASP

C-GRASP is a metaheuristic for finding optimal or near-optimal solutions to

Min f(x) subject to: $L \leq x \leq U$

where $x, L, U \in R^n$

and f(x) is continuous but can have discontinuities, be non-differentiable, be the output of a simulation, be the output of a decoder, etc.

C-GRASP

- C-GRASP is based on the discrete optimization metaheuristic GRASP
- It was proposed in 2006 by U. of FL PhD students Michael Hirsch and Claudio Meneses with Mauricio Resende and Panos Pardalos.

- M.J. Hirsch, C.N. Meneses, P.M. Pardalos, and M.G.C. Resende, "Global optimization by continuous GRASP," Optimization Letters, vol. 1, pp. 201-212, 2007.
- M.J. Hirsch, P.M. Pardalos, and M.G.C. Resende, "Speeding up continuous GRASP," European J. of Operational Research, vol. 205, pp. 507-521, 2010.

C-GRASP

- C-GRASP is a multi-start procedure, i.e. a major loop is repeated until some stopping criterion is satisfied.
- In each major iteration

➢ x is initialized with a solution randomly selected from the box defined by vectors L and U.

➢ Several minor iterations are carried out, where each minor iteration consists of a construction phase and a local improvement phase.

➢ Minor iterations are done on a dynamic grid and stop when the grid is too dense.

RK-GRASP

RK-GRASP, the GRASP random-key optimizer, is a C-GRASP, where

$L=(0,0,\ldots,0)$ and $U=(1,1,\ldots,1)$ ⟸ Unit *n*-dim hypercube

and function evaluations of $x \in \mathcal{H}_n$ are computed via a decoder $\mathcal{D}$ with objective function value $f(\mathcal{D}(x))$.

In addition to the standard grid local search, we propose two other local search heuristics to use in the local improvement phase of RK-GRASP.

RK-GRASP

f* = ∞

while (stopping criterion not satisfied) **do**

x = random[$\mathcal{H}_n$]; h = h(start);

while (h ≥ h(end)) **do**

x = ConstructGreedyRandomized(x)

x = LocalImprovement(x)

if (f($\mathcal{D}$(x)) < f*) **then** { x* = x; f* = f($\mathcal{D}$(x)) }

if (x did not improve this iteration) **then** { h = h/2 }

end while

end while

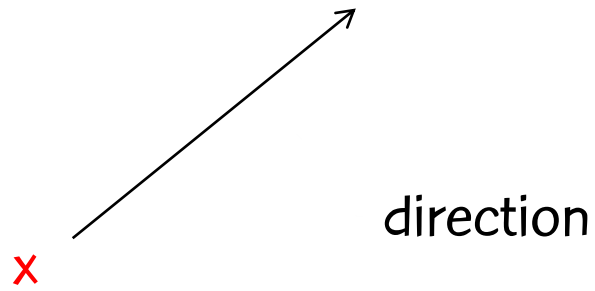
return x*

RK-GRASP line search

x

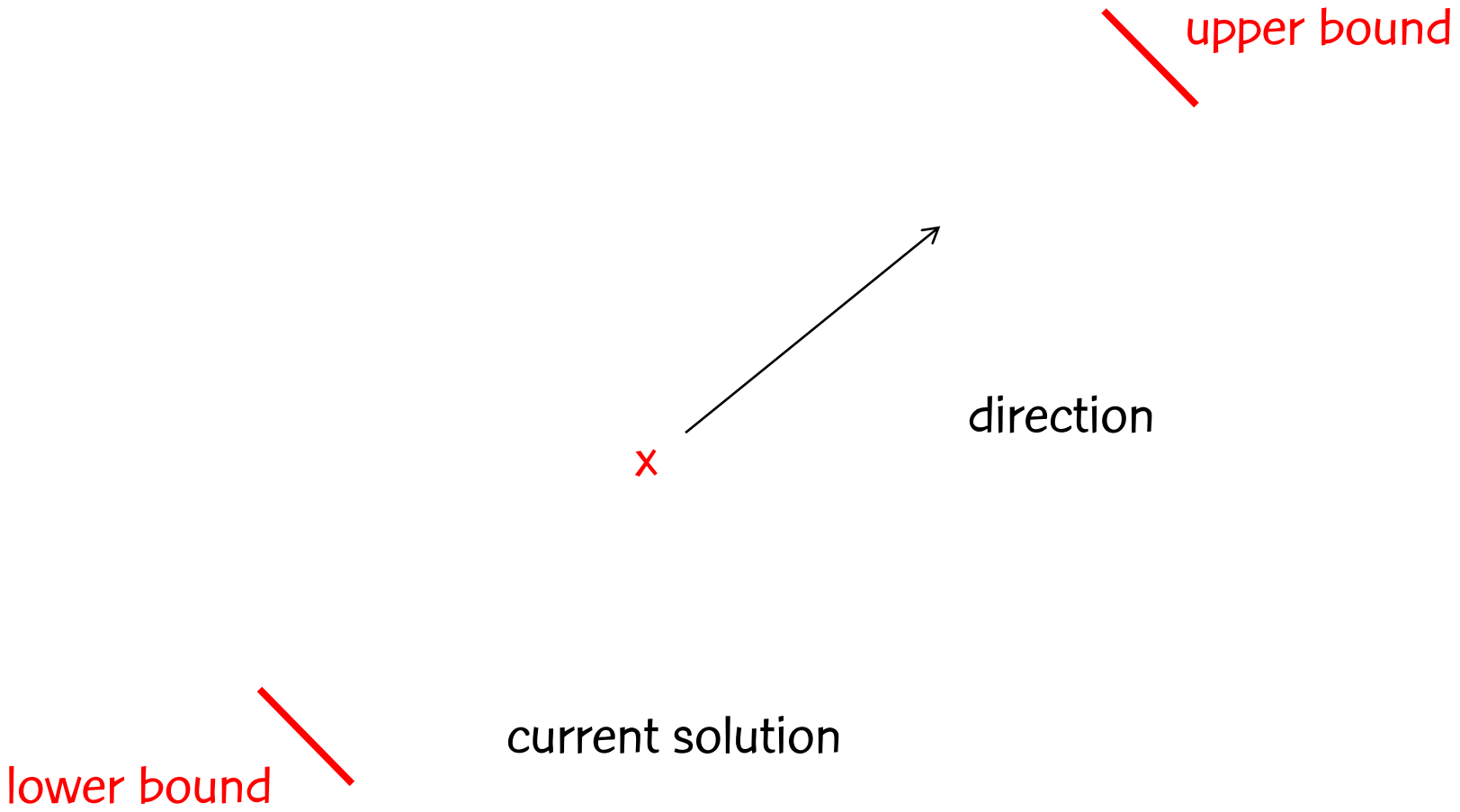
current solution

C-GRASP line search

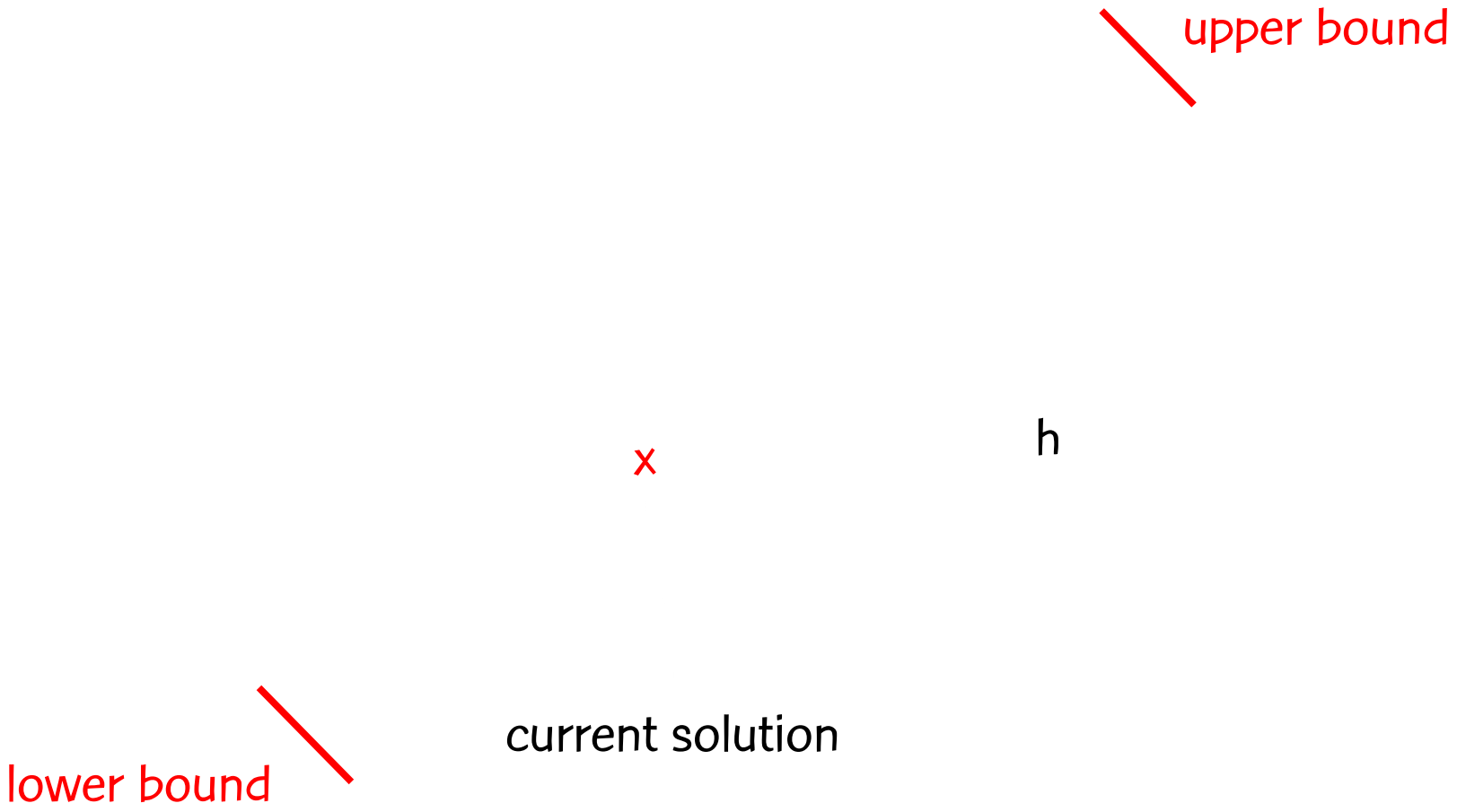


current solution

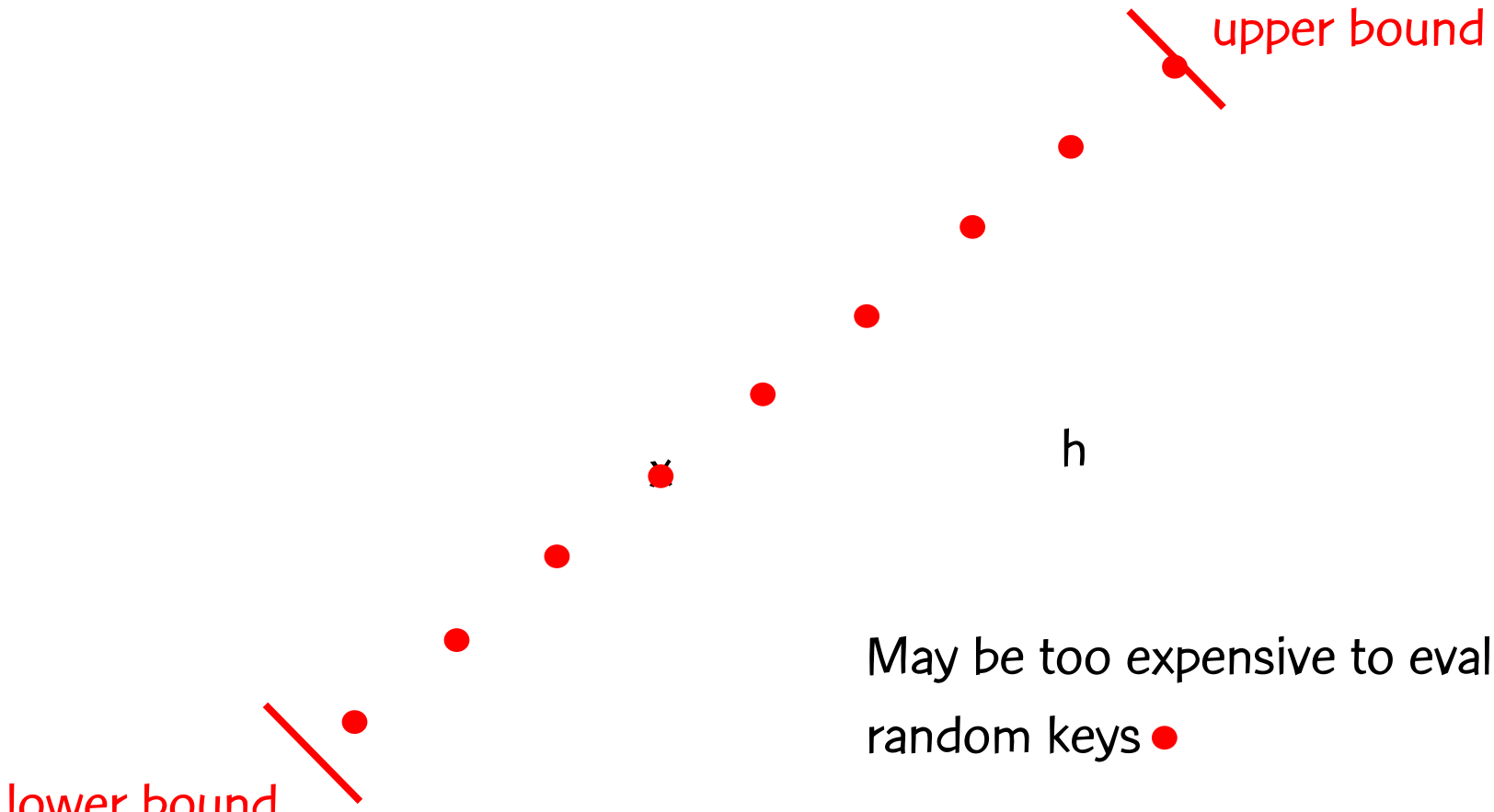
RK-GRASP line search



RK-GRASP line search



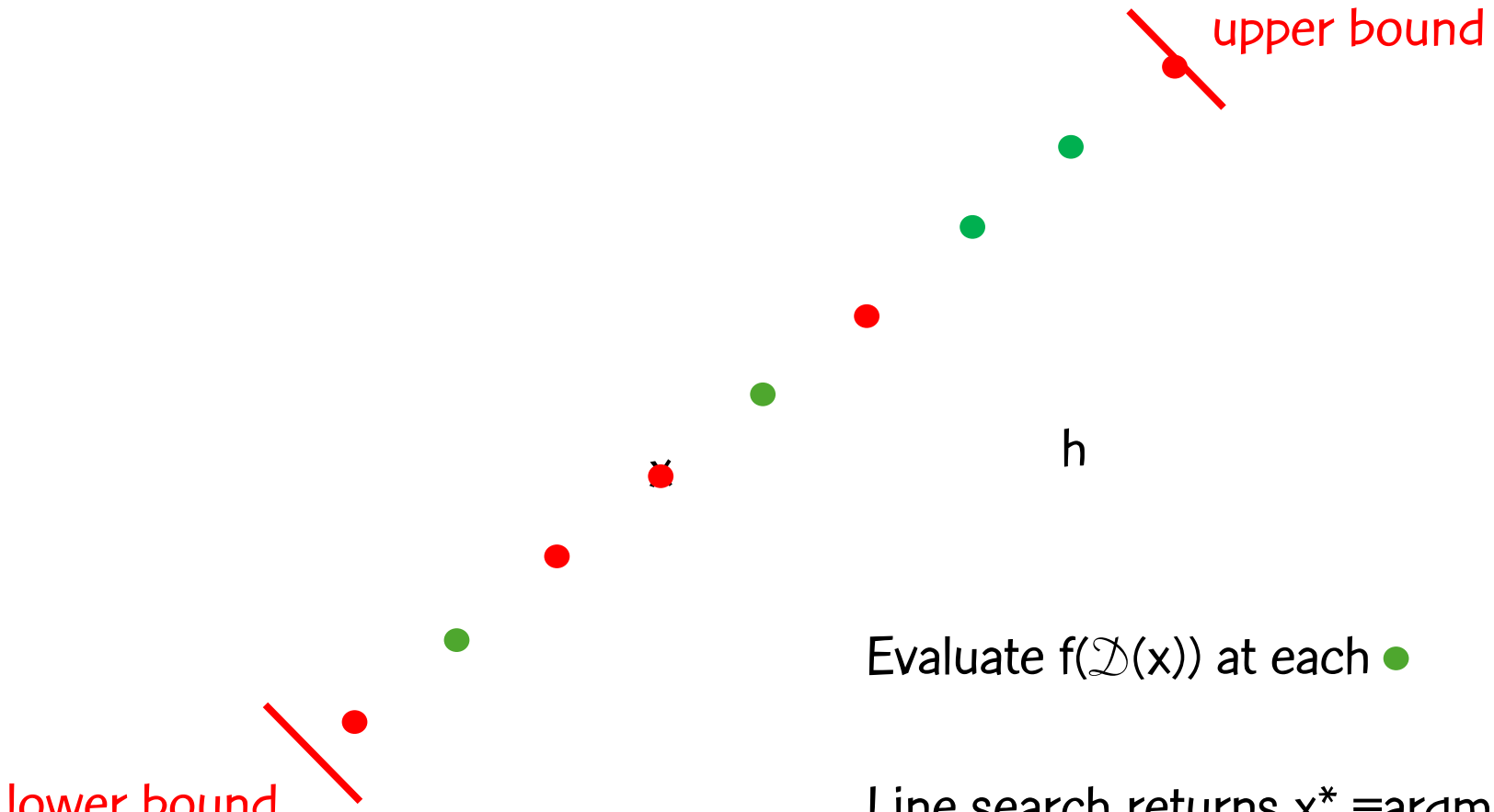
RK-GRASP line search



May be too expensive to evaluate $f(\mathcal{D}(x))$ at all random keys ●

Instead, sample a subset $\mathcal{K}_L$ of these random keys

RK-GRASP line search



Evaluate f($\mathcal{D}$(x)) at each ●

Line search returns x* =argmin{ f($\mathcal{D}$(x)): x ∈ $\mathcal{K}_L$ }

RK-GRASP greedy randomized construction

unset = {1, 2, 3, …, n }; x = x^0 i-th component

for (k = 1, 2, …, n) **do**

  for (all i ∈ unset) **do**

    z_i = line search in direction e_i = (0,0,…,1,….,0)

  end for

  RCL = { i ∈ unset | $f(z_i) < CUTOFF$ }

  Select at random i* ∈ RCL

  Set $x_{i*} = z_{i*}$; unset = unset \ {i*}

end for

C-GRASP local improvement

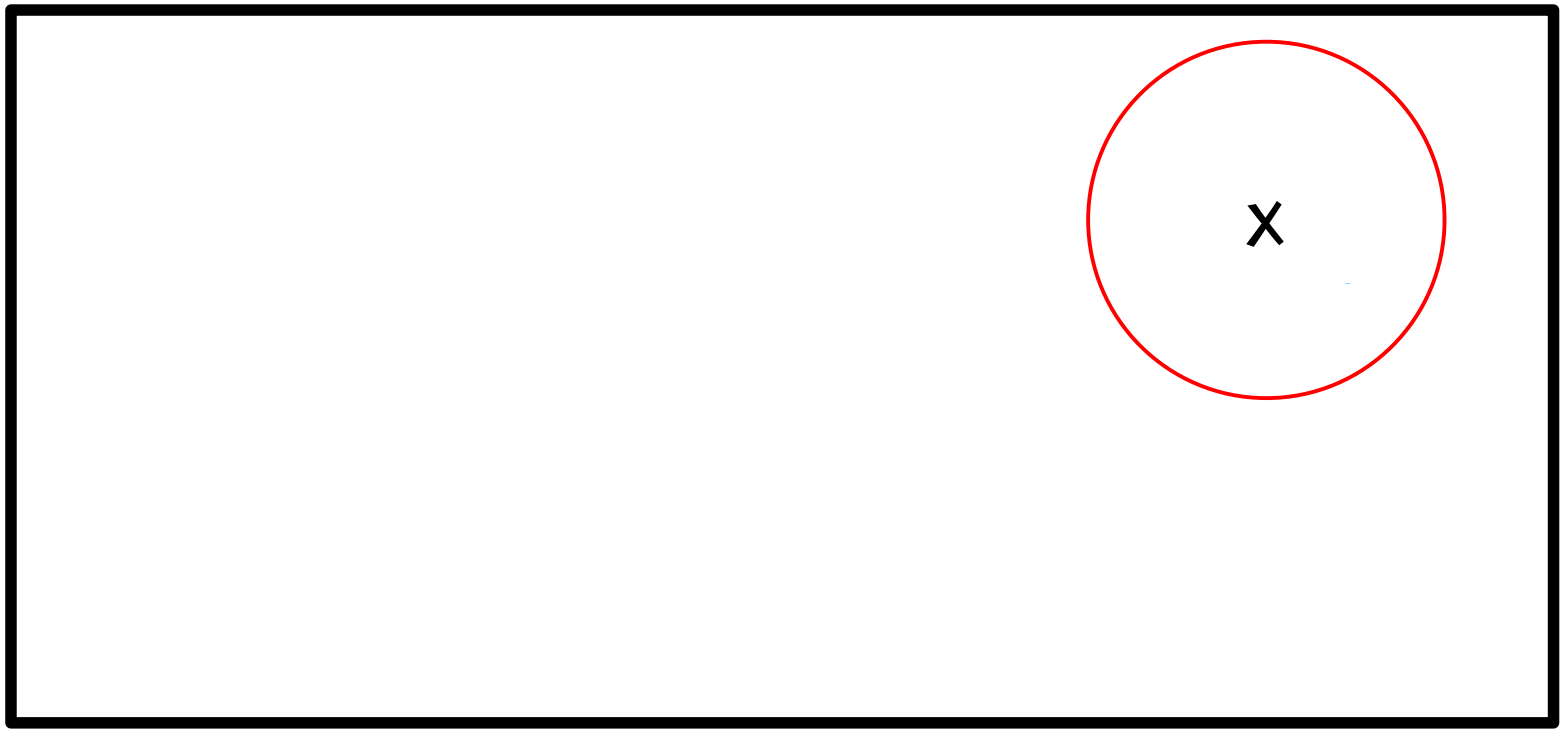
We propose three problem-independent local improvement algorithms for C-GRASP which we use to implement RK-GRASP:

1. Grid search
2. Nelder-Mead search with uniform crossover moves
3. Random Variable Neighborhood Descent

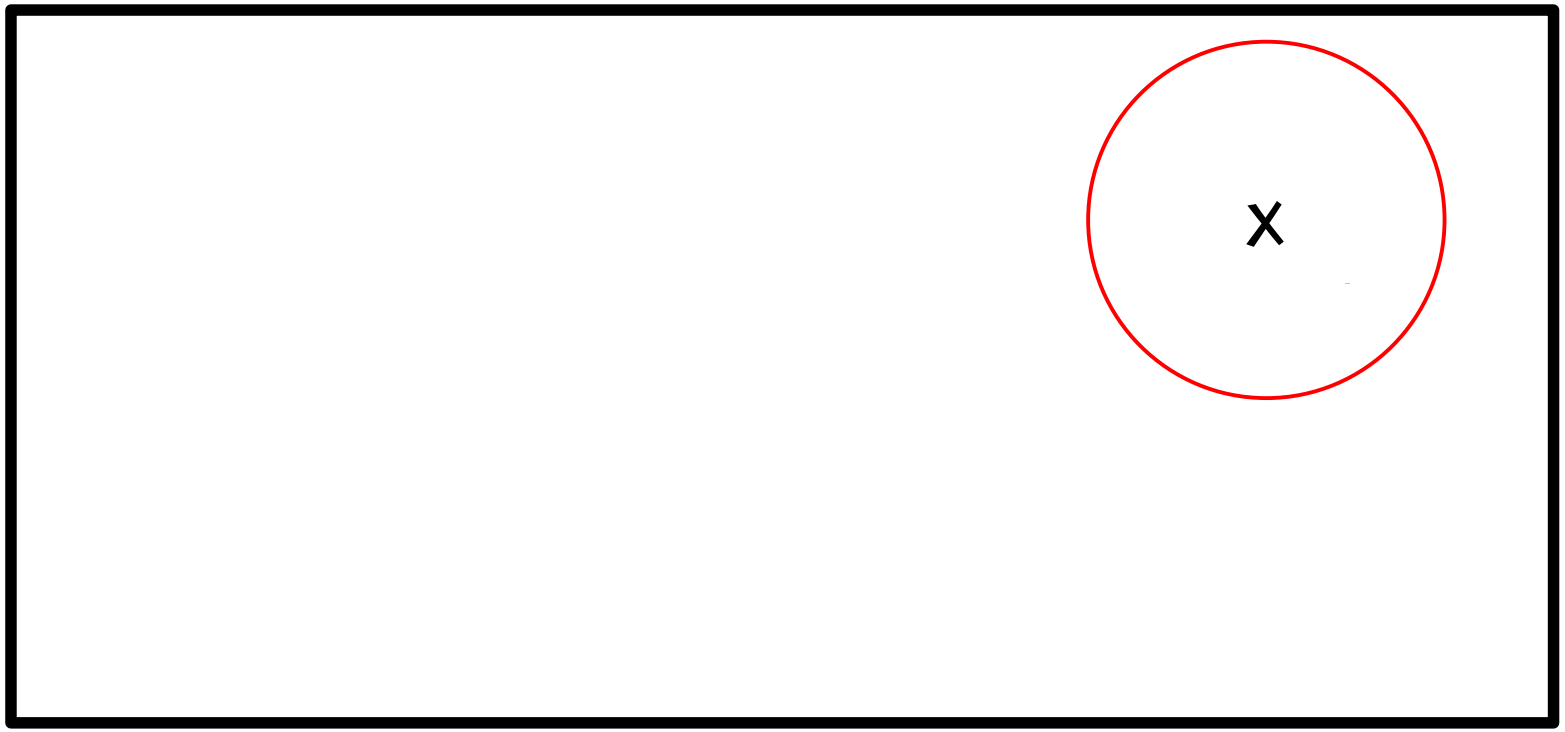
C-GRASP local improvement (grid search)



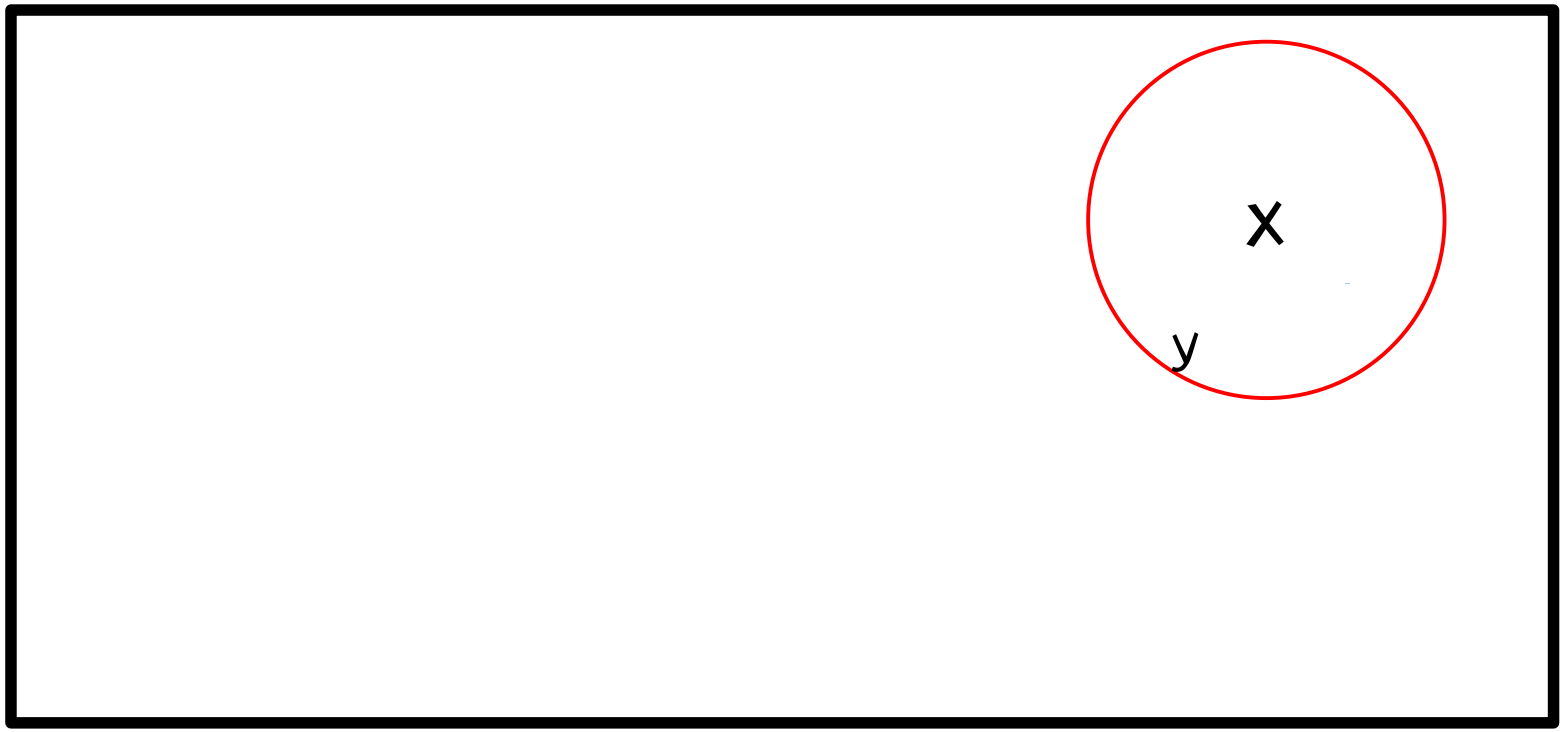
RK-GRASP local improvement (grid search)



RK-GRASP local improvement (grid search)



RK-GRASP local improvement (grid search)

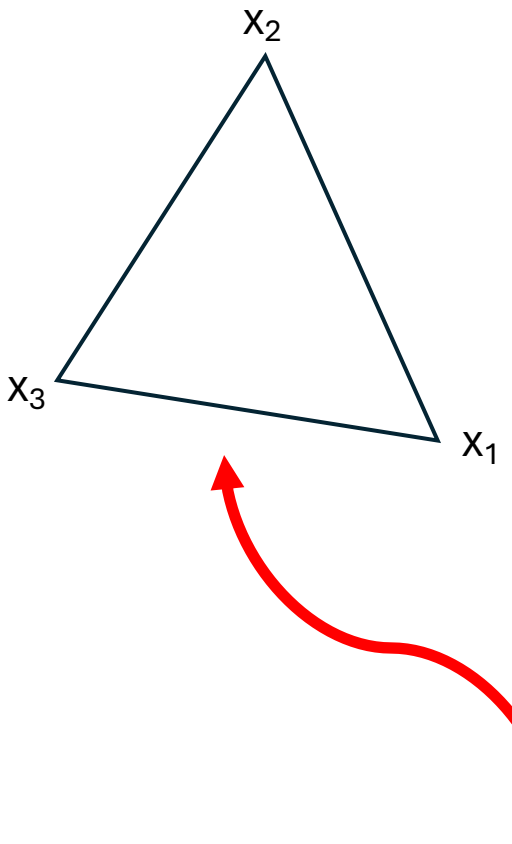


Sample projected point y on circle and evaluate $\mathcal{D}(y)$

If $f(\mathcal{D}(y)) < f(\mathcal{D}(x))$ then set $x \leftarrow y$, move grid to be coincide with x and restart local search from x

If max-points are examined without improvement: x is *h*-local min

Adapted Nelder-Mead search for RK-GRASP

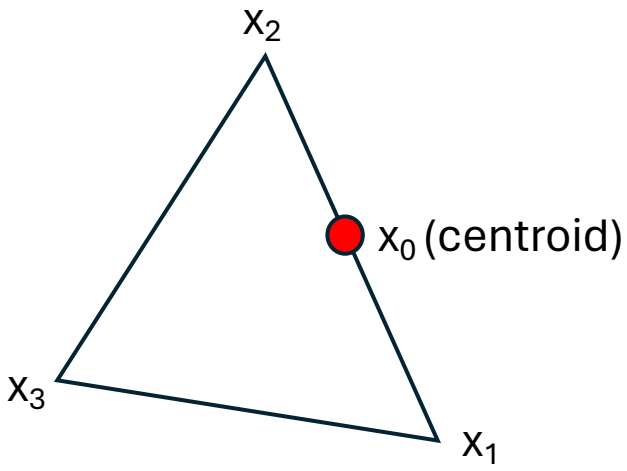


$$f(\mathcal{D}(x_1)) \leq f(\mathcal{D}(x_2)) \leq f(\mathcal{D}(x_3))$$

where one of the solutions is the semi-greedy solution and the other two are elite solutions found during the search.

The three solutions form the N-M simplex

Adapted Nelder-Mead search for RK-GRASP



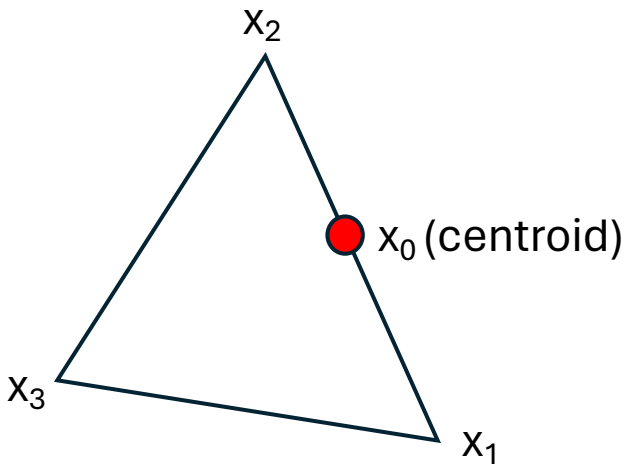
Five moves:

1. Reflection
2. Expansion
3. Inside contraction
4. Outside contraction
5. Shrinking

Instead of using standard Nelder-Mead moves, we use an adapted Uniform Crossover (UX) between solutions *A* and *B* to generate moves (C). There are two types of crossovers, UX(a,b,1) and UX(a,b, −1) defined by parameter *factor*:

factor = 1	*factor* = −1
A: $a_1\ a_2\ a_3\ a_4$	A: $a_1\ a_2\ a_3\ a_4$
B: $b_1\ b_2\ b_3\ b_4$	B: $b_1\ b_2\ b_3\ b_4$
C: $a_1\ a_2\ b_3\ a_4$	C: $a_1\ a_2\ 1-b_3\ a_4$

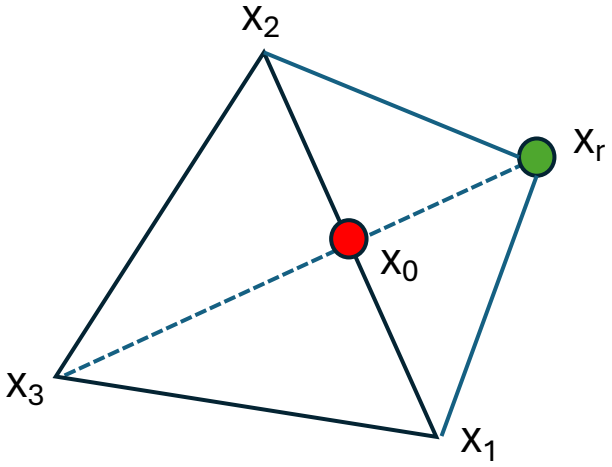
Adapted Nelder-Mead search for RK-GRASP



$x_0 = UX(x_1, x_2, 1)$

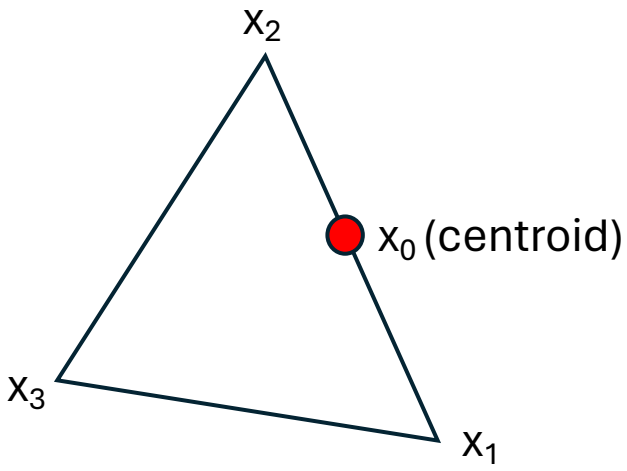
Five moves:

1. Reflection
2. Expansion
3. Inside contraction
4. Outside contraction
5. Shrinking



$x_r = UX(x_0, x_3, 1)$

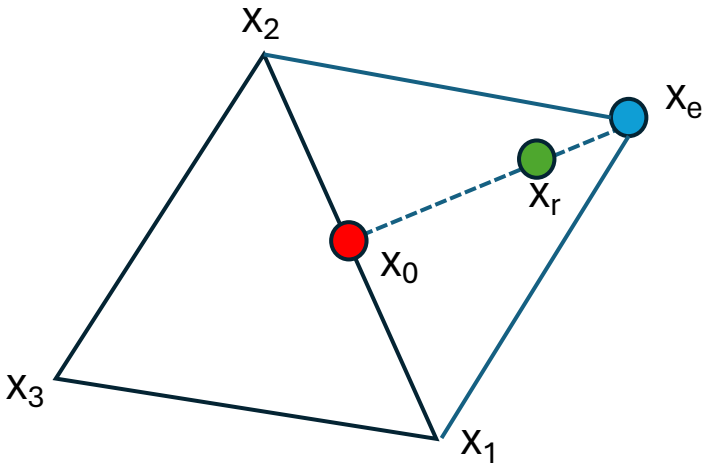
Adapted Nelder-Mead search for RK-GRASP



$x_0 = UX(x_1, x_2, 1)$

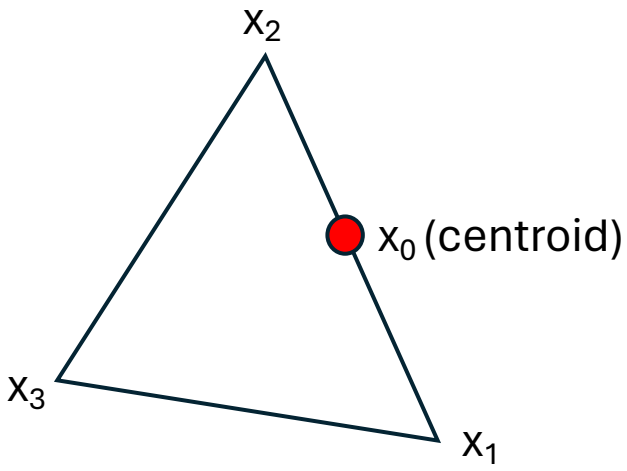
Five moves:

1. Reflection
2. Expansion
3. Inside contraction
4. Outside contraction
5. Shrinking



$x_e = UX(x_r, x_0, -1)$

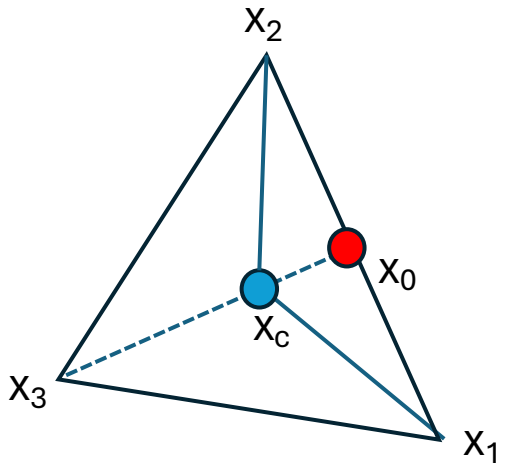
Adapted Nelder-Mead search for RK-GRASP



$x_0 = UX(x_1, x_2, 1)$

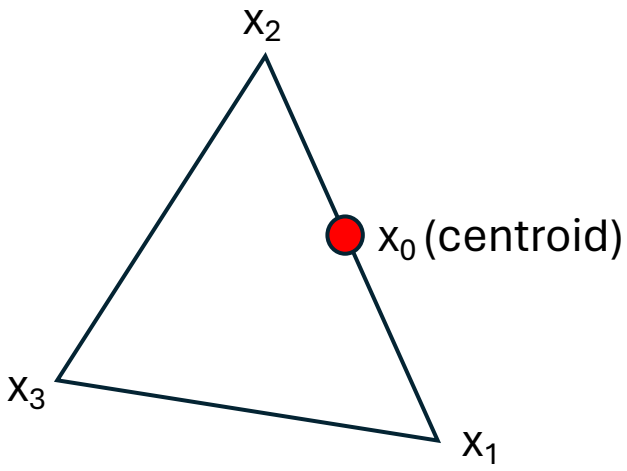
Five moves:

1. Reflection
2. Expansion
3. Inside contraction
4. Outside contraction
5. Shrinking



$x_c = UX(x_o, x_3, -1)$

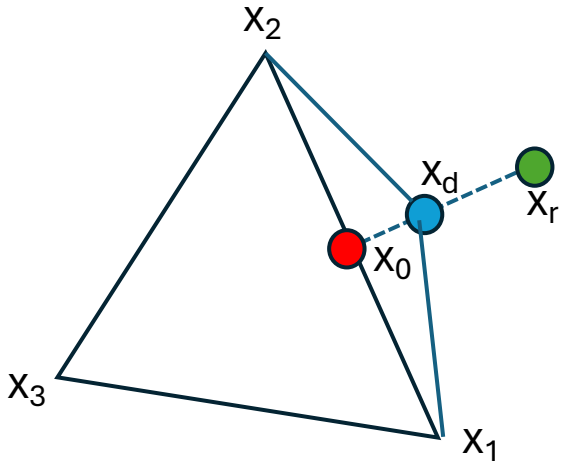
Adapted Nelder-Mead search for RK-GRASP



$x_0 = UX(x_1, x_2, 1)$

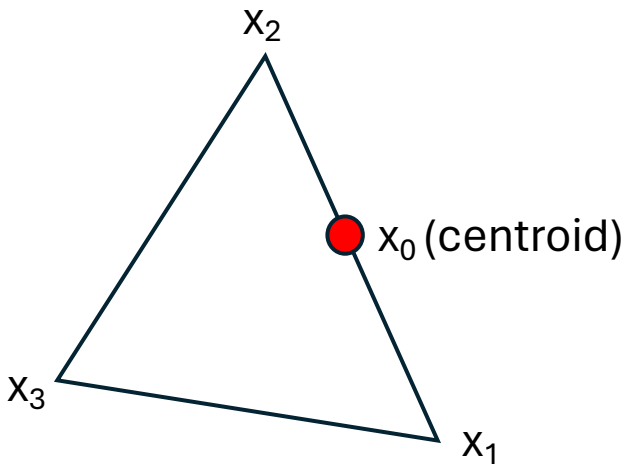
Five moves:

1. Reflection
2. Expansion
3. Inside contraction
4. Outside contraction
5. Shrinking



$x_d = UX(x_r, x_0, 1)$

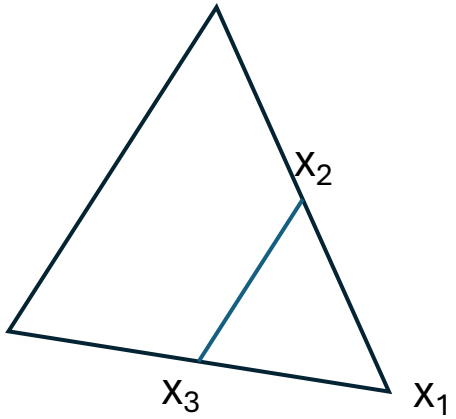
Adapted Nelder-Mead search for RK-GRASP



$x_0 = UX(x_1, x_2, 1)$

Five moves:

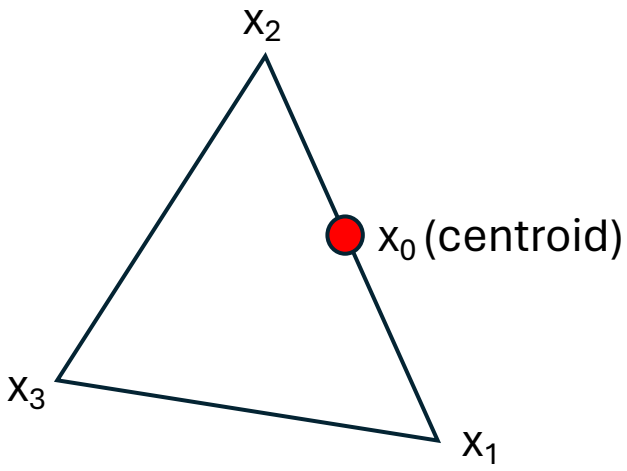
1. Reflection
2. Expansion
3. Inside contraction
4. Outside contraction
5. Shrinking



$x_2 = UX(x_1, x_2, 1)$

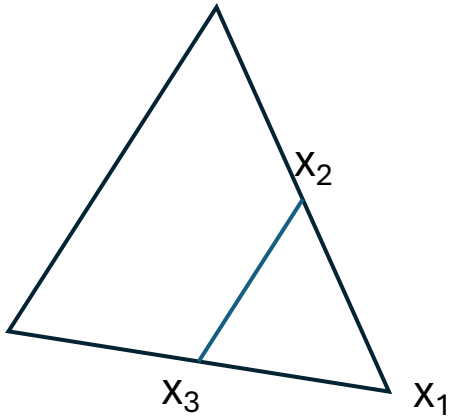
$x_3 = UX(x_1, x_3, 1)$

Adapted Nelder-Mead search for RK-GRASP



Five moves:

1. Reflection
2. Expansion
3. Inside contraction
4. Outside contraction
5. Shrinking



$x_0 = UX(x_1, x_2, 1)$

$x_2 = UX(x_1, x_2, 1)$

$x_3 = UX(x_1, x_3, 1)$

Use UX to guarantee that generated test solution is always in $\mathcal{H}_n$

Adapted Nelder-Mead search for RK-GRASP

Start with three solutions: the semi-greedy from the constructive phase, and two elite set solutions

While stopping criterion not satisfied

1. Sort simplex solutions and assign best to x_1, worst to x_3, and lousy to x_2
2. Generate centroid and reflection by applying UX between pairs of solutions in $\{ x_1, x_2, x_3\}$
3. Generate expansion and contractions
4. Update simplex if appropriate
5. Shrink if simplex is not updated

Return x_1

Random VND for RK-GRASP

There are five neighborhoods:

1. Grid search neighborhood
2. Nelder-Mead search neighborhood
3. SwapRK: Swaps pairs of random keys
4. InvertRK: Inverts random key (x becomes $1-x$)
5. Farey: For each key generate 18 random keys, each from interval $(\mathcal{F}_i, \mathcal{F}_{i+1})$, from the Farey F_7 sequence where F_7 = { 0, 1/7, 1/6, 1/5, ¼, 2/7, 1/3, 2/5, 3/7, ½, 4/7, 3/5, 2/3, 5/7, ¾, 4/5, 5/6, 6/7, 1}

Start with solution x
Initialize Neighbohood List (NL)
While NL not empty

1. Choose neighborhood $\mathcal{N}_i \in$ NL at random
2. Find best neighbor x' of x in $\mathcal{N}_i$
3. If $f(\mathcal{D}(x')) < f(\mathcal{D}(x))$ then x ←x' and restart NL
4. Else remove $\mathcal{N}_i$ from NL

Return x

Experimental results

As a proof of concept, we test our three RK-GRASP variants:

1. RK-GRASP with grid search
2. RK-GRASP with Nelder-Mead search
3. RK-GRASP with Random VND search

on five NP-hard combinatorial optimization problems:

1. Traveling salesman problem
2. Hub location problem in graphs
3. Steiner triple set covering problem
4. Node capacitated graph partitioning problem
5. Job sequencing and tool switching problem

For each problem we propose a decoder and use the same solver to solve all problems. Our objective is to show that RK-GRASP is problem agnostic.

Experimental results

Framework, instances and experimental results are available at
https://github.com/antoniochaves19/RK-GRASP

There are three parameters to be set:

1. Starting grid size h_s=0.125
2. Final grid size h_e = 0.00098
3. Grid search sampling density θ = rand[0.2, 0.4]

Experimental results

Framework, instances and experimental results are available at
https://github.com/antoniochaves19/RK-GRASP

There are three parameters to be set:

1. Starting grid size h_s=0.125
2. Final grid size h_e = 0.00098
3. Grid search sampling density θ = rand[0.2, 0.4]



Taken from previous papers on C-GRASP

Experimental results

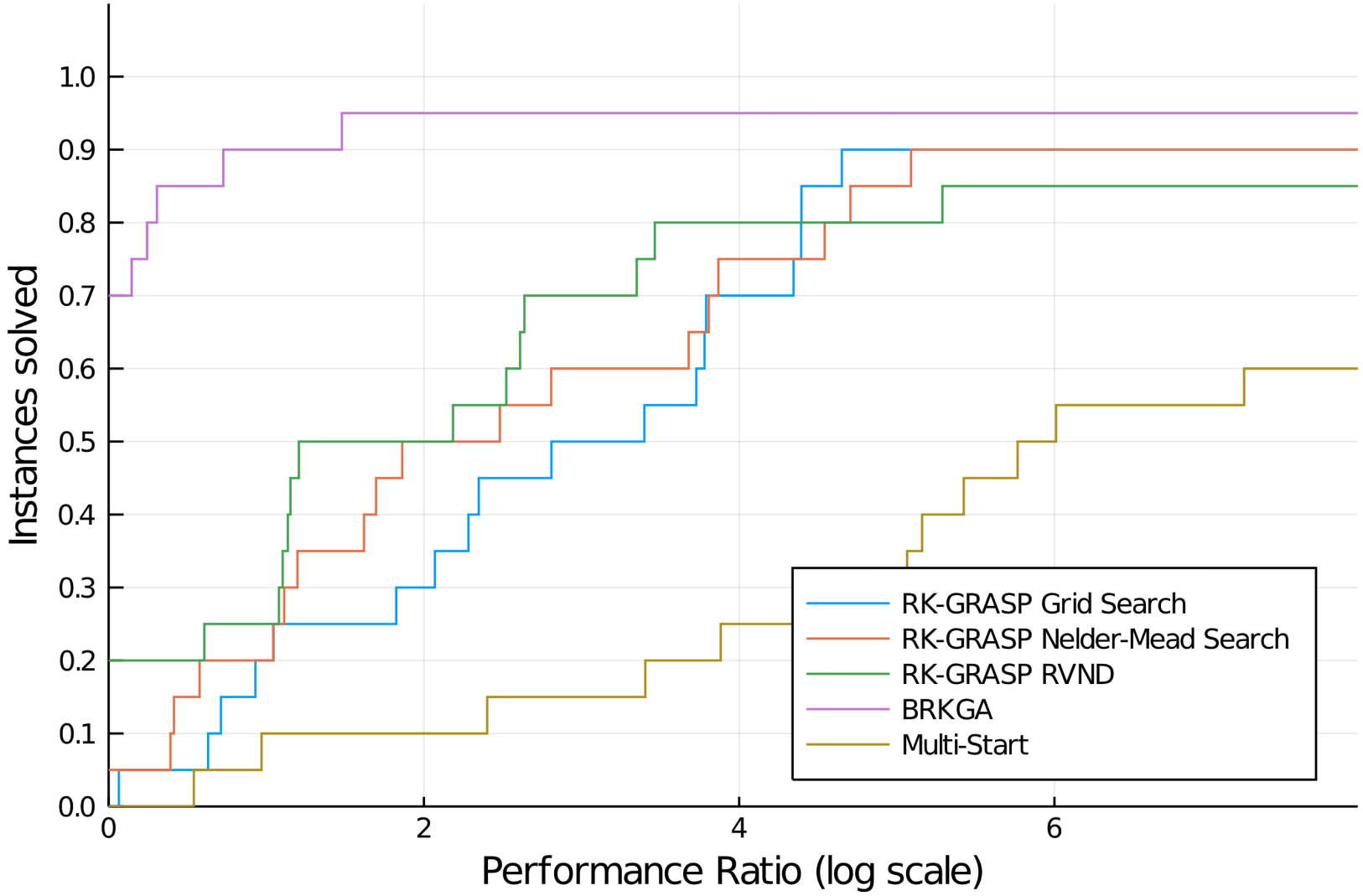
All algorithms were coded in C++ (compiled with GCC) and experiments were run on a Dual Xenon Silver 4114 20c/40t 2.2Ghz processor with 96 Gb of DDR4 RAM (running CentOS 8.0 x64 Linux).

For each instance, all algorithms were run five times for the same running time (in seconds, all a function of instance size):

1. Traveling salesman problem: (number of cities) seconds
2. Tree hub location problem: (number of nodes in graph) seconds
3. Steiner trip set covering: (number of columns) seconds
4. Node capacitated graph partitioning problem: (number of base stations) seconds
5. Job sequencing and tool switching problem: (number of jobs) seconds

TSP

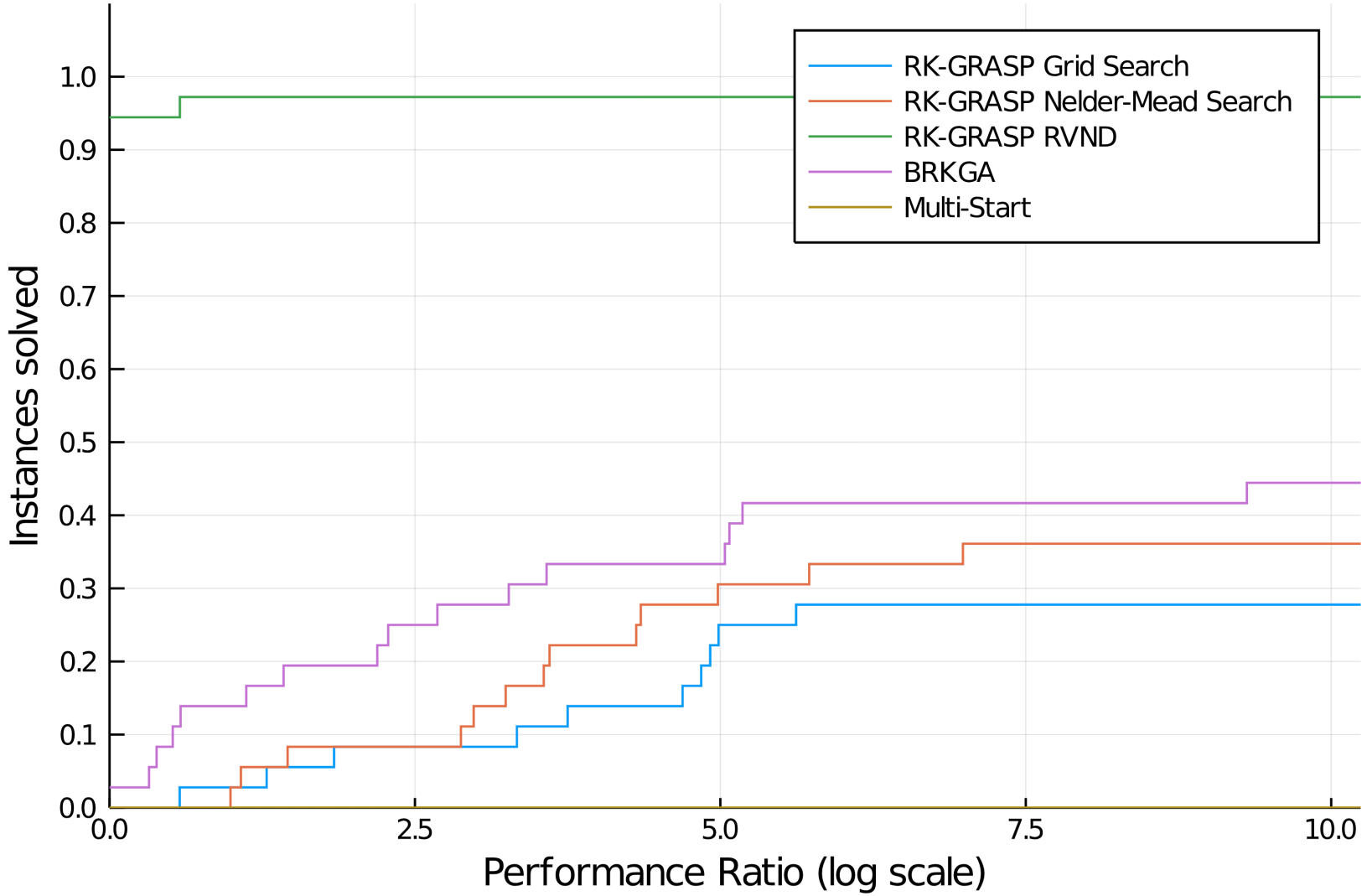
Performance profile of runtime



TSP

	RK-GRASP Grid search	RK-GRASP Nelder-Mead	RK-GRASP RVND	BRKGA	Multi-Start
median	38974.5	38872.5	38694.0	38727.0	39545.0
mean	45348.0	45214.8	45147.6	45166.5	45810.9
avg gap	0.25	0.24	0.07	0.12	1.66
#instances	20	20	20	20	20
#BKS	13	14	14	14	6
#newBKS	0	0	0	0	0

Tree Hub Location



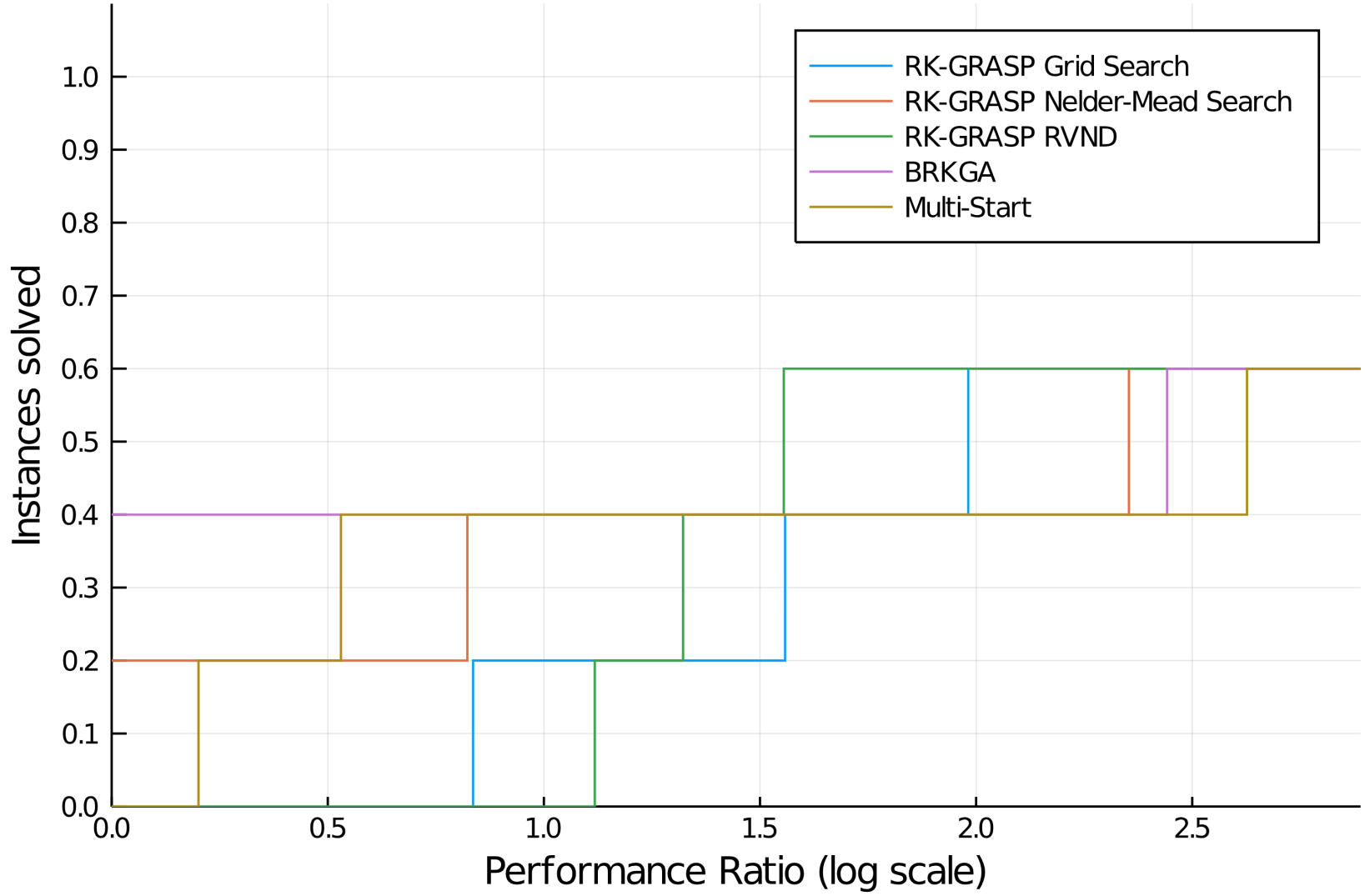
Performance profile of runtime

Tree Hub Location

	RK-GRASP Grid search	RK-GRASP Nelder-Mead	RK-GRASP RVND	BRKGA	Multi-Start
median	24811.1	22965.5	22610.4	22962.4	31175.4
mean	35575.4	34583.1	33363.1	34255.5	42441.3
avg gap	4.94	1.51	-0.06	1.39	28.72
#instances	36	36	36	36	36
#BKS	7	12	36	10	0
#newBKS	0	0	5	1	0

Steiner triple set covering

Performance profile of runtime

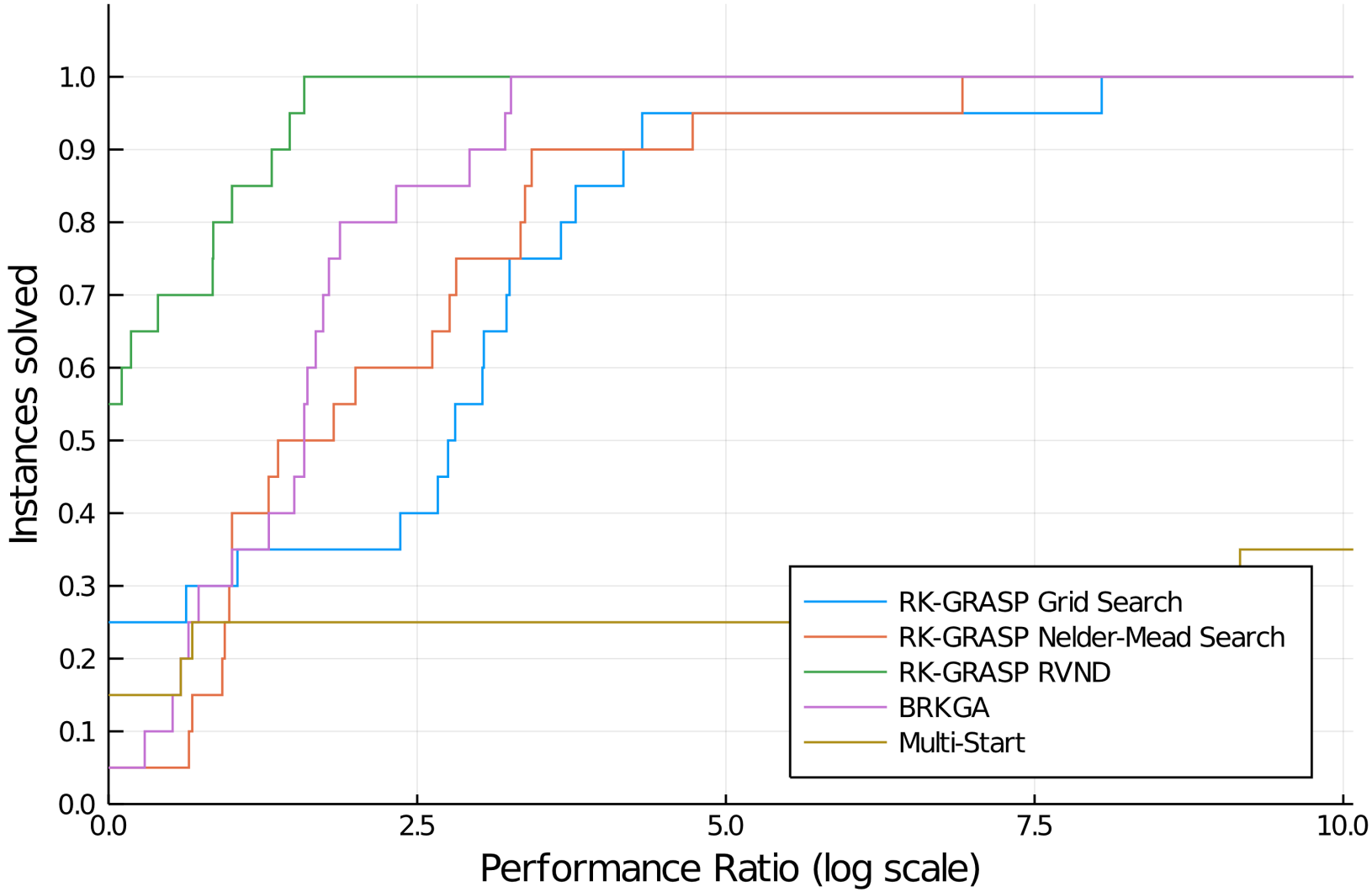


Steiner triple set covering

	RK-GRASP Grid search	RK-GRASP Nelder-Mead	RK-GRASP RVND	BRKGA	Multi-Start
median	203.0	198.0	198.0	198.0	202.0
mean	272.2	271.5	268.9	270.7	272.4
avg gap	1.74	1.68	0.98	1.40	1.83
#instances	5	5	5	5	5
#BKS	2	2	3	3	2
#newBKS	0	0	0	0	0

Node capacitated graph partitioning

Performance profile of runtime

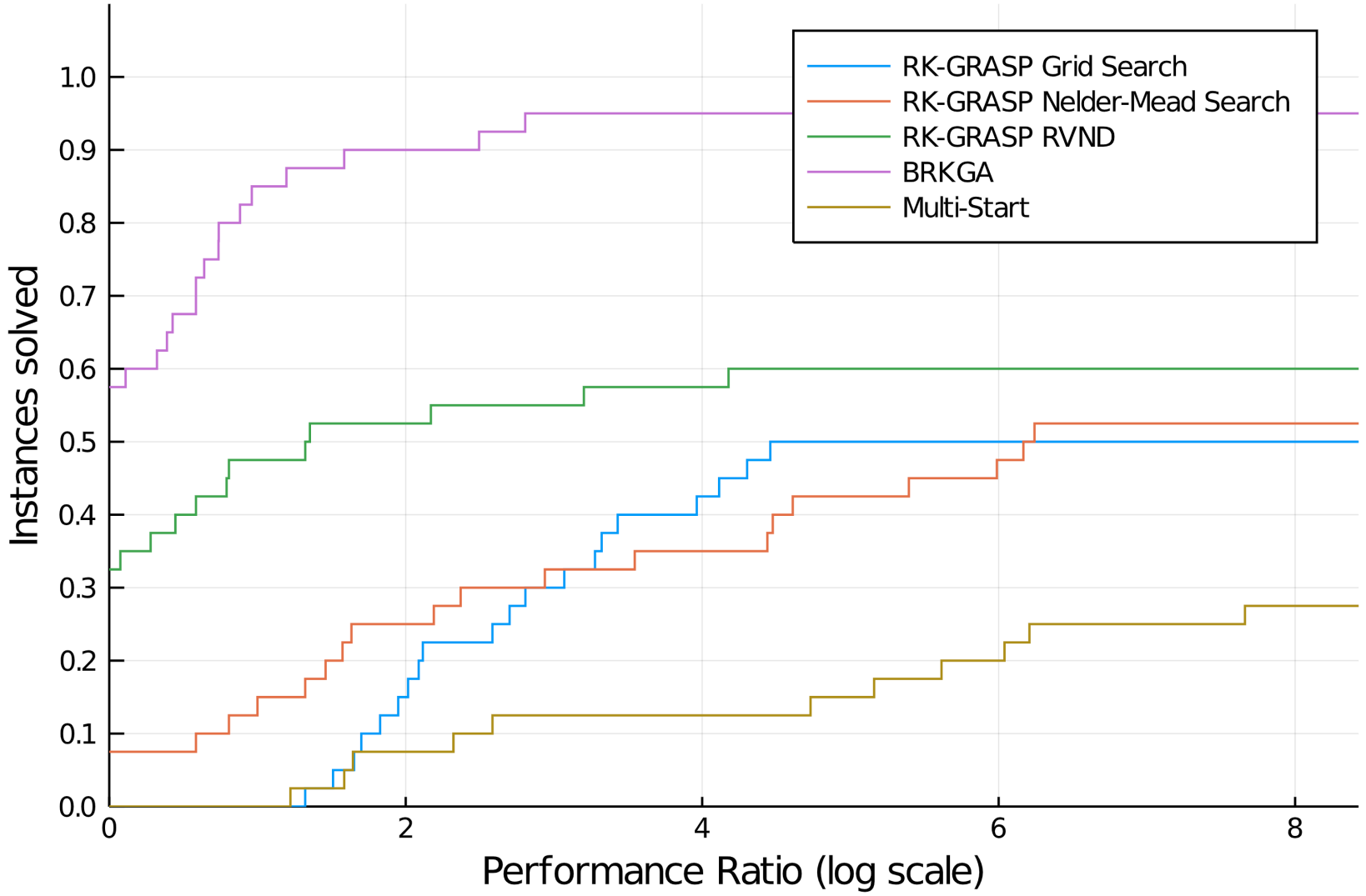


Node capacitated graph partitioning

	RK-GRASP Grid search	RK-GRASP Nelder-Mead	RK-GRASP RVND	BRKGA	Multi-Start
median	20699.0	20666.0	20666.0	20666.0	22756.0
mean	66228.0	65109.5	64552.6	64848.4	70850.2
avg gap	0.23	-0.44	-0.62	-0.53	4.62
#instances	20	20	20	20	20
#BKS	14	19	20	20	7
#newBKS	0	1	6	1	0

Jobs sequencing and tool switching problem

Performance profile of runtime



Jobs sequencing and tool switching problem

	RK-GRASP Grid search	RK-GRASP Nelder-Mead	RK-GRASP RVND	BRKGA	Multi-Start
median	52.0	52.5	51.0	51.0	64.0
mean	79.9	78.0	75.8	76.7	96.4
avg gap	1.75	0.57	-0.67	-0.33	14.95
#instances	40	40	40	40	40
#BKS	19	24	39	37	11
#newBKS	0	1	14	7	0

Concluding remarks

We adapted C-GRASP to work with random-key decoders and act as a Random-Key Optimizer (RKO).

This is a problem-agnostic GRASP, where only a decoder needs to be built for each problem that is solved. The solver can be reused when a new problem comes about.

We propose two new local search heuristics, adapted Nelder-Mead search and a Random Variable Neighborhood Descent, in addition to the standard grid search used in standard implementations of C-GRASP metaheuristic algorithms.

Concluding remarks

We tested the three variants of RK-GRASP, as well as a BRKGA and a random multi-start heuristic, on instances from five NP-hard combinatorial optimization algorithms.

RK-GRASP with RVND search emerged as the best method for the five classes of problems tested. The improvement is statistically significant when compared to the other two RK-GRASP variants, as well as BRKGA and Multi-Start.

The improvement of BRKGA over RK-GRASP with Nelder-Mead and Grid Search is statistically significant and hence BRKGA emerges as the second best algorithm. RK-GRASP with Nelder-Mead is better than RK-GRASP with Grid search in a statistically significant way.

Concluding remarks

All RK-GRASP variants, as well as BRKGA outperformed Multi-Start.

RK-GRASP with RVND improved the BKS on five instances of tree hub location (14%), six instances of node capacitated graph partitioning (30%), and 14 instances of job sequencing and tool switching (35%).

RK-GRASP with RVND found solutions at least as good as the BKS in 92.5% of the tested instances.

Thanks

A full version tech report is titled “A RANDOM-KEY GRASP FOR COMBINATORIAL OPTIMIZATION,” coauthored by ANTONIO A. CHAVES, MAURICIO G.C. RESENDE, and RICARDO M.A. SILVA is available at arXiv.org.



antonio.chaves@unifesp.br ■ mgcr@bekeley.edu ■ rmas@cin.ufpe.br